/ MIGRATING YOUR DATA FROM DRUPAL 7 TO DRUPAL 10/11

A Step by Step Guide

by Mauricio Dinarte &
Tag1 Consulting, Inc.

tag1.com TAG1

To Wendy.

Contents

Series Overview 1

Planning and Preparation Phase 2

01 - Getting to know your migration 3

02 - Source site audit - High level overview 7

03 - Source site audit - In-depth analysis 10

04 - Assumptions of the Migrate API 14

05 - Avoiding entity ID conflicts 18

06 - Known issues 23

07 - Drupal entities overview 27

Example Repository and Automated Migration 33

08 - Example repository setup and Drupal 7 site audit 34

09 - Performing an automated migration 40

10 - Generating migrations with Migrate Upgrade 49

11 - Customizing the generated migration 56

12 - Syntax and structure of migration files 64

13 - The migration process pipeline 70

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

Migration of Configuration 80

14 - Migrating content types 81

15 - Migrating vocabularies and Drupal 7 field collections into Drupal 10 paragraphs 95

16 - Preparing for field migrations 105

17 - Migrating field storage and instance settings 118

18 - Migrating field widgets settings 135

19 - Migrating view modes and field groups 146

20 - Migrating field formatter settings 154

21 - Applying Drupal recipes to add media types 164

22 - Configuring text formats, editors and user roles 170

Migration of Content 178

23 - Altering AUTO_INCREMENT table values to prevent entity ID conflicts 179

24 - Public and private file migrations 195

25 - User and taxonomy term migrations 217

26 - Entity type conversions. From nodes to user and taxonomy terms 237

27 - Paragraph migration. Creating custom process plugins 253

28 - Migrating media. Creating custom source plugins 267

29 - Migrating nodes - Part 1 287

30 - Node migrations - Part 2 306

31 - Debugging tips, performance considerations, Drupal CMS, AI-assisted migrations and more! 328

Series Overview

This series is designed to equip you with thorough explanations and step-by-step guidance for migrating data from Drupal 7 to Drupal 10 or 11, ensuring a seamless transition to the latest platform version.

As this series progresses, we will introduce a DDEV-based development environment, complete with instructions to set it up. This environment is prepared with a Drupal 7 site already installed and a Drupal 10 site set as the migration target. Perfect for developers and site owners alike, we will walk you through the entire data migration process, covering a wide range of topics along the way, empowering you to migrate your own sites with confidence. While the example site provided in the repository uses Drupal 10, all code examples are also compatible with Drupal 11. We will highlight any feature that is only available in a specific version of Drupal.

About the Author

Mauricio Dinarte is a senior software engineer at Tag1 Consulting, with more than a decade of web development experience. Since 2019, he has worked on a wide array of large scale Drupal migration projects, which has allowed him to amass a deep understanding of the Migrate API. He also enjoys teaching and distills his experience in written articles and public talks at Drupal events around the globe. Mauricio holds a Master's degree in Business Administration. He spends his free time reading, learning languages, and writing technical documentation.

About Tag1 Consulting, Inc.

Founded in 2007, Tag1 is a global technology consulting firm with over 100 team members across 20+ countries and 6 continents. Widely respected for open source leadership as the 2nd all-time leading contributor to the Drupal platform, Tag1 specializes in technical architecture, migrations, performance + scalability, and emergency support for large-scale, mission-critical technology systems.

Planning and Preparation Phase

- 01 - Getting to know your migration
- 02 - Source site audit - High level overview
- 03 - Source site audit - In-depth analysis
- 04 - Assumptions of the Migrate API
- 05 - Avoiding entity ID conflicts
- 06 - Known issues
- 07 - Drupal entities overview

01 - Getting to know your migration

Today, we are excited to launch a detailed blog series that serves as an in-depth guide on utilizing the Drupal 10 Migrate API. This series is designed to equip you with thorough explanations and step-by-step guidance for migrating all your data from a Drupal 7 site to Drupal 10, facilitating a smooth upgrade to the most current version of the platform.

As we progress, we will introduce a DDEV-based development environment, complete with instructions to set it up. This environment is prepared with a Drupal 7 site already installed and a Drupal 10 site set as the migration target. Through the course of this series, we will walk you through the entire data migration process, covering a wide range of topics along the way, empowering you to migrate your own sites confidently.

The Migrate API, created and co-maintained by Tag1 team members, is a flexible and powerful system that can be used to collect data from multiple sources and import it into Drupal. While migrating data from Drupal 7 to modern versions of Drupal is a common use case, it is just one of many supported scenarios. In fact, the Migrate API is a complete extract, transform, and load (ETL) framework that can be used to import data from CSV files into your site, among many other scenarios.

Many years ago, I worked on a Drupal 6 to 9 data migration that used three different sources: the Drupal 6 site, a Drupal 9 site used for content staging, and multiple CSV files, which contained multilingual content. That data migration is an example of how flexible the Migrate API is. While the focus of this series is the migration of data from a Drupal 7 site, any general knowledge you have about the API and ETL frameworks will prove useful. For more information, I suggest you read another series of articles I wrote on the topic: 31 Days of Drupal Migrations.

Before a deep dive into the technicalities of performing a data migration, it is crucial to understand your current site. In this blog post, we will take a step back and discuss the different components of a Drupal site. This will help you evaluate different approaches and tools that can be utilized for the migration. To keep things simple, we will not touch on additional services like

search, distributed content storage, caching layers, or proxies. Instead, we will focus on the Drupal application layer:

- **Configuration**: this determines the structure of the site. It includes all the different content types and the fields attached to them, taxonomy vocabularies, menus, user roles and permissions, views, media types, commerce product types, field collection and paragraph types, and webforms. General site settings are also part of the configuration. For example, site name, slogan, active default and administrative themes, enabled modules, date and time formats, text formats and WYSIWYG profiles, image styles, and pathauto patterns for url aliases.
- **Content**: this is user-submitted data. Nodes, taxonomy terms, files (addressed in the next point), menu items, users, media items, products, and webform submissions are all considered content. A single page on the website can contain a variety of content pieces. For instance, you can have an article (node) that includes a featured image (file), tags (taxonomy terms) related to the content presented, and a reference to the author (user).
- **Files**: these are a special type of content in Drupal. They are assets uploaded by users to be managed and served by Drupal. For example, images, text documents, and spreadsheets. While videos can be uploaded directly to the site, they are usually uploaded to dedicated video hosting providers such as YouTube and Vimeo. After they are uploaded, the videos can be embedded within Drupal.
- **Modules**: they provide functionality to the website. Drupal core itself comes with dozens of modules. You can also provide more functionality by adding modules from the contributed repository or by writing custom ones. Modules mostly consist of PHP code.
- **Themes**: they control the appearance of the site. Fonts, colors, and responsive behavior are all responsibilities of the theme. Similar to modules, Drupal core ships with a handful of themes, and more available in the contributed repository. Most sites use custom themes that align with the branding of the organization. Themes mostly consist of PHP code, CSS, JavaScript, and assets, such as images.

Configuration is usually created by site builders and developers. Content and files are generally added by content editors and, when allowed, by site visitors. Modules and themes are mostly the responsibility of backend and frontend developers, respectively.

Another aspect that is important to understand is where each of these site components live. Configuration and content are stored in the database. File references are stored in the database, but the assets themselves live in the file system. Modules and themes live in the file system.

Now, we can consider three different approaches for migrating to Drupal 10:

- **Manually create configuration and content in the new site.** This is a valid approach for small sites. If the site structure is simple and there is not much content, installing a fresh Drupal 10 site and recreating everything from scratch is achievable.
- **Manually create the configuration and automatically migrate the content.** This is the most common approach I have seen over the years. Organizations usually take the opportunity of migrating to Drupal 10 to make improvements and remove things that are no longer needed on their site. This is often reflected in changes to the content model and general site structure. For this approach, site builders and developers manually create the new site configuration per the new requirements. Then, content is moved into the new site structure.
- **Automatically migrate configuration and content.** With Drupal 7's end of life approaching, I have seen more organizations take this approach. If little or no configuration changes are required on the new site, you can automate the migration of the current configuration. If minor tweaks are needed, those can be handled manually on the new site. Once the content model is in place, the content is moved over.

Notice that in the three scenarios, the focus is on configuration and content. That is because the Migrate API is designed to help automate the migration of those two components only. Technically speaking, files are also considered content in Drupal. As such, they are also supported, but neither modules nor themes are part of the scope of the Migrate API.

If your site uses contributed modules and themes, you must verify that they have Drupal 10 compatible versions. If not, you can look for alternative projects that serve as replacements or offer similar functionality. For instance, the Drupal 10 entityqueue module is the successor of the Drupal 7 nodequeue module. If no replacement or alternative is found, consider help porting the module or theme to Drupal 10. As a last resort, you can write custom code for any required feature that is otherwise not readily available.

The code architecture in Drupal 10 is significantly different than that of Drupal 7. Because of this, any custom module or theme needs to be recreated. In the case of modules, you can use the Module Upgrader to get started in the porting process and finish with manual changes. As for themes, there are no tools that assist with the porting. Before writing any custom code, consider your alternatives. In many situations, that which required custom code before could be covered by new functionality added to Drupal core or contributed modules/themes. Alternatively, some features may no longer be needed and can be dropped from the migration.

Note: Matt Glaman has released a project named Retrofit. It aims to run Drupal 7 code on a Drupal 10 site without having to make modifications to it. While I have not tested it myself, Matt has an impressive track record of impactful contributions to Drupal. He also contributes to PHPStan, Drupal Rector, Upgrade Status, and more.

Before closing, it is appropriate to acknowledge that while the Migrate API is the most commonly used method, it is one of many ways to move data into Drupal. Another popular approach is the Feeds module and its ecosystem. There is also a module that uses the Feeds user interface on top of the Migrate API. Another option is to write custom code to get data into Drupal. No matter the approach, I highly recommend that when importing data, Drupal APIs are used instead of direct writes to database tables. Drupal is a flexible and complex system. When a piece of content is created it will likely end up in multiple tables. When using Drupal APIs, access controls, validation, caching, and other systems get a chance to act upon the data that is being imported. Writing to database tables directly bypasses a lot of the safety checks that Drupal implements.

In our next post, we will explain how to audit a Drupal 7 site in preparation for a Drupal 10 upgrade. We will also explain some assumptions and limitations in the Migrate API and how to work around them. And, right before diving into the hands-on exercises, we will explain a key concept in data migration projects: Drupal entities. Stay tuned!

02 - Source site audit - High level overview

One of the most impactful things to do in preparation for any migration project is to understand the source site. The more insight you gather into how the current site was built, the better equipped you will be to perform the migration. While reviewing the site, analyze what works well, what can be improved, and what can be dropped. The audit can be broken down in two parts: a high-level overview of the site architecture, and an in-depth analysis of the content model. This blog post covers the former and the next one in this series will explain the latter. Note that for better results, there must be fluid communication between the developers in charge of performing the migration and the product owners who use the site on a regular basis.

Let's start by noting all elements of the site that are no longer needed or functioning. Examples of this are: content types with no nodes associated, fields that were never used, webforms with no submissions, and disabled views or rules. This category should also include modules that integrate external services that have been shut down. In recent years, Google shut down the Sheets v3 API and Universal Analytics for Analytics. In cases like these, you might need to install newer versions of modules, apply patches to make them compatible with new versions of the APIs, or find alternative modules.

Next, identify cruft that does not need to be moved into the new site. Content types or fields labeled "old," "deprecated," "do not use," or similar, are glaring examples of things that do not have to be migrated to Drupal 10. Also, consider whether blocked users or unpublished nodes need to be migrated. In the case of nodes, the analysis should be done on a per content type basis. For example, if you are using workbench moderation to moderate content for a content type, it is expected to have unpublished nodes while they go through the publication workflow.

Also, consider how the business has changed since the site was originally launched. Previously, I worked on a migration from a Drupal 6 site that had been in operation for over a decade. A key aspect of the site was to present up-to-date information about retailers across multiple countries and languages.

By the time the migration project started, the organization no longer had representation in a handful of countries. Therefore, it was decided that any information associated with countries that had no current retailers should not be moved to the new site. Additionally, processes that have been offloaded to third-party services may help simplify the upgrade procedure. For instance, if order processing is handled by an external provider, it might not be necessary to implement a shopping cart in the new site nor migrate legacy order data.

If the current site is multilingual, do translations need to be carried over? In the Drupal 6 project I mentioned, some languages were dropped. If there were no retailers in a specific country, any language used exclusively in that country did not need to be migrated. Sometimes multilingual features are configured but never fully adopted, or abandoned due to lack of resources. Another scenario is when curated translations are discarded in favor of services like Google Translator. Whether for technical reasons, a change in business needs, or budget constraints, evaluate whether some, or all, translations can be left out of the upgrade.

Analyze why and how enabled modules are being used. I have seen sites use the Webform module to build a contact form. Another site used the Rules module to send an email when content is updated. And, there was a project that used the Flag module to mark a piece of content ready for review. While all are valid use cases, these examples introduced large codebases when barely utilizing the features each module provided. The Webform implementation can be replaced by Drupal core's Contact module and the Contact Storage contributed module if persisting submissions was a requirement. The rule to send an email can be implemented using custom code. And the flag to signal content that is ready for review can be substituted by a boolean field on the content type. Webform, Rules, and Flag are useful modules and their use is justified in many scenarios. The point here is to understand to what extent you are currently leveraging enabled modules. When planning the update, evaluate whether their implementation could be replaced by Core functionality, other contributed modules, or custom code.

Do you use the same installation to serve multiple sites? Is it a single codebase and multiple databases in a multisite Drupal installation? Or is it a single database serving multiple domains via the Domain access module? No matter the technical implementation, careful evaluation is needed when a single installation is used for more than one site. From a technical standpoint, check whether users' accounts and content are shared among sites. Business

needs, ease of maintenance, security implications, and complexity in the migration process should all be considered.

If your site was built using a Drupal distribution, first check if there is a Drupal 10 compatible version. If so, confirm that there is an automated upgrade path. Make sure to read the documentation regarding the upgrade procedure. It is likely that the content model and the modules to assemble the distribution have changed and manual tasks may need to be performed. An example is FarmOS, which provided an automated upgrade path from Drupal 7 to Drupal 9 (the current version at the time). If your distribution does not offer this, search for projects that could help with the migration. For instance, if your site was built on Commerce Kickstart for Drupal 7, the Commerce Migrate can help migrate commerce-related data into Commerce for Drupal 10, whether the new site uses the distribution or not.

Even though most of the things we have mentioned are technical in nature, it is important to present the results to different stakeholders. Their perspective and feedback will enrich the analysis. It will also help get the team aligned and keep expectations in check for the project. In the next article, we will continue to audit the site with an in-depth analysis of the site structure.

03 - Source site audit - In-depth analysis

In the previous article, we analyzed the site from a high-level perspective. It is time to dive deeper into the site structure. Use this source site audit template as a starting point. Feel free to add more elements or remove some if they are not relevant for your project. Filling the template can take a couple of hours, but it will save a lot of time down the road. Once the template is filled out, the developers performing the migration and the product owners should review it together and discuss the findings.

The first part of the template is an analysis of enabled projects: modules and themes. For each one, determine whether there is a version compatible with Drupal 10. If so, pay attention to the stability of the project. From highest to lowest stability, projects can have full releases, release candidates, beta, alpha, and development versions. Ideally, a project has a full release. Otherwise, beta versions tend to be stable enough and can be used. Note that from a security standpoint, only full releases are covered by the Drupal security team, so consider internal policies when conducting your evaluation. For instance, I have worked with organizations that only allow using stable modules with security coverage. In the list of projects, include any custom module or theme and add a note stating what each one is responsible for.

To fill out the module and themes tab in template, I recommend using the Upgrade Status module. Upon installation, it will automatically check for Drupal 10 compatibility for enabled projects and, in the cases where none is found, provide recommendations for alternative modules where available. For example, the Drupal 7 Address Field module is not available in Drupal 10, but the Address module can be used in its place. Most of the Date contributed modules made it into Drupal 10 core, but some features like recurring dates might need extra modules. Note that the availability of a module does not guarantee an automated upgrade path. Sometimes, one module provides the upgrade path for another. For instance, you can use Drupal 10 Paragraphs in place of Drupal 7 fields collections. Another example is the Migrate: URL2Link module, which provides a migration path for the URL module on Drupal 7 to the Link module

in Drupal 10 Core. Even if no upgrade path is available, being able to use the module in the new site may suffice. Either a manual configuration or a custom migration can make up for the lack of an upgrade path.

PROJECT	CURRENT VERSION	LATEST UPGRADE AVAILABLE
Address Field Includes: *Address Field*	7.x-1.3	Replaced by Address 2.0.1
Chaos Tool Suite (ctools) Includes: *Chaos tools*	7.x-1.21	In Drupal core since 8.x Mostly replaced by core APIs, including modal dialogs, exportables, and plugins. Excludes Page Manager and Form Wizard.
Date Includes: *Date, Date API, Date Views*	7.x-2.14	In Drupal core since 8.x No recurring dates support. See Date/time Extras: Provide a field for repeating / recurring dates and Recurring Dates Field.
Email Field Includes: *Email*	7.x-1.3	In Drupal core since 8.x E-mail address contact forms are not supported by core.
Entity API Includes: *Entity API*	7.x-1.10	In Drupal core since 8.x Replaced by core Entity system functionality.
Entity reference Includes: *Entity Reference*	7.x-1.9	In Drupal core since 8.x
Pathauto Includes: *Pathauto*	7.x-1.3	8.x-1.12
Telephone Includes: *Telephone*	7.x-1.0-alpha1	In Drupal core since 8.x

Figure 1. Partial screenshot of the report provided by the Upgrade Status module

Next, list and review all content types, how many nodes are for each type, and the percentage of nodes that are unpublished. Content types with few published nodes can be removed, or their content manually recreated, in the new site. If a content type has no nodes, remove it altogether. Always keep in mind how a content type is being used. For example, using the Workbench Moderation module can make a content type have a high percentage of unpublished nodes, yet the content type and the nodes need to be preserved.

Consider how the different content types are being used and whether they need to be recreated in the new site. It is possible that the content types are needed, but the nodes are not. For each content type, take note and provide examples of whether the node is used as a standalone page or as part of a listing page. This will help in other aspects of the migration project, such as estimating theming efforts. Even if obvious in most cases, be sure to note how nodes of

each type are created, and whether they are embedded as part of other content types or entities.

For taxonomy terms, it is useful to know how many terms are in each node. Also, take note of any vocabulary that has fields attached to them. All vocabularies that do not have fields can be migrated in a single migration. For those with fields, a separate migration is necessary. The same analysis can be applied to other fieldable content entities like paragraphs and field collections.

The next step is to create a list of all the fields on the site. Drupal 7 already includes a field list report at **`/admin/reports/fields`** which can be referenced directly. It is particularly important to note the different field types that exist and which modules provide them.

A technical note: In Drupal 7, a field could be shared across bundles of multiple entity types while, in Drupal 10, fields can only be shared among bundles of the same entity type. In Drupal 7, it was possible to share a field between a content type and a taxonomy vocabulary. In Drupal 10, each entity will have its own field, even if it has the same name. This might affect your migration strategy.

In the views portion of the template, make sure to take note of the base table. That is, whether the view was initially set up to list nodes, users, files, taxonomy terms, or another entity. Also, include information about plugins that are provided by modules outside of Drupal core and Views itself. For example, Views bulk operations, Views data export, Better exposed filters, and Views slideshow. Views migration is not supported as part of the core Migrate API. Modules like Views migration take a best-effort approach in migrating them. The generated migrations need to be reviewed and, where necessary, manually updated. The template suggests that you keep one record per view, but a view can have multiple displays. If you want a more granular analysis, add a column for the display.. If you decide to take a more granular approach, focus on the views that you know are critical for the project.

For the webforms tab, take note of how many submissions have been captured. Additionally, the publication status of the node and the webform itself are useful to determine whether they should be migrated or not. Webforms can include extra functionality, like sending the collected data via email or to an external API endpoint. The Webform: Migrate module does a very good job of automatting the migration of configuration and content related to webforms. Similar to Views, a manual review and update process may be necessary.

In the files tab, list all the schemes currently in use on the site: public,

private, temporary, etc. Are you using private files? Are you using an S3-like backend, for your public files with the S3 file system module? What are the mime types of uploaded files? If using the File Entity module in Drupal 7, what are the different media entities available? I tend to find unexpected things when reviewing Drupal's managed files on sites that have been running for a couple of years. At the very least, determine which schemes and mime types need to be migrated.

Regarding access controls, what roles are available on the site? Can some of them be merged, removed, or separated into multiple roles? What about the permissions for each of them? Should blocked users be migrated? Are there modules like Profile 2 that integrate with the user entity? If so, look at the Profile module in Drupal 10 for a feature parity and a migration path.

For text formats, ask yourself: what text formats are in use? Are they configured with a WYSIWYG editor? What text filters are being used? Which roles have access to which text formats? The `PHP code` filter provided by the PHP module should raise red flags in terms of security. If media is embedded as part of rich text fields, the Media Migration module can help with migrating those as well as converting the underlying references to media types.

Next, compile a list of image styles and effects being used. If the upgrade includes a redesign, some of them might not need to be migrated. Or, you can choose to use a completely different set of image styles in a responsive theme implementation. While not common, it is possible that a module relies on specific PHP extensions to work properly. That is, in addition to `ext-gd`, which is already a requirement for Drupal core.

When you fill out the template, aim to strike a balance between what is useful information and the time needed to collect it. Depending on the size of the project and the modules in use, it can take one to five hours to gather the data. Once collected, present it to your stakeholders and discuss it together.

04 - Assumptions of the Migrate API

As pointed out in the first article, the Migrate API scope is to move over content and configuration. Porting modules and themes are out of scope. With that in mind, let's take a closer look at some of the assumptions and limitations. In addition to what will be presented in today's article, I highly recommend reviewing the official documentation for upgrading from Drupal 7. Drupal is ever-evolving and the Migrate API documentation gets updated regularly. Of particular importance, review the page on known issues when upgrading from Drupal 7, as things get added and removed from that list as issues are discovered and fixed.

Stay up to date

The first assumption is that your Drupal 7 site is fully up to date. That is, Drupal core itself and any contributed module or theme should be in the latest available versions. Over the years, I have seen projects delay updates unless they are security-related. And, when many updates accumulated, they were skipped out of concern that changes could break existing functionality. I have also seen projects skip new releases altogether because everything was working, so there was no perceived reason to implement changes.

Running old versions of Drupal core or contributed modules and themes is not a good idea. If a new security release is available, it is safer and faster to apply the update if you are already on the latest stable version. You also get the benefits of new features and bug fixes.

To illustrate the importance of having your Drupal 7 site up to date, let's consider the Geofield module. The latest stable version in the 8.x branch is compatible with Drupal 10 and provides an automated upgrade path for the Geofield module in Drupal 7; specifically, the `7.x-2.x` branch. I once worked on a project that used the `7.x-1.x` branch in their Drupal 7 site. The automatic migration was failing because the field schema between the two Drupal 7 branches was different. The version compatible with Drupal 10 assumed you were using the latest release for Drupal 7. In this case, a custom migration was needed to migrate the data.

The same assumption applies for Drupal 10. Make sure you are on the latest stable version of Drupal core and any contributed module and theme you plan on using. It is common for migration projects to take months to complete, so make sure to apply updates as they become available during the lifespan of the project. When new releases are available, be sure to read the release notes for anything that might affect the migration and make changes as necessary. Also, stay current with Drupal's change records, as they might highlight modifications that impact the migration system. For instance, Drupal 10.1 introduced a new password hashing algorithm. In this case, sites upgrading from Drupal 7 need to enable a new Password Compatibility module so that old passwords continue to work. Note that contributed modules like Migrate Plus and Migrate Tools also publish change records.

Enable modules on the source and destination sites to attempt an automated migration

As part of the migration from Drupal 7 to Drupal 10, the system checks for which modules are enabled on both sites. From the list of enabled modules in Drupal 10, the system checks which ones provide an automated upgrade path from Drupal 7. In many cases, having the same module enabled on both sites is enough for the corresponding data to be migrated. Examples of this are the core Node and User modules and the contributed Pathauto and Redirect modules.

Drupal 10 availability does not guarantee an automated upgrade path. For example, the core Views module in Drupal 10 does not provide an upgrade path from the contributed Views module in Drupal 7. For migrating Views, the Views migration module goes a long way in automating the process. Similarly, the Webform module is available for both Drupal 7 and Drupal 10, but there is no automated upgrade path. The Webform migrate module can be used to port webform data.

It is also possible that a Drupal 7 module is no longer available in Drupal 10, but a new module offers similar functionality and an upgrade path. For example, the Address Field module for Drupal 7 was superseded by the Address module in Drupal 10. In this case, Address in Drupal 10 provides an automated upgrade path for data stores in Address Field in Drupal 7. Another example is the Paragraph module for Drupal 10, which provides an upgrade path for the

Field collection module in Drupal 7. Also, the Profile 2 module for Drupal 7 was replaced by the Profile module in Drupal 10. At the time of writing, the automated migration of profiles requires applying a patch.

In certain instances, a Drupal 7 module is replaced by a different module in Drupal 10, but no automatic upgrade path is provided. In those cases, a third module can be used to bridge the gap and automate the migration. For instance, the Nodequeue module in Drupal 7 was superseded by the Entityqueue module in Drupal 10. Here, the Nodequeue migrate module can be used to move the data over. At the time of writing, there is an open issue to add Drupal 10 compatibility to this module. Another example is the Migrate URL2Link module that provides an upgrade path from the contributed URL field module in Drupal 7 to the core Link module in Drupal 10.

Drupal modules can contain sub-modules, like the Webform module. It is possible that the automated upgrade path to Drupal 10 was separated into a sub-module that needs to be enabled separately. This is the case for the Field Group module, which is available for both Drupal 7 and in Drupal 10. To move field group data, in addition to the primary Field Group module for Drupal 10, it is necessary to enable the Field Group Migrate sub-module.

Also, modules in Drupal 7 could have been refactored and broken into multiple projects in Drupal 10. For example, in Drupal 7, there was a single core Block module. In Drupal 10, block functionality was broken into two modules: Block and Block Content modules. The former handles block configuration entities while the latter takes care of block content entities. To properly get block data migrated from Drupal 7, you need both modules enabled in Drupal 10.

Do not create any content or configuration in the destination site

By default, the automated upgrade procedure will attempt to create a verbatim copy of the legacy site. That is, Drupal 10 will try to recreate all configuration and content present in Drupal 7. In order to do this, the assumption is that Drupal 10 is installed with the minimal installation profile, followed by the upgrade procedure. It is important to manage expectations throughout the upgrade process. Unless there are automated upgrade paths for all modules used in Drupal 7, it is likely that there will be parts of the site that will not be

ported automatically. Also, as discussed in the first article, only content and configuration will be moved by the Migrate API. Porting custom modules or themes is a separate task.

To understand this assumption, let's consider what the Migrate API does under the hood. In terms of configuration, it will try to recreate all content types, fields, vocabularies, user roles, view modes, etc. Configuration already present on the site will be overwritten if the Drupal 10 identifier matches the one from Drupal 7. Be mindful that an installation profile can create configuration. For example, the standard installation profile in Drupal 10 creates the *Article* and *Basic page* content types, the *Tags* vocabulary, several fields, a comment type, a contact form, 3 text formats of which 2 use a WYSIWYG editor, 4 user roles, and more. If you were to run the automated migration on a Drupal 10 site that already had configuration, you might end up with seemingly repeated configuration; having two comments fields in a single content type, for instance. The recommendation is to install the site using the minimal installation profile, run the automated upgrade, and adjust the configuration afterward as needed.

With regard to content, the Migrate API will attempt to move all nodes, users, taxonomy terms, files, etc. Again, if the identifier of a Drupal 10 entity matches one from Drupal 7, it will be overwritten. This is very problematic because content entities tend to reference one another. For example, a node stores a reference to the user who created it. It can also hold references to files and taxonomy terms. If content is already present in the Drupal 10 site and it gets overwritten, those relationships will be lost or point to the wrong targets. To prevent data loss, the recommendation is to not create any content until the automated process is completed. There are workarounds to prevent ID conflicts that will be discussed in future articles.

As you can imagine, not being able to create content or configuration in the new Drupal 10 site is a big limitation. In fact, having a verbatim copy of the legacy Drupal 7 site is rarely the goal of a Drupal 10 migration project. As mentioned in the previous articles, it is common to change the site configuration and not migrate content that is no longer needed. As part of this series, we will teach you how to deviate from the recommendation to perform a custom migration. We will cover how to selectively use some of the configuration and content migrations and how to write custom ones when necessary.

05 - Avoiding entity ID conflicts

By default, the Drupal 7 to 10 upgrade path preserves entity IDs. In the previous article, we explained that this would cause problems if content or configuration already exists in the destination Drupal 10 site. Let's explore this further and evaluate ways to work around the issue.

For brevity, we are going to provide examples of content entities, but what we cover can be applied to configuration entities as well. We'll present three common scenarios that can produce entity ID conflicts and provide proven solutions.

Scenario 1: The Drupal 10 site had content before the migration process started

Let's say that in the Drupal 10 site, node 1 was created as an *article* by user ID 2, tagged with taxonomy term ID 3, and had an image attachment with file ID 5. Now consider that in the Drupal 7 site, node 1 was created as an *event* by user ID 8, tagged with taxonomy term ID 13, and had an image attachment with file ID 21. Node 1 in Drupal 10 will be overwritten with data from Drupal 7: the node type, title, publication status, author, creation timestamp, any field that also exists in Drupal 7, and more.

Explicit and implicit relationships would also be overwritten. **Explicit relationships** are entity reference fields to nodes, users, taxonomy terms, files, media entities, paragraphs, groups, commerce products, etc. **Implicit relationships** are those established by base field definitions, including the content type and the user who created the node. The next article will further delve into explicit and implicit relationships.

For now, consider how disruptive the overwrite can be. Changing the content type would break field data attached to the node. The overwritten values for entity references might point to non-existent entities in Drupal 10 and, even if they existed, the data could be invalid. Imagine a node reference field that allows referencing basic pages, but the ID that is retrieved from Drupal 7 points to a node of type event.

Scenario 2: Content is created in Drupal 10 in between incremental migrations

Migrating from Drupal 7 to 10 is an iterative process and, depending on the complexity and scale of the project, it can span multiple months. It is common to break up the migration into multiple stages and let content editors interact with the Drupal 10 site along the way.

Whether for testing the migration or trying out features being developed in the new site, content eventually gets created in Drupal 10. Concurrently, the Drupal 7 site continues functioning and new content is added. The next time an incremental migration is executed, the new content from Drupal 7 will override content created in Drupal 10.

Scenario 3: Content model updates require changes to entity types

Now let's look at another scenario where in Drupal 7 there was a *speaker* content type to collect information about those presenting at a conference. As part of the migration to Drupal 10, the speaker nodes want to be migrated as user entities. In Drupal, each entity type has its own auto-incremental counter. There might be users with ID matching the node IDs of the speaker nodes. Therefore, it is not possible to match the node ID to user ID as part of the entity type conversion.

There are other scenarios that can lead to potential ID conflicts. You can read more about them on the documentation page for known issues when upgrading from Drupal 6 or 7. For reference, some are:

- The source site might have data that did not specify an entity ID, but one is required in Drupal 10.
- A module, theme, or profile in Drupal 10 generates content upon installation.
- Translations added in Drupal 10 might be overwritten as part of an incremental migration.

We've covered three common entity ID conflict scenarios that can occur in Drupal 7 to Drupal 10 data migrations. Now, let's discuss the two primary ways to work around potential ID conflicts.

Solution 1: Customize the migrations to not preserve entity IDs

While it is possible to preserve the IDs for some entity/bundle combinations and not others, it is better to take an all-or-nothing approach. That way, you treat all migrations the same and can apply the same techniques for handling entity relationships. Most of the time, this will leverage the migration_lookup process plugin to establish the relationships among entities.

Here are three key considerations for this option:

1. Public-facing URLs might depend on entity IDs.

While it is common to have URL aliases for most content, I have seen sites that include the entity ID as part of the path or in query parameters. In cases like this, redirects need to be added so the Drupal 7 URLs redirect to the URL with the updated ID in Drupal 10.

The URL alias migration that ships with Drupal 7 already accounts for changes in entity IDs. If using a custom migration for URL aliases, you would have to map the old entity IDs to the new ones yourself. The migration_lookup process plugin can help with this.

Also consider the impact that changing URLs might have on SEO and inbound traffic from external sites pointing to Drupal 7 URLs. If changing URLs is necessary, it is essential to have a marketing plan in place to expect and address dips to organic traffic post-launch.

2. External services might depend on entity IDs.

The Drupal 7 site might expose an API for external services to consume. If the API exposes entity IDs, those services might rely on them to properly function. In cases like this, I have seen a new field added to Drupal 10 to store the legacy Drupal 7 entity ID. This new field is exposed in Drupal 10's API.

3. Content stored in rich text fields might depend on entity IDs.

Depending on the Drupal 7 configuration, some modules might embed entity IDs when storing references to files, images, or other types of media. Links to

content on the same Drupal 7 site might also use the internal entity ID. Special handling is required to update the entity IDs as part of rich text fields.

For updates to media references, the Media Migration and Migrate Media Handler modules can assist. For updates to links in rich text fields, creating custom process plugins to update the references is generally the best approach.

While it is possible to write some complex process pipelines to accomplish the task, it is better for the migration to remain easy to read and maintain. If you still want to follow this approach, make sure to use DOM proposing instead of regular expressions to update HTML markup in rich text fields.

Solution 2: artificially inflate the auto-increment value for all content entities in Drupal 10

Let's say that the current Drupal 7 site has 15,000 nodes. One could manipulate the counter for nodes in Drupal 10 to start at 100,000. Then, new content created directly in Drupal 10 will be assigned IDs high enough that they would not collide with the IDs of content imported from Drupal 7. This change would have to be applied to all content entities that can be created from migrated Drupal 7 data. We will demonstrate how to do this in a future article.

For now, remember to review these four considerations:

1. Don't forget revisionable entities

If the entity is revisionable, you need to manipulate the auto-increment value for the tables that store the revision data.

2. Different entity types may require different values

Different entity types will likely require a different value for their auto-increment. That is, the number used for nodes will not be the same as that of user, files, taxonomy terms, paragraphs, etc.

3. Content entities created as part of the migration

Account for all content entities that might be created as part of the data migration. You might not have a dedicated migration for URL aliases, but those might still be imported as part of the node migrations. If the Pathauto module is configured in Drupal 10, URL alias entities might be created even if you did not explicitly ask for it. These entities should also be considered when determining where the auto-increment value should be manipulated.

4. When reusing old entity IDs is not possible

When changing entity types, it might not be possible to reuse the old entity ID. As mentioned in Scenario 3 above, if changing nodes to users, the node ID value in Drupal 7 might already exist as a user ID value in Drupal 10. Another example might be files being migrated as media entities. If the site allows remote videos as media entities, the file ID value in Drupal 7 might already exist as a media ID value in Drupal 10.

Conclusion

We've discussed three common conflict scenarios in Drupal 7 to Drupal 10 migrations and two solution pathways to avoid them. It is important to note that, in my experience, entity ID conflicts in a data migration project are likely to occur. This can happen during the testing phase or after an initial migration is completed and incremental runs are needed. **It is not a matter of *if* they will happen, but of *when* they will happen.**

Therefore, it is best to plan for them from the beginning and define a strategy to work around them. Otherwise, it can be quite costly, if possible at all, to resolve such ID conflicts among multiple databases. As part of the upcoming example project in this series, we will demonstrate how such conflicts can be avoided.

06 - Known issues

In the previous article, we talked about avoiding entity ID conflicts which is one of the primary known issues when migrating to Drupal 10. In this section, we will discuss other issues that might arise and some limitations of the API. As time passes, issues are fixed and new ones might get discovered. While we strive to keep this series up to date, refer to the official documentation for the latest community aggregated resources.

Migrating Views

The migration of views is not supported by the core Migrate API. Among other reasons, changes to the site architecture and views plugins not being available in Drupal 10 are some of the reasons behind this decision. Outside of what is provided by Drupal core, multiple contributed modules were created to attempt an automated upgrade path. As of this writing, the only supported one is the Views Migration module. It takes a best-effort approach to performing an automated migration.

A few things to consider:

- There are many modules that integrate with Views to provide additional functionality. Those modules need to be enabled on Drupal 7 and Drupal 10 before attempting the automated upgrade. Otherwise, you will get broken or missing plugin handlers.
- If a Views plugin used in Drupal 7 is not found in Drupal 10, the automated migration will still attempt to create the view as complete as possible. In this case, when editing the view in Drupal 10 look for broken and missing plugin handlers and manually fix the issue.
- In some cases, it will not be possible to create the view at all. When this happens, check the migration messages to get a list of views that failed to migrate. A command similar to `drush migrate:messages d7_views_-migration` would show the relevant information.

- The module **only** works with Views_ *whose* **configuration is stored in the database_**. In Drupal 7, the Features module was a very popular approach to deploying configuration changes. When adding a view to a feature, its configuration is stored in code, not in the database. Such views would not be picked up by the automated upgrade path. To work around this, overwrite the view's configuration in Drupal 7. Making any change, like adding a space character to the description, will overwrite the view and store its new configuration in the database.

In my experience, the module does a good job. Simple views usually work out of the box in Drupal 10 and complex ones need a bit of tweaking to fix them. In all cases, I review each view and view display to make sure everything is working as expected.

Something to remember: you might not need all of your current views in the new Drupal 10 site. Always evaluate which views need to be replicated and which can be dropped. The site audit template contains a tab to keep track of views migrations.

PHP module

In Drupal 7, the PHP module was part of Drupal core. Many sites used it to embed PHP code to render dynamic content in rich text fields, control block visibility, and implement views plugins like contextual filters and argument validators. The module would store PHP code in the database and execute it on demand.

While a Drupal 10 version of the module is available, there are many reasons not to use it. The primary concerns are introducing security and performance issues and producing fatal errors and broken pages due to malformed code. It is **strongly advised not to use the PHP module** in Drupal 10.

Replacing the PHP module will depend on how it was used in Drupal 7. Custom tokens could be used to inject dynamic content in rich text fields. Custom condition plugins could replace block visibility rules. Custom views plugins could replace contextual filters and arguments validators. Note that you might not need to create custom code in all cases. Sometimes, the functionality provided by Drupal core or contributed modules suffices to replace instances of PHP module usage.

One aspect of the upgrade process that is not straightforward is finding all places in Drupal 7 where PHP code was used. I have seen code embedded in rich text fields attached nodes, block descriptions, views text area plugins, and more. Unfortunately, there is no single API call or query against the database that would retrieve places where code is embedded.

Consider the following to help determine where PHP is present in the database:

- Review text formats that have the PHP filter enabled. From there, review which roles have access to those text formats and what content they have permission to author. At the database level, the text format is a separate column in a table which can be used for filtering based on the identified text formats.
- Create a database dump and search for PHP opening tags in the result file. A tool commonly used for this is grep.
- Use an IDE capable of connecting to a database and perform a full-text search for PHP opening tags.
- Talk to the content editors. They are likely to know key pages or sections of the site that rely on PHP code.

Unsupported filter formats

Filters are used as part of input format configuration to clean up and enhance content entered by end-users in rich text fields. For example, you can configure one to restrict which HTML tags are allowed and another one to replace a token for a dynamic value. When migrating input formats Drupal will attempt to migrate all filters attached to them. If an equivalent Drupal 10 filter format is not found, it will be migrated as filter_null. This filter replaces all content for an empty string.

This can cause confusion because any input format using an unsupported filter will return an empty string. This effectively hides the content of any rich text fields that were saved with that input format even though it exists in the database. It is common to bump into this issue when using the PHP filter, but any unsupported filter format will produce the same result.

To work around this issue, you can:

- Check if a Drupal 10 module provides the missing filter format. If available, enable the module and run the migration for filter formats again.
- Edit and re-save the affected input formats to remove the references to `filter_null`. While this will make content appear, any functionality provided by the unsupported Drupal 7 filter will be lost. This could lead to unreplaced tokens and even security issues due to unfiltered markup.
- Edit the content where the unsupported filter is used and save it again using an input format. This case is similar to the previous one in the sense that we are bypassing the changes that the unsupported filter would produce.
- Implement the missing filter yourself and consider contributing it back.

Other issues

There are other known issues, but the ones above are the ones that I encounter more frequently. Other issues that you might encounter during your migration include:

- Conflicting text processing settings in text fields.
- Duplicate comment fields in content types.
- Inability to migrate files stored in a non-standard location.
- Field types, widgets, or formatters that have no Drupal 10 equivalent. The Migrate Skip Fields module will be covered in a future article to help with this issue.

Refer to the official documentation on known issues for more details on the above issues and other ones not listed in this article. Note that some issues are dependent on the version of Drupal you are migrating from. That is, some issues are only relevant when migrating from Drupal 6 while others manifest when the source site is Drupal 7.

Migrating from Drupal 7 to Drupal 10 presents several challenges, particularly with Views, the PHP module, and unsupported filters. To ensure a successful migration, leverage the migration modules available, replace PHP code with safer alternatives, address filter format issues, and consult official documentation for the latest updates. In our next part, we will provide an overview of Drupal entities.

07 - Drupal entities overview

Today we will take a step back from reviewing the Migrate API. Instead, we will have an overview of content and configuration entities in Drupal 10. This is important for two reasons. First, the Migrate API leverages the Entity API to create configuration and store content in Drupal via the destination plugins. Second, in the context of a migration, we will be creating Drupal 10 content and configuration entities based on Drupal 7 data, so having a high-level overview of how Drupal entities work will be very useful when we need to write custom migrations.

Configuration Entities

Drupal 10 configuration entities define the site structure. Some configuration entities included in Drupal core are:

- node types
- fields including their storage and instance settings
- taxonomy vocabularies
- media types
- block settings and block types
- comment types
- contact forms
- menus
- roles
- views
- image styles
- filter formats
- editor configuration

In the contributed modules space we also have:

- paragraph types
- pathauto patterns

- search API indexes and servers
- group types.

As part of Drupal commerce, some of the available configuration entities are:

- product types
- variation types
- product attributes
- order types
- order item types
- store types
- shipment types
- package types

Configuration entities are managed by the Configuration API. This will become important to understand when we work on migrating configuration. In a nutshell, Drupal's active configuration is stored in the database, namely the `config` table. For version control and deployment, configuration is exported to and imported from YAML files. This can be done via the administrative interface or using Drush configuration commands.

As we discussed in the first article, there are multiple migration strategies. An automated migration of configuration is optional. If not done automatically using the Migrate API, the configuration will have to be created manually. This is often the case when the site structure is going to be significantly different between the Drupal 7 and Drupal 10 sites.

A good way to imagine configuration entities is as a bucket you can put data into. For example, you can create nodes (content entities) of type article (configuration entity). While not strictly necessary, it is possible to have a site configuration complete before adding any content. However, that is rarely the case as, even when there is a clear plan of action in the beginning, project requirements often change and the site configuration needs to be modified.

For large projects in particular, it is not feasible to wait for all the site configuration to be completed before content migrations begin. In cases like this, it is common to have a group of developers building the site's configuration. As configuration is added to the site, another group of developers start working on migrating the content. **Communication is key during this process.** If changes are introduced to the content model, the migrations need to be updated accordingly.

Content Entities

Drupal 10 content entities store data to be displayed on the site. Some content entities included in Drupal core are:

- nodes
- users
- taxonomy terms
- files
- media
- menu items
- custom blocks
- contact messages
- comments
- path aliases

In the contributed modules space we have:

- paragraphs
- redirects
- search API tasks
- groups

As part of Drupal commerce, some of the available configuration entities are:

- products
- product variations
- product attribute values
- stores
- order
- order items
- stores
- shipments
- shipping methods
- licenses

Each content entity has a set of properties, known as base field definitions, to hold data. For instance, the node entity has the `nid` property to store the node id, `vid` to store the revision id, `type` to store the node type it belongs to, `status` to indicate whether the node is published or not, `uid` to store a reference to the user who created the node, and `title` to store the node's title. Note that while some property names might be the same across entities, they can be used to store different information. For example, in the case of the taxonomy term entity, the `vid` property stores the vocabulary the term belongs to. And in the user entity, the `status` property indicates whether a user account is active or blocked.

Technical note: Knowing all properties available for a content entity might not be straightforward. You can start by looking at the `baseFieldDefinitions` method of the class that defines the content entity to learn how the return value is constructed. There might be calls to the same method in the parent class. Some properties, like `published,` are added via traits. Others, like `workspace,` are added dynamically based on which modules are enabled on the site. You can search for online references such as these articles: properties for general content entities and for commerce related ones. For a complete list based on your current installation, you can use the entity_field.manager service to get information about an entity's base field definitions.

The following Drush command will show the properties for the `node` entity:

```
drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manager')->\
getBaseFieldDefinitions('node')));"
```

Content entities can be fieldable. Drupal fields are used to store discrete pieces of data which can be used for displaying, filtering, and sorting purposes. The main difference with entity properties is that fields are user-defined. That is, the developers working on the content model determine what information to collect and attach fields to entities accordingly, usually from the user interface. Examples of field types include: number, text, email, link, file, and address. Each field type can have multiple widgets and formatters, which determine how the data is collected and presented, respectively. Additionally, each field can have validation rules to enforce data integrity. All of this is provided by the Field API.

Content entities can also have bundles. A bundle is an abstraction used to define variations of an entity type. Bundles allow grouping entities that share

similar characteristics or describe the same idea. For the node entity, each content type is a bundle. For the taxonomy term entity, each vocabulary is a bundle. Because each entity bundle can have a different set of fields, you can think of them as templates to collect information.

Note: some entities, like users and files, do not allow bundles by default. Contributed modules can change this behavior. For example, the User bundle module allows bundles to be associated with users. For reference, the table below lists some content entities with a subset of their entity properties and lists whether they are fieldable and allow bundles.

Entity type	Entity properties (base field definitions)	Fieldable?	Allows bundles?
Node	nid, vid, langcode, type, status, uid, title, created	Yes	Yes – content type
Term	tid, langcode, vid, status, name, description, weight, parent	Yes	Yes – vocabulary
Media	mid, vid, langcode, bundle, status, uid, name, thumbnail	Yes	Yes – media type
BlockContent	id, langcode, type, status, info	Yes	Yes – custom block type
Comment	cid, langcode, comment_type, status, uid, pid, subject, name	Yes	Yes – comment type
User	uid, langcode, name, pass, mail, timezone, status, roles	Yes	No, out of the box Yes, with User bundle module
File	fid, langcode, uid, filename, uri, filemime, filesize, status	No	No

Figure 2. Content Entity Table

From a migration standpoint, having a clear understanding of the content model of the source and destination sites is key. We will also observe that a migration project consists of multiple individual migrations. **Each migration should only create one type of entity.** That is, a single migration should not create nodes and users at the same time. While not strictly necessary, it is common to have a single migration for each entity bundle. For instance, in the case of nodes, we would have one migration for basic pages and a different one for articles.

The main reason for this is that each content type has a different set of fields. If multiple bundles of the same entity do not have fields, they can share a migration. For example, all taxonomy terms that belong to vocabularies without fields attached to them can share a migration.

Another reason to understand how entities work is that many migration projects require changes to the content model between Drupal 7 and Drupal 10. For example, getting files into media entities. Converting nodes to users. Porting data from Drupal 7 organic groups module into Drupal 10 group module.

In some cases, knowing how fields store data in the source and destination sites is important. For example, if you want to combine or split fields attached to an entity bundle.

We quickly covered a lot about Drupal entities and the role they play in data migrations. Understanding entities, their types, and their relationships is important for a successful migration. This is a complex subject with many nuances, so stay tuned as we break it down further in upcoming articles.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

Example Repository and Automated Migration

- 08 - Example repository setup and Drupal 7 site audit
- 09 - Performing an automated migration
- 10 - Generating migrations with Migrate Upgrade
- 11 - Customizing the generated migration
- 12 - Syntax and structure of migration files
- 13 - The migration process pipeline

08 - Example repository setup and Drupal 7 site audit

Now that we have covered how to prepare for a migration, let's put that knowledge into practice. In this article we introduce the example project: a Drupal 7 site that we will be migrating to Drupal 10. After providing an overview of project setup, we will perform an audit of the Drupal 7 site and draft a migration plan to Drupal 10.

Example repository

The **repository is available on Github.** We will be using DDEV to set up local development environments for Drupal 7 and 10. Refer to DDEV's official documentation for installation instructions. If you choose to use a different development environment, adjust the commands accordingly.

To get the Drupal 7 site up and running, execute the following commands:

```
git clone https://github.com/tag1consulting/d7_to_d10_migration.git d7_to_d10\
_migration
cd d7_to_d10_migration/drupal7
ddev start
ddev import-db -f ../assets/drupal7_db.sql.gz
ddev import-files --source ../assets/drupal7_files.tar.gz
ddev restart
ddev launch
ddev drush uli
```

This will clone the repository into a folder named `d7_to_d10_migration`. Inside, you will find a `drupal7` folder with the code for a Drupal 7 installation including contrib modules. The commands also import an already populated database and user uploaded files from the `assets` folder.

The last command we used above (`ddev drush uli`) will give you a link to log in as an administrator. The Drupal 7 site will be available on your local machine at https://migration-drupal7.ddev.site/

We are going to provide instructions for setting up the Drupal 10 site in the next article. For now, let's focus on understanding the source of our migration.

Our example project is a website that shares information about Drupal meetups. It collects information about speakers and the sessions they present, venues where on-site events happen, and organizations that sponsor the meetups. To have a deeper understanding of the site, we are going to perform an audit following the template introduced in article 3 - Migrating Your Data from Drupal 7 to Drupal 10: Source Site Audit - An In depth Analysis.

Drupal 7 modules and themes analysis

The first tab in the template lists modules and themes. While it is possible to fill out the template manually, the example repository contains a file with commands that would help with the task. Execute the following command to get a list of projects with their version and status:

```
ddev drush pm:list --fields=package,project,name,type,status,version --format\
=csv | sort > d7_projects.csv
```

This will generate a file named `d7_projects.csv` Open it with any spreadsheet software or copy the data into the Google Sheet template itself.

In either case, what we want is the ability to filter the different columns to make analyzing the data easier. For example, you can filter for enabled projects only to determine what action to take for each in Drupal 10. Conversely, you can show a list of disabled (and not installed) projects to inquire why they exist in the code base if not in use.

Refer to article 3 - Migrating Your Data from Drupal 7 to Drupal 10: Source Site Audit - An In depth Analysis for an explanation of the template. For reference, some actions to take based on the Drupal 7 site are:

- Replace the addressfield module with the address module in Drupal 10.
- Replace the field_collection module with the paragraphs module in Drupal 10.
- Replace the youtube module with Core functionality provided by the media module in Drupal 10.

Drupal 7 content model audit

Let's proceed with an analysis of the content model. To make this process easier, we need access to the Drupal 7 database to issue SQL queries. When using DDEV, there are a few options:

- Use `ddev mysql` to access SQL prompt where you can perform the queries directly.
- Install a DDEV add-on to make a database GUI available. A common one is phpMyAdmin, a web-based SQL client, which can be launched using `ddev phpmyadmin` when installed.
- Use `ddev describe` to get connection information for the database managed by DDEV. You can use this information to connect to the database from a desktop application installed in your host machine. Also, some IDEs like PhpStorm contain database browsers which you could also leverage.

Independently of which way you prefer to run the SQL queries, refer to the D7 site audit document for examples that can be used to populate the template. The query below will get information regarding content types for the second tab in template:

```sql
SELECT
    nt.module AS provider_module,
    nt.type AS machine_name,
    nt.name,
    nt.description,
    COUNT(n.nid) AS node_count,
    COUNT(CASE WHEN n.status = 0 THEN 1 ELSE NULL END) AS unpublished_count
FROM node_type AS nt
LEFT JOIN node AS n ON n.type = nt.type
GROUP BY nt.type
ORDER BY node_count DESC;
```

This query will produce a list of all content types. For each of them, the total number of nodes and the count of unpublished nodes will be displayed. The template includes a formula to get a percentage of unpublished nodes. The query result also includes the module that provides the content type. In the example project, it is the `node` module for all of them because the content types were created using Drupal's admin interface. Some Drupal 7 modules like

user alert create a new content type to fulfill its functionality. In such cases, the machine name of the module that provides the content type will be listed.

Our example site does not include a lot of content. The number of nodes per content type is rather low, but even with the sample data we can identify some patterns that would affect the migration. Namely, all nodes of type `swag` are unpublished. Therefore, there is no need to carry this content type into the new Drupal 10 site. This was done intentionally to demonstrate how we can skip full content types from the migration.

A content type with a high percentage of unpublished nodes or a low number of total nodes should be reviewed to determine whether it is worth keeping. Refer to article 2 - Migrating Your Data from Drupal 7 to Drupal 10: Source Site Audit - A High Level Overview for more things to consider when deciding which content type to migrate.

For brevity, we will not list nor explain all the queries in the D7_SITE_-AUDIT.md document. Give them a try and populate a copy of the template with data from your project.

Migration plan for Drupal 10

Now that you have completed the audit of the example source site, let's draft a migration plan for Drupal 10.

Based on information collected in the site audit document, we will:

- Migrate nodes of type `speaker` as `user` entities.
- Migrate nodes of type `sponsor` as `taxonomy term` entities.
- Not migrate nodes of type `swag` and drop the content type altogether.
- Not migrate any unpublished node.
- Not migrate any blocked users.
- Not migrate revisions for any content entity.
- Migrate the `field_favorite_quote` field collection into `paragraph` entities.
- Migrate `field_video_recording` of type `youtube` into `media` entities of type `remote video`.
- Migrate `field_slides` of type `file` into `media` entities of type `document`.
- Migrate `field_profile_picture` of type `image` into `media` entities of type `image`.

- Merge the `field_drupal_org_profile`, `field_linkedin_profile`, and `field_x_twitter_profile` URL fields into a single field using the social link field module.
- Migrate the `field_website` URL field into a `link` field.
- Migrate `field_address` of type `addressfield` into Drupal 10's address module.

Some of the content model changes are arguably not necessary in our rather simple example project. Yet, we will go forward with them to have a broad array of examples. This will allow us to demonstrate different techniques that you will be able to reference (and adapt) on your own projects.

Before closing, a list of bullet points like the one presented above is far from being a useful migration plan. A proper one goes beyond configuration and content considerations, which are the focus of this migration series. Among other things, a proper plan should include:

1. An analysis of contributed modules and theme usage. When dropping or replacing anything, explain the rationale behind the decision.
2. An assessment of custom functionality and a plan to port custom modules and themes as necessary.
3. A list of existing third-party integrations and how they will be handled on the new site.
4. A review of the current server infrastructure and potential changes due to higher version requirements in Drupal 10.
5. Risk management, resource allocation, launch deadlines, and other business considerations.

For more information, we recommend watching this presentation on planning and estimating Drupal migrations presented by the Tag1 team at DrupalCon Portland 2024.

We covered a lot of territory quickly today. Take some time to explore the provided Drupal 7 site and practice running the queries in the *D7_SITE_AUDIT.md* document. Compare your migration plan with our example above. These are foundational planning steps that will be important when you begin your own migration. In our next article, we will provide instructions for setting up the Drupal 10 site.

Quick Resources

- Example Project repository (Github)
- DDEV official documentation
- D7 site audit document
- Planning and estimating Drupal migrations, DrupalCon Portland 2024 Presentation

09 - Performing an automated migration

In the previous article we looked at the example Drupal 7 site that will be migrated throughout this series. Today, we will start to interact with the Drupal 10 destination site. First, we will cover configuration that needs to be in place for the migration to work. Then, we will demonstrate how to perform an automated migration from the admin user interface.

Most of the information presented below is for reference only. The goal of this series is to teach you how to perform a custom migration, but there are still valuable lessons to learn from following the automated approach. Let's get started.

Migration related settings

The example repository includes a fully working Drupal 7 site as covered in the previous article. While it is very useful to have a local copy of the source site, it is technically not required. **The only hard requirement to perform the migration is having access to the Drupal 7 database.**

In our Drupal 10 site, we need to provide a new database connection pointing to the source we will be migrating from. This is normally done in `settings.php` or in a file included from it. In our example project, we have included the second database configuration in `settings.ddev.php`:

```
$databases['migrate']['default']['database'] = "db";
$databases['migrate']['default']['username'] = "db";
$databases['migrate']['default']['password'] = "db";
$databases['migrate']['default']['host'] = "ddev-migration-drupal7-db";
$databases['migrate']['default']['port'] = 3306;
$databases['migrate']['default']['driver'] = "mysql";
```

This establishes the `migrate` database connection. Refer to the official documentation on database configuration to learn more.

In our case, it is relevant to point out that from our *Drupal 10 DDEV project* we are connecting to the default database of the *Drupal 7 DDEV project*.

Alternatively, we could import a second database into the Drupal 10 DDEV project. Review this article to learn how to include extra databases in a single DDEV project. If you decide to do this, or use a different tool for local development, update the above connection settings accordingly. You could point the connection to a remote database if that is preferred or required.

To verify that the connection works, run this command in Drupal 10:

```
ddev drush sql:connect --database migrate
```

The output should be:

```
mysql --user=db --password=db --database=db --host=ddev-migration-drupal7-db \
--port=3306 -A
```

Another setting that should be considered before starting the migration is `migrate_node_migrate_type_classic`. This will determine how node migrations will be handled.

When set to `FALSE` a single migration will be generated for each content type. It will take care of importing the current revision, all past revisions, and all translations for the nodes. This is default behavior and it is referred to as the complete node migration approach.

When set to `TRUE` three different migrations will be generated for each content type: one for the current revision, one for past revisions, and one for the translations. Drupal sets this setting to `FALSE` by default.

In the previous article we mentioned that we will not be migrating revisions for any entity. Therefore, our example project overrides the default in `settings.migrate.php`:

```
$settings['migrate_node_migrate_type_classic'] = TRUE;
```

Later in this article, we will run the automated migration via the user interface provided by Core's Migrate Drupal UI module. One of the steps of the form wizard requires indicating what version of Drupal we are migrating

from, the database credentials, and the location of user uploaded files. As an alternative to entering this information manually in the browser, we can provide it via a group of four settings. In our example, they are defined in `settings.migrate.php`:

```
$settings['migrate_source_version'] = '7';
$settings['migrate_source_connection'] = 'migrate';
$settings['migrate_file_public_path'] = 'http://ddev-migration-drupal7-web';
$settings['migrate_file_private_path'] = '';
```

The value of the `migrate_source_connection` setting should match the name of the database connection pointing to Drupal 7. At the start of this section we named it `migrate` for our example project.

For migrating public files, we have two options: (a) either a fully qualified domain name of the Drupal 7 site to fetch the files from, or (b) a path to a folder in the same server with a copy of the user uploaded files.

Private files are by definition not web accessible. Therefore, if you need to migrate private files, they need to be copied to the server where the Drupal 10 migration will be executed.

We are going to expand on file migrations in a future article. For now, we want to point to this section of the official documentation on troubleshooting file migration errors in case you encounter issues.

Automated migration using the Migrate Drupal UI module

Drupal Core includes the following migration related modules:

- **Migrate:** consists of the core API that can be used to import data from any source.
- **Migrate Drupal:** allows connecting to a Drupal 6 or 7 site to import configuration and content.
- **Migrate Drupal UI:** exposes an admin user interface for the Migrate Drupal module. This will allow us to perform the automated migration from the browser.

To get started, let's execute the following commands from the root of our example repository to get a Drupal 10 site running:

```
cd drupal10
ddev start
ddev composer install
ddev drush --yes -v site:install minimal
ddev launch
ddev drush uli
```

Let's break down the group of commands. First it will download Core and other module dependencies via Composer. Then it installs Drupal using the `minimal` installation profile. As the name suggests, this is the minimal setup required to run a Drupal project. As discussed in article 4, for the migration we want a site with as little configuration as possible and no content at all. The last command will give you a link to log in as an administrator. The Drupal 10 site will be available at https://migration-drupal10.ddev.site/. Have you ever seen a Drupal site like this? :-)

To continue with the minimalistic motif, let's enable only the *three* migration modules mentioned at the beginning of this section. You can navigate your way to the `Extends` page available at `/admin/modules` or run the following command:

```
ddev drush --yes pm:enable migrate migrate_drupal migrate_drupal_ui
```

This will enable four modules. Wait, *what*?! Weren't we enabling *three* modules? Where did this fourth module come from? Glad you asked. Starting in Drupal 10.1, Migrate Drupal declares a dependency on the Password Compatibility (phpass) module. Refer to this change record for more details on why this is needed and when the extra module can be uninstalled. In a nutshell, a new password hashing algorithm was introduced and the `phpass` module allows old passwords from Drupal 7 to continue to work.

With the migration modules installed, let's make our way to the upgrade form available at `/upgrade`.

You will be greeted by links to the official documentation and preparation steps. Read the instructions *carefully* and click on the `Continue` button.

Figure 3. Upgrade Form-First Step of Wizard

The second step of the wizard will ask you to indicate which version of Drupal you intend to migrate from, the source database credentials, and the location for the public and private files.

In our example project, the form should be filled out already with the details provided in `settings.migrate.php`. These are the four settings we explained at the end of the previous section. If you are using a different setup, fill out the form manually. Then, click the `Review upgrade` button.

Figure 4. Second Step of Wizard

A possible third step in the wizard is a page titled "Upgrade analysis report." There are two conditions that would make this make appear: (a) you (or a module) have created content on the site that might get overridden by the migration, or (b) the Content Translation module is enabled.

In the second scenario, it might be a false positive that is explained in this Core issue. If you stumble upon this warning, verify that there is no content of any entity type that could be overridden. If that is the case, click the `I acknowledge I may lose data. Continue anyway.` button.

Figure 5. Third Step of Wizard

Then, you should land on a page titled "What will be upgraded?" To assemble this report, Drupal reads the modules that are enabled in the source and destination sites. The system will read every module enabled in Drupal 7 and check Drupal 10 modules that attempt to provide an automated upgrade path. We talked about this in article 4. Refer back to that article for more details on the automated migration.

For now, let me remind you to take this report with a grain of salt. Some modules that say they cannot be automatically migrated will, in fact, be migrated. And the opposite is also true. :-)

Technical note: To build this report data from a Drupal 10 module's `*.migrate_drupal.yml` file and available migration plugins will be used to decide the migration status for each enabled Drupal 7 module. Check the ReviewForm class in the Migrate Drupal UI module to see how the functionality is implemented.

To proceed with the example, let's install many more modules:

```
ddev drush --yes pm:enable address announcements_feed automated_cron big_pipe\
 \
  block block_content breakpoint ckeditor5 config contact contextual datetime\
 \
  dblog dynamic_page_cache editor entity_reference_revisions field field_grou\
p \
  field_group_migrate field_ui file filter help history image link media \
  media_library menu_link_content menu_ui mysql node options page_cache \
  paragraphs path path_alias pathauto shortcut social_link_field system taxon\
omy \
  telephone text token toolbar update user views views_ui
```

After doing so, refresh the page and you will see that the number of modules that (allegedly) can be automatically migrated will increase. And in the list of modules that appear not to have an automated migration, you will see modules like `field_collection` that will in fact be migrated to `paragraphs` automatically.

Figure 6. Fourth Step of Wizard

For now, let's proceed by clicking the `Perform upgrade` button.

Figure 7. Fifth Step of Wizard

Because we enabled the `dblog` module before performing the automated upgrade, you can go to `Reports :: Recent log messages` (`/admin/reports`) to read messages logged during the process. Pay particular attention to errors and warnings.

After doing so, have a look at the site to see how much content and configuration was possible to import automatically. In my experience, the results vary greatly from project to project. A common source of problems is Drupal 7 fields that do not have corresponding Drupal 10 versions. Because the field migrations happen relatively early in the process, errors in them might prevent subsequent migrations to work properly.

Even with an incomplete migration, the steps we took in this article have let us peek behind the curtain of the automated approach. We learned about settings that affect the migration process and some of its limitations. In the next article, we will execute an automated migration from the command line and explain how to leverage this option to create custom migrations.

10 - Generating migrations with Migrate Upgrade

In the previous article we performed an automated migration via the administration interface with tools provided by Drupal core out of the box. This gave us the opportunity to familiarize ourselves with the different modules that assist with the task, settings that affect the migration, and the results when there are no customizations in place.

Today we will leverage the Migrate Plus and Migrate Upgrade modules to generate migrations from the command line, which we can then pick from and customize.

Automated migration using the Migrate Upgrade module

Let's start by listing the modules that we will use to run the automated migration from the command line:

- **Migrate:** consists of the core API that can be used to import data from any source.
- **Migrate Drupal:** allows connecting to a Drupal 6 or 7 site to import configuration and content.
- **Migrate Plus:** provides extra source, process, and destination plugins; handles migrations as configuration entities; allows reusing of configuration via migration groups; and more. It is a dependency of Migrate Upgrade and a very useful module on its own.
- **Migrate Upgrade:** provides a Drush command to perform an upgrading from Drupal 6 or 7 to Drupal 10. Optionally, you can generate migrations without executing them.

When using Migrate Drupal UI, we can provide Drupal 7 database credentials via the user interface. However, this time, we will do it via the command line. In either case, we need to add an extra database connection via the `$databases` array in `settings.php` or in a file included from it.

In our example project, we have included the second database configuration in `settings.ddev.php`:

```
$databases['migrate']['default']['database'] = "db";
$databases['migrate']['default']['username'] = "db";
$databases['migrate']['default']['password'] = "db";
$databases['migrate']['default']['host'] = "ddev-migration-drupal7-db";
$databases['migrate']['default']['port'] = 3306;
$databases['migrate']['default']['driver'] = "mysql";
```

Refer to the **previous article** for an explanation of database configurations.

If you followed the instructions in our how-to series up to this point, the Composer command will likely report that there is nothing to install, update or remove. You likely have a partially migrated site. Let's discard it so we can learn about how to work from the command line. We will install the site again using the `minimal` installation profile.

To do that, run this command:

```
cd drupal10
ddev start
ddev composer install
ddev drush --yes -v site:install minimal
ddev launch
ddev drush uli
```

Now, let's enable the migration modules:

```
ddev drush --yes pm:enable migrate migrate_drupal migrate_plus migrate_upgrade
```

As mentioned in the last article, this will also enable the Password Compatibility (phpass) module. We also discussed how the Migrate API relies on modules being enabled in Drupal 10 to attempt an automated migration. Running the migration from the command line does not change this fact. So, let's enable the same set of modules that were enabled before the automated migration from the administration interface:

```
ddev drush --yes pm:enable address announcements_feed automated_cron big_pipe\
 \
  block block_content breakpoint ckeditor5 config contact contextual datetime\
 \
  dblog dynamic_page_cache editor entity_reference_revisions field field_grou\
p \
  field_group_migrate field_ui file filter help history image link media \
  media_library menu_link_content menu_ui mysql node options page_cache \
  paragraphs path path_alias pathauto shortcut social_link_field system taxon\
omy \
  telephone text token toolbar update user views views_ui
```

Finally, we can trigger the migration from the command line:

```
ddev drush --yes migrate:upgrade --legacy-db-key='migrate' \
  --legacy-root='http://ddev-migration-drupal7-web'
```

The `--legacy-db-key` flag of the `migrate:upgrade` command expects a database connection key from `settings.php` pointing to Drupal 7. Per our `$databases['migrate']` configuration, the value to use is `migrate`. The `--legacy-root` flag is used by the public files migration and it should contain (a) either a fully qualified domain name of the Drupal 7 site to fetch the files from, or (b) a path to a folder in the same server with a copy of the user uploaded files.

Notice there is no flag for private files. This issue aims to add one, but for now you can manually update the private files migration to specify the location of the files. Also, be mindful of other issues related to migrating (private) files.

After running the command you will see a list of executed migrations.

Figure 8. Executed Migrations

If you were to visit the site now, you would notice that it looks the same as if the migration had been executed from the user interface via the Migrate Drupal UI module. Then, what is the point of doing this from the command line?

For one, it is faster to perform the upgrade this way. It is common to have to run the automated migration multiple times while testing different modules that can help with the upgrade to Drupal 10. But the primary reason is that we can pass an additional flag to the `migrate:upgrade` command to **generate migration files, which we can then pick from and customize.**

```
ddev drush --yes migrate:upgrade --legacy-db-key='migrate' \
  --legacy-root='http://ddev-migration-drupal7-web' --configure-only
```

Working with migrations is a two step process: (1) write or generate the migration, and (2) execute/run the migration. When the `--configure-only` flag is not used, the Migrate Upgrade module will perform both actions in one go. When the flag is present, the module will only generate migrations as configuration entities without actually executing them. This will allow us to take the generated migrations and customize them as needed.

The output of the command is similar, but with some subtle differences. Instead of `Executed migrations`, the command will output `Generated migrations`. Also, the names of the migrations will be different, prefixed by the string `upgrade_`. This prefix can be changed by passing the `--migration-prefix` flag to the `migrate:upgrade` command.

Figure 9. Generated Migrations

Exporting generated migrations

If you were to visit the site after running the command with the `--configure-only`, you will see no configuration or content on the site. If you were using version control, you will notice that no new files are present on the file system. What gives? The command reported that the migrations were generated, but *where*? **The migrations were created in Drupal's active configuration storage which is the database.**

This is one of many examples in which we will see the Migrate API and related modules interacting with other Drupal core APIs. We are not going to get too deep into configuration management for now. Before exporting the generated migrations, let's define the location where the export should be placed:

```
$settings['config_sync_directory'] = '../config';
```

This is added to `settings.php` or in a file included from it. In our case, we indicated that the export location should be a folder named `config` at the same level of Drupal's docroot. Using the `tree` command inside `drupal10` we get the following output:

```
$ cd drupal10
$ tree -L 1
.
├── composer.json
├── composer.lock
├── config
├── vendor
└── web
```

The output has been trimmed, but you can see that `config` and `web` both exist at the same level under the `drupal10` folder at the root of our example repository. Finally, let's export the configuration:

```
ddev drush config:export
```

If you have exported configuration before and there are configuration changes, you will be prompted to confirm the operation. Otherwise, look at the `config` folder. Those familiar with configuration management will know that exporting configuration this way is an all or nothing operation. That is, you get the configuration for the whole site–not only those related to migrations.

Fortunately, the configuration files follow a pattern that will let us find the generated migrations. In fact, I like to move or copy the migration files to another folder to keep as reference. We will be reinstalling the site a few more times while setting up the project. And later on we will be exporting configuration regularly. Because configuration exports cleans the config sync folder before dumping the new configuration, **placing the generated migrations in a different location will let us refer back to them.**

```
cd drupal10
mkdir ref_migrations
cp config/migrate* ref_migrations
ls ref_migrations
```

We are going to explain the content of those files in a future article but feel free to open them in the text editor of your choice already. For now, a few notes regarding version control. I normally exclude this folder from version control. If using the same name across migration projects, you can define a global .gitignore rule:

```
touch ~/.gitignore
git config --global core.excludesfile ~/.gitignore
```

Those commands will create an empty `~/.gitignore` and configured it to include ignore rules to apply globally in Git. Edit the file to include `ref_migrations/`. Alternatively, you can add these files to a new `ref-migration` branch and even create a pull request from it. The branch can be closed without merging, but being there means you can refer back to the files in the future. Having a pull request is also a good opportunity to invite other team members to review and comment on the generated migrations.

While we will be providing tips regarding version control and configuration management where appropriate, the focus of this series is migrations. You are free to use whatever version control and configuration management strategies you prefer.

In the next article we will explain two different ways of working with migrations: as core plugins and as configure entities. We will also learn how to customize the migrations generated with Migrate Upgrade.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

11 - Customizing the generated migration

In the previous article we learned how to generate migrations using the Migrate Upgrade module. As explained, those migrations are generated as configuration entities managed by the Migrate Plus module. Today, we will learn about using migration plugins provided by Drupal Core. After learning about these two methods of handling migrations, we will explain how the example project is set up to perform configuration and content migrations. Finally, we will take a glance at customizing the automatically generated migrations.

Migrate Plus configuration entities

The two primary ways of handling migrations in Drupal 10 are using Migrate Plus configuration entities and Drupal Core's migration plugins. They differ in the filename pattern, the location where the migration files are placed, and how changes to the files are detected.

Let's start with configuration entities. When using this approach, the Migrate Plus module is required. Notice that once the migrations are generated, the Migrate Upgrade module can be uninstalled as the migrations will live in **the site's active configuration: the database.** When exported, the filename pattern will be `migrate_plus.migration.MIGRATON_ID.yml`.

It is useful to have all migration related functionality encapsulated in a custom module. When doing so, these migration files are placed inside the `config/install` folder of the custom module. When the module is **enabled for the first time**, the migrations will be copied to Drupal's active configuration, which is the database. Further changes to the files inside the module will be **ignored unless the configuration is synced again.**

This has many implications:

1. The migrations are part of the site's configuration. Doing a configuration export, with `drush config:export` for example, will create copies of the

migrations in the location specified by the `$settings['config_sync_directory']`. Having two copies of the same file might be confusing. Moreover, neither the version in the `config/install` folder nor the one in the `config_sync_directory` folder is the definitive representation of the migration. By default the active configuration is the **database**, not the exported files. It is possible that the database representation of the migration differs from one or both files.

2. If you need to update the migrations, you need to **update the site's configuration**. This can get messy quickly. Let's say that you need to update a field setting in a content type. Because of this change, the migration also needs to be updated. You need to be very mindful about the order in which configuration is exported and imported to prevent overwriting either the changes to the content type or the updates to the migration. There are multiple strategies to work around this. If you consider the copy of the migration inside your custom module the canonical version, you can use the `drush config:import --partial` command to execute a partial configuration import that only affects your custom migrations. You can also use modules like config_devel, config_update, and config_sync to apply updates. In the end, because you are dealing with configuration entities, you can use any workflow to apply configuration changes when the migrations need to be updated.
3. Because the migrations are configuration entities, you can leverage the configuration override system to alter the migrations dynamically. For example, you could use `$config['migrate_plus.migration.upgrade_d7_file']['source']['constants']['source_base_path'] = getenv('MIGRATE_D7_FILE_SOURCE');` in your `settings.php` to use an environment variable to specify the source (domain) to fetch files from in a public files migration. Modules like Configuration Split can also be used to alter the migrations. Again, because you are dealing with configuration entities, any module or workflow that you already implement can be used to manage and alter the migrations.

For reference, below is one way to manage migrations as configuration entities:

```
# 1) Run the migration.
$ drush migrate:import upgrade_d7_user

# 2) Rollback migration because the expected results were not obtained.
$ drush migrate:rollback upgrade_d7_user

# 3) Change the migration definition file in the "config/install" directory.

# 4) Sync configuration by folder using Drush.
$ drush config:import --partial --source="modules/custom/my_module/config/ins\
tall"

# 5) Run the migration again.
$ drush migrate:import upgrade_d7_user
```

Core migration plugins

Another way of handling migrations is using features provided by Drupal core out of the box. That is, using core migration plugins. When using this approach, the filename pattern is `MIGRATION_ID.yml` and the files are placed inside a `migrations` folder in your custom module. Rebuilding caches will apply the changes made to the migration files.

In recent years, this has been my preferred approach because making changes to the migration does not require updates to the site's configuration. Also, clearing caches is arguably more straightforward than having to be cautious when syncing updates to migration configuration entities.

For reference, below is one way to manage migrations as core plugins:

```
# 1) Run the migration.
$ drush migrate:import upgrade_d7_user

# 2) Rollback migration because the expected results were not obtained.
$ drush migrate:rollback upgrade_d7_user

# 3) Change the migration definition file.

# 4) Rebuild caches for changes to be picked up.
$ drush cache:rebuild

# 5) Run the migration again.
$ drush migrate:import upgrade_d7_user
```

Technical note: Migrations have a dedicated cache bin. During development, you could include the following snippet inside `settings.php` or in a file included from it to not require cache rebuilds when changes are made.

```
$settings['container_yamls'][] = DRUPAL_ROOT . '/sites/development.services.y\
ml';
$settings['cache']['bins']['discovery_migration'] = 'cache.backend.null';
```

Code organization for a custom migration

The example project has three elements that will help organize the code related to the custom migration:

- A custom installation profile: `tag1_profile`.
- A custom module to store content migrations and related code: `tag1_migration`.
- A custom module to store configuration migrations and related code: `tag1_migration_config`.

The custom installation profile is named `tag1_profile` and it is modeled after the minimal installation profile provided by Drupal core. In our example, sticking with `minimal` would probably suffice. But in large projects, a custom installation profile can be used to store code related to the site itself. **One key assumption of our example project is that the site can be reinstalled at any point using the exported configuration.**

While we progress through the examples, we will be migrating some configuration automatically and creating some manually. After every milestone, we will export the configuration. Then, we can use the `drush site:install tag1_profile --existing-config` command to get a fully configured site on top of which we can run content migrations.

Note: At the time of this writing, a new recipes API was recently introduced to Drupal core in version 10.3.0. Recipes can be used instead of installation profiles to fully configure a site. In our example project, we are sticking with installation profiles. Towards the end of the series, we will look at applying recipes to bring configuration into the site.

The `tag1_migration` module will contain all the **content migrations** as core plugins. If necessary, this module can be used to store custom source/process/destination migration plugins, hook implementations, registering event callbacks, and any other code related to migrating **content entities**.

The `tag1_migration_config` module will contain all the **configuration migrations** as core plugins. In fact, this is a submodule of the `tag1_migration` module. As said before, it is convenient to have all migration related code in one location. Remember that not all configuration will come from a migration. The `tag1_migration_config` could be removed after the migrated configuration makes its way to the `config_sync_directory` folder. Instead, I prefer to uninstall the module but keep the code around to have a reference of how some of the configuration was generated via migrations.

From config entities to core migration plugins

The example project will use core migration plugins. However, the generated migrations use configuration entities. Let's learn how to convert one approach to the other using the migration of taxonomy vocabularies.

After following the instructions in the previous article, you should have a file `config/migrate_plus.migration.upgrade_d7_taxonomy_vocabulary.yml` inside the Drupal 10 installation. The content of the file should look like:

```yaml
uuid: 79d5ff1c-7924-4508-99b5-ba0ad71d2960
langcode: en
status: true
dependencies: {  }
id: upgrade_d7_taxonomy_vocabulary
class: Drupal\migrate\Plugin\Migration
field_plugin_method: null
cck_plugin_method: null
migration_tags:
  - 'Drupal 7'
  - Configuration
migration_group: migrate_drupal_7
label: 'Taxonomy vocabularies'
source:
  plugin: d7_taxonomy_vocabulary
process:
  vid:
    -
      plugin: make_unique_entity_field
```

```
20        source: machine_name
21        entity_type: taxonomy_vocabulary
22        field: vid
23        length: 30
24        migrated: true
25    label:
26      -
27        plugin: get
28        source: name
29    name:
30      -
31        plugin: get
32        source: name
33    description:
34      -
35        plugin: get
36        source: description
37    weight:
38      -
39        plugin: get
40        source: weight
41  destination:
42    plugin: 'entity:taxonomy_vocabulary'
43  migration_dependencies:
44    required: {  }
45    optional: {  }
```

We will begin by moving this exported configuration entity into our custom module's `migrations` folder. At the same time, update the filename to remove the `migrate_plus.migration.` prefix. Then at the root of the Drupal 10 site, execute this command:

```
1  mkdir -p web/modules/custom/tag1_migration/tag1_migration_config/migrations/
2  mv config/migrate_plus.migration.upgrade_d7_taxonomy_vocabulary.yml \
3    web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
4  d7_taxonomy_vocabulary.yml
```

Note: Update the command accordingly if you opted out for moving the exported configuration to a `ref_migrations` folder as discussed in the previous article. What we need is to place the renamed file in the `migrations` folder of our custom module for configuration migrations.

Now, let's make changes to the file itself:

1. Remove the `uuid`, `langcode`, `status`, and `dependencies` keys at the start

of the file. These are settings related configuration entities. We are switching to using core migration plugins so they are not needed.

2. Remove the `field_plugin_method` and `cck_plugin_method` keys if set to `null`. These settings are used for field-related migrations, but are present in all the generated ones. When set to `null` they have no effect. Let's keep the migration files tidy by removing them.
3. Add two `migration_tags`. The first one corresponds to the type of entity being created, or the destination plugin if no entity is created. In this example, `taxonomy_vocabulary` would be our first custom tag as we are creating taxonomy vocabulary configuration entities. The second tag is either `tag1_configuration` or `tag1_content` depending on the type of entity being created. Adding these tags is not strictly necessary, but it helps understand what a file is migrating by scanning its content. Also, we could use these tags to import or rollback multiple migrations at once.
4. Remove the `migration_group`. Migration groups are a feature provided by the Migrate Plus module to share configuration among multiple migrations. When using Migrate Upgrade, a `migrate_drupal_7` group is automatically created and assigned to all the generated migrations. The only shared configuration this group provides is the name of the key to connect to the Drupal 7 database. To the extent possible, I prefer the migration files to be self-contained. That way, there is no need to look in multiple places to know what the migration is doing. We will see some exceptions when dealing with dynamic elements in some migrations. For now, let's remove the migration group.
5. Add a `source/key` configuration to account for removing the migration group. In our case, the value will be `migrate` which corresponds to the name of the connection key `$databases['migrate]` in the `settings.migrate.php` file. This was explained in article 9.

There are more changes that we could make. Most examples will make changes to the `process` section of the file. Some will also alter the `source` and `destination` sections. Each example will explain what customizations are needed and why we are making them.

For now, the final version of the migration file should look like this:

```yaml
id: upgrade_d7_taxonomy_vocabulary
class: Drupal\migrate\Plugin\Migration
migration_tags:
  - 'Drupal 7'
  - Configuration
  - taxonomy_vocabulary
  - tag1_configuration
label: 'Taxonomy vocabularies'
source:
  key: migrate
  plugin: d7_taxonomy_vocabulary
process: ... # Redacted for brevity
destination:
  plugin: 'entity:taxonomy_vocabulary'
migration_dependencies:
  required: {  }
  optional: {  }
```

Today we explored the differences between the two primary ways of handling migrations in Drupal 10 and demonstrated the process of converting migration configuration entities to core migration plugins. In the next article, we will take a deeper look at how the migrations files are written. Specifically, we will examine the syntax and structure of migration files.

12 - Syntax and structure of migration files

In the previous article, we saw what a migration file looks like. We made some changes without going too deep into explaining the syntax or structure of the file. Today, we are exploring the language in which migration files are written and the different sections it contains.

YAML Syntax

Before looking at the structure of a migration file, let's briefly discuss the language used for writing them. YAML stands for **YAML Ain't Markup Language**. It is a human-friendly data serialization language. Here is a basic example:

```yaml
# This is a comment
a_number: 1 # Comments can also appear at the end on lines
another_number: 3.14159
a_string: 'Hello world'
an_array: [1, 2, 3]
alternative_array_syntax:
  - 1
  - 2
  - 3
another_array: [1, two, 3.14159]
an_associative_array:
  one: 1
  two: 2
  three: 3
nested_arrays:
  numbers: [1, 2, 3]
  letters:
    vowels: [a, e, i, o, u]
    consonants: [b, c, d, f, g, h, j, k, l, m, n, o, p, q, r, s, t, v, w, x, \
y, z]
```

Now we will break down its syntax. Anything after a pound sign (#) is considered a comment. YAML uses key-value pairs separated by a colon (:)

with optional nesting of elements. While not a requirement by the language, Drupal uses lowercase letters and an underscore (_) to separate multiple words in a key. The value can be a scalar (like a number or string) or an array (with optional key names and/or nested elements). Drupal quotes string values when they contain more than one word. Array elements can be added in a single line when enclosed by square brackets (`[]`) and separated by commas (`,`). Alternatively, array elements can appear in their own line when indented and prefixed by a dash (`-`). Array elements can be of different types and they can also be nested.

YAML is sensitive to spaces and indentation. At least one space character is required to appear after the colon that separates the key and the value. In terms of indentation, the language does not impose how many space characters to use. To match Drupal's coding standards, we use two spaces for each indentation level.

Drupal uses the `.yml` extension for all files written in YAML syntax, but other projects might use a different one. For example, DDEV uses `.yaml` as the file extension. When writing Drupal migrations, make sure that your file name ends with `.yml`.

Technical note: Drupal uses the Symfony Yaml Component to load and dump YAML files. The YAML language supports more features than the ones presented above. While some features like multiline exports are supported by Drupal, others like parsing PHP constants and enums are not available at the moment. To keep things simple, we will focus on what Drupal is most relevant for writing migrations.

Structure of a migration file

Now, let's have a look at the structure of a migration file. We are going to analyze the modified taxonomy vocabulary migration from the previous article.

```yaml
id: upgrade_d7_taxonomy_vocabulary
class: Drupal\migrate\Plugin\Migration
migration_tags:
  - 'Drupal 7'
  - Configuration
  - taxonomy_vocabulary
  - tag1_configuration
label: 'Taxonomy vocabularies'
source:
  key: migrate
  plugin: d7_taxonomy_vocabulary
process:
  vid:
    -
      plugin: make_unique_entity_field
      source: machine_name
      entity_type: taxonomy_vocabulary
      field: vid
      length: 30
      migrated: true
  label:
    -
      plugin: get
      source: name
  name:
    -
      plugin: get
      source: name
  description:
    -
      plugin: get
      source: description
  weight:
    -
      plugin: get
      source: weight
destination:
  plugin: 'entity:taxonomy_vocabulary'
migration_dependencies:
  required: {  }
  optional: {  }
```

The three major parts of the migration are `source`, `process`, and `destination`. They correspond to the extract, transform, and load steps in ETL data migrations. In addition to these three, only the `id` and `label` keys are required. Other keys provide extra information to the system like dependencies among migrations. The keys presented above are the most frequently used. When working through the examples, we will highlight other keys as necessary.

Now we will review each key-value pair in the file.

The `id` key expects a string to be used as the internal identifier of the migration. Drupal and the Migrate API use this value to execute and keep track of the status of migration. It is customary for the `id` to match the migration file name, but without the `.yml` extension. The value should be alphanumeric characters, optionally using underscores to separate words.

As for the `label` key, it is a human-readable string used to name the migration in various interfaces.

The `source` key contains an associative array with the definition of the source plugin. The only required sub-key is `plugin` which indicates which source plugin to use. There are many plugins and each can contain different configuration options. Refer to this article for more information on source plugins. Our series focuses on migrating from Drupal 7 to Drupal 10, and this requires connecting to the Drupal 7 database. Internally, this uses the SqlBase abstract class which allows specifying a `key` to determine which database connection to use. This was explained in article 9. Some source plugins allow filtering of the data to migrate. For example, the `d7_taxonomy_term` used to migrate **taxonomy term content entities** uses the `bundle` key to limit which vocabularies to migrate terms from. Similarly, the `d7_node` plugin uses the `node_type` key to limit which content types to migrate nodes from.

The `process` key contains an associative array with the process pipeline the migration implements. This determines how source data will be processed and transformed to match the expected destination structure. That is a big enough topic to require a dedicated article, and so we will talk about this in the next article in our migration series.

The `destination` key contains an associative array with the definition of the destination plugin. The only required sub-key is `plugin`, which indicates which destination plugin to use. Similar to source plugins, there are many destination plugins and each can contain different configuration options. Refer to this article for more information on destination plugins. When migrating from Drupal 7 to Drupal 10, most plugins deal with creating entities. In general, the plugin id follows the pattern `entity:[ENTITY_TYPE]` where `[ENTITY_TYPE]` is the machine name of the entity being created. Some examples are `entity:node_type` to create **content type configuration entities** and `entity:node` to create **node content entities**. The Entity abstract class used for entity migrations allows setting the `default_bundle` to indicate which

entity bundle this migration is responsible for creating. For content entities in particular, it is also possible to use the `validate` key to indicate if we want to perform entity validation on the data that is being migrated.

The `class` key determines which PHP migration plugin class is used to execute the migration. A migration plugin acts like a container for the information about a single migration such as the source, process and destination plugins. Drupal\migrate\Plugin\Migration is the default for this key and could be left out if the default were to be used. A few examples exist in Drupal core in which this class is extended for extra customizations. Those deal with field migrations, node translations, and user profile data. When working with generated migrations, this `class` key is one to be aware of but will rarely change.

The `migration_tags` key contains an array of tags for the current migration. Tags can have an arbitrary value and in most cases they do not have special logic attached to them. That being said, when executing migrations, you can read the tags present and perform actions based on that. For instance, if you have a project specific tag, you can filter out any migration that does not include it to keep the list of migrations short in commands like `drush migrate:status`. Also, using the `--tag` flag in `drush migrate:import` and `drush migrate:rollback` you can execute those operations on multiple migrations using a single command. As explained in the previous article, our example project adds two tags to each migration. One tag indicates what we are creating and the other whether we are creating content or configuration. In the snippet above, they are `taxonomy_vocabulary` and `tag1_configuration` respectively.

The `migration_dependencies` key contains an array with two keys (`required` and `optional`) listing the migrations that this migration depends on. The required migrations must be run first and completed successfully. The optional migrations will be executed if they are present. Note that if a dependency is added, all the records for that migration must be processed. Let's say we are doing an incremental migration. Our migration for the event content type declares a dependency on the migration for the venue content type. If new venue nodes were added in the source site since the last time we executed a full migration, those venue nodes need to be imported prior to importing the new event nodes. Otherwise, you will get when trying to execute the event migration indicating that the dependencies have not been met. As part of the examples, we will be updating the dependencies of many

migrations. In some cases to remove duplicate entries. In others, to reflect content model changes.

All the keys covered so far are supported by Drupal core. As alluded in the previous article, more are available when using migration configuration entities with the Migrate Plus module. Refer to this article for more information on other root-level keys used in migration files.

Now that we have unpacked the YAML file structure and syntax, we are ready to tackle the next piece in the migration puzzle. In our upcoming article, we will dive into the migration process pipeline—where all these elements start to come together.

13 - The migration process pipeline

Our last article explored the syntax and structure of migration files. Today, we are diving deeper into the most important part of a migration: the process pipeline. This determines how source data will be processed and transformed to match the expected destination structure. We will learn how to configure and chain process plugins, how to set subfields and deltas for multi-value fields, and to work with source constants and pseudo-fields.

Let's get started.

From source to destination

The `process` section in a migration is responsible for transforming data as extracted from the source into a format that the destination expects. **The collection of all those data transformations is known as the migration process pipeline.** The Migrate API is a generic ETL framework. This means the source data can come from different types of sources like a database table; a CSV, JSON, or XML file; a remote API using JSON:API or GraphQL; or something else. The destination can be as diverse including databases, text files, and remote APIs. Because the series focuses on migrating from Drupal 7 to 10, most of our discussion will revolve around reading the Drupal 7 database and writing Drupal 10 entities.

Most of the `source` plugins we are going to see during the series extend the SqlBase abstract class, which requires implementing a `query` method. When migrating from Drupal 7, source plugins will use this method to fetch columns from one or multiple tables. For example, the d7_node source plugin fetches the latest node revision data from the `node` and `node_revision` Drupal 7 tables. It also handles node translations and exposes a `node_type` configuration option to limit which content types to fetch nodes from. The columns that the source plugins return become available in the process section. Source plugins can also implement a `fields` method to document which fields are available. You can use the `drush migrate:fields-source [MIGRATON_ID]` command to see a list of field names and their description for a given migration ID.

Note: The `drush migrate:fields-source` command is calling the `fields` method, not the `query` method. It is possible that extra data might be fetched even if not reported by the command. To get the complete picture of what is available you can use the Migrate Devel module or debug the migration import process. We are going to learn about debugging migrations in a future article.

Source plugins can optionally implement a `prepareRow` method to add additional data to each row being processed. In many cases, this comes from fetching data from other Drupal 7 tables. This is how the `d7_node` source plugin gets field API data attached to the content type. Field attachments are not reported by the `migrate:fields-source` drush command. The `prepareRow` can also be used to skip the whole row completely based on a specified condition.

This brings up a very important point about the migration process. **Each element to import is fetched, transformed, and stored one at a time.** In the case of nodes, each individual node is fetched from Drupal 7, the data transformed by the process pipeline, and the results sent to the destination plugin to create a Drupal 10 node (content) entity.

What if there is data you want or need to exclude from the migration? Fortunately, there are multiple methods to filter out data from a migration. A condition on the `query` method is ideal because the records are filtered out before any data related to them is fetched. If not possible, try to use `prepareRow`. In this case, Drupal 7 will be queried to get source data, but the record is skipped before sending it to the process pipeline. You can also use process plugins to skip records. Doing so as late as the destination plugin is also possible, but generally not necessary. When working through the examples, we will present multiple examples of filtering out Drupal 7 data.

Once the data is fetched from Drupal 7, it has to be transformed into a format that destination plugins can use to create Drupal 10 configuration and content entities. This is where we are going to spend most of the time when writing migrations. In some cases, we can copy data verbatim like a node title. Sometimes changes are necessary because the underlying storage model changed. For example, a date that was stored as a UNIX timestamp in Drupal 7 and now uses a string representation in the Drupal 10 database. When making changes to the content model, we might have to figure out a way to map old data to new one. This can happen at various levels: within a single field, within a single entity, or across multiple entities. Throughout the series we will present multiple examples of these types of transformations.

It is important to highlight that **during the process pipeline, Drupal does not yet know if the entity can be created or not.** This is especially true when entity validation is enabled in the destination plugin for content entities. Our job is to apply the necessary transformations to produce a valid representation of an entity. That is why it is imperative to have a good understanding of entities, their base properties, and the fields attached to them. We talked about this in article 7.

After all the transformations, the destination plugin grabs the data sent by the process pipeline and performs an import operation. In most cases, this means forwarding the data to the Entity API for it to create a configuration or content entity. As a result, we are not writing to Drupal 10 database tables directly but letting Drupal handle the operation based on existing APIs. Additionally, any hook or event that responds to entity actions will be triggered and executed.

While highly discouraged, it is possible to use destination plugins that write directly to database tables. We are not going to cover this as part of the series.

Process plugins

To make the necessary transformations, you need to apply one or more process plugins. Drupal core comes with many general purpose process plugins. It also ships with plugins aimed to assist with Drupal 7 to 10 migrations. The Migrate Plus module also offers very useful process plugins which we will use throughout the series.

Other modules that provide process plugins include migrate_conditions, migrate_process_extra, and migration_tools (not to be confused with the Migrate Tools module). It is important to know what transformations are possible with the process plugins that already exist so take time to review those resources.

That being said, it is quite common to write custom process plugins during Drupal 7 to 10 migrations. This approach will also be covered later in the series.

Note: Explaining what each process plugin does is beyond the scope of this series. To learn more, refer to the official documentation or other guides like the 31 days of migration series. When working on the examples, we will provide context and explanation for the plugins used in customizing the migrations.

Migration code snippets

Now let's review examples to understand how process plugins are configured.

```
process:
  nid:
    -
      plugin: get
      source: tnid
```

In this snippet, we are using the get process plugin, which expects a configuration option named `source`. The `get` plugin is used to make a verbatim copy of a **source field** as fetched by the source plugin. Here we are copying the `tnid` **source field** into the `nid` **destination property**. As we learned in the previous article, a dash (`-`) in YAML is used to represent an element of an array. It is possible, and common, to use multiple process plugins in order to make the necessary transformation on a single source field. When only a single transformation is needed, this could be simplify by removing the dash:

```
process:
  nid:
    plugin: get
    source: tnid
```

The plugin configuration is added as a direct child of destination property. Because `get` is a very common operation, the Migrate API considers it the default process plugin to apply. The above snippet can be further simplified to:

```
process:
  nid: tnid
```

This is syntactic sugar provided by the Migrate API. That is, alternative ways to write a pipeline with the objective of making it easier to read. **When writing migrations, prioritize readability as that will help others, including your future self, to understand what the migration is doing.** This extends to how the transformations are applied. If you need to use multiple process plugins and the pipeline gets long or hard to understand, consider writing a custom source plugin instead.

Each plugin offers a different set of configuration options. Some might be required while others are optional.

```
process:
  langcode:
      plugin: default_value
      source: language
      default_value: und
      strict: TRUE
```

The default_value process plugin checks if the specified `source` property contains a value. If not, whatever is indicated in the `default_value` configuration option is returned. An optional `strict` configuration option can be provided. When set to `TRUE`, the default is applied when input value is `NULL`. When set to `FALSE`, the default is applied when input contains a truthy PHP value. If not specified, `strict` is considered to be `FALSE`.

The two examples above populate entity base properties, which store a single value. When working with fields attached to a content entity, it is possible for a field to store more than one piece of data. Let's see how we can migrate a `Text (formatted, long)` field:

```
process:
  field_description/value:
    -
      plugin: get
      source: description
  field_description/format:
    -
      plugin: get
      source: format
```

A `Text (formatted, long)` field stores two pieces of data: the text to show (the `value` property) and the text format to apply when rendering it (the `format` property). Each piece of data a field stores is referred to as a **subfield**. In this case we have two but other field types can have more. For example, link fields have three subfields: `uri`, `title`, and `options`. And the last one stores a serialized array of options for the link.

It is important not only to know which subfields are available but also understand in which format we need to provide the data. In this case, it has to be a PHP serialized array. It would be impractical to list all subfields for

every possible field. Refer to this reference of sub-fields for some common field types.

Going back to our example, the syntax to use is `[FIELD_NAME]/[SUB_FIELD_NAME]` followed by the process pipeline necessary for the specified subfield. Note that some field types have a default subfield. When that is the case and you only want to set the default subfield, you can omit the `[SUB_FIELD_NAME]` part of the assignment:

```
process:
  field_description:
    -
      plugin: get
      source: description
  field_description/format:
    -
      plugin: get
      source: format
```

The above is a contrived example equivalent to the previous code snippet. Again, favor readability and clarity. If the above seems confusing, it is better to be verbose and indicate the subfield even when mapping the default one.

Technical note: The list of subfields available correspond to database columns used by the field type. They are listed in the `schema` method of the class that implements the field type. The default subfield is provided by the `mainPropertyName` method of the same class. For some field types, like double_field where there is no clear primary subfield, the `mainPropertyName` method will return `NULL`.

Another thing to consider with fields attached to content entities is the possibility of them having multiple values. Let's consider the following example for combining multiple link fields in Drupal 7 to a single multi-value link field in Drupal 10:

```yaml
process:
  field_online_profiles/0/uri: field_drupal/0/url
  field_online_profiles/0/title:
    plugin: default_value
    default_value: 'Drupal.org profile'
  field_online_profiles/1/uri: field_gitlab/0/url
  field_online_profiles/1/title:
    plugin: default_value
    default_value: 'GitLab profile'
  field_online_profiles/2/uri: field_github/0/url
  field_online_profiles/2/title:
    plugin: default_value
    default_value: 'GitHub profile'
```

The syntax for mapping multi-value fields is `[FIELD_NAME]/[DELTA]/[SUB_FIELD_NAME]` followed by the process pipeline, where `[DELTA]` is an integer starting in zero indicating the position of the element in the multi-value collection. Notice that the same syntax applies for extracting data from **Drupal 7 source fields** as to assigning data to **Drupal 10 destination properties**. If no subfield is specified, the default subfield is assumed as explained before. In the previous example, if there is no need to set the link title, the transformation can be simplified to:

```yaml
process:
  field_online_profiles/0/uri: field_drupal/0/url
  field_online_profiles/1/uri: field_gitlab/0/url
  field_online_profiles/2/uri: field_github/0/url
```

Having to specify the delta manually like this is not very flexible. The sub_process process plugin is often used for handling the assignment of multivalue fields, optionally setting more than one subfield in the same operation. For example:

```yaml
process:
  field_image:
    -
      plugin: sub_process
      source: field_image
      process:
        target_id: fid
        alt: alt
        title: title
        width: width
        height: height
```

Process plugin chains

In the code snippets presented so far, only one process plugin has been used for each destination property. Some transformations are more complex than others and using multiple plugins in a chain is a common practice. Chaining of process plugins works similarly to Unix pipelines in that the output of one process plugin becomes the input of the next one in the chain. When the last plugin in the chain completes its transformation, the return value is assigned to the destination property.

```yaml
process:
  source_full_path:
    -
      plugin: concat
      delimiter: /
      source:
        - constants/source_base_path
        - filepath
    -
      plugin: urlencode
```

In this example, the return value of the `concat` process plugin becomes the source for the `urlencode` plugin. Then, the output of `urlencode` is assigned to the `source_full_path` destination property. Most of the time, only the first plugin in the chain will have a `source` configured. For the rest, it is assumed that the output of the previous plugin should be the source. That being said, the migrate API will honor a `source` configuration in subsequent plugins in a process chain. An example would be using the skip_on_empty process plugin to check the presence of a field, but then do the transformation based on a different source field.

```
process:
  field_home_address:
    -
      plugin: skip_on_empty
      source: field_is_home_address/0/value
      method: process
    -
      plugin: addressfield
      source: field_address
```

Here we are checking if a boolean `field_is_home_address` is set to a non-empty value. In Drupal 7, a `TRUE` value will be stored as the integer `1`, while a `FALSE` value will be saved as the integer `0`. When the source field is `TRUE`, `skip_on_empty` allows the pipeline to continue, passing the `1` value to `addressfield`. But then, `addressfield` reads a different source field named `field_address` and that transformation is assigned to the `field_home_address` destination property.

Source constants and pseudo-fields

In the Migrate API, **source constants** are arbitrary values defined in the `source` section. **Pseudo-fields** are named variables defined in the `process` section to store the result of a plugin chain. In both cases, they can be used after being defined in the process pipeline. Let's look at an example:

```
source:
  key: migrate
  plugin: d7_file
  scheme: public
  constants:
    source_base_path: 'http://ddev-migration-drupal7-web/'
process:
  source_full_path:
    -
      plugin: concat
      delimiter: /
      source:
        - constants/source_base_path
        - filepath
    -
      plugin: urlencode
  uri:
    -
```

```
      plugin: file_copy
      source:
        - '@source_full_path'
        - uri
```

In this case, we are creating a **source constant** named `source_base_path`. This is used as one of the source arguments sent to the `contact` plugin. In the example, `source_full_path` is not an entity base property nor a field attached to the entity bundle. Therefore, it is considered a **pseudo-field** which stores the output of the process plugin chain composed of the `concat` and `urlencode` plugins. To reference the value of a pseudo-field, you put an at sign (`@`) and enclose it quotes (`'`).

When expanding the `source` configuration for a process plugin, if the name that starts with `@` it is read from the `process` section. Otherwise, it is read from the `source` section. A pseudo-field should appear in the process pipeline before it can be used. This is similar to how the rewrite results feature works in Views. Like pseudo-fields, you can also reference other destination properties using the `@` prefix as long as they have been defined in the process pipeline. In the snippet below, we reuse the `mail` property to assign the value of the `init` property.

```
process:
  mail:
    -
      plugin: get
      source: field_email/0/email
  init: '@mail'
```

As we have explored in this article, the Migrate API is flexible and impressive. Out of the box, you are able to write complex data transformations with process plugins. And when needed, writing custom process plugins is a straightforward process. However, remember to strive for writing process pipelines that are easy to read, understand, and maintain. Stay tuned as we build on these foundations and continue to explore the migration process.

Migration of Configuration

- 14 - Migrating content types
- 15 - Migrating vocabularies and Drupal 7 field collections into Drupal 10 paragraphs
- 16 - Preparing for field migrations
- 17 - Migrating field storage and instance settings
- 18 - Migrating field widgets settings
- 19 - Migrating view modes and field groups
- 20 - Migrating field formatter settings
- 21 - Applying Drupal recipes to add media types
- 22 - Configuring text formats, editors and user roles

14 - Migrating content types

This step-by-step series has covered a lot of ground on planning and preparing for a Drupal 7 to Drupal 10 migration. Today, we start putting that knowledge into practice by automatically migrating content types. First, we will execute a migration to pull all content types in Drupal 7. Then, we will customize the migration to remove unnecessary content types. Finally, we will learn how a separate node title label migration also affects content type configuration in Drupal 10.

Prerequisites

For this section, we are going to switch to the `start` branch in the repository. This will contain the basic project set up for the Drupal 7 and Drupal 10 sites, but no migrations. You can always refer to the `main` branch for the final version of the examples. If you have not done so, clone the repository and switch to the `start` branch.

```
git clone https://github.com/tag1consulting/d7_to_d10_migration.git d7_to_d10\
_migration
git checkout start
```

Moving forward, we will be executing migrations to migrate data from Drupal 7 into Drupal 10., This process requires both environments to be running. Refer to Article 8 -Example repository setup and Drupal 7 site audit - for an explanation on how to set up the Drupal 7 site and Article 9 - Performing an automated migration - for an explanation on how to set up the Drupal 10 site.

For reference, use the commands below to get both sites up and running.

For Drupal 7:

```
# From the root of the repository.
cd drupal7
ddev start
# See article 7 for importing the database and files.
ddev launch
ddev drush uli
```

For Drupal 10:

```
# From the root of the repository.
cd drupal10
ddev start
ddev composer install
# Install the site from existing configuration
ddev composer si
ddev launch
ddev drush uli
```

The `ddev composer si` is a custom Composer script that drops the current Drupal 10 database and installs the site from the existing configuration. Run this command before proceeding with today's example. Once the site is installed, there is no need to execute this command again unless otherwise instructed. The custom script is defined in the root `composer.json` file as such:

```
"scripts": {
    "si": "drush --yes sql:drop && drush --yes -v site:install tag1_profile -\
-existing-config"
},
```

Migrating content types

In Drupal 10, content types are configuration entities. Back in Article 1- Getting to Know Your Migration, we mentioned that migrating configuration automatically makes sense when there are little or no configuration changes compared to Drupal 7. Our example will include a significant number of content model changes. To account for that, we could create all the configuration manually and only automate the migration of content. Instead, we will cherry-pick and customize the automated migration of a subset of Drupal 7's configuration. After automating part of the process, we will manually create or update configuration in Drupal 10 to reflect the desired content model.

We learned how to generate automatic migrations in article 10, and we talked about code organization for a custom migration project in article 11.As mentioned previously, we will use a `tag1_migration_config` module to store configuration migrations and related code. To do so, copy the `upgrade_d7_node_type` migration from the reference folder into our custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_type.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_node_type.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our custom module. This was discussed in Article 11 - Customizing the generated migration as part of the changes to convert a migration configuration entity to a Core migration plugin. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node_type` and `tag1_configuration`.
- Add `key: migrate` under the `source` section.

For an explanation of these changes, refer back to article 11. After the modifications, the file should look like this:

```
id: upgrade_d7_node_type
class: Drupal\migrate\Plugin\Migration
migration_tags:
  - 'Drupal 7'
  - Configuration
  - node_type
  - tag1_configuration
label: 'Node type configuration'
source:
  key: migrate
  plugin: d7_node_type
  constants:
    preview: 1
process:
  type:
```

```
16      -
17        plugin: get
18        source: type
19    name:
20      -
21        plugin: get
22        source: name
23    description:
24      -
25        plugin: get
26        source: description
27    help:
28      -
29        plugin: get
30        source: help
31    title_label:
32      -
33        plugin: get
34        source: title_label
35    preview_mode:
36      -
37        plugin: get
38        source: constants/preview
39    display_submitted:
40      -
41        plugin: get
42        source: display_submitted
43    new_revision:
44      -
45        plugin: get
46        source: options/revision
47    create_body:
48      -
49        plugin: get
50        source: create_body
51    create_body_label:
52      -
53        plugin: get
54        source: body_label
55    third_party_settings/menu_ui/available_menus:
56      -
57        plugin: static_map
58        bypass: true
59        source: available_menus
60        map:
61          main-menu: main
62          management: admin
63          navigation: tools
64          user-menu: account
65    third_party_settings/menu_ui/parent:
```

```
    -
      plugin: static_map
      bypass: true
      source: parent
      map:
        'main-menu:0:': 'main:'
        'management:0:': 'admin:'
        'navigation:0:': 'tools:'
        'user-menu:0:': 'account:'
destination:
  plugin: 'entity:node_type'
migration_dependencies:
  required: {  }
  optional: {  }
```

Executing the content type migration

Drupal migrations are a two step process. You first write the migration plugin and then you execute it. As discussed in Article 11 - Customizing the generated migration, migrations can be written as Drupal Core plugins or as configuration entities using Migrate Plus. As for executing them, there are multiple migration runners. The most commonly used are Drush and the Migrate Tools module. Both options provide commands to execute migrations from the command line.

Migrate Tools also provides a user interface to run **migration configuration entities** from a web interface. See this Migrate Tools issue discussing support for Core plugins in the administration interface.

Technical note: The commands provided by Drush core and Migrate Plus are mostly the same. There are flags that are only present in one or the other. See this issue to learn about some differences between the two.

Migrate Tools provides extra functionality like shared configuration, which we will not implement. We do not need a web interface for running migrations and that feature does not currently support Core migration plugins. Therefore, we will use Drush to run the migrations from the command line throughout the whole series.

Execute the following commands to make sure our content type migration is being detected properly.

```
cd drupal10
ddev drush migrate:status upgrade_d7_node_type
```

You should see that there are eight records to process. Those correspond to the eight content types present in Drupal 7. You can see them at `https://migration-drupal7.ddev.site/admin/structure/types`

If the command fails saying `Invalid migration IDs: upgrade_d7_node_type`, make sure that the `tag1_migration_config` module is enabled and that the `upgrade_d7_node_type.yml` file is placed in the `migrations` folder inside the module. The full location within the repository should be `drupal10/web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_d7_node_type.yml`

If the command does not return any output other than table headers, it means it is not possible to connect to the Drupal 7 database. Make sure the Drupal 7 DDEV site is running and that a connection string named `migrate` exists in `settings.ddev.php`. See article 9 - Performing an Automated Migration - for details on setting up a database connection string.

Now, let's execute the migration. Moving forward, it is assumed that the commands are executed within the `drupal10` folder at the root of the example project repository.

```
ddev drush migrate:import upgrade_d7_node_type
```

You should see that eight items were processed. The report should also indicate that eight items were created—content types in this case. You can see the migrated content types at `https://migration-drupal10.ddev.site/admin/structure/types`

Customizing the content type migration

In Article 8 - Example repository setup and Drupal 7 site audit, we drafted a lightweight migration plan. We specified that three content types from Drupal 7 would not be migrated to Drupal 10:

- Nodes of type `speaker` will be migrated as `user` entities.
- Nodes of type `sponsor` will be migrated as `taxonomy term` entities.

- Nodes of type `swag` will not be migrated and the content type will be dropped altogether.

These content model changes are also reflected in the source site audit document discussed in Article 3 - Source Site Audit - An In depth Analysis and populated in Article 8 - Example repository setup and Drupal 7 site audit.

Now we need to update the migration to prevent the three content types listed above from being migrated. Identify the following lines in the `upgrade_d7_node_type` migration:

```
process:
  type:
    -
      plugin: get
      source: type
```

And change them to:

```
process:
  # @modified
  type:
    -
      plugin: skip_on_value
      source: type
      value:
        - speaker
        - sponsor
        - swag
      method: row
      message: 'Content type not needed in new site.'
```

As discussed before, the migrate API will fetch and process one record at a time. In this case, we are processing content types. For each Drupal 7 content type, the `d7_node_type` **source plugin** will populate the `type` **source property** with the **machine name** of the content type.

Using the skip_on_value process plugin, we prevent content types from being imported if their machine name matches one of the array elements defined in the `value` configuration option. We skip the whole record from being processed by setting `method: row`. Additionally, we set a custom `message` to explain the reason why the content type was skipped.

It is important to know what process plugins are provided by Drupal core and contributed modules like Migrate Plus. There are many operations that can be performed without writing custom code. During the series, we explain those relevant to customizing the migrations in the example project , but we recommend researching what is available on your own. Refer to the 31 days of migration series for examples and explanations on the use of various process plugins. We also encourage you to review the previous article, which explains in more detail the syntax of migration files and the process pipeline.

If we were to run the `migrate:status` command now, we would see that eight records had been imported and that there are 0 unprocessed records. If we were to run the `migrate:import` command, you would see that no items are processed. Instead of doing either, execute the following commands:

```
# Rollback the migration. Removes all previously imported content types.
ddev drush migrate:rollback upgrade_d7_node_type

# Rebuild caches. This makes changes to the migration file to be detected.
ddev drush cache:rebuild

# Import the migration again. This time, only a subset of the content types w\
ill be migrated.
ddev drush migrate:import upgrade_d7_node_type

# Check the messages related to this migration.
ddev drush migrate:messages upgrade_d7_node_type

# Check the status of this migration.
ddev drush migrate:status upgrade_d7_node_type
```

Let's break down these commands.

First, we rollback the `upgrade_d7_node_type` migration. This removes the content types that were previously imported. Rolling back a migration does not always result in deleting the imported entities. For example, when reverting a node translation, the primary entity might remain in the system. Also, the deletion of entities can be queued to be performed at a later time. We will highlight the cases when rolling back a migration does not result in the immediate deletion of the entity.

Because we are using Drupal core migration plugins, we need to rebuild caches for our changes to the file to take effect. Alternatively, during development we could set the `discovery_migration` cache bin

to `cache.backend.null`. This was discussed in Article 11 - Customizing the generated migration as a way to bypass the need to rebuild caches. Throughout the series, we will be explicit as to when the migration cache is cleared and will rebuild caches manually.

Then we execute the migration again using the `migrate:import` command. This time you will see that out of the eight processed items, five were created and three were ignored. With the `migrate:messages` command we can check that records with **Source ID(s)** `speaker`, `sponsor`, and `swag` have no **Destination ID(s)**. This means that those records were not imported into Drupal 10. The **Message** column should contain a familiar message: "Content type not needed in new site."

```
understanddrupal.com drupal10 % ddev drush migrate:messages upgrade_d7_node_type

 Level  Source ID(s)  Destination ID(s)  Message
 4      speaker                           upgrade_d7_node_type:type: Content type not needed in new site.
 4      sponsor                           upgrade_d7_node_type:type: Content type not needed in new site.
 4      swag                              upgrade_d7_node_type:type: Content type not needed in new site.
understanddrupal.com drupal10 %
```

Figure 10. Console message

Technical note: The `Level` column in the `migrate:messages` command is populated by a call to `\Drupal\migrate\MigrateException::getLevel()`. Possible values are constants in the `\Drupal\migrate\Plugin\MigrationInterface` whose name starts with `MESSAGE_`. An error is stored as `1`, a warning as `2`, a notice as `3`, and an informational message as `4`.

Finally, we execute `migrate:status` to verify there are no pending records to process. It is possible to restrict how many records to import by using the `--limit` flag of the migrate:import command or by dynamically altering the migration query to add a range. Unless you are manually limiting the number of records to process, you should see 0 unprocessed records.

A number other than 0 usually indicates a problem somewhere in the migration:

- If the number is positive, it is possible that the **source plugin**'s configured ID columns are not unique.
- Another option for a positive value is that the column used as a highwater property is not unique.

- If negative, it usually means that records in Drupal 7 have been deleted since the last time the migration was executed. In this case, deleting a content type in Drupal 7 after we execute this migration would result in a negative number.

At this point, there should be 5 content types in the Drupal 10 site. Except for the `body` field attached to the `Article` (`article`) and `Basic page` (`page`), no other field has been added to any content types. Migrating fields require four different migrations which will be discussed in future articles.

Those paying very close attention will notice there is a setting in the `Venue` content type in Drupal 10 that does not match the corresponding Drupal 7 value. That is, the label of the title field. In Drupal 7, it is `Name` while in Drupal 10 it is `Title`.

Migrating node title labels

That brings us to migrating node title labels. There is a separate migration for importing the label for node titles: `upgrade_d7_node_title_label`. Copy it from the reference migrations folder into our custom module and rebuild caches for the new migration to be detected:

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_title_label.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_node_title_label.yml
ddev drush cache:rebuild
```

After copying the file, make the following changes:

1. Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
2. Add two migration tags: `base_field_override` and `tag1_configuration`.
3. Add `key: migrate` under the `source` section.

Now, execute the migration:

```
ddev drush migrate:import upgrade_d7_node_title_label
```

You should see the following notice in the console: `Processed 8 items (1 created, 0 updated, 3 failed, 4 ignored)`. There will also be this error: `upgrade_d7_node_title_label migration: 3 failed.` The eight items correspond to the eight content types from Drupal 7. For each of them, the migration will fetch the Drupal 7 content type label and create a `base_field_override` configuration if it differs from the default `Title` value. The `1 created` corresponds to the `Venue` content type. If you go to `https://migration-drupal10.ddev.site/admin/structure/types/manage/venue`, the `Title field label` shall now be set to `Name` just like in Drupal 7. The `3 failed` correspond to the three content types that were skipped in the `upgrade_d7_node_type`. If the content type does not exist in Drupal 10, it is not possible to override the title label. The `4 ignored` corresponds to other four content types that were migrated: `Article`, `Basic page`, `Event`, and `Session`. Because they use `Title` as the `Title field label`, there is no need to provide a `base_field_override` configuration for them.

If we execute the `ddev drush migrate:messages upgrade_d7_node_title_label` command, we will see there are three messages for this migration. The level `1` for **Source ID(s)** `speaker`, `sponsor`, and `swag` indicate an error as described in the **Message** column.

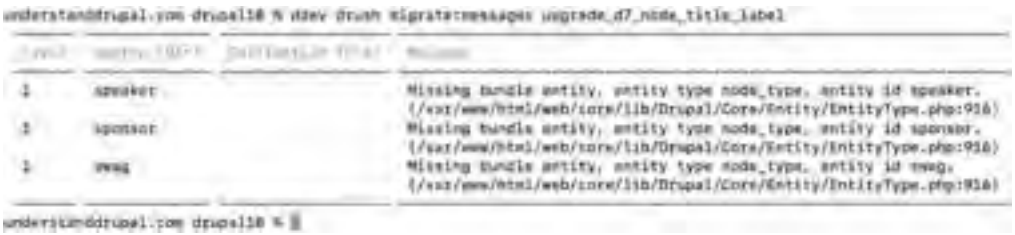


Figure 11. Console notice

What about the four records that were ignored? They are explicitly skipped in the migration:

```yaml
process:
  label:
    -
      plugin: static_map
      source: title_label
      bypass: true
      map:
        Title: 0
    -
      plugin: skip_on_empty
      method: row
```

The process pipeline for the `label` **destination property** skips the whole row if the source `title_label` has a value of `Title`. It first uses the static_map plugin to convert the label `Title` to the number zero. This is later passed to the skip_on_empty plugin, which treats the zero as an empty value. The whole row is skipped by `method: row`.

Ideally, everytime we use `skip_on_empty` or `skip_on_value` to skip a whole row, we should set the `message` configuration option. That way, the map table for this migration stores the reason why a row was skipped. See this issue on improving exception messages in process plugins.

In the meantime, because we are customizing the migrations, we could add the `message` configuration option to the `skip_on_empty` plugin ourselves. Doing so will be left as an independent exercise for you to do. To give it a try, rollback the migration, add the `message` configuration, import the migration again, and review the messages.

Technical note: The migrate messages table is truncated when the migration is rolled back. If you execute `migrate:messages` right after `migrate:rollback` there will not be any messages. You need to run `migrate:import` again for messages to appear, when properly configured.

Striking a balance

We added a whole new migration to update the `Title field label` of the `Venue` content type. **Was that really needed?** Probably not. We could have made this change directly in Drupal 10 after the `Venue` content type was migrated. We did it here to illustrate a point.

Multiple migrations can affect the same concept in Drupal 10. In this case, both `upgrade_d7_node_type` and `upgrade_d7_node_title_label` affect how content types are imported. Under the hood, they create different configuration entities, but the point still stands. Another example of this is migration fields which consist of four different migrations: storage, instance, widget, and formatter. We are going to cover field migrations in future articles.

That being said, we need to strike a balance between what to automate and what changes to do manually in Drupal 10. In cases like the above, it will be evident that updating the configuration manually is better. In other cases, like migrating fields, it is beneficial to run an automated migration even if the results are not 100% perfect. Past experience and familiarity with writing custom migrations will go a long way in making the right decision.

In my experience, menus and blocks are rarely migrated automatically. For permissions, an initial automated migration is performed and they are later adjusted manually. For fields, try to migrate as much as possible automatically. In a field migration for a recent project I was able to automate: dropping fields, renaming fields, moving fields from one entity to another, changing field types, changing field widgets/formatters, and more. This required a good amount of customizations, but it was worth doing considering the number of fields to migrate and the content model changes that needed to be applied. Explaining how this was accomplished is outside the scope of this series, but we will touch on some techniques that could be used to perform these and other customizations.

Ultimately, **our goal is to teach you how to create custom migrations for your own projects**.

What about configuration changes and version control?

We are not going too deep into configuration management nor version control in this series. Those are very important topics to carefully consider in your real life projects. In the interest of brevity, we focus on explaining how to write custom migrations.

That being said, below are some tips for you to consider:

- When executing migrations that create configuration, export that configuration and commit it to the repository.

- When a configuration migration is rolled back and imported again, the `uuid` key of the `*.yml` files will change. You might want to wait for all/most customizations to be completed before committing the resulting configuration.
- If you reinstall the site from the existing configuration, the `_core/default_config_hash` value will change. Also, the migration map and messages tables will be lost. For configuration migrations, this is acceptable as long as your content migrations do not perform lookups against configuration migrations. We are going to follow this approach during the series.
- A single migration can produce multiple configuration entities. In today's example, the `upgrade_d7_node_type` migration created 5 `node.type.*.yml` configuration entities.
- When customizing migrations, it is a good idea to commit the generated migration as-is. Then, add the modifications to the migration files as separate commits. This will make it easier to identify what changes were made.

In the next article, we will learn to migrate taxonomy vocabularies and convert Drupal 7 field collections to Drupal 10 paragraphs.

15 - Migrating vocabularies and Drupal 7 field collections into Drupal 10 paragraphs

In the previous article, we began migrating configuration from Drupal 7 example site to our Drupal 10 instance, specifically content types. In today's article, we will continue with two more D7 entities: taxonomy vocabularies and field collections. The latter will be imported as Paragraphs in Drupal 10. Along the way, we will review the content model and the migration plan. This will help us determine what parts of the migration should be automated and what can be performed manually.

Before we begin

This series is designed to build on top of explanations and examples presented in previous articles. Moving forward, we will assume familiarity with the topics discussed in these five articles:

- Article 8: Site audit and lightweight migration plan
- Article 11: How to customize and execute generated migrations
- Article 12: The syntax of migration files
- Article 13: Source constants, pseudo-fields, and the migration process pipeline
- Article 14: Troubleshooting instructions for common errors when working with migrations

Troubleshooting instructions will be included only for issues not yet covered. Likewise, commands will be presented without much explanation if they had already been discussed. New commands or new flags for previously introduced commands will be explained when presented for the first time. The same applies for other migration concepts that have been discussed. If necessary, take the time to go back in the series and revisit those articles.

Migrating taxonomy vocabularies

We continue the process of migrating Drupal 7 configuration with taxonomy vocabularies. For this we use `upgrade_d7_taxonomy_vocabulary`, the same migration to explain how to customize generated migrations from article 11. Let's copy it from the reference folder into our custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_taxonomy_vocabulary.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_taxonomy_vocabulary.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a migrations folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `taxonomy_vocabulary` and `tag1_configuration`.
- Add `key: migrate` under the `source` section.

After the modifications, the file should look like this:

```
id: upgrade_d7_taxonomy_vocabulary
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - taxonomy_vocabulary
 - tag1_configuration
label: 'Taxonomy vocabularies'
source:
 key: migrate
 plugin: d7_taxonomy_vocabulary
process:
 vid:
   -
     plugin: make_unique_entity_field
```

```
16        source: machine_name
17        entity_type: taxonomy_vocabulary
18        field: vid
19        length: 30
20        migrated: true
21    label:
22      -
23        plugin: get
24        source: name
25    name:
26      -
27        plugin: get
28        source: name
29    description:
30      -
31        plugin: get
32        source: description
33    weight:
34      -
35        plugin: get
36        source: weight
37  destination:
38    plugin: 'entity:taxonomy_vocabulary'
39  migration_dependencies:
40    required: {  }
41    optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7 and the number of vocabularies to import is correct. Then, run `migrate:import` to perform the import operation.

```
1  ddev drush cache:rebuild
2  ddev drush migrate:status upgrade_d7_taxonomy_vocabulary
3  ddev drush migrate:import upgrade_d7_taxonomy_vocabulary
```

This should create the two taxonomy vocabularies from Drupal 7: `Article Tags` (`tags`) and `Session topics` (`session_topics`). You can check them at `https://migration-drupal10.ddev.site/admin/structure/taxonomy` and compare them to their Drupal 7 equivalent at `https://migration-drupal7.ddev.site/admin/structure/taxonomy`.

Adding taxonomy vocabularies creates configuration entities. Remember to export configuration and commit the changes. Also, commit the new `upgrade_d7_taxonomy_vocabulary` migration.

From content type to taxonomy vocabulary

In the lightweight migration plan we drafted in article 8, We specified that nodes of type `sponsor` will be migrated as `taxonomy term` entities. Therefore, we need to convert the `sponsor` content type into a taxonomy vocabulary.

Pay close attention to the terminology used. Nodes and taxonomy terms are **content entities**. Content types and taxonomy vocabularies are **configuration entities**. Refer to article 7 for a refresher on these concepts and the difference among them.

Before we do anything else, let's have a look at the `sponsor` content type in Drupal 7. Take time to thoroughly review how the content type is configured. It is important to check the fields attached to it: `https://migration-drupal7.ddev.site/admin/structure/types/manage/sponsor/fields`

There are two fields we need to account for: `field_logo` of type `Image` and `field_description` of type `Long text`. After further inspecting `field_description`, we can see that it is a rich text field that allows selecting a text format. Additionally, we have the node properties `title` and `path`. The latter stores a path alias for the node, if one is set.

Before thinking about migrating either configuration or content related to sponsors, we need to come up with a plan on how to convert the Drupal 7 content type into a Drupal 10 taxonomy vocabulary. Check this article to learn about base field definitions for Drupal 10 taxonomy term content entities.

Now, consider the following:

- The Drupal 7 node `title` will be migrated as the Drupal 10 taxonomy term `name`.
- The Drupal 7 node `field_description` will be migrated as the Drupal 10 taxonomy term `description` property. No need to create a new field for this because the term's description is already a rich text field.
- We would need to add an image field to the vocabulary in Drupal 10 to map `field_logo` from Drupal 7. We could use a media entity reference field here, but let's keep things simple and use a regular image field instead.
- Drupal 10 taxonomy terms have a `path` property which could be used to map the node's `path` property from Drupal 7. We will talk about URL alias

migration later in the series. For now, the Drupal 7 path alias will not be considered.

In addition to the above, it is important to review other fields and properties either from Drupal 7 or Drupal 10. For example, we will *not* migrate node IDs (`nid`) as taxonomy term IDs (`tid`). It is possible that in Drupal 7, the value of the node id of a sponsor matches the value of an existing taxonomy term id. If we were to map the `nid` as `tid`, we can potentially overwrite and lose data as discussed in article 5. We will work on preventing entity ID conflicts during the future content migrations.

To automate or not to automate, that is the question. We could probably come up with a way to automate the creation of the new `sponsor` taxonomy vocabulary in Drupal 10 and add an image field, but shall we? No. As described in the previous article, we need to strike a balance between what to automate and what changes to do manually in Drupal 10.

In this case, it is easy enough to create a new vocabulary via the interface and subsequently add the image field. Basic site-building knowledge is assumed for those following this series. We will not list every step of the process but start by visiting `https://migration-drupal10.ddev.site/admin/structure/taxonomy/add` to create the new vocabulary. Then proceed to attach the image field for the logo and configure the form and view modes. Refer to the final example to see how everything is set up.

Creating a new `sponsor` taxonomy term in Drupal 10 should look like the above. Notice that the description does not offer a way to select a text format. We have yet to migrate them so at the moment only the default `Plain text` format exists. When more formats become available, it will be possible to choose from them based on the user's permissions.

From field collections to paragraphs

Field collection was a very popular module in Drupal 7. It has no Drupal 10 version, but the Paragraphs module can be used as a replacement. Paragraphs also offers an automated migration for moving configuration and content from Drupal 7 field collections. This time, we will take advantage of the automation available. This was part of the lightweight migration plan from article 8.

As a reminder, our use of field collections in Drupal 7 is rather contrived. We have a single field collection named `field_favorite_quote` attached to the `speaker` content type. The field collection itself has only two fields. We could have planned a different way to migrate this data, but bare with us. We want to include examples of paragraph migrations. That being said, **always evaluate what is the best content model taking into account new features or modules that become available in the most recent version of Drupal**. For example, I have migrated Drupal 7 field collections into Drupal 10 double fields or other entity types altogether.

For now, copy the `upgrade_d7_field_collection_type` migration from the reference folder into our custom module and rebuild caches for the migration to be detected. For this migration to exist, you should have had the Paragraphs module enabled in Drupal 10 before generating the migrations with the Migrate Upgrade module as discussed in article 10.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_collection_type.yml\
 \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field_collection_type.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a migrations folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `paragraphs_type` and `tag1_configuration`.
- Add `key: migrate` under the `source` section.

After the modifications, the file should look like this:

```
id: upgrade_d7_field_collection_type
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - paragraphs_type
 - tag1_configuration
label: 'Paragraphs - Field Collection type configuration'
source:
 key: migrate
 plugin: d7_field_collection_type
 add_description: true
process:
 id:
   -
     plugin: get
     source: bundle
 label:
   -
```

```
20        plugin: get
21        source: name
22    description:
23      -
24        plugin: get
25        source: description
26  destination:
27    plugin: 'entity:paragraphs_type'
28  migration_dependencies:
29    required: {  }
30    optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7 and the number of field collections to import is correct. Then, run `migrate:import` to perform the import operation.

```
1  ddev drush cache:rebuild
2  ddev drush migrate:status upgrade_d7_field_collection_type
3  ddev drush migrate:import upgrade_d7_field_collection_type
```

If you get an `Invalid migration IDs: upgrade_d7_field_collection_type` error when trying to execute any of the migrate commands, make sure the `paragraphs` module is enabled in Drupal 10. This migration uses the `d7_field_collection_type` source plugin provided by that module. If the `paragraphs` module is uninstalled or not present at all in the codebase, the migrate API will filter out the migration even if the

`upgrade_d7_field_collection_type.yml` file exists in the expected location. The example repository already adds the `paragraphs` module to the project. If using a different setup, install it via Composer: `composer require 'drupal/paragraphs:^1.17'`.

This should create one paragraph in Drupal 10 which you can review at `https://migration-drupal10.ddev.site/admin/structure/paragraphs_-type`. Note that the name of the paragraph is derived from the Drupal 7 field name. As for the machine name, it is the Drupal 7 field name without the `field_` prefix. In our case, the Drupal 7 `field_favorite_quote` field collection was imported as the `favorite_quote` paragraph in Drupal 10.

Great job! Three entity types are now migrated. In the next article, we will begin migrating the fields attached to these and other entities.

16 - Preparing for field migrations

So far we have migrated three entity types: content types, taxonomy vocabularies, and paragraphs. It is very common that fields are attached to those, and other entities, to collect and display data. **Field migrations can be tricky.** For one, it is a multi-step process that requires, at a minimum, four different migrations. Additionally, it is common to find errors because field related configuration used in Drupal 7 is not available in Drupal 10. In this article, we take a pause from executing migrations to understand how fields work in Drupal. The information presented today will prove useful for custom migrations, especially when they include content model changes.

Understanding Drupal fields

Drupal fields are used to provide structure to the information the CMS stores. They save discrete data, which can be used for displaying, filtering, and sorting purposes. Fields are attached to entities like nodes, users, taxonomy terms, blocks, etc. For entities that can have bundles, each bundle can have a different set of fields attached to them. The node entity, for example, almost always has a different set of fields attached to each content type. Read this article for more information on fields from a site building perspective.

Fields can be of multiple types: number, text, file, date, telephone, reference to other Drupal entities, and more. The **field type** determines the underlying database table structure used to store the field data. For example, Drupal offers `Date` and `Timestamp` fields to store date and time information. The `Date` field uses a string representation in ISO 8601 format while the `Timestamp` field stores an integer corresponding to the UNIX timestamp. Each field can be of only one type. For example, a field can be of type `Date` or `Timestamp`, but not both.

Under the hood, a field type consists of two components: **storage settings** and **instance settings**. To provide some context, let's review the settings of a `Date` field attached to the `Event` content type. Below is an screenshot of the configuration form:

In the screenshot there are two **storage settings** in the `Field Storage` section: `Date type` and `Allowed number of values`. The former determines if we want to save `Date and time` values or `Date only` values, without a time component. The latter is internally named **field cardinality** and indicates how many values to store for this field. The cardinality can be a fixed integer value or unlimited if we want to allow an arbitrary number of values. In our example,

we only deal with single date events so a cardinality of one suffices. If we were to allow for multi-day events, increasing the cardinality of this field is not the best approach. Instead, use a `Date range` field which allows specifying start and end dates as part of a single value. Back to field storage settings, some apply globally for every field type while others are dependent on the field type in question. For example, cardinality is a global field storage setting that can be set independently on the field type. On the other hand, the date type storage setting is only available for fields of type `Date` and `Date range` fields.

The rest of the settings in the screenshot above are **instance settings**: `Label`, `Help text`, `Required field`, `Set default value`, and `Default value`. Similar to storage settings, instance settings can be global or specific to field types. All the settings in this `Date` field example are global because the field type itself does not provide any field-specific setting. Other field types do. For example, the `Image` field provides settings to restrict the allowed file extensions, specify maximum and minimum image dimensions, enforce a maximum upload size, enable and optionally require `title` and `alt` HTML attributes.

Both storage and instance settings provide validation. On top of this, each field type has some intrinsic validation logic. For example, a required `Date` field not only checks that a value is specified, but also that such value is a valid date. February 31, 2024 would yield a validation error if the user tries to enter that date. Drupal APIs enforce data validation at different layers. Explaining them is beyond the scope of this article, but for reference you can research about Field API, Entity API, Form API, and Typed Data API, and validation constraints.

Drupal 10 allows sharing fields across bundles of the same entity. **Storage settings apply to all usages of the field across bundles of the same entity. Instance settings can be different for each bundle where the field is used.** Consider an `Image` field attached to multiple content types. One storage setting for images is `Upload destination` which determines if files uploaded via this field are public or private. You cannot have the same field, identified by the same machine name, as public in one content type and as private in another. In fact, if no content has been added for that field, you will be allowed to change the field storage value in any of the content types. But remember that the change will affect all the content types where the field is used. If there is already data for the field, Drupal might lock some storage settings to maintain data integrity and prevent data loss. For example, if an option of a `List (text)` field has already been used, Drupal will not let you remove that option nor

change its machine name.

On top of field types, Drupal also leverages **field widgets** for capturing data and **field formatters** for displaying data. The field widgets are used in the **edit** interface for an entity while the field formatters are used in the **view** interface for an entity. Going back to content types and nodes, field widgets are configured as part of the `Manage form display` section of the content type. You see them in action when **editing** an entity. As for field formatters, they are configured as part of the `Manage display` section of the content type, and you see them in action when **viewing** an entity. The available widgets and formatters will vary per field type. For instance, out of the box a `List (text)` field can choose between two field widgets: a select list or check boxes/radio buttons. The same field type can choose between two field formatters: `key` to display the machine name of the option or `default` to show the name/label of the option.

As it is often the case, Drupal can be extended via modules. New modules can add new field types, alter storage or instance settings, provide extra widgets and formatters for existing fields, and more. Remember these are provided by modules and such modules need to be available in order for the functionality to be available. Drupal core ships with the Telephone and Datetime range modules, but they are disabled by default. Unless you enable them, field related functionality provided by them will not be available.

Field-related migrations

We could probably write a whole series on Drupal fields alone. For now, let's shift our focus back to migrations. You can certainly go a long way without understanding how Drupal fields work. If your Drupal 7 project only uses core fields or contributed fields with an automated upgrade path, the migrate API will take care of everything for you. But that is not the case with most real life projects. **Investing time learning about the different aspects of fields is very valuable, especially in custom migrations.** If you need to make content model changes – like changing from one entity to another, changing from one field type to another, or combining/splitting fields – it will be necessary to understand how fields work.

As we begin, it's important to note that migrating fields requires at least four migrations:

- `upgrade_d7_field` migrates **field storage settings**.
- `upgrade_d7_field_instance` migrates **field instance settings**.
- `upgrade_d7_field_instance_widget_settings` migrates **field widget settings**.
- `upgrade_d7_field_formatter_settings` migrates **field formatter settings**.

Each of those migrations can specify one or more migration dependencies. For example, it is common for the `upgrade_d7_field_instance` migration to depend on the `upgrade_d7_node_type`, because field instances are associated with bundles of the node entity, that is content types. `upgrade_d7_field_instance` also requires the field storage settings to be present. For this reason, it declares a dependency on `upgrade_d7_field`. Both `upgrade_d7_field_instance_widget_settings` and `upgrade_d7_field_formatter_settings` depend on field instance settings thus they declare a dependency on `upgrade_d7_field_instance`. In the case of `upgrade_d7_field_formatter_settings`, formatter settings are tied to view modes so there is an extra dependency on the `upgrade_d7_view_modes` migration. Other dependencies might be specified.

Field migrations are very dynamic and depend on the entities that are used on your project. If using paragraphs, the field migrations will likely depend on the migration that created the paragraph types, `upgrade_d7_field_collection_type` in our example. We are going to cover which migration needs to be executed and in which order as we progress through the examples.

Other modules can introduce field related configuration that also needs to be migrated. For example, if using the field group module in Drupal 7, you can have extra migrations for each entity/bundle combination that uses field groups. Also remember that the automated upgrade path requires modules to be enabled both in Drupal 7 and Drupal 10. This also applies for modules that provide field related functionality. In fact, some upgrade paths might require two or more modules in Drupal 10 to properly migrate Drupal 7 data. For instance, the date module in Drupal 7 provides field types that can store both start and end dates. In Drupal 10, you have the Datetime module if you only need to store the start date and the Datetime range module if you need to store start and end dates. If you have a Drupal 7 field that accepts end dates, but the `Datetime Range` is not enabled in Drupal 10, you will get an error indicating that such a field cannot be migrated unless the extra module is enabled. These

errors are logged to the messages tables. They can be accessed using the drush migrate:messages command or the migration messages admin interface.

Custom field migrations

A custom field migration is necessary when there are content model changes or when your Drupal 10 implementation uses a different field configuration than the one used in Drupal 7. Consider the following examples:

- You want to convert embedded images in rich text fields in Drupal 7 into media entities in Drupal 10.
- You want to extract content from rich text fields in Drupal 7 and save it into paragraphs or layout builder in Drupal 10.
- You have multiple single value link fields in Drupal 7 that should be consolidated into one social link field in Drupal 10.
- You have multiple single value file fields in Drupal 7 that should be consolidated into one multi-value file field in Drupal 10.
- A field type, widget, or formatter used in Drupal 7 is not available in Drupal 10. A new configuration is implemented and you need to map Drupal 7 settings into Drupal 10 settings.
- You want to convert nodes in Drupal 7 to users in Drupal 10. An email field attached to the node in D7 should be mapped to the `mail` and `init` properties of the user entity in D10.
- You need to provide an upgrade path for a module that does not offer it out of the box. For example, at the time of publishing this article the Radioactivity module does not provide an upgrade path from Drupal 7 to Drupal 10.
- You have a date field using the ISO 8601 representation to store the value in Drupal 7 that should be converted to a UNIX timestamp field in Drupal 10. One reason for such change could be to improve performance when doing date and time calculations or comparisons.

Broadly speaking, these transformations can be grouped into converting from one field/entity type to another, combining/splitting fields, and extracting data to save it in a structured way. There are two primary ways of performing these custom field migrations.

The first approach consists of implementing a MigrateField plugin to provide the mapping between Drupal 7 and Drupal 10 settings. This can dynamically alter the four field-related migrations we listed in the previous section. It can also alter the process plugin chain applied to individual fields based on their types.

The second approach involves manually performing the necessary data transformation as part of the process pipeline. This often means creating a custom process plugin to transform data extracted from Drupal 7 into the format the Drupal 10 field expects. For this approach, it is also possible to implement hook_migrate_prepare_row to transform data as it is fetched by the source plugin.

Database table structure for Drupal fields

A recurrent theme in today's article is data transformation. Now, we will take a look at how Drupal stores field data. Along the way, we will highlight some technical details on how fields are implemented.

Let's start with things that are common between Drupal 7 and Drupal 10 fields. Each field has a unique machine name. When a field is attached to an entity bundle, it creates two tables in the database. One table stores the current value for the latest revision of the content entity. The other table stores the past values used by previous revisions of the content entity. In both cases, the table names include the machine name of the field they store data for. In both cases, there is a column in the table to store delta values. Every field can potentially have an unlimited cardinality. When a field allows for multiple values to be stored, this delta column keeps track of the order in which values were added to the field.

In Drupal 7, fields can be shared across multiple entities. The pattern for the database tables' names is `field_data_[FIELD_MACHINE_NAME]` and `field_revision_[FIELD_MACHINE_NAME]`. In both cases, the following columns are available: `entity_type`, `bundle`, `deleted`, `entity_id`, `revision_id`, `language`, `delta`. Notice that language information is also stored at the field level. More columns will be created based on what the field type deems necessary to store. Drupal 7 fields are exposed by hook_field_info and their database schema is determined by hook_field_schema. As an example, consider the `image` field that ships with Drupal 7 core. In image_field_schema

we can see that this field type requires five columns to store its data: `fid`, `alt`, `title`, `width`, and `height`. When the Field API creates this field, the column names follow the pattern `[FIELD_MACHINE_NAME]_[COLUMN_NAME]`. Assuming you are using MySQL, MariaDB, or any of its variants you can use the EXPLAIN statement to see the structure of a table. Below is the output of running `EXPLAIN field_data_field_image;` on the SQL server of the example Drupal 7 site.

```
+--------------------+------------------+------+-----+---------+-------+
| Field              | Type             | Null | Key | Default | Extra |
+--------------------+------------------+------+-----+---------+-------+
| entity_type        | varchar(128)     | NO   | PRI |         |       |
| bundle             | varchar(128)     | NO   | MUL |         |       |
| deleted            | tinyint(4)       | NO   | PRI | 0       |       |
| entity_id          | int(10) unsigned | NO   | PRI | NULL    |       |
| revision_id        | int(10) unsigned | YES  | MUL | NULL    |       |
| language           | varchar(32)      | NO   | PRI |         |       |
| delta              | int(10) unsigned | NO   | PRI | NULL    |       |
| field_image_fid    | int(10) unsigned | YES  | MUL | NULL    |       |
| field_image_alt    | varchar(512)     | YES  |     | NULL    |       |
| field_image_title  | varchar(1024)    | YES  |     | NULL    |       |
| field_image_width  | int(10) unsigned | YES  |     | NULL    |       |
| field_image_height | int(10) unsigned | YES  |     | NULL    |       |
+--------------------+------------------+------+-----+---------+-------+
```

Note: In the example project, you can execute `ddev mysql` to access a database client to execute SQL statements. Alternatively, you can use one of the many database GUIs available for DDEV.

In Drupal 10, fields can only be shared across bundles of the same entity. The pattern for the database tables' names is `[ENTITY_TYPE]__[FIELD_MACHINE_NAME]` and `[ENTITY_TYPE]_revision__[FIELD_MACHINE_NAME]`. **Notice the double underscores before the machine name of the field.** In both cases, the following columns are available: `entity_type`, `bundle`, `entity_id`, `revision_id`, `language`, `delta`. Compared to Drupal 7, only the `deleted` column is missing. Like in Drupal 7, more columns will be created based on what the field type deems necessary to store. In Drupal 10, field types are exposed by FieldType plugins and their database schema is determined by the `schema` method of the class providing the field type. Study the `image` field that ships with Drupal 10 core. It is provided by the ImageItem class, which in its `schema` method lists some familiar column names: `target_id`, `alt`, `title`, `width`, `height`. Compared to Drupal 7, the `fid` column was renamed to `target_id`.

Using `target_id` as the column to store the ID of the entity the field is pointing to is a common pattern in all entity reference fields. As with Drupal 7, these column names translate to `[FIELD_MACHINE_NAME]_[COLUMN_NAME]` columns when the field is created by the Field API. For reference, below is the output of running `EXPLAIN node__field_image;` on the SQL server of the example Drupal 10 site.

```
+-----------------------+------------------+------+-----+---------+-------+
| Field                 | Type             | Null | Key | Default | Extra |
+-----------------------+------------------+------+-----+---------+-------+
| bundle                | varchar(128)     | NO   | MUL |         |       |
| deleted               | tinyint(4)       | NO   | PRI | 0       |       |
| entity_id             | int(10) unsigned | NO   | PRI | NULL    |       |
| revision_id           | int(10) unsigned | NO   | MUL | NULL    |       |
| langcode              | varchar(32)      | NO   | PRI |         |       |
| delta                 | int(10) unsigned | NO   | PRI | NULL    |       |
| field_image_target_id | int(10) unsigned | NO   | MUL | NULL    |       |
| field_image_alt       | varchar(512)     | YES  |     | NULL    |       |
| field_image_title     | varchar(1024)    | YES  |     | NULL    |       |
| field_image_width     | int(10) unsigned | YES  |     | NULL    |       |
| field_image_height    | int(10) unsigned | YES  |     | NULL    |       |
+-----------------------+------------------+------+-----+---------+-------+
```

Technical note: The columns listed in the `schema` method of the class that implements the field type correspond to the sub-fields covered in article 13. The default sub-field is the column specified in the `mainPropertyName` method of the same class. In the case of the ImageItem class, the `mainPropertyName` method returns `target_id` making it the default sub-field. For some field types, like double_field where there is no clear primary subfield, the `mainPropertyName` method will return `NULL`.

To put this into practice, consider the following field mapping for an image field:

```yaml
process:
 field_image:
   -
     plugin: sub_process
     source: field_image
     process:
       target_id: fid
       alt: alt
       title: title
       width: width
       height: height
```

Technical note: Image is a core field both in Drupal 7 and Drupal 10. In this case, the transformation is provided by Drupal 10 core via the `defineValueProcessPipeline` method of the ImageField migrate field plugin.

Because all fields can potentially have multiple values, when the migrate API fetches field data from Drupal 7, it will return a nested array structure. At the top level the nested array is keyed by the deltas and each value includes a full set of columns used by the field type. See the following example of a content entity with two values for the `field_image` field:

```
[
  0 => [
    fid = "3"
    alt = "DrupalCon Portland 2024 logo"
    title = null
    width = "800"
    height = "400"
  ],
  1 => [
    fid = "7"
    alt = "DrupalCon Barcelona 2024 logo"
    title = null
    width = "800"
    height = "400"
  ],
]
```

When we have nested structures like the above, we can use the sub_process process plugin to iterate over them. In the `process` configuration of this plugin we specify the mapping from Drupal 7 column names to Drupal 10 column names. Notably, the `fid` column from Drupal 7 is mapped to `target_id` in Drupal 10. The rest of the columns are mapped verbatim without the need of name changes.

In the case of image fields, Drupal takes care of implementing the necessary transformations. But the same logic applies for custom field migrations. We read data from Drupal 7 and transform it, one way or another, into what Drupal 10 expects. We will cover different examples as progress through the series.

Drupal 10 field and migrate plugins

To wrap up today's article, I want to highlight that in Drupal 10, field types, widgets, and formatters are all implemented via plugins. This will become evident when we execute the field-related migrations. Very likely you will get errors mentioning that a plugin was not found for a field type, widget, or formatter used in Drupal 7 that is not available in Drupal 10. Based on the enabled modules in your Drupal 10 installation, you can check which field plugins are available using the following Drush commands:

```
# List of field type plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.fiel\
d.field_type')->getDefinitions()));"

# Details on a specific field type plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.field.field_typ\
e')->getDefinitions()['FIELD_TYPE_ID']);"

# List of field widget plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.fiel\
d.widget')->getDefinitions()));"

# Details on a specific field widget plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.field.widget')-\
>getDefinitions()['FIELD_WIDGET_ID']);"

# List of field formatter plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.fiel\
d.formatter')->getDefinitions()));"

# Details on a specific field formatter plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.field.formatter\
')->getDefinitions()['FIELD_FORMATTER_ID']);"
```

Migrations, source, process, destination, and field plugins can be inspected in a similar manner:

```
# List of migration plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ation')->getDefinitions()));"

# Details on a specific migration plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migration')->ge\
tDefinitions()['MIGRATION_ID']);"

# List of migrate source plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.source')->getDefinitions()));"

# Details on a specific migrate source plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.source'\
)->getDefinitions()['SOURCE_PLUGIN_ID']);"

# List of migrate process plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.process')->getDefinitions()));"

# Details on a specific migrate process plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.process\
')->getDefinitions()['PROCESS_PLUGIN_ID']);"

# List of migrate destination plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.destination')->getDefinitions()));"

# Details on a specific migrate destination plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.destina\
tion')->getDefinitions()['DESTINATION_PLUGIN_ID']);"

# List of migrate field plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.field')->getDefinitions()));"

# Details on a specific migrate field plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.field')\
->getDefinitions()['MIGRATE_FIELD_PLUGIN_ID']);"
```

Technical note: There are more migration plugin types. Drupal core provides migrate ID maps. The Migrate Plus module adds authentication, data_fetcher, and data_parser plugins. The Migrate Tools module includes shared configuration plugins. Other modules can extend the migration system with more migration-related plugin types. The ones listed in the snippet above are the most frequently used when writing custom migrations.

We covered a lot of ground today. This article is arguably the most technical

in the series so far. It is intended to act as a reference for upcoming articles covering configuration and content migrations. While we are going to include many examples of custom field migrations, the information presented today should help inform your real-world migrations.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

17 - Migrating field storage and instance settings

In the previous article, we learn about Drupal fields, their underlying data structure at the database level, and some considerations for creating custom field migrations. We also pointed out that migrating fields require at least four different migrations. Today, we will start the multi-step process of migrating fields by taking care of the first two components: storage and instance settings.

We are going to initially run the automated migrations with no significant modifications to familiarize ourselves with the process and to identify some of the errors that come up when migrating fields. Then, we are going to use the Migrate Skip Fields module and perform manual configuration updates to our field configuration to account for content model changes.

Before we begin

Our example project involves many content model changes that require adjustments to field configurations. Refer to article 8 for a high level overview of the migration plan. Also refer to the `field_instance` tab of the Drupal Site Audit Template for a summary of the content model changes related to fields.

In article 14, we talked about exporting and committing configuration changes to version control. This is useful in case something goes wrong and you need to revert to a previous state of the site configuration. You could also go one step further and create a full database dump after completing or before starting a migration. Importing a database dump is generally easier than restoring a previous state of the configuration, especially if you have uncommitted configuration files. It also has the added benefit of including any content that exists on the site. Some content migrations can take a very long time to complete. Having a snapshot of the (partially) migrated database makes it easier to start from a known checkpoint in case something goes wrong with the next migration. You can use the following DDEV commands to create, list, restore, and delete database backups.

```
# Create database backup.
ddev snapshot --name [BACKUP_NAME]

# List database backups.
ddev snapshot --list

# Restore database backup.
ddev snapshot restore [BACKUP_NAME]

# Delete all database backups.
ddev snapshot --cleanup

# Delete single database backup.
ls .ddev/db_snapshots/
rm [FILENAME]
```

Replace `[BACKUP_NAME]` by an identifier using underscores (_) instead of spaces ( ) to separate words. The commands above were provided because the example project is set up with DDEV already. If preferred, you could use other tools to create and restore database dumps. For example, drush sql:dump to create a backup and ddev import-db to restore it.

Technical note: The snapshot command in DDEV backs up the entire state of the database server for the project. If you have two or more databases, all of them will be included in the backup. When restoring, all databases will be recreated based on the content of the backup file. Snapshots are stored in the `.ddev/db_snapshots` folder as gzipped files. They can be manually and individually deleted if needed.

Field storage and instance configuration

Drupal fields consist of four different configuration elements: storage, instance, widget, and formatter settings. We are migrating the first two today. As a quick reminder, field storage settings are closely related to the underlying structure Drupal uses to save information at the database level. If you re-use fields across multiple bundles of the same entity, all those fields will share the same storage configuration. On the other hand, field instance settings can vary per entity/bundle combination. In many cases, field instance settings are related to validation logic when collecting data. Refer to the previous article for a more thorough explanation of Drupal fields.

We use `upgrade_d7_field` and `upgrade_d7_field_instance` to migrate field storage and instance settings respectively. Let's copy these migrations

from the reference folder into our custom module and rebuild caches for the migrations to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field.yml
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_instance.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field_instance.yml
ddev drush cache:rebuild
```

Note that while copying the files, we also changed their names and placed them in a migrations folder inside our custom module. After copying the files, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `cck_plugin_method`, and `migration_group`. Notice that for field migrations, we preserve the `field_plugin_method` key.
- Add the `field_storage_config` tag to the `upgrade_d7_field` migration and the `field_config` to the `upgrade_d7_field_instance`. Add the `tag1_configuration` tag to both migrations.
- Add `key: migrate` under the source section.
- Update the required `migration_dependencies` to include the migrations that create entities. Given that fields are attached to entities, we want to be explicit about establishing dependencies among migrations. In this case, add `upgrade_d7_node_type`, `upgrade_d7_taxonomy_vocabulary`, and `upgrade_d7_field_collection_type` to both field storage and instance migrations.

After the modifications, the `upgrade_d7_field.yml` file should look like this:

```yaml
id: upgrade_d7_field
class: Drupal\migrate_drupal\Plugin\migrate\FieldMigration
field_plugin_method: alterFieldMigration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - field_storage_config
 - tag1_configuration
label: 'Field configuration'
source:
 key: migrate
 plugin: d7_field
 constants:
   status: true
   langcode: und
process:
 entity_type:
   -
     plugin: get
     source: entity_type
   -
     plugin: static_map
     map:
       field_collection_item: paragraph
       paragraphs_item: paragraph
     bypass: true
 status:
   -
     plugin: get
     source: constants/status
 langcode:
   -
     plugin: get
     source: constants/langcode
 field_name:
   -
     plugin: get
     source: field_name
 type:
   -
     plugin: process_field
     source: type
     method: getFieldType
     map:
       d7_text:
         d7_text: d7_text
       taxonomy_term_reference:
         taxonomy_term_reference: taxonomy_term_reference
       image:
         image: image
```

```
51        email:
52          email: email
53        field_url:
54          field_url: field_url
55        addressfield:
56          addressfield: addressfield
57        telephone:
58          telephone: telephone
59        entityreference:
60          entityreference: entityreference
61        file:
62          file: file
63        list:
64          list: list
65        datetime:
66          datetime: datetime
67        number_default:
68          number_default: number_default
69  cardinality:
70    -
71      plugin: get
72      source: cardinality
73  settings:
74    0:
75      plugin: d7_field_settings
76    field_collection:
77      plugin: field_collection_field_settings
78 destination:
79  plugin: 'entity:field_storage_config'
80 migration_dependencies:
81  required:
82    - upgrade_d7_field_collection_type
83    - upgrade_d7_node_type
84    - upgrade_d7_taxonomy_vocabulary
85  optional: {  }
```

And the `upgrade_d7_field_instance.yml` file should look like this:

```yaml
id: upgrade_d7_field_instance
class: Drupal\migrate_drupal\Plugin\migrate\FieldMigration
field_plugin_method: alterFieldInstanceMigration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - field_config
 - tag1_configuration
label: 'Field instance configuration'
source:
 key: migrate
 plugin: d7_field_instance
 constants:
   status: true
   comment_node: comment_node_
process:
 type:
   -
     plugin: process_field
     source: type
     method: getFieldType
 entity_type:
   -
     plugin: get
     source: entity_type
   -
     plugin: static_map
     map:
       field_collection_item: paragraph
       paragraphs_item: paragraph
     bypass: true
 field_name:
   -
     plugin: get
     source: field_name
 bundle_mapped:
   -
     plugin: static_map
     source: bundle
     bypass: true
     map:
       comment_node_forum: comment_forum
   -
     plugin: paragraphs_process_on_value
     source_value: entity_type
     expected_value: field_collection_item
     process:
       plugin: paragraphs_strip_field_prefix
 _comment_type:
   -
```

```
      plugin: explode
      source: bundle
      delimiter: comment_node_
    -
      plugin: extract
      index:
        - 1
      default: false
    -
      plugin: skip_on_empty
      method: process
    -
      plugin: migration_lookup
      migration: {  }
  bundle:
    -
      plugin: field_bundle
      source:
        - entity_type
        - '@bundle_mapped'
  label:
    -
      plugin: get
      source: label
  description:
    -
      plugin: get
      source: description
  required:
    -
      plugin: get
      source: required
  status:
    -
      plugin: get
      source: constants/status
  allowed_values:
    -
      plugin: sub_process
      source: allowed_vid
      process:
        -
          plugin: migration_lookup
          migration: upgrade_d7_taxonomy_vocabulary
          source: vid
  settings:
    0:
      plugin: d7_field_instance_settings
      source:
        - settings
```

```
101         - widget
102         - field_definition
103     field_collection:
104       plugin: field_collection_field_instance_settings
105   default_value_function:
106     -
107       plugin: get
108       source: ''
109   default_value:
110     -
111       plugin: d7_field_instance_defaults
112       source:
113         - default_value
114         - widget
115   translatable:
116     -
117       plugin: get
118       source: translatable
119 destination:
120   plugin: 'entity:field_config'
121 migration_dependencies:
122   required:
123     - upgrade_d7_field
124     - upgrade_d7_field_collection_type
125     - upgrade_d7_node_type
126     - upgrade_d7_taxonomy_vocabulary
127   optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migrations. RWe run `migrate:status` to make sure we can connect to Drupal 7. Then, we run `migrate:import` to perform the import operations. Notice that both commands allow specifying multiple migrations when separated by a comma without spaces between migration IDs. When you specify multiple migrations to be executed, the command will build a dependency tree and execute the migrations trying to respect what is specified in the `migration_dependencies` key.

```
1 ddev drush cache:rebuild
2 ddev drush migrate:status upgrade_d7_field,upgrade_d7_field_instance
3 ddev drush migrate:import upgrade_d7_field,upgrade_d7_field_instance
```

After executing the `migrate:import` operation, we encounter our first error:

```
[error]  Attempt to create a field storage field_video_recording with no type\
. (/var/www/html/web/core/modules/field/src/Entity/FieldStorageConfig.php:271\
)
[notice] Processed 28 items (27 created, 0 updated, 1 failed, 0 ignored) in 0\
.4 seconds (4578.2/min) - done with 'upgrade_d7_field'
In MigrateRunnerCommands.php line 414:
 upgrade_d7_field migration: 1 failed.
```

While it may sound obvious, it is very important to carefully read all messages logged to the console, especially when there are errors. Notice that the error states a problem with the `upgrade_d7_field` migration, but there is no reference to `upgrade_d7_field_instance`. If one of the migrations triggers an error, the `drush migrate:import` command exits immediately and any subsequent migrations are not executed.

For now, ignore the error and try to import field instances again by running `ddev drush migrate:import upgrade_d7_field_instance`. You will get a different set of errors:

```
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id speaker. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Attempted to create, modify or delete an instance of field with name\
 field_video_recording on entity type node when the field storage does not ex\
ist. (/var/www/html/web/core/modules/field/src/Entity/FieldConfig.php:316)
[error]  Missing bundle entity, entity type node_type, entity id sponsor. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id sponsor. (/v\
ar/www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id swag. (/var/\
www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id swag. (/var/\
www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[error]  Missing bundle entity, entity type node_type, entity id swag. (/var/\
```

```
www/html/web/core/lib/Drupal/Core/Entity/EntityType.php:916)
[notice] Processed 34 items (20 created, 0 updated, 14 failed, 0 ignored) in \
0.3 seconds (6051.3/min) - done with 'upgrade_d7_field_instance'
In MigrateRunnerCommands.php line 414:
 upgrade_d7_field_instance migration: 14 failed.
```

Let's analyze the output of executing both migrations. In the case of `upgrade_d7_field`, we get a very useful error message: `Attempt to create a field storage field_video_recording with no type.` This means that the field type of the `field_video_recording` is not supported in Drupal 10. Technically speaking, in Drupal 10 there is no field plugin with the same machine name as the field type name in Drupal 7 nor a field migrate plugin that maps the field type from the source site to one available in the target site. This migration also produced the following notice: `Processed 28 items (27 created, 0 updated, 1 failed, 0 ignored)`. 27 out of 28 fields created without errors is not bad at all. It is important to note that this field storage settings was created for all entities that had field attachments in Drupal 7. In our case, that includes content types and paragraph types.

As for the `upgrade_d7_field_instance` migration, the error messages are a bit more cryptic. The error `Missing bundle entity, entity type node_type, entity id speaker.` means that we are trying to migrate a field that belongs to an entity/bundle combination that does not exist in Drupal 10. In our example, this is the result of skipping some content types in the `upgrade_d7_node_type` migration. Any Drupal 7 field attached to a content type that was skipped in Drupal 10 will yield this error. The other error – `Attempted to create, modify or delete an instance of field with name field_video_recording on entity type node when the field storage does not exist.` – is more self explanatory. A field instance setting depends on the corresponding field storage setting to exist. In the `upgrade_d7_field`, it was not possible to migrate the storage settings for the `field_video_recording`; therefore, it is not possible to migrate the instance settings for a non-existent field. The field instance migration produced the following notice: `Processed 34 items (20 created, 0 updated, 14 failed, 0 ignored)`. A higher percentage of failed records, but that is expected given we skipped three content types will all their fields as part of our custom migration. All the fields attached to those content types will yield an error when trying to migrate them.

Making sense of error messages might be challenging. Reading online documentation, asking for support in community channels, following guides

like this series, debugging migrations, and a lot of practice will help you understand the different error messages that you will inevitably encounter in real life migration projects.

To get more context on a failed record, check the migration messages logged during the import operation. You can do that using the `drush migrate:messages` command or the migration messages admin interface. Executing the `ddev drush migrate:messages upgrade_d7_field` command we get... *nothing*. Even though there were records that failed to process, the migrate API did not log the reasons they failed. The only information available is the error message printed to the console. . To find the source of the error, we can search the codebase for the error message. In this case, the error is produced in the \Drupal\field\Entity\FieldStorage class. While this technical detail might not provide a lot of context, it is always helpful to identify **where** errors originate. This helps in understanding why it happens and makes it easier to find a solution.

Now we will check if the `upgrade_d7_field_instance` migration logged any errors. Feel free to execute the `ddev drush migrate:messages upgrade_d7_field_instance` command and see the results in the command line. Alternatively, go to `https://migration-drupal10.ddev.site/admin/reports/migration-messages` and click on the link indicating the message count for the `upgrade_d7_field_instance`.

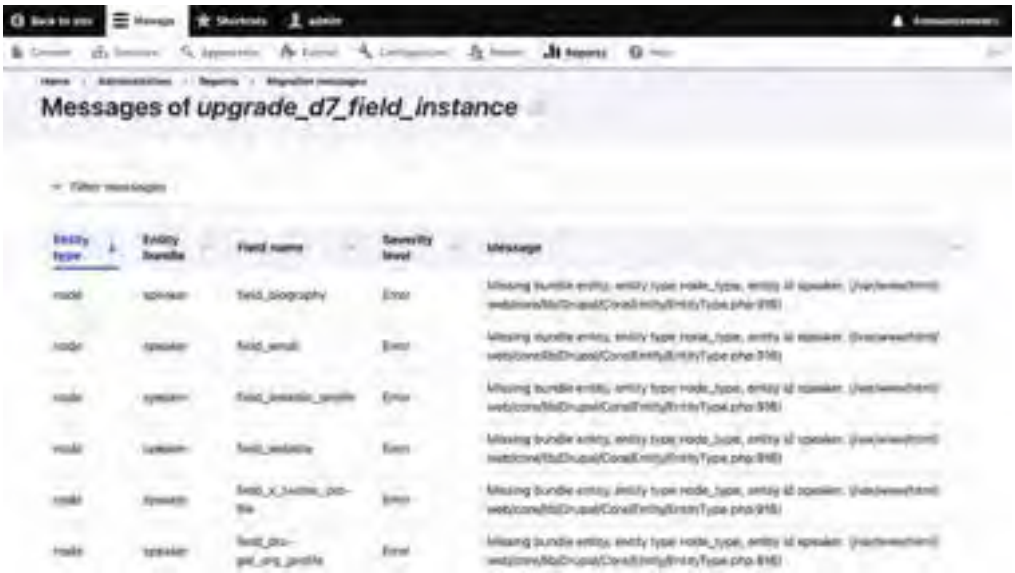
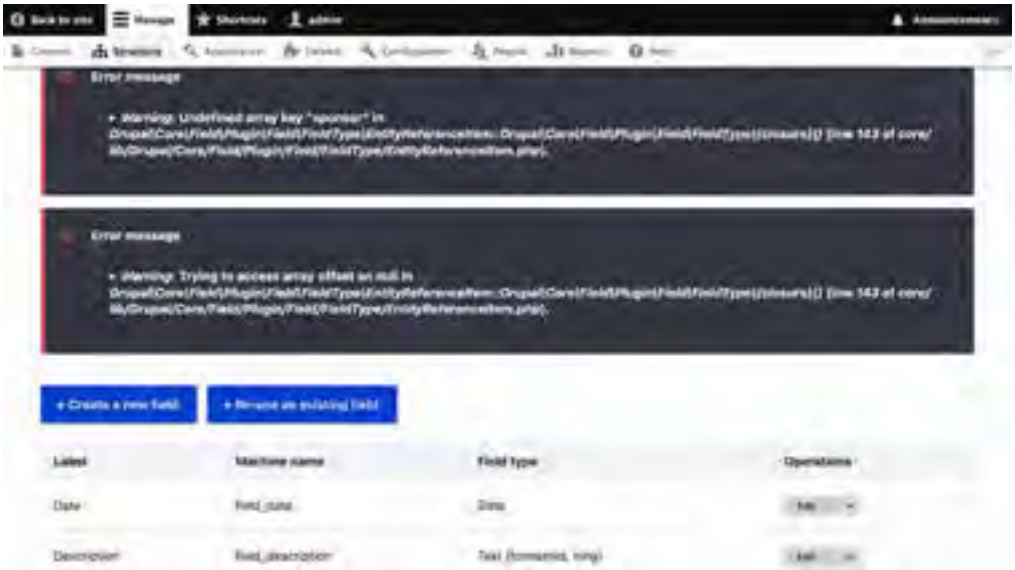


Figure 12. Message Count

The report provides more context as to why an error was thrown. Namely, you can see which entity type / bundle / field name combination triggered the error. Here, all the `Missing bundle entity, entity type node_type, entity id` errors are related to fields attached to content types we explicitly skipped: `speaker`, `session`, and `swag`. We will learn how to prevent these errors from happening, but for now they are to be expected.

Before moving forward, let's review the result of our migrations. The migrations reported that most field storage and instance settings were created. You can go to the content types and paragraph types administration pages and review the fields attached to them. For example, below is a screenshot of the fields in the `Event` content type.



The good news is that many fields are already added to the content type. The bad news is that we received a couple of cryptic error messages. Again, prior experience and some intuition will go a long way into understanding and fixing these errors. Pay close attention to the `Field type` configuration for `field_sponsors`. It says it is an entity reference field, allowing references to nodes (content), of type... *nothing*? If we click on the edit button, we see that there is no content type selected. When we review this field in Drupal 7, we notice that it allows referencing nodes of type `Sponsor`, but we skipped that content type altogether in Drupal 10. Sponsor nodes are going to be migrated as taxonomy vocabularies. Therefore, we can update this field configuration

to point to taxonomy terms instead of nodes. In the `Field Storage` section, change `Type of item to reference` to `Taxonomy term`. In the `Reference type` section select the `Sponsor` vocabulary. Save the field configuration and the errors we saw before will be gone.

If you want to practice, fix a similar error in the `Session` content type. These errors were the result of a mismatch between the content model in Drupal 7 compared to the content model in Drupal 10. After executing each migration, it is good practice to verify it is producing the expected results. In the case of the `upgrade_d7_field` and `upgrade_d7_field_instance` migrations, this means reviewing the field attached to all entity/bundle combinations.

Before continuing, let me share one of my favorite administration pages in all of Drupal: **the field list report**. Go to `https://migration-drupal10.ddev.site/admin/reports/fields` and you will be greeted by a lot of errors. At this point, some of these errors should start to feel familiar. Let's ignore the errors for now and have a look at the report itself.

You will see every field attached to every entity type. For each field, there is the field type and the module that provides it. The `Used in` column indicates all the bundles of an entity type where a field is used. The `Summary` column will include extra information about the field. For example, for entity reference fields, you will see which entity type the field allows to reference. The ideal scenario is that you do not see any blank spaces in the `Used in` column. If

there is a row without a value in `Used in` or if there are commas without text in between like `Session, Event, ,` this is a signal that something went wrong with the field instance migrations.

Rolling back migrations

Migrations are an iterative process. It is common to update a migration multiple times until you get the desired results. That process requires rolling back and importing the migration again. Normally, rolling back a migration does not affect content or configuration that was imported by a different migration. But that is not the case with field-related migrations. If we were to rollback the field instance migration, the operation also deletes some of the field storage settings.

Consider the following with regards to rolling back migrations:

- The migration destination plugin determines the action to take when a migration is rolled back. For example, when using the migrating nodes with revisions and translations, the migrate API would not delete a node if there are still revisions or translations in the system. This is done on a per row basis, not for the migration as a whole.
- When rolling back a migration, other Drupal APIs are invoked. In the case of content and configuration migrations, Drupal would call the entity delete operations, where many checks and actions are performed before and after an entity is deleted. For example, the field instance post-delete actions deletes the field storage configuration if the field is not used by other other bundles and it is not marked as persistent.
- Some configuration attached to an entity might prevent it from being deleted. For example, a field storage setting can be marked as persistent meaning it would not be deleted even if it is no longer used. An example of this is the node body field storage, which gets reused when a new content type is added via the user interface.

Getting back to field-related migrations, if you rollback the `upgrade_d7_field_instance` migration, we know that at least some field storage settings will also be removed. But, if you were to run `ddev drush migrate:status upgrade_d7_field` you would see that it reports that all fields had been migrated and that there are no records to process. This means that field storage

settings were deleted from the system, but the Migrate API is not aware of that. This happens because the migration system keeps its own mapping database tables to keep track of the status of each migration. In this case, rolling back the `upgrade_d7_field_instance` migration did not affect the mapping tables of the `upgrade_d7_field` migration. So, to work around this, we need to rollback both `upgrade_d7_field_instance` and `upgrade_d7_field`, and import them again. Otherwise, when trying to import field instance settings you will get errors related to missing field storage settings. Trying this out will be left as an exercise to the reader.

Skipping fields in Drupal migrations

The content model changes include dropping a whole content type, changing from one entity to another, and changing from one field type to another. This means that a one-to-one copy of the Drupal 7 field configuration is not possible. To make the transition easier, we are going to prevent some fields from being migrated. Then, we will manually create or update field settings as needed to match the new content model. The Migrate Skip Fields module can be used to prevent migrating fields that are not necessary. Make sure to include the module via Composer and enable it.

```
ddev composer require 'drupal/migrate_skip_fields:^1.0@alpha'
ddev drush pm:enable migrate_skip_fields
```

Once enabled, you can skip fields by entity, bundle, name, or type with the following settings:

```
// Drupal core version of source site. Either '6' or '7'.
$settings['migrate_skip_field_source_version'] = '7';

// Skip by entity type, bundle, and field name. An asterisk serves as a wildc\
ard to skip all values of one component.
$settings['migrate_skip_fields_by_name'] = [
  'entity_type:bundle:field_name',
  'entity_type:bundle:*',
  'entity_type:*:field_name',
  '*:*:field_name',
];

// Skip by field type. See hook_field_info. Alternatively, see the content_no\
```

```
de_field table in Drupal 6 or the field_config table in Drupal 7.
$settings['migrate_skip_fields_by_type'] = [
  'field_type_1',
  'field_type_2',
];
```

The `migrate_skip_fields_by_name` setting allows specifying multiple values each using three components (entity type, bundle, and name) separated by a colon (`:`) to indicate which fields should be skipped. You can use asterisk (*) as a wildcard for any of the components. That means that you want to skip fields for all values of that component. For example, `node:swag:*` means that you want to skip *all fields* attached to the `swag` *bundle* (content type) of the `node` *entity type*. For `migrate_skip_fields_by_type`, you need to use the machine name of the Drupal 7 field type to exclude. This can be found in the `hook_field_info` implementation of the module that provides the field. Alternatively, you can query the `field_config` table in the Drupal 7 database to find details on field machine names and their types. For those following the series, article 8 included a Drupal 7 site audit in which we populated a source site template introduced in article 3. You can use the `field_instance` tab of that document as a reference of what we need to do in our example project.

The above settings need to be added in `settings.php` or in a file included from it. In our example project, we use `settings.migrate.php` for migration-related settings. Add the following settings to that file:

```
// Migrate skip fields settings.
$settings['migrate_skip_fields_source_version'] = '7';

$settings['migrate_skip_fields_by_name'] = [
  'node:speaker:*', // Changed to user entity.
  'node:sponsor:*', // Changed to taxonomy term entity.
  'node:swag:*', // Content type no longer needed.
  '*:*:field_image', // Changed to media reference field.
];

$settings['migrate_skip_fields_by_type'] = [
  'youtube', // Changed to media reference field.
];
```

The above translates to skipping all fields attached to the `speaker`, `sponsor`, and `swag` content types; all fields whose machine name is `field_image` no matter the entity type or bundle; and all fields of type `youtube`. Later in the

series, we will use a combination of Drupal recipes and manual updates to bring back some of the fields that were skipped with the updated configuration for Drupal 10.

The Migrate Skip Fields module acts on the import operation of the field-related migrations. Therefore, we need to rollback and import them again.

```
ddev drush migrate:rollback upgrade_d7_field,upgrade_d7_field_instance
ddev drush cache:rebuild
ddev drush migrate:import upgrade_d7_field,upgrade_d7_field_instance
```

There should not be any failed records in the import operation report. Use the `drush migrate:messages` command or the migration messages admin interface to see helpful log data added by the Migrate Skip Fields module. Then, visit the field list report at `https://migration-drupal10.ddev.site/admin/reports/fields` Notice that the number of fields is significantly lower and that there are no error messages at the top of the page. A big win already, but we are not done yet.

Remember manual configuration updates we did to the `field_sponsors` field in the `Event` content type earlier? Because we rolled back the migration and the field was deleted, we need to do these manual updates again. As mentioned before, after each field migration, you should review the associated entities and bundles to verify the migrated configuration is free of errors and in line with the new content model.

Let's make the following updates:

- In the `Event` content type, edit the `field_sponsors` field. Change `Type of item to reference` to `Taxonomy Term` and for `Vocabulary` select `Sponsor`. Then save the field configuration.
- In the `Session` content type, edit the `field_speakers` field. Change `Type of item to reference` to `User` and for `Sort by` select `Name`. Then save the field configuration.

With these two manual updates complete, we have finished migrating field storage and instance settings. What a journey! In your own projects, be sure to export the new, fleshed out configuration and commit it to version control. In our next articles, we will walk through migrating field widgets, formatters, and groups.

18 - Migrating field widgets settings

In the previous article, we migrated field storage and instance settings. Along the way, we used the Migrate Skip Fields module to account for content model changes in the new site. Today, we continue with the next field-related migration: **widgets**. While doing so, we will find out that new migrations might uncover issues or misconfigurations with already executed migrations.

Before we begin

Familiarity with site building concepts related to fields is assumed. Refer to article 16 for a high level overview.

Drupal themes control the appearance of the website and how user interface elements are presented. In a standard Drupal 10 installation, Olivero is the default theme and Claro is the administration theme. Additionally, the administration theme is used when editing or creating content. Our example Drupal 10 project follows this setup. All of this can be configured from the `Apperance` page at `https://migration-drupal10.ddev.site/admin/appearance` Alternatively, you could run the following commands:

```
ddev drush theme:enable olivero
ddev drush --yes config:set system.theme default olivero
ddev drush theme:enable claro
ddev drush --yes config:set system.theme admin claro
ddev drush --yes config:set --input-format=yaml node.settings use_admin_theme\
 TRUE
```

Field widgets migrations

A widget determines the form elements used to capture field-related data. Each field type can be supported by one or more field widgets. For example, an entity reference field can use an HTML input with autocomplete functionality or a select list. Configuration for the field can affect which form elements are

used. Following the same example, an entity reference field can be configured to use the `Check boxes/radio buttons` widget. In this case, a cardinality of one renders radio buttons. Check boxes are presented when the cardinality is greater than one or when set to unlimited.

When multiple widgets are available for the same field type, Drupal takes care of standardizing how the data is stored in the database. No matter which field type is used for the entity reference field, they are saved in the database using the same data structure.

We use `upgrade_d7_field_instance_widget_settings` to migrate field widget settings. Copy it from the reference folder into our custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_instance_widget_set\
tings.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field_instance_widget_settings.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `cck_plugin_method`, and `migration_group`. Notice that for field migrations, we preserve the `field_plugin_method` key.
- Add two migration tags: `component_entity_form_display` and `tag1_configuration`.
- Add `key: migrate` under the source section.
- Update the required `migration_dependencies` to include `upgrade_d7_field`, `upgrade_d7_field_instance`, and the migrations that create configuration entities.

In addition to the updates above, remember that the Migrate Skip Fields module is installed on the site. Via hook_migration_plugins_alter, it modifies the field widget migration to prevent some fields being imported. In particular, those excluded by the `migrate_skip_fields_by_name` setting as explained in the previous article. For field widgets, the module does not act on the

`migrate_skip_fields_by_type` setting. The original migration already accounts for not importing widget data for a field whose storage setting was not migrated. This happens in the `field_type_exists` pipeline of the `process` section.

We are going to make a small addition here. The last process plugin in this pipeline is `skip_on_empty`. We are going to add a `message` configuration to log when a field is skipped because its storage setting is missing. The logs are added to the migration tables and can be viewed from the command line or the migration messages admin interface.

After the modifications, the `upgrade_d7_field_instance_widget_settings.yml` file should look like this:

```yaml
id: upgrade_d7_field_instance_widget_settings
class: Drupal\migrate_drupal\Plugin\migrate\FieldMigration
field_plugin_method: alterFieldWidgetMigration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - component_entity_form_display
 - tag1_configuration
label: 'Field instance widget configuration'
source:
 key: migrate
 plugin: d7_field_instance_per_form_display
 constants:
   form_mode: default
   third_party_settings: {  }
process:
 # @modified
 field_type_exists:
   -
     plugin: migration_lookup
     migration: upgrade_d7_field
     source:
       - field_name
       - entity_type
   -
     plugin: extract
     index:
       - 0
   -
     plugin: skip_on_empty
     method: row
     message: "Field storage configuration does not exist."
 bundle:
```

```
    -
      plugin: migration_lookup
      migration: upgrade_d7_field_instance
      source:
        - entity_type
        - bundle
        - field_name
    -
      plugin: extract
      index:
        - 1
  form_mode:
    -
      plugin: get
      source: constants/form_mode
  field_name:
    -
      plugin: get
      source: field_name
  entity_type:
    -
      plugin: get
      source: entity_type
    -
      plugin: static_map
      map:
        field_collection_item: paragraph
        paragraphs_item: paragraph
      bypass: true
  options/weight:
    -
      plugin: get
      source: widget/weight
  widget_type:
    -
      plugin: process_field
      source: type
      method: getFieldWidgetType
  options/type:
    type:
      plugin: static_map
      bypass: true
      source: '@widget_type'
      map:
        link_field: link_default
        email_textfield: email_default
        date_select: datetime_default
        date_text: datetime_default
        date_popup: datetime_default
        media_generic: file_generic
```

```
84          phone_textfield: telephone_default
85          options_onoff: boolean_checkbox
86          entityreference_autocomplete: entity_reference_autocomplete
87          entityreference_autocomplete_tags: entity_reference_autocomplete_tags
88          taxonomy_autocomplete: entity_reference_autocomplete
89          d7_text: d7_text_default
90          taxonomy_term_reference: taxonomy_term_reference_default
91          image: image_default
92          image_miw: image_image
93          url_external: link_default
94          field_collection_embed: entity_reference_paragraphs
95          field_collection: field_collection_default
96          addressfield_standard: address_default
97          entityreference: entityreference_default
98          file_mfw: file_generic
99          filefield_widget: file_generic
100         list: list_default
101         date: datetime_default
102         datetime: datetime_default
103         datestamp: datetime_timestamp
104         number_default: number_default_default
105   options/settings:
106     -
107       plugin: field_instance_widget_settings
108       source:
109         - widget/type
110         - widget/settings
111   options/third_party_settings:
112     -
113       plugin: get
114       source: constants/third_party_settings
115 destination:
116   plugin: component_entity_form_display
117 migration_dependencies:
118   required:
119     - upgrade_d7_field
120     - upgrade_d7_field_collection_type
121     - upgrade_d7_field_instance
122     - upgrade_d7_node_type
123     - upgrade_d7_taxonomy_vocabulary
124   optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_field_instance_widget_settings
ddev drush migrate:import upgrade_d7_field_instance_widget_settings
```

No errors after running the migration. Great! Feel free to have a look at the logs using the `drush migrate:messages` command.

Similar to the migrations of storage and instance settings, **we should review that the result of migrating field widget settings is correct.** There are two places you should look at. Using the `Article` content type as an example, go to:

1. The manage form display page where you can make adjustments to the migrated field widget configuration.
2. The add article page where you can see the current state of the field widget configuration.

This is where having familiarity with site building concepts and past experience will help. Below is a screenshot of the `Add content` page for the `Article content` type.

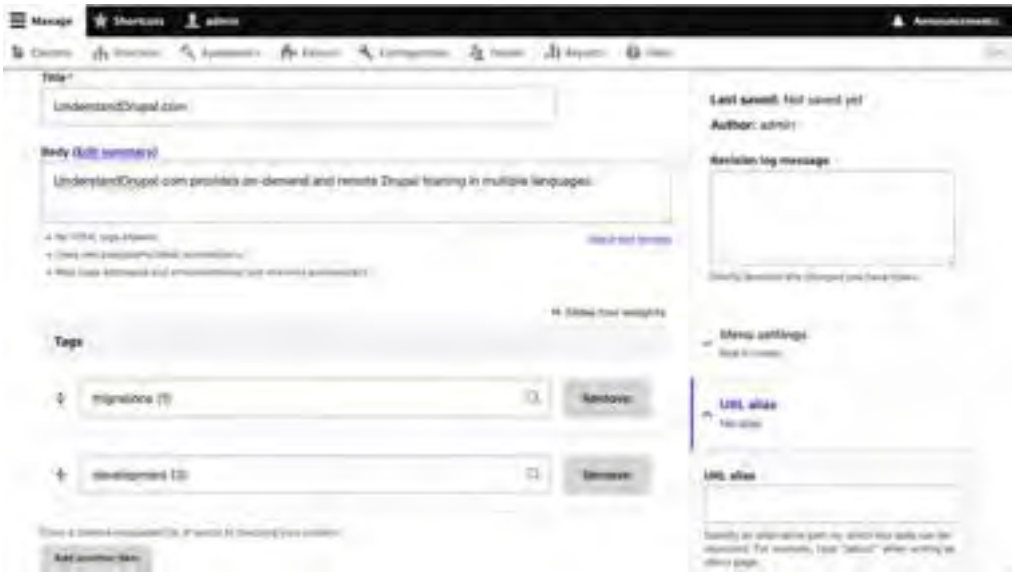


Figure 13. Add Content Page

Compare it with the equivalent page in Drupal 7: `https://migration-drupal7.ddev.site/node/add/article` With regards to fields, there are three things to note:

1. **The `Image` (`field_image`) field is missing.** This is to be expected because we will be using media reference fields instead. All uses of the `field_image` field were skipped while working on the storage and instance migrations. We are going to add media reference fields in a future article.
2. **The `Tags` (`field_tags`) field does not behave the same compared to the source site.** In Drupal 7, there is a single HTML input element that permits entering a comma-separated list of taxonomy terms. Additionally, if a term does not exist, it will be created on the fly in the `Article Tags` taxonomy vocabulary. In Drupal 10, the field is currently configured to use a separate HTML input element for each term we want to reference. Moreover, creating terms on the fly is not allowed.
3. **The `Body` (`body`) field does not offer the option to choose a text format.** At the moment only the `Plain text` format exists on the site. When there are no other formats available, rich text fields do not show the select list for choosing among the available options. We are going to cover migrating text formats in a future article.

Understanding how field widget settings are mapped

The entity reference system received a major overhaul in Drupal 8 and that has carried over ever since. For brevity, we are only going to highlight what is relevant to the example at hand.

In Drupal 7, there were at least two ways to reference taxonomy terms. One is using `Term reference` fields provided by the core Taxonomy module and the other is using `Entity Reference` fields provided by the contributed Entity Reference module. The former provides one widget called `Autocomplete term widget (tagging)`. The latter provides two widgets: `Autocomplete` and `Autocomplete (Tags style)`.

Drupal 10 follows the pattern of the contributed module by providing two widgets for referencing entities and even reuses the same labels. Below are the machine names and labels for all five widgets:

1. **`taxonomy_autocomplete`:** Drupal 7's core `taxonomy` module `Autocomplete term widget (tagging)` widget.
2. **`entityreference_autocomplete`:** Drupal 7's contributed `entityreference` module `Autocomplete` widget.

3. **`entityreference_autocomplete_tags`:** Drupal 7's contributed `entityreference` module `Autocomplete (Tags style)` widget.
4. **`entity_reference_autocomplete`:** Drupal 10 core `Autocomplete` widget.
5. **`entity_reference_autocomplete_tags`:** Drupal 10 core `Autocomplete (Tags style)` widget.

Take a look at the following snippet extracted from the `upgrade_d7_field_instance_widget_settings` migration:

```
process:
 options/type:
   type:
     plugin: static_map
     bypass: true
     source: '@widget_type'
     map:
       entityreference_autocomplete: entity_reference_autocomplete
       entityreference_autocomplete_tags: entity_reference_autocomplete_tags
       taxonomy_autocomplete: entity_reference_autocomplete
```

This is where the mapping of widgets settings from source to destination happens. Notice that `taxonomy_autocomplete` from Drupal 7 is mapped to `entity_reference_autocomplete` in Drupal 10. This is why the `Tags` (`field_tags`) field in Drupal 10 does not offer the *tags style* behavior. For that to work, we need to update its instance settings to leverage the `entity_reference_autocomplete_tags`.

As for creating terms on the fly, Drupal 7 `Term reference` fields could only include terms from a single vocabulary. In Drupal 10, entity reference fields allow including entities from multiple bundles of the same entity type. When multiple bundles can be referenced and creating entities that do not exist is allowed, you need to pick one bundle where the new entities will be added. In the case of the `Tags` (`field_tags`) field in Drupal 10, we need to update **field instance settings** to allow creating taxonomy terms on the fly.

That was quite a technical detour from reviewing the result of the field widget migration. In most cases, you do not need to know the technicalities of how the Migrate API maps configuration from Drupal 7 to Drupal 10. That being said, we have leveled up by gaining a deeper understanding of how the transformation process works.

Manual updates to field widget settings

In your own projects, take the time to review the migrated widget settings for all fields across all entity types. For our example project, below is the list of adjustments that need to be made manually:

- **`field_tags` field in `article` content type:** change field storage to create referenced entities if they do not already exist and update the widget to use the `Autocomplete (Tags style)` one.
- **`field_topics` field in `session` content type:** change field storage to create referenced entities if they do not already exist and update the widget to use the `Autocomplete (Tags style)` one.
- **`field_address` field in `venue` content type:** update the field instance settings to make the `United States` as the only available country and hide some address components. Namely, add an override to hide: `First name`, `Middle name`, `Last name`, `Organization`, `Address line 3`.

To replicate the behavior for the `field_tags` field, go to `Manage form display` page for the `Article` content type: `https://migration-drupal10.ddev.site/admin/structure/types/manage/article/form-display` and select the `Autocomplete (Tags style)` widgets for the `Tags` field.

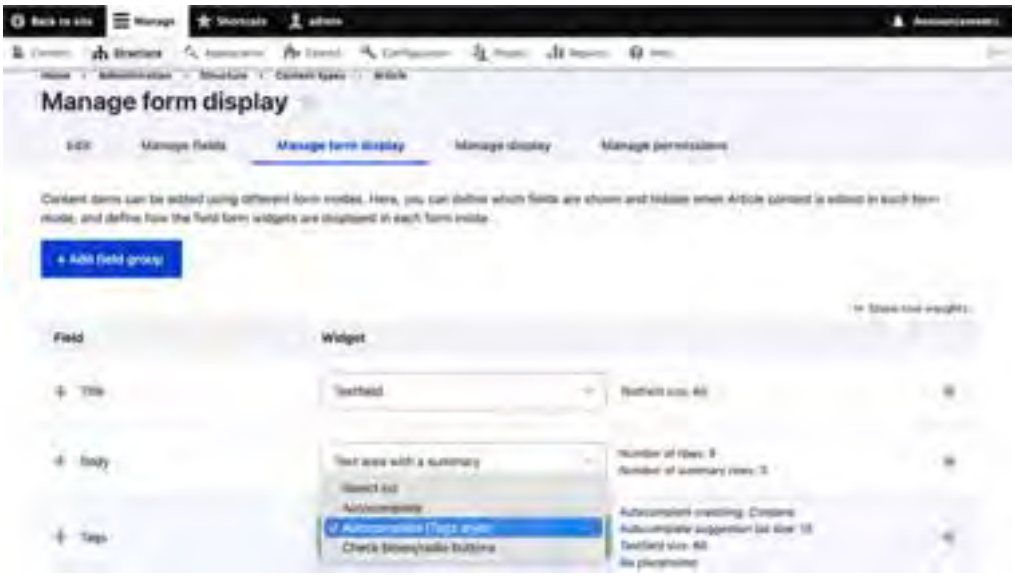


Figure 14. Article tags

To allow creating terms on the fly in `field_tags` field go to the `Manage fields` page for the `Article` content type, go to `https://migration-drupal10.ddev.site/admin/structure/types/manage/article/fields` Then, edit the `Tags` field and in the `Reference type` section check the box for `Create referenced entities if they don't already exist`.

If we were to allow referencing terms from multiple vocabularies, you will get a prompt labeled `Store new items in` to indicate which vocabulary should contain the newly created terms. The image below shows this interface for reference, but in our final example, the `Tags` field only allows referencing terms from one vocabulary: `Article Tags`.

Figure 15. Manage form display page

The changes to `field_topics` and `field_address` will be left as an exercise to the reader. Refer to the code repository for the final results of the migration of configuration.

While reviewing the field widget migration, we identified instance settings issues that required manual updates. This is common in the migration process, where adjustments to previous migrations are necessary when working on new ones. Remember that field migrations are complex, with storage, instance, widget, and formatter settings being closely interconnected.

We've covered significant ground in our comprehensive Drupal 7 to 10 migration series to date, but there is more to explore. Our next article will

focus on migrating view modes and field groups. We encourage you to apply these insights to your own migration projects and look forward to guiding you through the next steps.

19 - Migrating view modes and field groups

Today, we're building on our previous work with field widget settings. We will cover migrating **view modes**, a prerequisite for migrating field groups and field formatter settings. We'll then walk through migrating field groups. Field formatter settings will be addressed in our next article.

Before we begin

Familiarity with site building concepts is assumed. Namely, view modes and form modes. They are collectively known as display modes in Drupal 10 and you can learn more about them in this article on Drupal.org.

For today's discussions, it is worth noting that:

- View modes determine how content is **viewed**. Form modes determine how content is **edited**. In both cases, visibility can be controlled on a per field basis.
- View modes use **field formatters** to determine how individual fields are presented. Form modes use **field widgets** to determine how to collect data for each field.
- Each entity type can have a different set of display modes. Additionally, each bundle for each entity type can have a particular display mode enabled or disabled.
- The concept of view modes existed in Drupal 7, but form modes were introduced in Drupal 8. That is why we have a view modes migration but not a form modes migration.
- Field groups in Drupal 7 provided a way to group fields for viewing and editing purposes. That is why the field group migration can affect view mode and form mode configuration in Drupal 10.

Migrating view modes

Our example Drupal 7 project does not use any custom view mode other than what comes available out of the box in a standard installation or provided by enabled modules. Three view modes in total: `default` and `teaser` for the `node` entity and `default` for the `field_collection_item` entity. Equivalent view modes are available in Drupal 10.

We are still going to run the migration for two reasons:

1. To learn how to do it for projects with custom view modes.
2. Other migrations specify a dependency on this one. They rely on a `migration_lookup` against `upgrade_d7_view_modes` which would yield no results if this migration is not executed.

Let's get started. Copy the `upgrade_d7_view_modes` migration from the reference folder into our custom module and rebuild caches for it to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_view_modes.yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_view_modes.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a migrations folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `cck_plugin_method`, `field_plugin_method`, and `migration_group`.
- Add two migration tags: `entity_view_mode` and `tag1_configuration`.
- Add `key: migrate` under the source section.
- Update the required `migration_dependencies` to include the migrations that create entities.

After the modifications, the `upgrade_d7_view_modes.yml` file should look like this:

```yaml
id: upgrade_d7_view_modes
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - entity_view_mode
 - tag1_configuration
label: 'View modes'
source:
 key: migrate
 plugin: d7_view_mode
process:
 mode:
   -
     plugin: static_map
     source: view_mode
     bypass: true
     map:
       default: full
 label:
   -
     plugin: static_map
     source: view_mode
     bypass: true
     map:
       search_index: 'Search index'
       search_result: 'Search result'
       rss: RSS
       print: Print
       teaser: Teaser
       full: Full
       default: Full
 targetEntityType:
   -
     plugin: get
     source: entity_type
   -
     plugin: static_map
     map:
       field_collection_item: paragraph
       paragraphs_item: paragraph
     bypass: true
destination:
 plugin: 'entity:entity_view_mode'
migration_dependencies:
 required:
   - upgrade_d7_field_collection_type
   - upgrade_d7_node_type
   - upgrade_d7_taxonomy_vocabulary
 optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_view_modes
ddev drush migrate:import upgrade_d7_view_modes
```

View modes are configuration entities in Drupal 10. After running this migration, review the site's configuration. Even in cases like ours (without custom view modes), you might get configuration changes.

One such change is related to the labels of the view modes. This migration imports the `default` view mode from Drupal 7 as the `full` view mode in Drupal 10. In all cases, the label is set to `Full` no matter the entity type associated with the view mode. By default the `node` entity in Drupal 10 uses `Full content` as the label for the `full` view mode. After running this migration, its label will be changed to `Full`. You are free to keep or discard the label change as long as the migrated view modes remain in the system. If you are certain that a migrated view mode will not be used in the new site, you can update the migration to prevent it from being migrated or manually delete it after executing the migration.

In Drupal 7, field formatters settings, field groups, and views are some examples of places where view modes can be used.

Migrating field groups

Our source site includes only one field group and it contains two fields. One of those is a YouTube field that we skipped because it will be replaced by a media reference field in a future article. It is a rather contrived example, but we can use it to learn how to migrate field groups.

Field groups migrations are provided by the `field_group_migrate` submodule inside the main `field_group` project. The submodule needs to be enabled before generating the migrations using the Migrate Upgrade module. When that is the case, the `field_group_migrate` submodule uses a deriver to create one migration for each entity type and bundle combination that uses field groups in the source site.

For our example, copy the `upgrade_d7_field_group_node_session` migration from the reference folder into our custom module and rebuild caches for it to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_group_node_session.\
yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field_group_node_session.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a migrations folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `cck_plugin_method`, `field_plugin_method`, and `migration_group`.
- Add two migration tags: `d7_field_group` and `tag1_configuration`.
- Add `key: migrate` under the source section.
- Update the required `migration_dependencies` to include the migration responsible for migrating the entity type for the current field group migration. In this case, that would be `upgrade_d7_node_type`.

After the modifications, the `upgrade_d7_field_group_node_session.yml` file should look like this:

```yaml
id: upgrade_d7_field_group_node_session
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - d7_field_group
 - tag1_configuration
label: 'Field groups of node (bundle: session)'
source:
 key: migrate
 plugin: d7_field_group
 entity_type: node
 bundle: session
process:
 entity_type:
   -
     plugin: get
```

```
18        source: entity_type
19      -
20        plugin: static_map
21        map:
22          field_collection_item: paragraph
23          paragraphs_item: paragraph
24        bypass: true
25    bundle:
26      -
27        plugin: get
28        source: bundle
29    mode:
30      -
31        plugin: static_map
32        source: mode
33        bypass: true
34        map:
35          form: default
36    type:
37      -
38        plugin: static_map
39        source: mode
40        default_value: entity_view_display
41        map:
42          form: entity_form_display
43    group_name:
44      -
45        plugin: get
46        source: group_name
47    settings:
48      -
49        plugin: get
50        source: settings
51  destination:
52    plugin: d7_field_group
53  migration_dependencies:
54    required:
55      - upgrade_d7_node_type
56      - upgrade_d7_view_modes
57    optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_field_group_node_session
ddev drush migrate:import upgrade_d7_field_group_node_session
```

The imported migration group can be seen in the `Manage form display` tab for the `Session` content type: `https://migration-drupal10.ddev.site/admin/structure/types/manage/session/form-display`

As usual, make sure to review and confirm the result of the migration is what you expect. At first sight, things look good. The `Manage form display` tab shows the `Resources` group of type `Fieldset` containing the `Slides` field. But if you try to add a node of type `Session`, you will notice that the group does not behave like it does in the source site. In Drupal 7, the field group is collapsible. In Drupal 10, it is not.

Figure 16. Manage Form Display page

In many cases even when an automated migration exists, implementation details could have changed between Drupal 7 and Drupal 10. As documented in this issue, the option to make fieldsets collapsible was removed in Drupal 10. If you want to get similar functionality, you need to change the group type to `Details`. Other changes include merging the `Vertical tab` and `Horizontal tab` group types from Drupal 7 into a single `Tab` group in Drupal 10. Look at the project page of the Field Group module for a list of available groups and notes about their usage.

Because field group migrations are generated for each entity type and bundle combination, you have the flexibility to choose which ones to execute and which to discard. These migrations are particularly helpful when the groups contain a large number of fields. As demonstrated in our example, you might have to tweak the settings for the group itself but getting the fields inside the groups saves time.

Consider whether to recreate some group configurations manually or drop them entirely. Remember to always strike a balance between what to automate and what to redo manually.

In the next article, we will explore migrating field formatter settings, building on what we've learned so far.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

20 - Migrating field formatter settings

In this article, we are completing field-related migrations by importing formatter settings. This step builds on our previous work with view modes and field groups, bringing us closer to a functional Drupal 10 site. As we go through the process we will also identify other configuration elements used by field formatters that don't have an automatic migration.

Remember, if you have been following our series, you have already migrated view modes—a prerequisite for field formatters. The Migrate Skip Fields module will also be used to handle content model changes in the new site.

Before we begin

Familiarity with site building concepts related to fields is assumed. Refer to article 16 for a high level overview. Today's article also touches on view modes and date formats.

Field formatter migrations

A formatter determines how field data is rendered by Drupal. Each field type can be supported by one or more field formatters. For example, an entity reference field can print only the entity ID, its label as plain text or as a link, or the representation of the entity as specified by a view mode. Some field configuration might affect which formatters are available. For example, entity reference fields that point to taxonomy terms can use the RSS category formatter.

Technical note: The FormatterInterface interface includes a isApplicable method that determines if a formatter can be used for a field taking into account its configuration.

We use `upgrade_d7_field_formatter_settings` to migrate field formatter settings. Copy it from the reference folder into our custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_formatter_settings.\
yml \
  web/modules/custom/tag1_migration/tag1_migration_config/migrations/upgrade_\
d7_field_formatter_settings.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `cck_plugin_method`, and `migration_group`. Notice that for field migrations, we preserve the `field_plugin_method` key.
- Add two migration tags: `component_entity_display` and `tag1_configuration`.
- Add `key: migrate` under the source section.
- Update the required `migration_dependencies` to include `upgrade_d7_field`, `upgrade_d7_field_instance`, `upgrade_d7_view_modes`, and the migrations that create configuration entities.

In addition to the updates above, remember that the Migrate Skip Fields module is installed on the site. Via hook_migration_plugins_alter, it modifies the field formatter migration to prevent some fields being imported. In particular, those excluded by the `migrate_skip_fields_by_name` setting as explained in article 17.

For field formatters, the module does not act on the `migrate_skip_fields_by_type` setting. The original migration already accounts for not importing formatter data for a field whose storage setting was not migrated. This happens in the field_type_exists pipeline of the process section.

We are going to make a small addition here. The last process plugin in this pipeline is `skip_on_empty`. We will add a `message` configuration to log when a field is skipped because its storage setting is missing. The logs are added to the migration tables and can be viewed from the command line or the migration messages admin interface.

After the modifications, the `upgrade_d7_field_formatter_settings.yml` file should look like this:

```yaml
id: upgrade_d7_field_formatter_settings
class: Drupal\migrate_drupal\Plugin\migrate\FieldMigration
field_plugin_method: alterFieldFormatterMigration
migration_tags:
 - 'Drupal 7'
 - Configuration
 - component_entity_display
 - tag1_configuration
label: 'Field formatter configuration'
source:
 key: migrate
 plugin: d7_field_instance_per_view_mode
 constants:
   third_party_settings: {  }
process:
 # @modified
 field_type_exists:
   -
     plugin: migration_lookup
     migration: upgrade_d7_field
     source:
       - field_name
       - entity_type
   -
     plugin: extract
     index:
       - 0
   -
     plugin: skip_on_empty
     method: row
     message: "Field storage configuration does not exist."
 entity_type:
   -
     plugin: get
     source: entity_type
   -
     plugin: static_map
     map:
       field_collection_item: paragraph
       paragraphs_item: paragraph
     bypass: true
 bundle:
   -
     plugin: migration_lookup
     migration: upgrade_d7_field_instance
     source:
       - entity_type
       - bundle
       - field_name
   -
```

```
51        plugin: extract
52        index:
53          - 1
54    view_mode:
55      -
56        plugin: migration_lookup
57        migration: upgrade_d7_view_modes
58        source:
59          - entity_type
60          - view_mode
61      -
62        plugin: extract
63        index:
64          - 1
65      -
66        plugin: static_map
67        bypass: true
68        map:
69          full: default
70    field_name:
71      -
72        plugin: get
73        source: field_name
74    options/label:
75      -
76        plugin: get
77        source: formatter/label
78    options/weight:
79      -
80        plugin: get
81        source: formatter/weight
82    plugin_id:
83      -
84        plugin: process_field
85        source: type
86        method: getPluginId
87    formatter_type:
88      -
89        plugin: process_field
90        source: type
91        method: getFieldFormatterType
92    options/type:
93      -
94        plugin: static_map
95        bypass: true
96        source:
97          - '@plugin_id'
98          - '@formatter_type'
99        map:
100         taxonomy_term_reference:
```

```
101          taxonomy_term_reference_link: entity_reference_label
102          taxonomy_term_reference_plain: entity_reference_label
103          taxonomy_term_reference_rss_category: entity_reference_label
104          i18n_taxonomy_term_reference_link: entity_reference_label
105          i18n_taxonomy_term_reference_plain: entity_reference_label
106          entityreference_entity_view: entity_reference_entity_view
107        email:
108          email_formatter_default: email_mailto
109          email_formatter_contact: basic_string
110          email_formatter_plain: basic_string
111          email_formatter_spamspan: basic_string
112          email_default: email_mailto
113          email_contact: basic_string
114          email_plain: basic_string
115          email_spamspan: basic_string
116        field_url:
117          url_default: link
118          url_plain: link
119        field_collection:
120          field_collection_view: entity_reference_revisions_entity_view
121        addressfield:
122          addressfield_default: address_default
123        telephone:
124          text_plain: string
125          telephone_link: telephone_link
126        entityreference:
127          entityreference_label: entity_reference_label
128          entityreference_entity_id: entity_reference_entity_id
129          entityreference_entity_view: entity_reference_entity_view
130        file:
131          default: file_default
132          url_plain: file_url_plain
133          path_plain: file_url_plain
134          image_plain: image
135          image_nodelink: image
136          image_imagelink: image
137        datetime:
138          date_default: datetime_default
139          format_interval: datetime_time_ago
140          date_plain: datetime_plain
141    -
142      plugin: d7_field_type_defaults
143    -
144      plugin: skip_on_empty
145      method: row
146  hidden:
147    -
148      plugin: static_map
149      source: '@options/type'
150      map:
```

```
151         hidden: true
152       default_value: false
153   options/settings:
154     -
155       plugin: default_value
156       source: formatter/settings
157       default_value: {  }
158   options/third_party_settings:
159     -
160       plugin: get
161       source: constants/third_party_settings
162   options/settings/view_mode:
163     field_collection:
164       plugin: paragraphs_process_on_value
165       source_value: type
166       expected_value: field_collection
167       process:
168         plugin: get
169         source: formatter/settings/view_mode
170 destination:
171   plugin: component_entity_display
172 migration_dependencies:
173   required:
174     - upgrade_d7_field
175     - upgrade_d7_field_collection_type
176     - upgrade_d7_field_instance
177     - upgrade_d7_node_type
178     - upgrade_d7_taxonomy_vocabulary
179     - upgrade_d7_view_modes
180   optional: { }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
1 ddev drush cache:rebuild
2 ddev drush migrate:status upgrade_d7_field_formatter_settings
3 ddev drush migrate:import upgrade_d7_field_formatter_settings
```

No errors after running the migration. Great! Feel free to have a look at the logs using the `drush migrate:messages` command.

Similar to other field-related migrations, **we should confirm that the result of migrating field formatter settings is correct.** There are two places you should look at. Using the `Event` content type as an example, go to:

1. The manage display page where you can make adjustments to the migrated field formatter configuration.
2. The view page for a session node where you can see the current state of the field formatter configuration. At the moment, we have not migrated nodes nor other content entities. Feel free to create some content to see how it is rendered.

This is where having familiarity with site building concepts and past experience will help. Below is a screenshot of an `Event` node comparing how some of the fields attached to the content type are rendered.

Figure 17. Manage Display page

Notice that the `Date` field in Drupal 10 shows the time even though the field type is supposed to store a date value only. This is because the `Default long date` format used to render the field is configured to render a time component. In Drupal 7, the `Date` field is configured to use `Long` date format, which can also render a time component; however, it is hidden when the date field is configured to store a date value only. This highlights how formatters can behave differently between Drupal versions and the importance of carefully reviewing the migrated configuration.

For the example above, we could hide the time component for the `Date` field in multiple ways. One option is to create a new date format which does

not print the time. Another option is the `Custom` date formatter for the date field and use a PHP datetime pattern that does not print the time like `l, F j, Y`. The first option is recommended if you plan to reuse the same date format in multiple places. The second option can be used if you are certain that there is no need to reuse the pattern.

For the purpose of this example, we are going to go with a third option: use a different date format. Because we are using the Olivero theme, we can use the `Olivero Medium` date format which will hide the time and the day of the week. An acceptable compromise for the example.

Technical note: Drupal 10 does not provide an automated upgrade path for custom date types and formats from Drupal 7. They are stored in the `date_format_type` and `date_formats` tables respectively. You can write a custom migration to read from those Drupal 7 tables and create DateFormat configuration entities in Drupal 10. Or you can recreate them manually on the new site. In either case, you will have to identify where the custom date formats were used in Drupal 7 and manually update the corresponding configuration in Drupal 10. At the time of this writing, this issue contains a patch for migrating custom date formats, but no date types.

Below is a screenshot of an `Session` node comparing how some of the fields attached to the content type are rendered.

Figure 18. Session Node

Notice that for the `Topics` field, the label is rendered in bold in Drupal 10, but not in Drupal 7. As for the terms themselves, in Drupal 10 they appear stacked on top of each other and, in Drupal 7, they are listed next to one another. For the `Slides` field, Drupal 10 shows the file size in parenthesis and not the file icon. In Drupal 7, you see an icon that varies per file type and there

is no reference to the file size. These differences are a combination of changes to how formatters are implemented in different versions of Drupal and how themes decide to render some user interface elements.

For our example project, we are not going to take any action here. In real life projects, you are likely going to use a custom theme that will control how interface elements are rendered. Contributed modules like Fences can also manipulate the output of fields. **As a rule of thumb, before adding any module to your project, evaluate if the functionality provided outweighs the extra configuration and maintenance required to use it.**

Below is a screenshot of a `Venue` node rendered in Drupal 10.

Figure 19. Venue Node

The `Venue` content type has a `Phone` field that is rendered as plain text. This matches the configuration from Drupal 7, but that does not mean that we cannot make improvements as part of the migration. In this case, we can update the field to use the `Telephone link` formatter to render a link using the tel URL scheme. Users visiting the site with a mobile phone will be able to place the call by clicking on the link. Drupal is producing semantic HTML markup and it is up to the visitor's browser to determine what action to take when the link is clicked. Not obvious from the screenshot above, but the `Address` field shows the country in Drupal 10. The field formatter in Drupal 7 hides it when the field only allows a single country to be selected. No action to take as part of the example project.

While reviewing the field formatter migration, we identified that date formats are not migrated automatically. **Migrations can be complex, and not every Drupal 7 setting has an automated upgrade path.** Make sure to thoroughly review the migrated configuration and perform manual updates as needed. To wrap up the migration of fields, remember that storage, instance, widget, and formatter settings are closely interconnected.

Coming up in our step-by-step migration series, we will complete configuration migrations by walking through: media types and image styles, roles and permissions, and text and input formats. Once the remaining configuration is in place, we will be ready to work with content.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

21 - Applying Drupal recipes to add media types

We executed the last field-related migrations in the previous article, but we are not done with field configuration yet. Back in article 17, we used the Migrate Skip Fields module to prevent the automatic migration from importing image and YouTube fields. Today, we are going to use Drupal recipes to create media types and manually add media reference fields where needed.

Drupal recipes

The Recipes API,introduced in Drupal 10.3 allows installing modules and themes, importing configuration and content, and altering existing configuration. Recipes can also depend on other recipes, making them composable and highly flexible. Another benefit is the potential to replace installation profiles. This not only saves time but can bring consistency across different projects. In fact, Core's standard installation profile was replicated as a set of recipes. We are going to use a couple of those recipes to add media types to our Drupal 10 project.

Drupal recipes can validate that configuration elements are in a known state *before* the recipe is applied. Once a recipe is applied, the imported configuration is handled by the site itself. Any updates should follow standard configuration management practices.

Take a look at the `core/recipes` folder in Drupal 10's docroot to better understand the anatomy of a recipe. Each one is contained in a folder with a required `recipe.yml` file. They can optionally contain `config` and `content` folders to import configuration and content respectively. We could certainly write a whole series on Drupal recipes, but for now we are going to focus on what is applicable to our example project.

Technical note: Recipes are a first-class Composer project type: `drupal-recipe`. They can be required as any other Composer package.

Applying a Drupal recipe

Drupal Core provides a built-in script for applying recipes. Starting with Drush 13, a recipe command is available. When executed from a Drupal 10 docroot, any of these commands can be used to apply a recipe:

```
php core/scripts/drupal recipe /path/to/recipe_folder
drush recipe /path/to/recipe_folder
```

We are going to use the Drush approach since applying recipes from the host into the Docker container requires some extra arguments. For reference, refer to this documentation to apply recipes when using DDEV.

First, we will apply a recipe to use images as media entities. From the Drupal 10 folder in your host machine, run the following command:

```
ddev drush recipe core/recipes/image_media_type
```

If things go as expected, you will see a message indicating that the `Image media type applied successfully`. This should create an `image` media type, add an image field to it, and configure the `default` and `media_library` view and form modes. Visit https://migration-drupal10.ddev.site/admin/structure/media/manage/image/fields to see how things were set up.

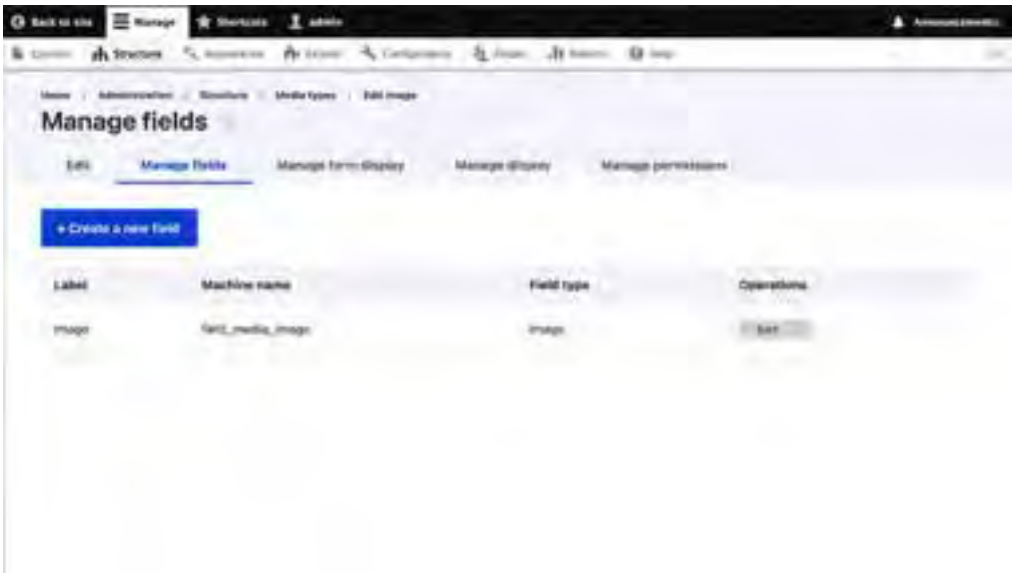


Figure 20. Manage Form Display page

These changes required adding new configuration to the site. Remember to export and commit the changes as with any other configuration change. If the recipe failed to apply, refer to the troubleshooting section below.

With one recipe applied, we will now do another that will allow YouTube videos to be embedded. From the Drupal 10 folder in your host machine, run the following command:

```
ddev drush recipe core/recipes/remote_video_media_type
```

When applied, the recipe will create a `remote_video` media type, add a plain text field to it, and configure the `default` and `media_library` view and form modes. The text field is used to store a link to the remote video URL. YouTube and Vimeo are supported out of the box. Visit https://migration-drupal10.ddev.site/admin/structure/media/manage/remote_video/fields to see how things were set up. As you would expect, the site configuration has been updated. Export the changes and commit them to the repository.

With media types already created, we are going to manually add media reference fields where appropriate. Per the site audit document referenced in article 3, these are the fields to add:

1. In the `Article` content type, add a media reference field to images.
2. In the `Venue` content type, add a media reference field to images.
3. In the `Session` content type, add a media reference field to remote videos. Add this field to the `Resources` group.

This is a site building exercise readers should be able to do on their own. Refer to the code repository for the final configuration.

Troubleshooting recipe application

Recipes verify that included configuration does not exist or that existing configuration matches the recipe definition *exactly*. This check is also performed for dependent recipes.

In the case of existing configuration, the check is based on machine names. Any variation will prevent the recipe from applying. For example, the `image_media_type` recipe depends on the `large` image style. If we update its label from `Large (480×480)` to `Large 480×480 px`, the following error will be triggered when trying to apply the recipe:

```
In ConfigConfigurator.php line 47:

 The configuration 'image.style.large' exists already and does not match the \
recipe's configuration
```

Our label change might not be meaningful, but what if we had made changes to the image effects used in that image style? We certainly would not want to lose those customizations! As a general rule, recipes would not overwrite any configuration element if its current value differs from the recipe definition. Instead, a validation error is yielded. This might seem limiting. What is the point of using recipes then?

Remember that recipes are meant to be starting points. You can install a site from recipes. When there is no existing configuration, no conflicts can arise. That being said, **applying recipes on existing sites is a valid and useful feature.** In addition to those provided by Core, there are contributed recipes that you can use on your projects today.

Note: As of Drupal 10.4, recipes can opt out of strict comparisons with existing config. At the time of publishing, a strict comparison is still the default. Changing to making the configuration comparison lenient by default is being discussed in this issue.

Should we use recipes in migration projects?

It depends on your project requirements and the desired content model.

In our case, our Drupal 7 project did not use the Media module, but we still wanted to use media entities in Drupal 10. Recipes are available for creating media types modeled after the `standard` installation profile. No need to reinvent the wheel here!

Even so, put on your site builder hat as we take a deeper look at the implications of using these recipes.

Our example Drupal 7 project is arguably very simple. It is based on the `standard` installation profile and there are not many customizations. No custom image styles or text formats, nor changes to those provided out of the box. Only one custom role (`Editor`) and no WYSIWYG configuration. We could definitely use many Core recipes modeled after the `standard` installation profile, but should we?

For the sake of the example, consider image styles. Drupal Core provides a migration for image styles from Drupal 7. If we were to use it, the migration will overwrite any image style to match its Drupal 7 definition. In the case of Core image styles, Drupal 10 uses a WebP converter. That feature did not exist before. Since we are not customizing the Core image styles, and we do not have any custom one, for our example project *it is better not to migrate image styles.* That way, **we take advantage of the optimizations that come with WebP in Drupal 10.**

If you have custom image styles, use the `upgrade_d7_image_styles` migration from the `ref_migrations` folder. In this case, the image style migration needs to happen *after* applying the `image_media_type` recipe. Otherwise, updates to existing image styles will make the configuration validation fail and the recipe would not be applied. While this might seem like a chicken and egg problem, good planning goes a long way into determining the correct order to apply recipes, migrate configuration, and do manual configuration changes.

Technical note: Most source plugins for upgrading from Drupal 7 connect directly to the database to fetch data. Drupal 7 allowed some configuration to be defined in code, either by modules or features. When that configuration is in its default state, only the code representation exists. The database would not hold a reference to it. That is the case for the image styles in our example. Checking the status of the `upgrade_d7_image_styles` reveals there are no records to migrate, but we know that image styles do exist in Drupal 7. Those present only exist in code. For them to exist in the database, you would need to overwrite them. This also happens with views, field groups, and other configurations that can be represented in code. Take this into account if a migration reports no records to migrate or the total count is lower than expected.

To answer the question posed at the start of this section, it is ok to use recipes when they can be cleanly applied. If subsequent migrations might overwrite configuration elements created by the recipes, make sure the new configuration is truly what you want. If we export and commit configuration changes after every step, git should show us if anything was overridden. Keep in eye on functionality that did not exist in Drupal 7 and could be lost. That should not stop you from using the migration. If anything, manually apply the configuration that was overwritten by the migration.

Today, we've covered the basics of the Recipes API, how to apply recipes, and used them for creating media types. In our upcoming articles, we'll walk

through migrating user roles, permissions, and text and input formats to your new Drupal 10 site.

22 - Configuring text formats, editors and user roles

In the previous article, we learned to apply Drupal recipes to add configuration to our Drupal 10 site. Today, we are going to continue this process to bring in more configuration related to text formats and editors, user roles, and user fields. We will explain the reasons why recipes are used instead of migrations and provide guidelines for running an automated migration in your own projects if needed.

Applying text format and editor recipes

Our example Drupal 7 site was installed using the `standard` installation profile and made no customizations to text formats. Moreover, it did not use contributed modules to provide a WYSIWYG editor. We could use the `upgrade_d7_filter_format` to migrate the text formats and manually add editor configuration afterwards. Instead, we will apply recipes that come with Drupal core to add text formats already configured with an editor.

From the Drupal 10 folder in your host machine, run the following commands:

```
ddev drush recipe core/recipes/basic_html_format_editor
ddev drush recipe core/recipes/restricted_html_format
ddev drush recipe core/recipes/full_html_format_editor
```

If things go as expected, you will see messages indicating that the recipes were applied successfully. Otherwise, refer to the previous article for information on troubleshooting recipes application.

The `basic_html_format_editor` recipe creates the `basic_html` text format, a corresponding editor configuration, and grants the `authenticated` user role access to the new text format. The `restricted_html_format` recipe creates the `restricted_html` text format, no editor configuration,

and grants the `anonymous` user role access to the new text format. The `full_html_format_editor` recipe creates the `full_html` text format, a corresponding editor configuration, and no changes to user roles or permissions. In addition to these three text formats that were created, the `plain_text` text format already existed. You can see the results at `https://migration-drupal10.ddev.site/admin/config/content/formats`

Figure 21. Manage Form Display page

Note: The image above includes additional roles that are created in the next section.

In Drupal 7, the available text formats are: `plain_text`, `filtered_html`, and `full_html`. As part of the content migrations, `filtered_html` will be mapped to `basic_html`. The other two will be migrated to their counterparts in Drupal 10.

If you had custom text formats, the `upgrade_d7_filter_format` would take care of migrating them. Their text filter configuration will also be migrated provided equivalent Drupal 10 plugins exist for those used in Drupal 7. The `upgrade_d7_filter_format` migration uses the filter_id process plugin to allow mapping text filters between both versions of Drupal. In our case, we decided to go with the new defaults in Drupal 10 and map text format values as needed.

Applying user role recipes

The `standard` installation profile in Drupal 7 gives us three roles: `anonymous user`, `authenticated user`, and `administrator`. In addition to these, we added a fourth one: `Editor`.

In Drupal 10, we get four roles out of the box: `anonymous`, `authenticated`, `administrator`, and `content_editor`. The first two are present in our existing site. We are going to apply recipes to add the last two. Later on, we will do the corresponding mapping of roles from Drupal 7 to 10.

Technical note: The roles that are initially available depend on how the site was originally installed. Our example project uses a custom installation profile named `tag1_profile`, modeled after the minimal installation profile. The profile depends on the `node` module, which itself depends on the `user` module. When enabled, the `user` module provides the anonymous and authenticated roles via the module's required configuration. If a site is installed using the umami installation profile, the administrator role is also created via the profile's required configuration. The standard installation profile, on the other hand, creates both the administrator and content_editor roles via the profile's required configuration. Note that both umami and standard include definitions for the `anonymous` and `authenticated` roles to assign permissions they have access to.

From the Drupal 10 folder in your host machine, run the following commands:

```
ddev drush recipe core/recipes/administrator_role
ddev drush recipe core/recipes/content_editor_role
```

If the recipes apply cleanly, we should see two new roles at `https://migration-drupal10.ddev.site/admin/people/roles`
The first one is the `Content editor` role with permission to use the administration pages and view own unpublished content. The second one is the `Administrator` role with all permissions granted. The latter automatically receives all current and future permissins as determined by the `Role settings` page at `https://migration-drupal10.ddev.site/admin/people/role-settings`

Figure 22. Manage Form Display page

At this point you might be wondering, why are we using recipes instead of a migration to import roles and permissions? Because our example is very simple and it adds little value to run a migration. Remember that the automatic migration is trying to provide (to the extent possible) a verbatim copy of the configuration and content that existed in Drupal 7. That is not desired in most real life projects. You might want to drop outdated functionality, take advantage of new Drupal 10 features, or change the site's architecture.

The fact the migrations are generated automatically does not mean that you need to execute them all. As we have seen so far, and will continue to do, **we will cherry pick and customize the migrations that are relevant to accomplish the goals set in our migration plan.**

If you want to run the role migration nonetheless, or have a project with a lot of custom roles, run the `upgrade_d7_user_role` from the `ref_migrations` folder. **Keep in mind that running this migration will replace all permissions already assigned to Drupal 10 roles with only those present in Drupal 7.** This is generally not an issue because configuration migrations happen early on. If you are using a custom approach, be mindful that the user role migration should be executed before assigning permissions manually.

The `upgrade_d7_user_role` migration does two things:

1. **Creates Drupal 10 user role configuration entities based on the Drupal**

7 role names. It is worth noting that in Drupal 7, roles were identified by a numeric role id. In Drupal 10, roles are identified by a machine name string. The migration will create the machine name based on the name value from Drupal 7.

2. **Try to migrate permissions.** In both versions of Drupal, permissions are identified by strings, but their value can vary greatly for the "same permission". For example, the `edit terms in 1` permission in Drupal 7 would be the equivalent of the `edit terms in tags` permission in Drupal 10. For context, vocabularies in Drupal 7 were identified by numeric ids while machine name strings are used in Drupal 10. In order to account for changes in permission names, the `upgrade_d7_user_role` provides a static map of old permissions to new ones. We could manually update this map if necessary. Depending on how many roles and permissions are in place, it might be easier and faster to manually update the permissions in Drupal 10 after all roles are created in the new site. At the end of this process, export the new configuration and commit the changes.

Technical note: Drupal 10 migrations that create user role entities will check for invalid permissions which would be removed before being assigned to a role. In this context, an invalid role is one whose Drupal 7 name, after the mapping mentioned above, does not match an existing Drupal 10 role. The migration will log a message for each role that contains invalid permissions. Refer to this change record for more information.

Below is the output of executing `ddev drush migrate:messages --format=yaml upgrade_d7_user_role` after running the user role migration in our example project:

```
-
 level: '2'
 source_ids: '3'
 destination_ids: administrator
 message: "Permission(s) 'access all views', 'access dashboard', 'administer \
field collections', 'administer fieldgroups', 'administer fields', 'administe\
r youtube', 'block IP addresses', 'delete revisions', 'delete terms in 1', 'e\
dit field collections', 'edit terms in 1', 'revert revisions', 'use ctools im\
port', 'view revisions' not found."
-
 level: '2'
 source_ids: '4'
 destination_ids: editor
 message: "Permission(s) 'create speaker content', 'create sponsor content', \
```

```
15 'delete any speaker content', 'delete any sponsor content', 'delete own speak\
16 er content', 'delete own sponsor content', 'delete revisions', 'delete terms \
17 in 1', 'delete terms in 2', 'edit any speaker content', 'edit any sponsor con\
18 tent', 'edit own speaker content', 'edit own sponsor content', 'edit terms in\
19  1', 'edit terms in 2', 'revert revisions', 'view revisions' not found."
```

While on the topic of roles and permissions, let's do the following configuration changes manually:

- Add a `Speaker` role. This role will be assigned to users that are created from `Speaker` nodes in Drupal 7.
- Update the `Speakers` field in the `Session` content type to only allow referencing users with the `Speaker` role we just created.
- Update the `Basic HTML` text format so that it can be used by the `Content editor` and `Speaker` roles.

Other permissions will be assigned to different roles to mimic the configuration from Drupal 7. This is a site building exercise readers can do on their own. Refer to the example repository for the final set of permissions.

Adding fields to user accounts

Back in article 8, we determined that Drupal 7 nodes of type `Speaker` will be migrated as user entities in Drupal 10. To accomplish this content model change, we need to add fields so that user accounts can store data previously present in nodes. The interface to add fields to the user entity is available at `https://migration-drupal10.ddev.site/admin/config/people/accounts/fields` The fields to add are:

- A `User picture` field of type `File upload > Image`.
- A `Biography` field of type `Plain text > Text (plain, long)`.
- A `Favorite quote` field of type `Paragraphs`. This will store a reference to paragraphs of type `Favorite quote`.
- A `Social media links` field of type `Social Links`. This field type is provided by the Social Link Field module which is already available on the site.

- A `Website` field of type `Link`. Only external links should be allowed and the link text should be disabled.

For the `User picture` field apply the `user_picture` recipe: `ddev drush recipe core/recipes/user_picture`. All other fields should be created manually. None of them is required. All except for the `Social media links` field should have a cardinality of one, that is, the allowed number of values should be limited to one. In the case of the `Social media links` field, set the cardinality to unlimited and allow the following platforms: `Drupal User Profile`, `LinkedIn`, and `X (Twitter)`.

Refer to the example repository for the final field configuration including widget and formatter settings. Below is an image of the fields that were added to the user entity:

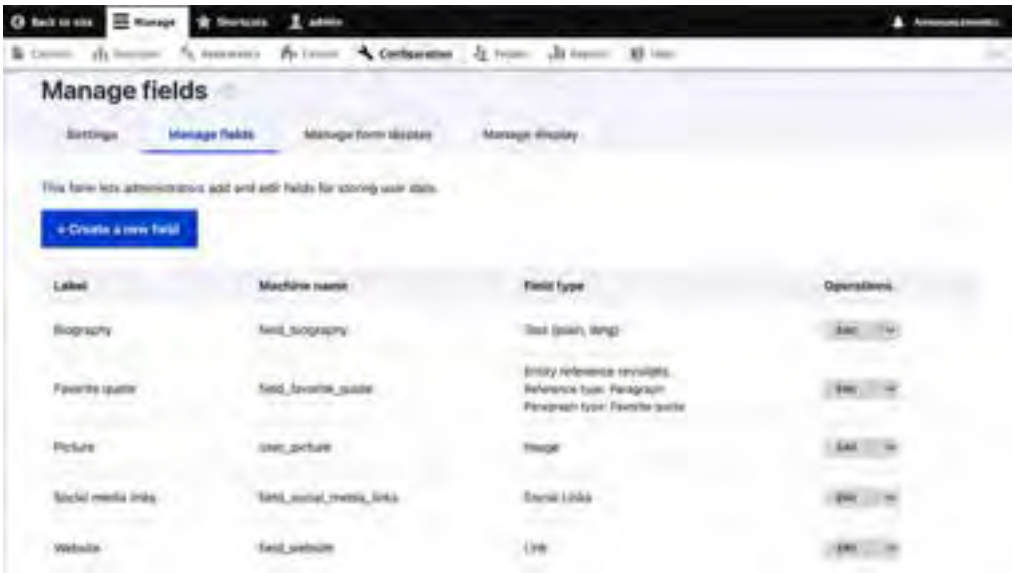


Figure 23. Manage Form Display page

We have seen that there are multiple ways to bring configuration into a new Drupal 10 site:

- running an automatic migration
- applying recipes
- manually creating configuration from the administration interface

Remember to always strike a balance among the different options. As an anecdote, I worked on a Drupal 7 migration for a media outlet that contained over 150 image styles. In this case, it made sense to automate their migration. I created a custom process plugin to convert the image effects configuration provided by the ImageCache Actions Drupal 7 module into image effect plugins provided by the Image Effects Drupal 10 module. I used techniques similar to the ones described in this video presentation to do the mapping between the both versions of Drupal.

In the `ref_migrations` folder, you will find more migrations related to configuration. They will have the `Configuration` migration tag. Feel free to have a look at them and continue exploring on your own. For the purpose of our example, the current state of the configuration suffices to accomplish our goals:

1. understand the process of migrating configuration
2. have an updated content model we can migrate into

In the next section, we will explain how to adapt content migrations to match the site's structural changes.

Migration of Content

- 23 - Altering AUTO_INCREMENT table values to prevent entity ID conflicts
- 24 - Public and private file migrations
- 25 - User and taxonomy term migrations
- 26 - Entity type conversions. From nodes to user and taxonomy terms
- 27 - Paragraph migration. Creating custom process plugins
- 28 - Migrating media. Creating custom source plugins
- 29 - Migrating nodes - Part 1
- 30 - Node migrations - Part 2
- 31 - Debugging tips, performance considerations, Drupal CMS, AI-assisted migrations and more!

23 - Altering AUTO_INCREMENT table values to prevent entity ID conflicts

In the last article, we wrapped up migrating configuration to match the content model we specified in our upgrade plan. Ready to start migrating content? Hang in there – it's coming up next. But first, we are addressing one of the hardest issues to resolve when they show up: *entity ID conflicts in content migrations*.

We introduced this topic in article 4 and expanded on it in article 5. Today, we'll detail one way to prevent these problems. We'll first tackle the issue using a custom approach, then show how the AUTO_INCREMENT Alter module can help.

Revisiting entity ID conflicts

Imagine you are close to wrapping up a large migration project. Editors have been testing the new content authoring experience, but they have not had the opportunity to try the full editorial workflow. A Drupal 10 staging environment is provisioned for them to do so. Having content and users in the staging environment would speed up the testing process so a full migration is executed. In addition to checking migrated content, editors also create new test content: nodes, users, files, etc. All this time, the Drupal 7 site continues to be available and content is created there as well.

Two weeks later, an incremental migration is run to test other features and that's when things start to break. **Drupal assigns entity IDs automatically when new content is created.** The Drupal 10 staging environment and the Drupal 7 production environment ended up with conflicting entity IDs. The editorial team is informed that to overcome problems with the incremental migration, the site needs to be re-installed and a full migration executed *again*. After this, all the test content that editors created in the staging environment is lost.

Editors are not happy, but they take the loss with grace.

A month later, and after a lot of testing and updates, the upgraded Drupal 10 site is almost ready to launch. During this time, editors go above and beyond. They not only finish testing the editorial workflow but also create complex landing pages to get familiar with the new components-based page building system. To avoid having to recreate this content again after the new site is launched, editors ask the developer team if it is possible to somehow preserve the landing pages created in the staging environment. Developers give it a shot, but it ends up being too hard to reconcile the staging database with the freshly migrated database that will be used to launch the new site. Editors are informed that they will have to recreate the landing pages manually after the Drupal 10 site launches.

This time the pill is harder to swallow.

This fictitious case exposes two critical issues:

1. **No precautions were taken** to account for entity ID conflicts *before* running content migrations.
2. **Miscommunications** between the development and editorial teams led to losses in time, effort, and trust.

Both issues affect the project's budget and create friction among the team. We discussed the technical side of the entity ID conflicts previously in article 5. This time, we are highlighting non-technical challenges. **Careful planning, or lack thereof, have big consequences in migration projects**. So, take advantage of guides like ours and plan your Drupal 7 migration diligently. *Measure twice, cut once.*

Preventing entity ID conflicts

From a technical standpoint, there are two primary ways to prevent entity ID conflicts:

1. **Do not reuse the entity IDs from Drupal 7 in Drupal 10.** This requires extra customizations of generated migrations because the migrate API assumes entity IDs are kept the same.
2. **Artificially inflate the auto-increment value for all content entities in Drupal 10.** Arguably easier to do, but you need to pay very close attention to how entities are created in the new site.

Our recommendation? No matter if you decide to reuse entity IDs from Drupal 7 or not, **always manipulate the auto-increment value of content entities in Drupal 10.** Doing this will allow for incremental migrations to run without conflicting with manually created content before the Drupal 10 site is launched.

Most real life migration projects include changes to the content model. In that case, it is not possible to preserve the IDs for some entities from Drupal 7. So, altering the auto-increment value of content entities will prevent issues between incremental migration runs.

Content entity IDs auto-increment values

Before explaining how to alter auto-increment values, let's talk about entity IDs. As explained in article 7, Drupal entities have unique identifiers. Configuration entities usually leverage machine names stored as string values. Content entities favor numeric identifiers stored as integer values. It is possible to have other formats (like UUIDs) or data types (like floats) to uniquely identify entities as long as their values are unique. That being said, strings for configuration entities and integers for content entities is the common practice.

Technical note: Both content and configuration entities use `entity_keys` to determine the name of their identifiers. Take the Node content entity and NodeType configuration entities as examples. For the `Node` content entity, the ContentEntityBase::baseFieldDefinitions method reads the `id` entity key and creates an unsigned integer identifier name nid. For the `NodeType` configuration entity the node.schema.yml schema reads the `id` entity key and validates an identifier named type exists. In this case, `type` is a machine name, an extension of strings with additional constraints.

Integer identifiers, as those used by content entities, have the following behavior:

- Start with 1, unless otherwise instructed. This is what we will alter in a bit.
- Increment by 1 each time a new entity is created. It is possible to change this to a different value.
- If an entity is deleted, its identifier cannot be reused. It is disposed of altogether.

Consider nodes as an example. The first node created on the site will have a **node id (nid)** value of 1, the second of 2, third of 3, and so on. If the node with `nid` 2 is deleted and a new one is created, the `nid` of the new node will not be 2 but the value of the highest node id at the time plus one.

It is important to note that **each content entity type has a separate counter for their identifiers**. That is, it is fine to have a node with `nid` 1, a user with `uid` 1, a taxonomy term with `tid` 1, etc.

Also, **bundles within the same entity type share the same counter for identifiers**. For example, it is not possible to have a node of type article with `nid` 1 and at the same time a node of type of basic page with `nid` 1. No matter the content type, all nodes follow the same counter.

Altering auto-increment values for content entity tables

To prevent entity ID conflicts, we need to alter the auto-increment value of tables holding content entity data. This is a three step process:

1. Identify which Drupal 10 tables to alter.
2. Find out the new auto-increment value for each table.
3. Connect to the database to perform the `ALTER` operation on each table.

With regards to the first step, **the tables to alter are those defined in the `base_table` and, if used, the `revision_table` keys of the annotation that defines the configuration entity**.** The code below is part of the ContentEntityType annotation for the `node` entity:

```
/**
* Defines the node entity class.
*
* @ContentEntityType(
*   id = "node",
*   label = @Translation("Content"),
*   base_table = "node",
*   data_table = "node_field_data",
*   revision_table = "node_revision",
*   revision_data_table = "node_field_revision",
*   entity_keys = {
*     "id" = "nid",
*     "revision" = "vid",
```

```
*     "bundle" = "type",
*     "label" = "title",
*     "langcode" = "langcode",
*     "uuid" = "uuid",
*     "status" = "status",
*     "published" = "status",
*     "uid" = "uid",
*     "owner" = "uid",
*   }
* )
*/
class Node extends EditorialContentEntityBase implements NodeInterface {
   ...
}
```

Note: There are ongoing efforts to convert doctrine annotations to native PHP attributes. A lot of progress was made in the 10.3.x release cycle. At the time of writing, an issue to convert ContentEntityType and ConfigEntityType from annotations to PHP attributes is very close to landing in Drupal core. It is very likely that this change will be introduced in Drupal 11.1.0.

For the `node` entity, `node` and `node_revision` are the tables to alter as specified by the `base_table` and `revision_table` annotation keys respectively. The `user` and `file` entities do not have `revision_table` key in their corresponding classes: User and File respectively. For these two entities, the data tables to alter are `users` and `file_managed`.

It would be impractical to list the base and revision tables for all possible content entities. That varies per project and depends on the set of enabled modules. Instead, open an interactive PHP shell with Drush using `ddev drush php:cli` in the Drupal 10 folder. Then, run the following code:

```
// Get all entities Drupal 10 entities in the current installation.
$entities = \Drupal::entityTypeManager()->getDefinitions();

// Filter entity list to only include content entities.
$content_entities = array_filter($entities, function ($entity_definition) {
  return $entity_definition->getGroup() === 'content';
});

// Find the class that defines each content entity.
$entity_classes = array_map(function ($entity_definition) {
  return $entity_definition->getClass();
}, $content_entities);

```

```
// Find the base tables of content entities.
$base_tables = array_map(function ($entity_definition) {
  $table = $entity_definition->getBaseTable();
  $label = $entity_definition->getLabel();
  return $table
    ? "The $label content entity uses '$table' as its data table."
    : "The $label content entity does NOT use a base table.";
}, $content_entities);

// Find the revision tables of content entities.
$revision_tables = array_map(function ($entity_definition) {
  $table = $entity_definition->getRevisionTable();
  $label = $entity_definition->getLabel();
  return $table
    ? "The $label content entity uses '$table' as its revision table."
    : "The $label content entity DOES NOT use a revision table.";
}, $content_entities);

// Print the class that defines each content entity.
$entity_classes

// Print the base tables.
$base_tables

// Print the revision tables.
$revision_tables
```

In our example Drupal 10 project, you will get results similar to the following inside the interactive PHP shell:

```
> $entity_classes
= [
   "block_content" => "Drupal\block_content\Entity\BlockContent",
   "contact_message" => "Drupal\contact\Entity\Message",
   "file" => "Drupal\file\Entity\File",
   "media" => "Drupal\media\Entity\Media",
   "menu_link_content" => "Drupal\menu_link_content\Entity\MenuLinkContent",
   "node" => "Drupal\node\Entity\Node",
   "path_alias" => "Drupal\path_alias\Entity\PathAlias",
   "shortcut" => "Drupal\shortcut\Entity\Shortcut",
   "taxonomy_term" => "Drupal\taxonomy\Entity\Term",
   "user" => "Drupal\user\Entity\User",
   "paragraph" => "Drupal\paragraphs\Entity\Paragraph",
 ]
> $base_tables
= [
   "block_content" => "The Content block content entity uses 'block_content' \
as its data table.",
```

```
   "contact_message" => "The Contact message content entity DOES NOT use a ba\
se table.",
   "file" => "The File content entity uses 'file_managed' as its data table.",
   "media" => "The Media content entity uses 'media' as its data table.",
   "menu_link_content" => "The Custom menu link content entity uses 'menu_lin\
k_content' as its data table.",
   "node" => "The Content content entity uses 'node' as its data table.",
   "path_alias" => "The URL alias content entity uses 'path_alias' as its dat\
a table.",
   "shortcut" => "The Shortcut link content entity uses 'shortcut' as its dat\
a table.",
   "taxonomy_term" => "The Taxonomy term content entity uses 'taxonomy_term_d\
ata' as its data table.",
   "user" => "The User content entity uses 'users' as its data table.",
   "paragraph" => "The Paragraph content entity uses 'paragraphs_item' as its\
 data table.",
 ]
> $revision_tables
= [
   "block_content" => "The Content block content entity uses 'block_content_r\
evision' as its revision table.",
   "contact_message" => "The Contact message content entity DOES NOT use a re\
vision table.",
   "file" => "The File content entity DOES NOT use a revision table.",
   "media" => "The Media content entity uses 'media_revision' as its revision\
 table.",
   "menu_link_content" => "The Custom menu link content entity uses 'menu_lin\
k_content_revision' as its revision table.",
   "node" => "The Content content entity uses 'node_revision' as its revision\
 table.",
   "path_alias" => "The URL alias content entity uses 'path_alias_revision' a\
s its revision table.",
   "shortcut" => "The Shortcut link content entity DOES NOT use a revision ta\
ble.",
   "taxonomy_term" => "The Taxonomy term content entity uses 'taxonomy_term_r\
evision' as its revision table.",
   "user" => "The User content entity DOES NOT use a revision table.",
   "paragraph" => "The Paragraph content entity uses 'paragraphs_item_revisio\
n' as its revision table.",
 ]
> exit
```

Use `exit` to leave the interactive PHP shell. For all three variables, the array keys are the **machine names** of the content entities we are gathering information about. The array values for `$entity_classes`, `$base_tables`, and `$revision_tables` correspond to the class that defines the entity, its base table, and its revision table respectively.

Finding out the new auto-increment value for each content entity

Now that we know which table to alter, how do we find out the new auto-increment value for each? We need to check the Drupal 7 database. Unfortunately, the entity API in Drupal 7 is not as mature as that in Drupal 10. Moreover, many Drupal 10 content entities were not entities at all in Drupal 7. This makes the process a lot more complicated.

That being said, let's try to extract as much useful information as possible using Drupal 7 APIs. Open an interactive PHP shell with Drush using `ddev drush php:cli` in the Drupal 7 folder. Then, run the following code:

```
// Get Drupal 7 entity defintions.
$entities = entity_get_info();

// Get entity IDs (machine names).
$entities_ids = array_keys($entities);

// Find the class that defines each entity.
$entity_classes = array_map(function ($entity_definition) {
  return $entity_definition['controller class'];
}, $entities);
// Find the base tables of entities.

$base_tables = array_map(function ($entity_definition) {
  return $entity_definition['base table'];
}, $entities);

// Find the revision tables of content entities.
$id_columns = array_map(function ($entity_definition) {
  return $entity_definition['base table'] . '.' . $entity_definition['entity \
keys']['id'];
}, $entities);

// Find the revision tables of content entities.
$revision_columns = array_map(function ($entity_definition) {
  return $entity_definition['entity keys']['revision']
    ? $entity_definition['base table'] . "." . $entity_definition['entity key\
s']['revision']
    : 'NO revision column';
}, $entities);

// Print the class that defines each entity.
$entity_classes

```

```
// Print the base tables.
$base_tables

// Print the id columns from the base tables.
$id_columns

// Print the revision columns from the base tables.
$revision_columns
```

In our example Drupal 7 project, you will get results similar to the following inside the interactive PHP shell:

```
>>> $entity_classes
=> [
    "field_collection_item" => "EntityAPIController",
    "node" => "NodeController",
    "file" => "DrupalDefaultEntityController",
    "taxonomy_term" => "TaxonomyTermController",
    "taxonomy_vocabulary" => "TaxonomyVocabularyController",
    "user" => "UserController",
  ]
>>> $base_tables
=> [
    "field_collection_item" => "field_collection_item",
    "node" => "node",
    "file" => "file_managed",
    "taxonomy_term" => "taxonomy_term_data",
    "taxonomy_vocabulary" => "taxonomy_vocabulary",
    "user" => "users",
  ]
>>> $id_columns
=> [
    "field_collection_item" => "field_collection_item.item_id",
    "node" => "node.nid",
    "file" => "file_managed.fid",
    "taxonomy_term" => "taxonomy_term_data.tid",
    "taxonomy_vocabulary" => "taxonomy_vocabulary.vid",
    "user" => "users.uid",
  ]
>>> $revision_columns
=> [
    "field_collection_item" => "field_collection_item.revision_id",
    "node" => "node.vid",
    "file" => "NO revision column",
    "taxonomy_term" => "NO revision column",
    "taxonomy_vocabulary" => "NO revision column",
    "user" => "NO revision column",
  ]
>>> exit
```

Use `exit` to leave the interactive PHP shell. The first thing to note is that the number of entities in Drupal 7 is much lower compared to Drupal 10. Also, the list makes no distinction between content and configuration entities. For all four variables, the array keys are the **machine names** of the entities we are gathering information about. The array values for `$entity_classes` and `$base_tables` correspond to the class that defines the entity and its base table. The array values of `$id_columns` and `$revision_columns` correspond to the **id** and **revision** columns of the entity's **base table**..

With the information above, we can reach the following conclusion: nodes and field collections were revisionable in Drupal 7, while taxonomy terms, users, and files were not.

We can also build the following SQL queries to run against the Drupal 7 database.

```
-- Get higest node id value.
SELECT nid FROM node ORDER BY nid DESC LIMIT 1;

-- Get higest node revision id value.
SELECT vid FROM node ORDER BY vid DESC LIMIT 1;

-- Get higest taxonomy term id value.
SELECT tid FROM taxonomy_term_data ORDER BY tid DESC LIMIT 1;

-- Get higest user id value.
SELECT uid FROM users ORDER BY uid DESC LIMIT 1;

-- Get higest file id value.
SELECT fid FROM file_managed ORDER BY fid DESC LIMIT 1;

-- Get higest field collection id value.
SELECT item_id FROM field_collection_item ORDER BY item_id DESC LIMIT 1;

-- Get higest field collection revision id value.
SELECT revision_id FROM field_collection_item ORDER BY revision_id DESC LIMIT\
 1;
```

You can run these queries in multiple ways. One of them is opening a SQL command-line interface with Drush: `ddev drush sql:cli` from the Drupal 7 folder. Save the results of executing these queries.

But this may not be enough. What about URL aliases, blocks, menu items, or redirects? The list of entities you need to consider will vary per project, and you need to account for any content model changes. For example, our example

project converts field collections to paragraphs and nodes to type `Speaker` to user entities. We need to further review the Drupal 7 database to collect the new auto-increment values for more content entities.

We'll start with getting a list of all tables. Depending on the database server used to run the Drupal 7 website, you will have different CLI clients. DDEV bundles the following:

- `ddev mysql` for interacting with MySQL/MariaDB. Use `SHOW TABLES;` in the CLI to list all tables.
- `ddev psql` for interacting with PostgreSQL. Use `\dt` in the CLI to list all tables.
- `ddev . sqlite3` for interacting with SQLite. Use `.tables` in the CLI to list all tables.

If you are not comfortable with a CLI database client, you can use one of the many GUI database clients available in DDEV. Of course, you are free to use any other database client you are familiar with. Many IDEs provide SQL clients out of the box or via plugins.

We will proceed using the command-line interface for MySQL. Execute `ddev mysql` from the Drupal 7 folder and run the following commands to get relevant information for our example project:

```sql
-- Get a list of all tables.
SHOW TABLES;

-- Get information about the url_alias table and get the higest id value.
DESCRIBE url_alias;
SELECT pid FROM url_alias ORDER BY pid DESC LIMIT 1;

-- Get information about the block_custom table and get the higest id value.
DESCRIBE block_custom;
SELECT bid FROM block_custom ORDER BY bid DESC LIMIT 1;

-- Get information about the menu_links table and get the higest id value.
DESCRIBE menu_links;
SELECT mlid FROM menu_links ORDER BY mlid DESC LIMIT 1;
```

After getting a list of all Drupal 7 tables, we look for those that are likely to store data that would be migrated as content entities in Drupal 10. In our example, for blocks the table to query is `block_custom`, not `block`. And for

menu links, the right table is `menu_links`, not `menu_custom` nor `menu_router`. Identifying the correct tables requires experience and a bit of trial and error.

Technical note: In Drupal 7, hook_schema was used to declare tables that needed to be created when modules were installed for them to work properly. Look for implementations of this hook. If the Devel module is available in Drupal 7, you can use `drush hook schema` to get a list of implementations by currently enabled modules.

After the highest ID value has been gathered from all relevant Drupal 7 tables, you need to come up with the new auto-increment value for Drupal 10. When doing so, keep the following considerations in mind:

- **Choose a high enough value that would make it unlikely for Drupal 7 entities to have the same ID as those created in Drupal 10 before the new site launches.** For example, if over the lifespan of the Drupal 7 site `52,123` nodes were created, you can double and round the value to `100,000`.
- **The value for the Drupal 10 base and revisions tables will likely be different.** For example, in Drupal 7 there might be `52,123` nodes created but content updates produced `231,051` revisions. So, you can set the node base table in Drupal 10 to start at `100,000` while the revision table starts at `500,000`.
- **The value will likely be different for each entity type.** For example, in Drupal 7 you can have `52,123` nodes and `3,701` users. So, you can set the node base table in Drupal 10 to start at `100,000` while the user base table starts at `7,500`.
- **Think how content model changes might affect values to use.** For example, we are going to convert `Speaker` nodes to user entities. If in Drupal 7 you have `52,123` nodes and `3,701 users`, for Drupal 10 your reference value should be the higher of the two. One of the `Speaker` nodes can have node id `3`, or `512`, or `5,250`, or `7,503`, or `12,001` ...you get the point. If you decide to map the Drupal 7 node id to a Drupal 10 user id, make sure that current and potential future node ids values are considered. When one Drupal 10 entity can be populated from two or more Drupal 7 entities, use the highest value among them as the base for the calculations. In this example, we can set the base tables for both the node and user entities to `100,000`.
- **Be familiar with all entities that are created during an import operation.** For example, it is possible, and sometimes desired, to automatically create URL aliases when nodes are created. This can be done explicitly as

part of the migration or implicitly by modules like the Pathauto module implementing entity insert hooks. So, consider using the value of the Drupal 7 `url_alias` table to set a new auto-increment for the base table of the `path_alias` content entity in Drupal 10.

Technical note: Drupal 10 stores content entity IDs using the `serial` data type. This will be mapped to a native type provided by the database engine used to power the site. The MySQL/MariaDB driver converts `serial` to an unsigned `INT` (4 bytes) that accepts a maximum value of `4,294,967,295`. The PostgreSQL driver transforms it to a `serial` (4 bytes) that accepts a maximum value of `2,147,483,647`. The SQLite driver maps it to a signed `INTEGER` (64 bytes) that accepts a maximum value of `9,223,372,036,854,775,807`. Take these numbers with a grain of salt. Above all, **use common sense when deciding the new auto-increment values.**

Performing the ALTER operation on Drupal 10 tables

We have had very lengthy and highly technical discussions. Let's take a shortcut and see how we can perform the ALTER operation in our example, which uses a MySQL/MariaDB engine. Remember that other database engines might require a different approach.

```
/**
* Alter AUTO_INCREMENT value for base and revision tables of content entities.
*
* This helps prevent entity ID conflicts during D7 to D10 migrations.
*/
function tag1_migration_table_auto_increment_alter() {
 // Associative array whose keys represent the table name and values indicate
 // the new auto-increment value for the table to alter.
 $auto_increment = [
   // Drupal 10 class: \Drupal\node\Entity\Node
   // Drupal 7 tables: node and node_revision
   'node' => 450,
   'node_revision' => 1000,

   // Drupal 10 class: \Drupal\user\Entity\User
   // Drupal 7 tables: users
   'users' => 350,

   // Drupal 10 class: \Drupal\taxonomy\Entity\Term
   // Drupal 7 tables: taxonomy_term_data
```

```
    'taxonomy_term_data' => 200,
    'taxonomy_term_revision' => 200,

    // Drupal 10 class: \Drupal\path_alias\Entity\PathAlias
    // Drupal 7 tables: url_alias, node, node_revision, users, and taxonomy_te\
rm_data
    'path_alias' => 1000,
    'path_alias_revision' => 1000,

    // Drupal 10 class: \Drupal\file\Entity\File
    // Drupal 7 tables: file_managed
    'file_managed' => 350,

    // Drupal 10 class: \Drupal\media\Entity\Media
    // Drupal 7 tables: file_managed, node, and node_revision
    'media' => 1000,
    'media_revision' => 1000,

    // Drupal 10 class: \Drupal\paragraphs\Entity\Paragraph
    // Drupal 7 tables: paragraphs_item and paragraphs_item_revision
    'paragraphs_item' => 750,
    'paragraphs_item_revision' => 1500,

    // Drupal 10 class: \Drupal\menu_link_content\Entity\MenuLinkContent
    // Drupal 7 tables: menu_links
    'menu_link_content' => 500,
    'menu_link_content_revision' => 500,
  ];

  foreach ($auto_increment as $table => $new_value) {
    \Drupal::database()->query(\sprintf('ALTER TABLE {%s} AUTO_INCREMENT = %s;\
', $table, $new_value));
  }
}
```

Add the above code to the `tag1_migration.module` file, make sure the module is enabled, and call the function responsible for making the alterations:

```
ddev drush pm:enable tag1_migration
ddev drush php:eval "tag1_migration_table_auto_increment_alter();"
```

Wait, not too fast! Let's review what we did here:

1. Create the `tag1_migration_table_auto_increment_alter` PHP function in our custom `tag1_migration` module that uses Drupal 10's database API to execute the table ALTER statements.

2. Store in an associative array named `$auto_increment` the name of the Drupal 10 tables to alter with their new auto-increment value.
3. Iterate over the array to dynamically produce SQL statements similar to `ALTER TABLE node AUTO_INCREMENT = 450;`. **Note**: You can manually create and execute these queries directly in the database server if desired.
4. Use Drupal 10's database API to execute the ALTER statements.
5. Enable our custom `tag1_migration` so our new function is available.
6. Use Drush to execute our new `tag1_migration_table_auto_increment_alter` function.
7. Hope for the best... I mean, profit!

To make sure our operation works, you can verify the current auto-increment value of tables in MySQL/MariaDB as follows:

```
-- In all statements, replace 'db' with the name of your database and
-- replace 'node' with the table to check.
USE db;
SHOW CREATE TABLE node\G
SHOW TABLE STATUS FROM db WHERE name = 'node'\G
SELECT `AUTO_INCREMENT` FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = '\
db' AND TABLE_NAME = 'node'\G
```

Before wrapping up this section, we want to highlight *two very, very important points*. In MySQL/MariaDB:

1. **It is not possible to reset the counter to a value less than or equal to the value that is currently in use.** If the current auto-increment value for the `node` table is 453 and you try to set it to 450, it will remain at 453.
2. **Alterations to auto-increment values need to be executed in each database server instance.** In our case, we should call the `tag1_migration_table_auto_increment_alter` function in our local development environment and in any remote (staging) environment. A SQL dump can capture the `AUTO_INCREMENT` value, but it is generally a terrible idea to upload a local database to a remote environment..

The AUTO_INCREMENT Alter module

This article has been packed with in-depth SQL content, but we aren't done yet. We have **one more thing...** introducing the AUTO_INCREMENT Alter module. Our custom function can be replaced with the following setting:

```
$settings['auto_increment_alter_content_entities'] = [
  'node' => [450, 1000], // Alter the tables for the node content entity.
  'user' => [350], // Alter the tables for the user content entity.
  'taxonomy_term' => [200], // Alter the tables for the taxonomy term content\
 entity.
  'path_alias' => [1000], // Alter the tables for the URL alias content entit\
y.
  'file' => [350], // Alter the tables for the file content entity.
  'paragraph' => [750, 1500], // Alter the tables for the paragraph content e\
ntity.
  'media' => [1000], // Alter the tables for the media content entity.
  'menu_link_content' => [500], // Alter the tables for the custom menu link \
content entity.
];
```

Add the above to `settings.migrate.php`. Then, use the following snippet in the Drupal 10 folder to trigger the ALTER operations:

```
ddev drush auto-increment-alter:content-entities
```

The benefit of using this approach is that there is no need to know which tables to alter. The module leverages Drupal 10 APIs to introspect entity definitions and determine which are the base and revisions tables to alter. You do need to know the machine name of the content entity and the new auto-increment values for tables. Refer to the project README.me file provided by the module to learn more about its features.

You can go a long way in a migration project writing very little SQL. But the reality is that being familiar with the database structure and knowing how to query data is very handy.

And don't worry! Future articles will not be as technical as this one. When necessary, we will address database questions in the context of the content migration at hand. In the next article, we start with migrating content. Ready, set, go!

24 - Public and private file migrations

After discussing how to avoid entity ID conflicts in the previous article, we are finally ready to start migrating content. The first entity we will focus on is files, covering both public and private file migrations. We will share tips and hacks related to performance optimizations and discuss how to handle files hosted outside of Drupal.

Prerequisites

For this section, we are going to switch to the `configuration` branch in the repository. This will contain the migrated configuration from the previous section **exported to the `drupal10/config` directory and the `tag1_migration_config` module disabled.** The code for the `tag1_migration_config` module is still there and can be reviewed for reference, but the migrations related to configuration will not be available in the Drupal installation. This will require customizations to content migrations, which will be explained as needed.

Remember that in addition to using migrations, we also added configuration by applying recipes and making manual changes in the administration interface. The migrations alone cannot reproduce the current state of the site's configuration.

Instead of starting from scratch, **we will use the migrations generated by the Migrate Upgrade module**. This was explained in article 10. The migration files were copied into the `drupal10/ref_migrations` folder of the repository. More content migrations were generated than what we will use. Similar to working with the configuration migrations, having them available does not force us to execute them all. We will cherry-pick and customize the generated migrations that will let us import content in accordance with our migration plan.

If you have not done so, clone the repository and switch to the `configuration` branch. You can always refer to the `main` branch for the final version of the examples.

```
git clone https://github.com/tag1consulting/d7_to_d10_migration.git d7_to_d10\
_migration
git checkout configuration
```

Moving forward, we will be executing migrations to migrate content from Drupal 7 into Drupal 10. So we need both environments running. Refer to article 8 for an explanation on how to set up the Drupal 7 site. Refer to article 9 for an explanation on how to set up the Drupal 10 site. For a quick reference, use the commands below to get both sites up and running.

For Drupal 7:

```
cd drupal7
ddev start
ddev launch
ddev drush uli
```

For Drupal 10:

```
cd drupal10
ddev start
ddev composer install
ddev composer si # Install the site from existing configuration
ddev launch
ddev drush uli
```

Entity ID and high water mark considerations

For each content migration, we will have a short section dedicated to considerations about entity ID conflicts and setting a high water mark value. The former we covered in great detail in the previous article. The latter is an optimization for performing incremental migrations.

To avoid entity ID conflicts, we are going to artificially inflate the AUTO_INCREMENT value of entity tables so that entity IDs of imported content from Drupal 7 does not collide with entity IDs of newly created content in Drupal 10.

The base table for the `file` entity in Drupal 10 is `files_managed`. There is no revision table for this entity. When deciding the new AUTO_INCREMENT for the table in Drupal 10, consider the highest file ID (`fid`) in the source site's `file_managed` table. Execute the following query against the Drupal 7 project:

```sql
-- Get the highest file id value.
SELECT fid FROM file_managed ORDER BY fid DESC LIMIT 1;
```

For brevity, we are going to use the AUTO_INCREMENT Alter module to apply the new value. Refer to the previous article for more information of how this module works or how to perform the operation in custom code.

```
$settings['auto_increment_alter_content_entities'] = [
  'file' => [350], // Alter the tables for the file content entity.
];
```

Now execute the command provided by the AUTO_INCREMENT Alter module to trigger the alter operation in the Drupal 10 project.

```
ddev drush auto-increment-alter:content-entities
```

As for the high water property, let's have some context to understand how it is useful. When executing content migrations, the source plugin will build a SQL query to determine what records to fetch and process. When the `high_water_property` property is set, the migrate API keeps track of the last record that was processed. This is used to limit the total number of records that need to be processed in subsequent runs.

When performing incremental migrations, instead of fetching all records, a condition is added to the query so that only records above the last saved water mark value are retrieved. Records with lower values are assumed to have been processed already.

In our small example project, the performance gains are negligible. In real migrations projects, this could save hours – or quite literally days – when a fully migrated Drupal 10 database is needed.

While premature optimization might not be necessary, it is good to be familiar with performance optimizations baked into the systems we use. **Performance, like security, should not be an afterthought**.

We need to identify one field returned by the source plugin whose value always increases. Usually, the `high_water_property` property is set to a timestamp or serial identifier. In the case of files, we can use the file ID (`fid`). Being the entity's identifier, its value will keep growing as new files are added

to the systems. If an initial migration imports all files up to `fid` 42, it is safe for an incremental run to only fetch records whose `fid` is greater than 42.

For files, we will include the following snippet in the source plugin configuration:

```
source:
 plugin: d7_file
 high_water_property:
   name: fid
   alias: f
```

Technical note: When entities are processed in batches, it is possible for them to be created or updated so fast that timestamp values end up being the same. If available, favor entity revisions over timestamps when selecting the high water property for the content migration. Table primary keys are also good candidates for high water properties.

Please note that the high water property is one of multiple performance optimizations that can be implemented for content migrations. Review this article for more configuration options offered by different source plugins. Some like `track_changes` and `batch_size` can also have an impact on performance. We are discussing the `high_water_property` configuration because it is one of the most impactful for large, incremental migrations.

If you decide to use another approach, be mindful that different configuration options might not play well together. Also, it is totally valid to implement your own performance optimizations via custom source plugins or migration runners. The core API aims to be a general purpose tool and cannot cover every edge case. You and your team have the domain knowledge of the project at hand and can implement tailor-made solutions.

Migrating public files

We use `upgrade_d7_file` to migrate public files. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected. If you just reinstalled the site from the `configuration` branch, the module should already be enabled. If you have been following along performing each migration along the way, you need to disable the `tag1_migration_config` module and enable the `tag1_migration` module.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_file.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_file.yml
ddev drush cache:rebuild

# Run if the site was not installed using the configuration branch.
ddev drush pm:uninstall tag1_migration_config
ddev drush pm:enable tag1_migration
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: uuid, langcode, status, dependencies, field_-plugin_method, cck_plugin_method, and migration_group.
- Add two migration tags: `file` and `tag1_content`.
- Add `key: migrate` under the source section.
- Add the `high_water_property` property as demonstrated above.

After the modifications, the `upgrade_d7_file.yml` file should look like this:

```yaml
id: upgrade_d7_file
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - file
 - tag1_content
label: 'Public files'
source:
 key: migrate
 plugin: d7_file
 scheme: public
 high_water_property:
   name: fid
   alias: f
 constants:
   source_base_path: 'http://ddev-migration-drupal7-web/'
process:
 fid:
   -
     plugin: get
     source: fid
 filename:
```

```
    -
      plugin: get
      source: filename
  source_full_path:
    -
      plugin: concat
      delimiter: /
      source:
        - constants/new_base_path
        - filepath
      file_exists: 'use existing'
    -
      plugin: urlencode
  uri: uri
  filemime:
    -
      plugin: get
      source: filemime
  status:
    -
      plugin: get
      source: status
  created:
    -
      plugin: get
      source: timestamp
  changed:
    -
      plugin: get
      source: timestamp
  uid:
    -
      plugin: get
      source: uid
destination:
 plugin: 'entity:file'
migration_dependencies:
 required: {  }
 optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_file
ddev drush migrate:import upgrade_d7_file
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content/files` and look at the list of migrated files.

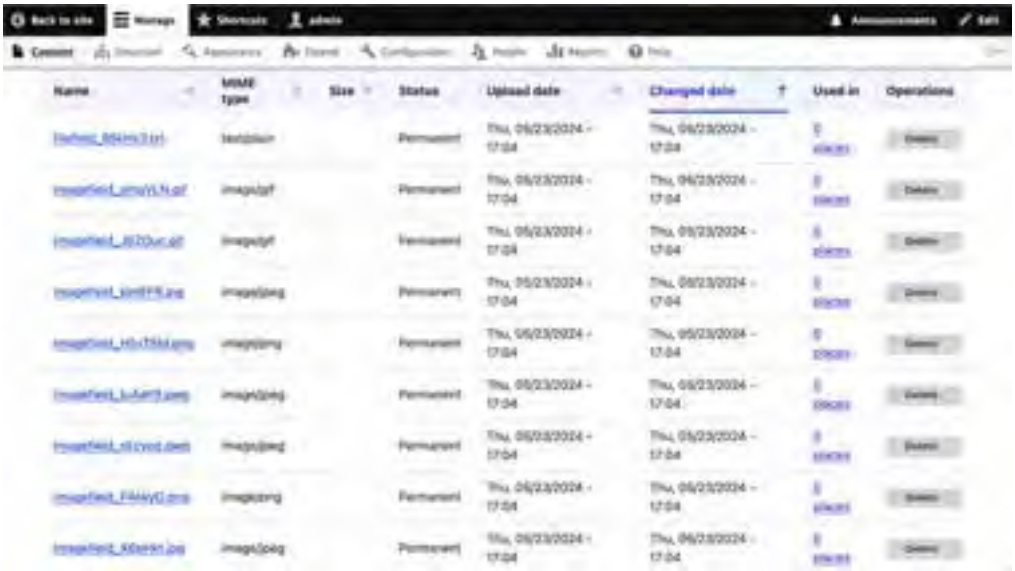


Figure 24. Configuration form

Understanding how Drupal file migrations work

Are we done already? Yes, we've completed migrating files. Congratulations! But you're here to gain a deeper understanding of the migrate API, and our next focus will be to address where the files are coming from.

Take a look a closer look at the `source` configuration of the `upgrade_d7_file` migration:

```
source:
 key: migrate
 plugin: d7_file
 scheme: public
 constants:
   source_base_path: 'http://ddev-migration-drupal7-web/'
```

To have more context, consider the result of the following query against the Drupal 7 database `SELECT * FROM file_managed WHERE uri LIKE "public:/%" LIMIT 1\G`:

```
*************************** 1. row ***************************
     fid: 1
     uid: 1
filename: druplicon.jpg
     uri: public://article/druplicon.jpg
filemime: image/jpeg
filesize: 2890
  status: 1
timestamp: 280299600
```

The `scheme` configuration option can be set to a string or an array to filter which files are retrieved from the Drupal 7 site. Common values for this setting are `public` and `private` to fetch public and private files respectively. If `scheme` is omitted when using the `d7_file` source plugin, all files in all supported schemes are retrieved, with the exception of those in the `temporary` scheme. The `upgrade_d7_file` migration is configured to retrieve public files only. In the example above, the file id (`fid`) 1 belongs to the `public` scheme as evidenced by the `uri` value: `public://article/druplicon.jpg`.

The `d7_file` source plugin expects a value for the `constant/source_base_path` configuration option. For public files, this can be set to a site address or a local file directory **relative to which URIs in the files table are specified.** Example values are `https://www.tag1consulting.com` or `/var/www/html/drupal7`, respectively. The latter will be much faster than the former – especially if downloading over public internet.

But what does URIs relative to the files table mean? For public files, the `d7_file` source plugin will perform the following operations when determining the location to retrieve the file from:

1. Determine the public file directory in the source site. It will read the `file_public_path` Drupal 7 variable or default to `sites/default/files`.

2. Read the value of the `uri` column and replace `public:/` with the location of the public file directory identified in the previous step.
3. Convert the result of the previous step to a relative path. The new value is assigned to the `filepath` source property.
4. Back in our `upgrade_d7_file` migration, we concatenate `constants/source_base_path` and `filepath` to obtain the full path of the file to import. The result is stored in the `source_full_path` pseudofield.
5. The `source_full_path` pseudofield is passed to the file_copy process plugin which takes care of getting the file into Drupal 10. The plugin includes logic for performing a download or copy operation depending on whether the received path represents a remote resource or not.

By applying the above logic, each public file's URI retrieved from the database will be transformed as follow:

- `public://article/druplicon.jpg` gets converted to `sites/default/files/article/druplicon.jpg` inside the `d7_file` source plugin.
- `sites/default/files/article/druplicon.jpg` gets converted to `http://ddev-migration-drupal7-web//sites/default/files/article/druplic` as part of the process pipeline of the `upgrade_d7_file` migration.
- The file at `http://ddev-migration-drupal7-web//sites/default/files/articl` is copied into `public://article/druplicon.jpg`, which translates to `/var/www/html/web/sites/default/files/article/druplicon.jpg`.

The location in Drupal 10 would be different if you configured the `file_public_path` setting in `web/sites/default/settings.php` file inside the `drupal10` folder. Otherwise, this settings defaults to `sites/default/files`.

Technical note: The `d7_file` source plugin is provided by the \Drupal\file\Plugin\migrate\source\d7\File class. The `upgrade_d7_file` migration is derived from core/modules/file/migrations/d7_file.yml. We highly recommend looking at the source code for both to get a better idea of what is going on under the hood. Even if you do not understand all the technical details, the files include helpful comments about how things work and how they can be configured.

One more thing to point out is where `http://ddev-migration-drupal7-web/` came from. That is the result of an action we took back in article 10 when

we executed the following Drush command to generate the migrations with the Migrate Upgrade module:

```
ddev drush --yes migrate:upgrade --legacy-db-key='migrate'
  --legacy-root='http://ddev-migration-drupal7-web'
```

During the generation of migration configuration entities, the value of the `--legacy-root` command line option is injected into the `source/constants/source_base_path` configuration of migrations that create file entities like `upgrade_d7_file` and `upgrade_d7_file_private`. See \Drupal\migrate_upgrade\MigrateUpgradeDrushRunner::applyFilePath for details.

Migrating private files

Migrating private files follows a similar approach. The primary difference is that **private files, in theory, are not web accessible**. As such, the `file_copy` process plugin will not be able to retrieve the file if `constants/source_base_path` is set to a public domain. Instead, the Drupal 7 files need to be copied over to the same server where Drupal 10 is running. And then, configure the private files migration to request files via a local path like `/var/www/html/private_files`.

When copying the files, it is important that **the private files are placed outside Drupal 10's webroot or otherwise prevent access to that folder at the web server level**. Before discussing how to migrate private files, note that you can also migrate public files from a local directory in Drupal 10's server. A *local file copy operation will be much faster than having to download a remote file*, especially when considering thousands of files and hundreds of GBs of data.

Our example repository does not make use of private files so setting up private files will be left as an exercise to the curious reader. See https://www.drupal.org/docs/7/core/modules/file/overview for instructions on how to do it in Drupal 7. We are going to proceed under the assumption that the private file system path is set to `/var/www/html/private_files`.

The first step to migrate private files would be to copy the folder containing them. A common way to do it is using `rsync` to move files from one server to another or from a server to a local environment. Some hosting providers will provide a compressed `*.zip` or `*.tar.gz` file with the content of user

uploaded files. One way or another, we need to get the Drupal 7 private files into a location where Drupal 10 can access them.

In our example project, Drupal 7's `/var/www/html/private_files` path inside DDEV's web container will map to `drupal7/private_files` in your host machine. And Drupal 10's `/var/www/html/d7_private_files` path inside DDEV's web container will map to `drupal10/d7_private_files` in your host machine.

The following commands can be used to simulate having Drupal 7 private files and move them to a local folder in Drupal 10's server. We are creating a file that will eventually be migrated.

```
# Location of private files in Drupal 7.
mkdir -p drupal7/private_files

# Location of Drupal 7 private files that were moved into Drupal 10's web ser\
ver.
mkdir drupal10/d7_private_files

# Location of private files in Drupal 10.
mkdir drupal10/new_private_files_location

# Create a test private file.
echo "Do not tell anyone." >> drupal7/private_files/secret.txt

# Copy private files from Drupal 7 into Drupal 10's web server.
cp -r drupal7/private_files/. drupal10/d7_private_files

# Verify that the files were copied into the location where the private files\
 migration expects it.
cat drupal10/d7_private_files/secret.txt
```

Assuming we have records for private files in the database, consider the result of the following query `SELECT * FROM file_managed WHERE uri LIKE "private:/%" LIMIT 1\G`:

```
*************************** 1. row ***************************
     fid: 54
     uid: 1
filename: secret.txt
     uri: private://secret.txt
filemime: text/plain
filesize: 4
  status: 1
timestamp: 979534800
```

Now we need to configure private files in Drupal 10. Refer to the online documentation for more information on configuring and securing access to private files. In short, we will add the following to the `web/sites/default-/settings.php` file inside the `drupal10` folder.

```
$settings['file_private_path'] = $app_root . '/../new_private_files_location';
```

Now that we have private files configured in Drupal 7 and 10 plus records in the `file_managed` table in the source site, we can proceed with the migration. We use `upgrade_d7_file_private` to migrate private files. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_file_private.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_file_private.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: uuid, langcode, status, dependencies, field_-plugin_method, cck_plugin_method, and migration_group.
- Add two migration tags: `file` and `tag1_content`.
- Add `key: migrate` under the source section.
- Add the `high_water_property` property as demonstrated above.
- Change the value of `source/constants/source_base_path` to `/var/www/html/private_files`.

- Add a new source constant named `new_base_path` and set its value to `/var/www/html/d7_private_files`.
- Update the first element of the `process/source_full_path/0/source` configuration to `- constants/new_base_path`.

The last three steps are very important. `source_base_path` defines the location where the private files were in Drupal 7's server. `new_base_path` defines the location where the private files were copied into Drupal 10's server. After the modifications, the `upgrade_d7_file_private.yml` file should look like this:

```yaml
id: upgrade_d7_file_private
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - file
 - tag1_content
label: 'Private files'
source:
 key: migrate
 plugin: d7_file
 scheme: private
 high_water_property:
   name: fid
   alias: f
 constants:
   source_base_path: '/var/www/html/private_files'
   new_base_path: '/var/www/html/d7_private_files'
process:
 fid:
   -
     plugin: get
     source: fid
 filename:
   -
     plugin: get
     source: filename
 source_full_path:
   -
     plugin: concat
     delimiter: /
     source:
       - constants/new_base_path
       - filepath
 uri:
   -
```

```
      plugin: file_copy
      source:
        - '@source_full_path'
        - uri
  filemime:
    -
      plugin: get
      source: filemime
  status:
    -
      plugin: get
      source: status
  created:
    -
      plugin: get
      source: timestamp
  changed:
    -
      plugin: get
      source: timestamp
  uid:
    -
      plugin: get
      source: uid
destination:
 plugin: 'entity:file'
migration_dependencies:
 required: {  }
 optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_file_private
ddev drush migrate:import upgrade_d7_file_private
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content/files` and look at newly migrated private files.

For private files, the `d7_file` source plugin will perform the following operation when determining the location to retrieve the file from:

1. Determine the private file directory in the source site. It will read the `file_private_path` Drupal 7 variable. No default is used if the variable is not set.
2. Read the value of the `uri` column and replace `private:/` with the location of the private file directory identified in the previous step.
3. Convert the result of the previous step to a relative path. The new value is assigned to the `filepath` source property.
4. Back in our `upgrade_d7_file_private` migration, we concatenate `constants/new_base_path` and `filepath` to obtain the full path of the file to import **inside Drupal 10's web server**. The result is stored in the `source_full_path` pseudofield.
5. The `source_full_path` pseudofield is passed to the file_copy process plugin which takes care of getting the file into Drupal 10. In this case, the plugin will understand that the location is not a remote resource.

A file copy operation will place the file into Drupal 10's private file folder: `/var/www/html/web/../new_private_files_location`. This is the same as:

- `/var/www/html/new_private_files_location` inside DDEV's web container, and
- `drupal10/new_private_files_location` in your host machine.

By applying the above logic, each private file's URI retrieved from the database will be transformed as follow:

- `private://secret.txt` gets converted to `/var/www/html/private_files/secret.txt` inside the `d7_file` source plugin.
- `/var/www/html/private_files/secret.txt` gets converted to `/secret.txt` inside the `d7_file` source plugin.
- `/secret.txt` gets converted to `/var/www/html/d7_private_files//secret.txt` as part of the process pipeline of the `upgrade_d7_file_private` migration.
- The file at `/var/www/html/d7_private_files//secret.txt` in Drupal 10's server is copied into `private://secret.txt`.

The `private://secret.txt` Drupal 10 URI the same as:

- `/var/www/html/new_private_files_location/secret.txt` inside DDEV's web container, and
- `drupal10/new_private_files_location/secret.txt` in your host machine.

Importing files from a local copy

The `new_base_path` source property is *only necessary if the location of the private files in Drupal 7 is different to the location where the such files are copied inside Drupal 10's web server*. In our example, the Drupal 7 private files lived in `/var/www/html/private_files` and were copied into `/var/www/html/d7_private_files` in preparation for the migration.

You might be tempted to place the files directly where they will ultimately live in Drupal 10 and bypass the `file_copy` operation. In our example, that would be `/var/www/html/new_private_files_location`. However, it is better not to do this, because there might be modules altering the `uri` property of the retrieved file before saving the file entity. Use cases for this include changing the schema of files or reorganizing their folder structure. In such cases, the real location on disk will not match the expected location as stored in Drupal 10's `file_managed` table.

A similar approach of copying the files into Drupal 10's web server can be used to speed up public file migrations. Apply the following changes to the `upgrade_d7_file` migration:

- Change the value of `source/constants/source_base_path` to `sites/default/files`. This should match the value of Drupal 7's `file_public_path` variable. Use the `sites/default/files` if the variable is not set.
- Add a new source constant named `new_base_path` and set its value to `/var/www/html/d7_public_files` contains a copy of Drupal 7's public file folder.
- Update the first element of the `process/source_full_path/0/source` configuration to `- constants/new_base_path`.

Before going any further, I want to assert that, yes, keeping track of all the different locations involved file migrations can be quite hard. While writing this article, I had to thoroughly review the code of the d7_file source plugin and the file_copy process plugin, inspect how the file migrations were generated, tweak such migrations, and step debug through the whole process to piece together something that works.

One thing is for sure. You can come up with a different set of file locations and still migrate files properly. Nothing stops you from creating custom source

or process plugins, or coming up with different process pipelines. If so, refer back to this article for inspiration.

Remember to review the migration system's issue queue if you encounter problems. Asking for support in the `#migration` channel in Drupal slack is a good way to get community support.

Performance considerations

When there is a large number of files amounting to hundreds of GBs of data, the file migrations can take a very long time to complete. The best way to speed up the process is copying the Drupal 7 files into the Drupal 10 server and using that as the source of the migration.

Another optimization is passing an additional configuration option to the `file_copy` plugin to prevent multiple downloads of the same file. Adding `file_exists: 'use existing'` will check if a file with the same name exists in the destination location. If so, the `file_copy` plugin bails out without downloading or copying the file again. Using previously migrated files should not be a problem because, at the moment, it is not possible to replace files once they are uploaded to Drupal. For reference on how to apply this configuration, see the following snippet:

```
process:
 uri:
   -
     plugin: file_copy
     source:
       - '@source_full_path'
       - uri
     file_exists: 'use existing'
```

The two techniques explained above apply to public and private migrations. They are also safe to use when running a full migration to prepare a production-like Drupal 10 database. If you want to know some tips and hacks for developing locally, continue reading.

Note: While not related to the above, if you are using mutagen with DDEV on macOS, configure the upload_dirs setting.

Don't try these techniques at home

Congrats on making it this far into the article! As a reward, I will share some insights that might reduce the wait times for file migrations during local development. Just promise me two things:

1. **Never** do this when running a migration in production. **Never.**
2. If someone asks, don't tell them that you heard of these techniques from me. ;-)

Tip 1: Not migrating files

The first tip to speed up the migration of files is **not migrating any file**. Let me elaborate. It is possible to **migrate a reference to a file without getting the file on disk**. Instead of using the `file_copy` plugin, you make a verbatim assignment of the `uri` property from source to destination like this:

```
process:
 uri: uri
```

Assuming a public file located at `public://dinarcon.jpg` in Drupal 7, the migration will store a reference in Drupal 10 with the same URI value in its `file_managed` table. Any user trying to retrieve the file will get a file not found error. But as far as the migration is concerned, the references to the files have been migrated and the file entities created. This is also enough to consider the migration *completed*. Other migrations that depend on files will not fail with the error: `did not meet the requirements. Missing migrations upgrade_d7_file`.

The above technique can be supplemented with the Stage File Proxy module when performing public file migrations. In our example, you can install, enable, and configure the module with the following commands:

```
composer require drupal/stage_file_proxy
drush pm:enable stage_file_proxy
drush config:set stage_file_proxy.settings origin 'http://ddev-migration-drup\
al7-web'
```

When a request comes for a file missing in Drupal 10, the module will try to retrieve it from the origin domain. This would not work for migrating private files because they are, in theory, *not* web accessible. I have seen conversations about "temporarily" removing access control to private files, but that is an action with very real security risks. **Don't do that.** Please do not shoot yourself in the foot. If you decide to do it anyways, put the source and destination sites behind a firewall when executing the private files migrations.

Tip 2: Hacking Drupal core

The next technique involves *hacking Drupal core*. What? I didn't say it would be pretty. Joking aside, hacking Drupal core means modifying the files that comprise the base framework. This is useful if you need to rollback file migrations. In many cases, rolling back a migration triggers the delete operation on the imported entities. Deleting a file entity removes all references to the file and **removes the file from disk**.

When a file migration is rolled back, the files need to be downloaded again the next time the migration is executed. Not a big deal in our rather simple example. But when dealing with large file directories deleting and re-downloading files might equate to *hours* of waiting.

To prevent the files from being removed from disk upon rollback, edit the `\Drupal\file\Entity\File::preDelete` method at web/core/modules/-file/src/Entity/File.php inside the `drupal10` folder:

```
public static function preDelete(EntityStorageInterface $storage, array $enti\
ties) {
   parent::preDelete($storage, $entities);
   foreach ($entities as $entity) {
     // ...
     // Delete the actual file. Failures due to invalid files and files that
     // were already deleted are logged to watchdog but ignored, the
     // corresponding file entity will be deleted.
     try {
       // Comment out or remove the line below.
       // \Drupal::service('file_system')->delete($entity->getFileUri());
     }
     catch (FileException $e) {
       // Ignore and continue.
     }
   }
 }
```

This is a bad idea on so many levels. You are altering a core behaviour. The removal of disk will be skipped not only during migration rollbacks, but *whenever* Drupal invokes an entity delete operation on files. You could forget that you made this change and introduce bugs in other parts of the system. You could lose your hack in the next update of Composer packages. You should definitely not try this – but if you do, create a patch with the hack and apply it using cweagans/composer-patches. A composer update won't override the patch *and* it provides more visibility. You're less likely to forget the hack was applied when you see the patch in your composer file.

I guess I cannot take back the tips that I shared. If you decide to implement any of the techniques discussed in this section, exercise extra caution. **Remember: never do any of them in a production environment.** They are meant to be used exclusively during local development.

Migrating files hosted outside of Drupal

If you are using an external service to host your files, you need to account for more things when migrating them. Let's say you are using the S3 File System module to store files in an Amazon S3 bucket. Ask yourself the following questions:

- Will you share the **same bucket** between Drupal 7 and 10? If you do, and you do not need to change the path structure, you could migrate

the `uri` property verbatim as discussed above. **Warning:** When using shared buckets, rolling back the migration in Drupal 10 will trigger the deletion of the file in the bucket. *This will also affect Drupal 7!* If you decide to use shared buckets, make sure to **never** rollback a migration in an environment that points to a production bucket. This is easier said than done. You could probably subscribe to the MigrateEvents::PRE_-ROLLBACK event and throw an exception to prevent further processing. Or better yet, *never use shared buckets*.

- If not using the shared buckets, how will you move the files from one to another? Many providers offer tools to bulk copy files between buckets. Some providers even offer migration tools from external providers.
- Does Drupal 10 have write access to the new bucket? It might sound obvious, but verify access controls are properly set up. You could have copied an image file into the new bucket with an external tool. When there is a need to generate an image style, Drupal might not be able to create it if permissions are misconfigured.
- Do you need to enable CORS-based uploads? Not directly related to the migration operation, but it might be a requirement carried over from the Drupal 7 site. The S3 File System CORS Upload module can help with this.

Is there more to file migrations?

We can certainly keep going, but this article is long enough already. Instead, I invite you to listen this podcast that discusses complex file and media migrations: https://www.tag1consulting.com/blog/moving-drupal-7-drupal-10-managing-complex-file-and-media-migrations

Before wrapping up, let me highlight one more thing. Take another look at the final `upgrade_d7_file.yml` file. The changes we made to that migration do not change the import logic in a meaningful way. We could have been able to import the public files *without making any modifications* to the generated migration. Drupal can be adapted to fulfill very niche requirements, but it can do so much with little or no customization.

We covered a lot of ground today, but don't feel that you need to understand every detail of how the migrate API works. Yes, it is a fantastic, flexible, and powerful tool. It is also a complex one. In my case, *I learn something new every time I interact with it.*

Whew! We're done with files for now. In the next article, we will move on to migrating users and taxonomy terms.

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

25 - User and taxonomy term migrations

In the previous article, we learned how to migrate files. Today, we continue with two more entities: users and taxonomy terms. First, we'll talk about preventing entity ID conflicts and water mark considerations. Then, we'll discuss why our content migrations will not have dependencies on configuration migrations. Next, we'll write the migrations for the two entities. Finally, we'll cover stylistic choices we can leverage to make our migration files clean and compact.

Entity ID and high water mark considerations

Drupal 7's user and taxonomy term entities are revisionable entities out of the box. Usually you would only have to review the users and taxonomy_term_data Drupal 7 database tables to determine the new AUTO_INCREMENT values in Drupal 10, but things are not so simple in our example project. Our migration plan dictates that nodes of type `speaker` should be migrated as user entities and nodes of type `sponsor` should become taxonomy term entities. In today's article, we are going to migrate users and taxonomy terms as they exist in Drupal 7. In the next one, we will take care of converting nodes to other entity types.

In Drupal 10, the user entity is still not revisionable, but the taxonomy entity was changed to store revision data. *To determine the new AUTO_INCREMENT values for one entity in Drupal 10, you might need to review multiple entities and tables in Drupal 7.*

For the *base table* of the `user` entity in Drupal 10, consider the highest user ID (`uid`) and node ID (`nid`) of `speaker` nodes in Drupal 7. For the *base table* of `taxonomy_term` entity in Drupal 10, consider the highest taxonomy term ID (`tid`) and node ID of `sponsor` nodes in Drupal 7. For the *revision table* of the `taxonomy_term` entity in Drupal 10 consider the highest taxonomy term ID and revision ID of `sponsor` nodes (`vid`) in Drupal 7.

Refer to article 23 for information on how to query the database to retrieve this information. The same article explains how to set the new AUTO_INCREMENT values in Drupal 10 via custom code and using the AUTO_INCREMENT Alter module.

For brevity, we are going to use the latter:

```
$settings['auto_increment_alter_content_entities'] = [
  'user' => [350], // Alter the tables for the user content entity.
  'taxonomy_term' => [200], // Alter the tables for the taxonomy term content\
 entity.
];
```

Now execute the Drush command provided by the AUTO_INCREMENT Alter module to trigger the alter operation in the Drupal 10 project.

```
ddev drush auto-increment-alter:content-entities
```

To select the appropriate column for the `high_water_property` property, verify the available columns in the `users` and `taxonomy_term_data` tables:

```
DESCRIBE users;
DESCRIBE taxonomy_term_data;
```

Technical note: The `high_water_property` configuration option should be one of the columns returned by the source plugin. It is common that all fields for the primary source table are returned. That being said, a source plugin can provide more fields by performing JOIN operations on multiple tables or adding custom fields when preparing the row. This is how node migrations attach field API data to the records being fetched. Source plugins can also limit the fields to retrieve from a table or unset values that were already fetched. The drush migrate:fields-source command might help identify which fields are available. That is usually a static list and might not contain every possible option. One way to know for sure is available is to debug the migration.

For users, there are two possible columns to use. `uid` which stores the user ID and `changed` which stores the last time the account was updated. If you choose `uid`, incremental migrations will pick up new users, but they will not be able to update previously migrated users. This might be a problem if

you need to run the Drupal 7 and 10 sites in parallel during a phased launch strategy. Changes to user accounts like credentials or roles updates will not be synced during incremental runs.

If you decide to use `changed`, new users and updates to existing users will be migrated. The caveat is that `changed` can contain duplicate values and that does not play well with the high water property. Under certain circumstances, the last saved value in the high water mark can match the value of records that still need to be processed. When this happens, the migration will skip some records. At the end of the migration, you will have pending records to process. This will prevent other migrations from running due to unmet dependencies. This problem might arise when running incremental migrations or when batching large migrations into smaller pieces. In such cases, implement a different optimization strategy.

For our example, we will proceed with `changed` for the `high_water_property` property in the user migration.

```
source:
 key: migrate
 plugin: d7_user
 high_water_property:
   name: changed
   alias: u
```

In the case of taxonomy terms, Drupal 7 does not store information about when the entity was created or updated. The only viable option for the `high_water_property` property is the taxonomy term ID (`tid`):

```
source:
 key: migrate
 plugin: d7_taxonomy_term
 high_water_property:
   name: tid
   alias: td
```

Dependencies on configuration migrations

We generated the content migrations back in article 10 using the Migrate Upgrade module. One key assumption of this approach is that all configuration

migrations are executed, and their map tables are present, by the time the content migrations are executed. Because of this, many of the generated content migrations have dependencies on configuration migrations.

Our example project implemented various content model changes between Drupal 7 and Drupal 10. We used a combination of migrations, recipes, and manual configuration changes to build the new content model. Once the site was configured as needed, we deactivated the `tag1_migration_config` module, because the migrations alone did not reflect the final state of the site's configuration. Instead, we exported and updated configuration at different stages. The objective was to be able to install the site from the existing configuration. This gave us a fully configured site with no content that can be used as the starting point for our content migration efforts.

Re-installing Drupal from existing configuration means that the map and message tables that the migrate API uses to keep track of imports are no longer available. Lookup operations against configuration migrations will not work. This will require adjustments to the generated content migrations. It will be more work for us, but that will better reflect how custom migrations are performed. Replicating Drupal 7's configuration verbatim is rarely the goal of most real life migration projects.

To get started, we will have to update the `migration_dependencies`. We will also have to make adjustments to the process pipeline.

Migrating users

We use `upgrade_d7_user` to migrate users. Copy it from the reference folder into our custom module `tag1_migration` and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_user.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_user.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `user` and `tag1_content`.
- Add `key: migrate` under the source section.
- Add the `high_water_property` property as demonstrated above.
- Map the `changed` property in the `process` section. This is optional, but we might as well migrate when an account was last updated if the data is available.
- Update the migration dependencies so that only a *required dependency* on `upgrade_d7_file` remains.

In addition to the above, we need to make two more updates to the migration. First, our migration plan says that blocked users should not be migrated. In Drupal 7, the `status` column of the `users` table stores the relevant information. A value of `1` indicates the user is active while `0` means the user is blocked. We are going to use the skip_on_empty process plugin to prevent blocked users from being migrated:

```
process:
 status:
   -
     plugin: skip_on_empty
     source: status
     method: row
     message: 'User was not migrated because the account is blocked.'
```

Note: It is important to get familiar with process plugins available in Drupal core and contributed modules like Migrate Plus. We will demonstrate the usage of many throughout the series, but it is not practical nor feasible to cover them all. Read this documentation page to get acquainted with existing process plugins.

The second destination property that we need to update is `roles`. The generated migration attempts a lookup operation against the `upgrade_d7_user_role` migration. As explained earlier, this will not work. To address this situation, we need to know the IDs of the Drupal 7 roles. In article 3, we introduced a site audit template. Then, in article 8, we filled out the template with the help of document containing SQL queries to execute against the Drupal 7 database. Let's get the role IDs and update the role mapping accordingly.

```sql
SELECT rid AS role_id, name FROM role ORDER BY rid ASC;
```

```yaml
process:
 roles:
   -
     plugin: static_map
     source: roles
     map:
       3: administrator
       4: content_editor
     bypass: TRUE
```

Let's break down how we end up with the above code snippet. In Drupal 7, roles were identified by an **integer role ID** (`rid`) value. In Drupal 10, roles are identified by a **string machine name** value. So, we need to *map the Drupal 7 role id to the Drupal 10 role machine name*. Pay close attention to possible name changes in roles. In Drupal 7, role id 4 was labeled `Editor`. In Drupal 10, the corresponding role's machine name is `content_editor`. Refer back to article 22 for context as to how this came to be.

The `static_map` process plugin can be used to make the necessary transformation from Drupal 7 role IDs to Drupal 10 role machine names. The usage of `bypass` is not needed in this case but was included for completeness. If there are roles that should not be migrated, you can exclude them from the assignment of the `roles` destination property. The `static_map` process plugin will throw an exception if it receives a value not present in the `map` configuration. To avoid this, we can let through any source role ID without a corresponding mapping. As long as there are no Drupal 10 roles matching such integer identifiers, there is no need to take extra measures to filter them out. The user save operation will only assign roles whose machine names exist in Drupal 10.

After the modifications, the `upgrade_d7_user.yml` file should look like this:

```yaml
id: upgrade_d7_user
class: Drupal\user\Plugin\migrate\User
migration_tags:
 - 'Drupal 7'
 - Content
 - user
 - tag1_content
label: 'User accounts'
source:
 key: migrate
 plugin: d7_user
 high_water_property:
   name: changed
   alias: u
process:
 uid:
   -
     plugin: get
     source: uid
 name:
   -
     plugin: get
     source: name
 pass:
   -
     plugin: get
     source: pass
 mail:
   -
     plugin: get
     source: mail
 created:
   -
     plugin: get
     source: created
 changed:
   -
     plugin: get
     source: changed
 access:
   -
     plugin: get
     source: access
 login:
   -
     plugin: get
     source: login
 status:
   -
     plugin: skip_on_empty
```

```
      source: status
      method: row
      message: 'User was not migrated because the account is blocked.'
  timezone:
    -
      plugin: get
      source: timezone
  langcode:
    -
      plugin: user_langcode
      source: entity_language
      fallback_to_site_default: false
  preferred_langcode:
    -
      plugin: user_langcode
      source: language
      fallback_to_site_default: true
  preferred_admin_langcode:
    -
      plugin: user_langcode
      source: language
      fallback_to_site_default: true
  init:
    -
      plugin: get
      source: init
  roles:
    -
      plugin: static_map
      source: roles
      map:
        3: administrator
        4: content_editor
      bypass: TRUE
  user_picture:
    -
      plugin: default_value
      source: picture
      default_value: null
    -
      plugin: migration_lookup
      migration: upgrade_d7_file
destination:
 plugin: 'entity:user'
migration_dependencies:
 required:
   - upgrade_d7_file
 optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the

migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_user
ddev drush migrate:import upgrade_d7_user
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/people` and look at the list of migrated users.

You can also see the messages related to users being skipped by running the `ddev drush migrate:messages upgrade_d7_user --fields=source_ids,message` or visiting the reports page at `https://migration-drupal10.ddev.site/admin/reports/migration-messages/upgrade_d7_user`. The source ID for the `upgrade_d7_user` migration is Drupal 7's user ID. The reports indicate that users with `uid` 10, 11, and 17 had their account deactivated.

```
--------- ------------------------------------------------------------------\
------------
 Source    Message                                                          \

 ID(s)
--------- ------------------------------------------------------------------\
------------
 10        upgrade_d7_user:status: User was not migrated because the account \
is blocked.
 11        upgrade_d7_user:status: User was not migrated because the account \
is blocked.
 17        upgrade_d7_user:status: User was not migrated because the account \
is blocked.
--------- ------------------------------------------------------------------\
------------
```

Technical note: Source plugins need a way to uniquely identify each record to be processed. They implement the \Drupal\migrate\Plugin\MigrateSourceInterface::getIds method to get a set of one or more source fields, and their data types, that can be used for identification purposes.

Preventing records from being migrated

There are two common ways to **prevent records from being migrated**:

1. Via the process pipeline.
2. By modifying the query of the source plugin. See \Drupal\migrate\Plugin\migrate\source\SqlBase::query.

When using the process pipeline, we pass source data to process plugins to determine if a record should be migrated or not. This is what we did in today's example. When filtering out via source plugins, we add extra conditions to the SQL query that fetches the data. That way, unwanted records are removed directly by the source plugin without ever reaching Drupal.

Each approach has something in their favor. Filtering by modifying the source plugin's query is more performant, because you are reducing the total number of records that need to be processed. This can lead to big performance gains when migrating large datasets. Filtering via the process pipeline is more flexible as you have access to Drupal APIs – and the whole PHP language –

to implement the necessary logic that determines which records should not be migrated. This might be easier than modifying the source plugin's query to produce an equivalent exclusion logic.

Both are valid approaches. Implement the one that makes sense in each situation. We will demonstrate source plugin filtering when migrating media and node entities.

Migrating taxonomy terms

The automated upgrade process generates one migration for each Drupal 7 vocabulary. Having separate migrations is useful when each bundle of the content entity has a different set of fields. That is usually the case for content types, which are bundles of the `node` entity. Very often vocabularies, which are bundles of the `taxonomy_term` entity, do not have any field attached to them.

When migrating taxonomy terms:

- Use a single migration for **all vocabularies that do not have any fields**.
- Plus, one separate migration for **each vocabulary that has one or more fields**.

Our example project has two vocabularies: `Session topics` and `Article Tags`. None of them has fields so we will merge the two generated migrations into one. But first, let's compare the two generated migrations.

From the `drupal10` folder, execute the following command:

```
ddev exec diff ref_migrations/migrate_plus.migration.upgrade_d7_taxonomy_term\
_session_topics.yml \
  ref_migrations/migrate_plus.migration.upgrade_d7_taxonomy_term_tags.yml
```

The output of should be similar to:

```
1c1
< uuid: cae873b1-70c6-4843-901d-9e2f0077e73a
---
> uuid: 0610670f-d2eb-40a0-a5b2-4ba51f750902
5c5
< id: upgrade_d7_taxonomy_term_session_topics
---
> id: upgrade_d7_taxonomy_term_tags
13c13
< label: 'Taxonomy terms (Session topics)'
---
> label: 'Taxonomy terms (Article Tags)'
16c16
<   bundle: session_topics
---
>   bundle: tags
72c72
<   default_bundle: session_topics
---
>   default_bundle: tags
```

Technical note: If you get an error after executing the diff command, it is fine to ignore it. When differences are found between the files being compared, `diff` will exit with status code `1`. This is reported as an error by `ddev exec`, but in reality that is the expected result. You can suppress the error by running the command inside DDEV's web container (`ddev ssh`), by running the `diff` command from your host machine if it is available, or by redirecting the STDERR output to the `/dev/null` pseudo-device. Explaining this is outside the scope of the series.

Without context or a detailed explanation it is certainly hard to make sense of the `diff` command's output. Feel free to use a program that can provide visual differences between two files. Or better yet, check if your IDE of choice includes this functionality out of the box. Both PHPStorm and VSCode can compare files visually. Below is an image showing of leveraging a visual diff tool:

```
1- uuid: cae873b1-70c6-4643-08b8-0c2f0077e73a
2  langcode: en
3  status: true
4  dependencies: {  }
5- id: upgrade_d7_taxonomy_term_session_topics
6  class: Drupal\migrate\Plugin\Migration
7  field_plugin_method: null
8  cck_plugin_method: null
9  migration_tags:
10   - 'Drupal 7'
11   - Content
12 migration_group: migrate_drupal_7
13- label: 'Taxonomy terms (Session topics)'
14 source:
15   plugin: d7_taxonomy_term
16-  bundle: session_topics
17 process: [...]
18 destination:
19   plugin: 'entity:taxonomy_term'
20-  default_bundle: session_topics
21 migration_dependencies:
22   required:
23     - upgrade_d7_taxonomy_vocabulary
24   optional:
25     - upgrade_d7_field_instance
26     - upgrade_d7_taxonomy_vocabulary
27
```

```
1+ uuid: 0610670f-d2eb-48a0-a5b2-4ba51f750002
2  langcode: en
3  status: true
4  dependencies: {  }
5+ id: upgrade_d7_taxonomy_term_tags
6  class: Drupal\migrate\Plugin\Migration
7  field_plugin_method: null
8  cck_plugin_method: null
9  migration_tags:
10   - 'Drupal 7'
11   - Content
12 migration_group: migrate_drupal_7
13+ label: 'Taxonomy terms (Article Tags)'
14 source:
15   plugin: d7_taxonomy_term
16+  bundle: tags
17 process: [...]
18 destination:
19   plugin: 'entity:taxonomy_term'
20-  default_bundle: tags
21 migration_dependencies:
22   required:
23     - upgrade_d7_taxonomy_vocabulary
24   optional:
25     - upgrade_d7_field_instance
26     - upgrade_d7_taxonomy_vocabulary
27
```

Figure 25. Visual Diff Tool

One way or another, it is clear that there are very little differences between `upgrade_d7_taxonomy_term_session_topics.yml` and `upgrade_d7_taxonomy_term_tags.yml`. Let's analyze each one:

- `uuid` is a root property. It can be discarded when converting configuration entities to core migration plugins.
- `id` is a root property. For the combined migration we can use `upgrade_d7_taxonomy_term` as the migration ID.
- `label` is a root property. For the combined migration we can use `Taxonomy terms` as the migration ID.
- `bundle` is a configuration option for the `d7_taxonomy_term` source plugin. When set, it limits the vocabularies to fetch taxonomy terms from. For the combined migration we do not have to filter out any vocabulary. `source/bundle` will be deleted.
- `default_bundle` is a configuration option for the `entity:taxonomy_term` destination plugin. When set, it hints which vocabulary the taxonomy term that is being imported belongs to. For the combined migration we will use the `vid` destination property for this purpose. `destination/default_bundle` will be deleted.

Technical note: The `default_value` configuration in entity destination plugin if there is a destination property for the entity's `bundle` key. In the

case of taxonomy terms, it would be the `vid` destination property. For nodes, it would be the `type` property. This is implemented in the \Drupal\migrate\Plugin\migrate\destination\Entity::getBundle method.

Merging taxonomy term migrations

Because the `session_topics` and `tags` Drupal 7 vocabularies do not have fields, we can have a single migration for both. We will use `upgrade_d7_taxonomy_term_session_topics.yml` as our starting point to migrate taxonomy terms. Copy it from the reference folder into our custom module `tag1_migration` and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_taxonomy_term_session_top\
ics.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_taxonomy_term.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Change the `id` to `upgrade_d7_taxonomy_term`.
- Change the `label` to `Taxonomy terms`.
- Add two migration tags: `taxonomy_term` and `tag1_content`.
- Add `key: migrate` under the source section.
- Add the `high_water_property` property as demonstrated above.
- Remove the `bundle` key from the `source` section.
- Remove the `default_bundle` from the `destination` section. We will set the `vid` destination property.
- Remove all migration dependencies.

Note: The d7_taxonomy_term source plugin can use an optional `bundle` configuration option. If it is missing, like what we are suggesting above, it will fetch all terms no matter the vocabulary they belong to. Alternatively, you can

explicitly list one or more Drupal 7 vocabulary machine names to fetch terms from.

In addition to the above, we need to make two more changes to the migration. First, we need to update the mapping of the `vid` destination property. This is used to determine which Drupal 10 vocabulary the term being processed belongs to. At the moment, the mapping of `vid` uses a migration lookup against a configuration migration. As noted earlier in the article, we are not going to rely on configuration migrations being executed in the current installation.

Below is a snippet of the original mapping:

```
process:
 vid:
   -
     plugin: migration_lookup
     migration: upgrade_d7_taxonomy_vocabulary
     source: vid
```

To replace the lookup operation, we can implement a static map similar to the one used for assigning roles in the user migration. In this case, we will map the **integer vocabulary IDs** from Drupal 7 to **string vocabulary machines** in Drupal 10. To obtain the map values refer to the filled out site audit template or execute the taxonomy term query against the Drupal 7 database.

```
SELECT
   tv.module,
   tv.vid,
   tv.machine_name,
   tv.name,
   tv.description,
   COUNT(td.tid) AS term_count
FROM taxonomy_vocabulary AS tv
LEFT JOIN taxonomy_term_data AS td ON td.vid = tv.vid
GROUP BY tv.vid
ORDER BY term_count DESC;
```

```
process:
 vid:
   -
     plugin: static_map
     source: vid
     map:
       1: tags
       2: session_topics
```

Notice that we are not setting the `bypass` configuration in the `static_map` process plugin. When missing, a row will fail to import if the value of `source` property is not present in the `map`. In our case, this prevents creating a taxonomy term with an invalid Drupal 10 vocabulary. You can run the `drush migrate:messages` command to see the taxonomy term IDs of the records that are skipped for this reason. When this happens, we are either missing a vocabulary in the static map or the source plugin is fetching terms from vocabularies that we did not intend to.

Technical note: The `d7_taxonomy_term` source plugin *used in the content migrations* fetches the `machine_name` field from the `taxonomy_vocabulary` Drupal 7 table. In most cases, that value will match the machine name of the Drupal 10 vocabulary created by `<em>upgrade_d7_taxonomy_vocabulary configuration migration</em>`. If you review the vocabulary migration discussed in article #15, you will see that the `vid` property uses the make_unique_entity_field process plugin. This can lead to different vocabulary machine names between the two versions of Drupal. If you are certain that the machine names are the same, you can assign the `vid` property as follows: `vid: machine_name`. Otherwise, a `static_map` will get the job done.

The second change we need to make is to the `parent_id` pseudo-field. This is read by the `parent` destination property to indicate the parent term when a vocabulary has a nested hierarchy. Currently, the second transformation of its process plugin chain performs a lookup operation against all the generated taxonomy term content migration. We are customizing and merging these migrations into one. Therefore, we need to update the value of `process/parent_id/1/migration` to `upgrade_d7_taxonomy_term`, which is the ID of the migration we are writing. This means we have a migration that performs a lookup operation against itself to determine the parent taxonomy term.

Note: If a vocabulary is not hierarchical, the `parent_id` pseudo-field and the `parent` destination property can be removed altogether. When merging term migrations like in our example, do not remove them unless you are certain

that none of the vocabularies being handled by the merged migration makes use of term hierarchies.

After the modifications, the `upgrade_d7_taxonomy_term.yml` file should look like this:

```yaml
id: upgrade_d7_taxonomy_term
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - taxonomy_term
 - tag1_content
label: 'Taxonomy terms'
source:
 key: migrate
 plugin: d7_taxonomy_term
 high_water_property:
   name: tid
   alias: td
process:
 tid:
   -
     plugin: get
     source: tid
 revision_id:
   -
     plugin: get
     source: tid
 vid:
   -
     plugin: static_map
     source: vid
     map:
       1: tags
       2: session_topics
 name:
   -
     plugin: get
     source: name
 description/value:
   -
     plugin: get
     source: description
 description/format:
   -
     plugin: get
     source: format
 weight:
```

```
    -
      plugin: get
      source: weight
  parent_id:
    -
      plugin: skip_on_empty
      method: process
      source: parent
    -
      plugin: migration_lookup
      migration:
        - upgrade_d7_taxonomy_term
  parent:
    -
      plugin: default_value
      default_value: 0
      source: '@parent_id'
  changed:
    -
      plugin: get
      source: timestamp
  langcode:
    -
      plugin: get
      source: language
destination:
  plugin: 'entity:taxonomy_term'
migration_dependencies:
  required: { }
  optional: { }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_taxonomy_term
ddev drush migrate:import upgrade_d7_taxonomy_term
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/structure/taxonomy/manage/tags/` and `https://migration-drupal10.ddev.site/admin/structure/taxonomy/manage/s` `topics/overview` to see the migrated terms in their corresponding vocabularies.

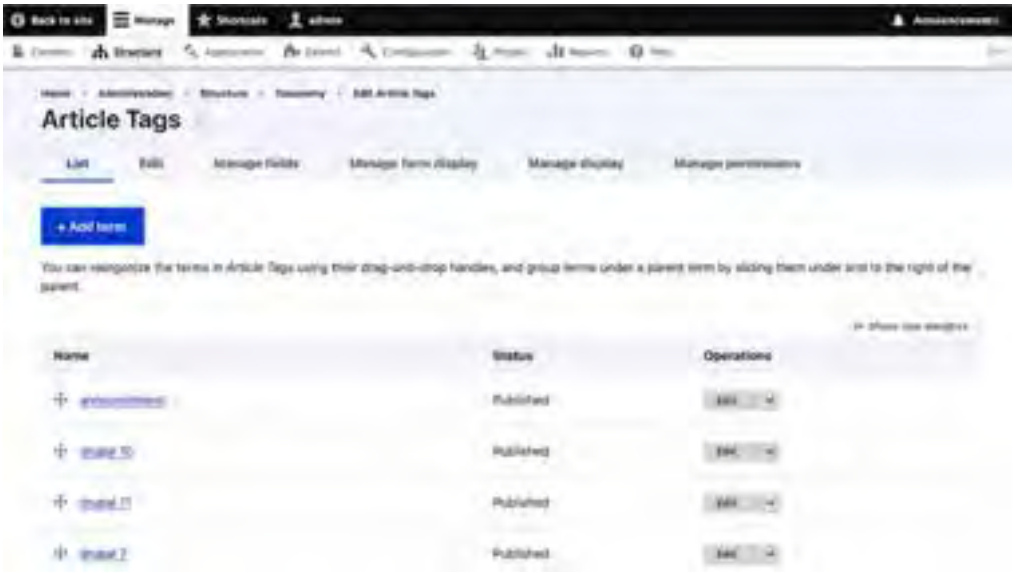


Figure 26. Article Tag Terms

Clean and compact migrations

Before closing, I would like to acknowledge that the two migrations we wrote in this article are not as clean and compact as they could be. The former refers to a migration cleaned of unnecessary destination properties. The latter refers to using syntactic sugar in the migration files to make them more compact and easier to read.

Consider this alternative `process` section for the `upgrade_d7_taxonomy_term` migration:

```
process:
 tid: tid
 revision_id: tid
 vid:
   plugin: static_map
   source: vid
   map:
     1: tags
     2: session_topics
 name: name
 description/value: description
 description/format: format
 weight: weight
```

```
14  changed: timestamp
15  langcode: language
```

The `parent_id` and `parent` destination properties are removed because none of our vocabularies are hierarchical. In the case of the `upgrade_d7_user` migration, we could have removed the `user_picture` destination property and the corresponding dependency on `upgrade_d7_file`, because none of Drupal 7 users had a picture associated with them. Removing unnecessary destination properties requires an understanding of the project requirements and the source data. It is not possible for us to give generalized advice on this matter. **Carefully review any destination property that you are planning to remove to make sure it is indeed not needed.**

In the snippet above, all destination properties that use a single `get` process plugin are reduced to a one-line mapping. The `vid` uses a single process plugin so there is no need to use an array syntax for it. This syntactic sugar makes the migration files more compact. I would also argue that this format is easier to read. **Whichever style you use, be consistent throughout the whole project.** Above all, favor what would make it easier for you and your team to understand what the migration is doing. Ultimately, you are the ones responsible for maintaining the project.

In the next article, we will learn how one entity type in Drupal 7 can be migrated into a different entity type in Drupal 10. That is useful when we need to implement content model changes.

26 - Entity type conversions. From nodes to user and taxonomy terms

In this article, we start implementing content model changes. Namely, we'll migrate Drupal 7 nodes as Drupal 10 user and taxonomy term entities. After covering entity ID and high water mark considerations, we will explain how to map data between two different entity types. We'll also show how to introspect the Drupal 10 site to determine which properties and fields are available in target entity types. Then, we'll write the two migrations by hand to practice what we have learned so far in the series. Finally, we'll share all properties and fields for users and taxonomy terms in our example to serve as reference in similar projects.

Entity ID and high water mark considerations

Let's begin with migrating nodes into users and taxonomy terms. In the previous article, we discussed the need to account for node IDs and revision IDs when deciding on the new AUTO_INCREMENT values for the two entities. No need to repeat that today. Just remember that when incorporating content model changes, multiple Drupal 7 entities and tables should be considered.

Also note that when changing entity types, it will not be possible to reuse the same identifier from the original entity in most cases. Imagine that a node with `nid` 3 in Drupal 7 belongs to a content type that will be converted to the user entity in Drupal 10. And in Drupal 7 we already had a user with `uid` 3. If we preserve user IDs, trying to migrate `nid` 3 on the already migrated `uid` 3 will cause data loss. This is extra problematic when there are entity references to preserve. This was discussed in article 4 and later expanded in article 5.

Two possible ways to avoid problems in this type of situation are:

1. **Do not migrate the identifier of the Drupal 7 entity** that will be migrated into a different Drupal 10 entity. In the scenario described above, we would not migrate the `nid` property of the nodes that will be converted to users. This is the approach we will follow in our example project.

2. **Apply an offset to the identifier of the Drupal 7 entity** that will be migrated into a different Drupal 10 entity. The offset should be high enough to avoid conflicts with the two entities at play. In the scenario described above, we could apply an offset of 1,000,000 so that `nid` 3 will be imported as `uid` 1,000,003. This would work under the assumption that, by the time we run the final migration prior to launch, there are less than 1,000,000 users in Drupal 7. When migrating revisionable entities, a different offset value might be required for the revision identifier. In the case of nodes, that is the version id (`vid`) property.

When we have a migration that involves an entity type change, the `high_water_property` property should be selected based on the original entity type from Drupal 7, not the new one in Drupal 10. This is because **the high water is a configuration option of the source plugin**. Our source is Drupal 7 nodes; therefore, we need to choose a node property to use as the high water. We will use the node revision ID (`vid`) as follow:

```
source:
 key: migrate
 plugin: d7_node
 high_water_property:
   name: vid
   alias: nr
```

Technical note: Per our migration plan, we do not need to migrate past node revisions. Because of this, we used the node classic migration when generating the migrations. This is being highlighted because the *node classic migration* and the *node complete migration* use different source plugins: d7_node and d7_node_complete, respectively. Different source plugins will expose different sets of fields. Not all of them are listed by the drush migrate:fields-source command. Debugging the migration is the best way to find all available fields.

Destination entity properties and fields

When implementing content model changes, it is important to have a good understanding of Drupal entities. As part of the process pipeline, we need to **set properties and fields in the Drupal 10 destination entity using data from the Drupal 7 entity retrieved by the source plugin**. In today's example, we will

set properties and fields for the Drupal 10 `user` and `taxonomy_term` entities using data from the Drupal 7 `node` entities.

But, how do you know what properties and fields need to be set? There are multiple ways:

1. Use the generated migrations (in the `ref_migrations` folder) to get a sense of what properties are expected based on the *destination plugin* of each migration.
2. Consult online references. This article has a list of properties for various Drupal core content entities. This other article has a list of properties for various Drupal Commerce content entities.
3. Ask the Drupal API. The commands below will extract data for your current Drupal 10 installation based on the modules that are enabled:

```
# Get all entities Drupal 10 entities in the current installation.
ddev drush php:eval "print_r(array_keys(\Drupal::entityTypeManager()->getDefi\
nitions()));"

# List only content entities.
# The use of single quotes to pass the code snippet to php:eval is important.
ddev drush php:eval 'print_r(array_keys(array_filter(\Drupal::entityTypeManag\
er()->getDefinitions(), fn($entity) => $entity->getGroup() === "content")));'

# List all properties and fields for an entity. Replace ENTITY_ID with the ma\
chine name of the entity. Example: 'node'.
# Only content entities are supported.
ddev drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manage\
r')->getFieldStorageDefinitions('ENTITY_ID')));"

# List all properties and fields for a bundle of an entity. Replace ENTITY_ID\
 and BUNDLE with the machine name of the entity and bundle, respectively. Exa\
mple: 'node' and 'article'.
# Only entity types that implement \Drupal\Core\Entity\FieldableEntityInterfa\
ce are supported.
ddev drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manage\
r')->getFieldDefinitions('ENTITY_ID', 'BUNDLE')));"

# List fields across bundles of an entity. Replace ENTITY_ID with the machine\
 name of the entity. Example: 'node'.
ddev drush php:eval "print_r(\Drupal::service('entity_field.manager')->getFie\
ldMap()['ENTITY_ID']);"
```

Knowing the names of an entity's properties and fields is the first step. You also need to know what type of data is expected. Is it a scalar value like an

integer, a string, or a boolean? Is it an array? If so, what is the structure of the array? When assigning values to fields, it is important to know their cardinality and what sub-fields are available. This article includes a list of sub-fields per field type. It is also useful to know what is the default sub-field, if one exists, for the field type.

Again, looking at the generated migrations and consulting online references can help with this. When in doubt, ask the Drupal API:

```
# Get a list of sub-fields. Replace ENTITY_ID and PROPERTY_OR_FIELD with the \
machine name of the entity and machine name of the property/field, respective\
ly. Example: 'node' and 'nid'/'body'.
ddev drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manage\
r')->getFieldStorageDefinitions('ENTITY_ID')['PROPERTY_OR_FIELD']->getColumns\
()));"

# Get details about the sub-field, including database base schema information.
ddev drush php:eval "print_r(\Drupal::service('entity_field.manager')->getFie\
ldStorageDefinitions('ENTITY_ID')['PROPERTY_OR_FIELD']->getColumns());"

# Get the default sub-field, if one exists for the field type.
ddev drush php:eval "var_dump(\Drupal::service('entity_field.manager')->getFi\
eldStorageDefinitions('ENTITY_ID')['PROPERTY_OR_FIELD']->getMainPropertyName(\
));"
```

Properties and fields for user and taxonomy term entities

When creating Drupal 10 entities, it is not necessary to provide a value for every property and field. In fact, in some cases we will not have suitable Drupal 7 data to populate some Drupa 10 properties and fields. In other cases, we will intentionally not set a destination property. In our example, we will not set the primary identifiers of the user and taxonomy term entities (`uid` and `tid`) to avoid potential ID conflicts as explained above.

You can get a list of available destination properties and fields for the `user` and `taxonomy_term` entities in your current installation execute with the following commands inside the `drupal10` folder:

```
ddev drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manage\
r')->getFieldStorageDefinitions('user')));"
ddev drush php:eval "print_r(array_keys(\Drupal::service('entity_field.manage\
r')->getFieldStorageDefinitions('taxonomy_term')));"
```

If you want to obtain even more details, open an interactive PHP shell executing `ddev drush php:cli` in the Drupal 10 folder. Then, run the following code:

```
// Get all property and field definitions for an entity. Pick one of the foll\
owing.
$field_storage_definitions = \Drupal::service('entity_field.manager')->getFie\
ldStorageDefinitions('user');
$field_storage_definitions = \Drupal::service('entity_field.manager')->getFie\
ldStorageDefinitions('taxonomy_term');

// Find more details about each property and field.
$field_storage_data = array_map(function ($field_definition) {
 $module = ($field_definition instanceof \Drupal\Core\Field\BaseFieldDefiniti\
on)
   ? $field_definition->getProvider()
   : $field_definition->get('module');
 $label = $field_definition->getLabel();
 $description = $field_definition->getDescription();

 if ($field_definition instanceof \Drupal\field\Entity\FieldStorageConfig) {
   $label = 'Field ' . $field_definition->getName();
   $description = 'Attached to bundle(s): ' . implode(', ', $field_definition\
->getBundles()) . '.';
 }

 return [
   'module' => $module,
   'type' => $field_definition->getType(),
   'label' => ($label instanceof \Drupal\Core\StringTranslation\TranslatableM\
arkup)
     ? $label->render()
     : $label,
   'description' => ($description instanceof \Drupal\Core\StringTranslation\T\
ranslatableMarkup)
     ? $description->render()
     : $description,
   'cardinality' => $field_definition->getCardinality(),
   'default_subfield' => $field_definition->getMainPropertyName(),
   'subfields' => array_keys($field_definition->getColumns())
     ?? $field_storage->getPropertyNames(),
 ];
}, $field_storage_definitions);
```

```

// Print the names of the entity's properties and fields.
array_keys($field_storage_definitions)

// Print details about the entity's properties and fields.
$field_storage_data

// Print details for a single property or field in the entity.
$field_storage_data['user_picture']

// The output of the $field_storage_data['user_picture'] is:
 [
   "module" => "image",
   "type" => "image",
   "label" => "Field user_picture",
   "description" => "Attached to bundle(s): user.",
   "cardinality" => 1,
   "default_subfield" => "target_id",
   "subfields" => [
     "target_id",
     "alt",
     "title",
     "width",
     "height",
   ],
 ]
```

Migrating nodes as taxonomy terms

We could use the `migrate_plus.migration.upgrade_d7_node_sponsor.yml` and `migrate_plus.migration.upgrade_d7_taxonomy_term_tags.yml` files in the `ref_migrations` as a reference to migrate Drupal 7 nodes into Drupal 10 taxonomy terms. We could also review the `upgrade_d7_taxonomy_term` migration we created in the previous article.

Instead, we will create the migration file from scratch. We have seen and customized many migrations already in the series. At this point, you should have gained familiarity with the structure of migration files.

As a reminder, we want to migrate nodes of type `sponsor` as taxonomy term entities. A corresponding `sponsor` Drupal 10 vocabulary was created back in article 15. Below is a summary of how Drupal 7 data will be migrated into Drupal 10:

- The node title will be migrated as the taxonomy term name.

- The description field will be migrated as the taxonomy term description.
- The logo field will be migrated into a newly created image field attached to the `sponsor` taxonomy vocabulary.

Create an `upgrade_d7_node_sponsor_to_taxonomy_term` file in the `web/modules/custom/tag1_migration/migrations` folder of our Drupal 10 project. The same file name we will use as the migration ID. You can come up with any name as long as it is unique. When creating migrations that involve entity type conversions, I recommend including the source and destination entity machine names in the migration ID.

Below is the content of the file:

```
id: upgrade_d7_node_sponsor_to_taxonomy_term
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - taxonomy_term
 - tag1_content
label: 'Nodes (Sponsor) to taxonomy terms'
source:
 key: migrate
 plugin: d7_node
 node_type: sponsor
 high_water_property:
   name: vid
   alias: nr
process:
 name:
   -
     plugin: get
     source: title
 description:
   -
     plugin: sub_process
     source: field_description
     process:
       value: value
       format:
         -
           plugin: static_map
           source: format
           map:
             filtered_html: restricted_html
           bypass: TRUE
 changed:
```

```
    -
      plugin: get
      source: changed
  langcode:
    -
      plugin: default_value
      source: language
      default_value: und
  field_logo:
    -
      plugin: sub_process
      source: field_logo
      process:
        target_id: fid
        alt: alt
        title: title
        width: width
        height: height
destination:
  plugin: 'entity:taxonomy_term'
  default_bundle: 'sponsors'
migration_dependencies:
  required:
    - upgrade_d7_file
    - upgrade_d7_taxonomy_term
  optional: { }
```

Note that the source plugin fetches nodes and the destination plugin is configured to create taxonomy terms. Then, in the process section, we map available Drupal 7 node data to Drupal 10 taxonomy term data.

Drupal 7 uses a rich text field for the description. This means that the `field_description` stores information about the text format used. As noted in article 22, in our example project text formats between Drupal 7 and 10 do not match verbatim. In the migration above, if the Drupal 7 `filtered_html` format is used, it will be migrated as the `restricted_html` in Drupal 10.

Text formats play an important role in the security of a Drupal website. They can filter out malicious markup that can be used to breach into a website and compromise their users. Security hardening is outside the scope of this series. Yet, we want to make sure you consider how text formats have an impact on security and incorporate that in your migration plan.

To specify the vocabulary a term belongs to, you can either use the `vid` property as we did in the `upgrade_d7_taxonomy_term` or use the specify the `default_bundle` property in the `entity:taxonomy_term` destination plugin.

If both are set, the `vid` property will take precedence.

Before executing the `upgrade_d7_node_sponsor_to_taxonomy_term` migration, make sure to account for potential entity ID conflicts. This is particularly important when a migration performs content model changes. We covered this in great detail in article 23. You can use the AUTO_INCREMENT Alter module with the configuration from the start of this article.

Now, rebuild caches for our new migration to be detected and execute it. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_sponsor_to_taxonomy_term
ddev drush migrate:import upgrade_d7_node_sponsor_to_taxonomy_term
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/structure/taxonomy/manage/spons` and look at the list of migrated taxonomy terms.

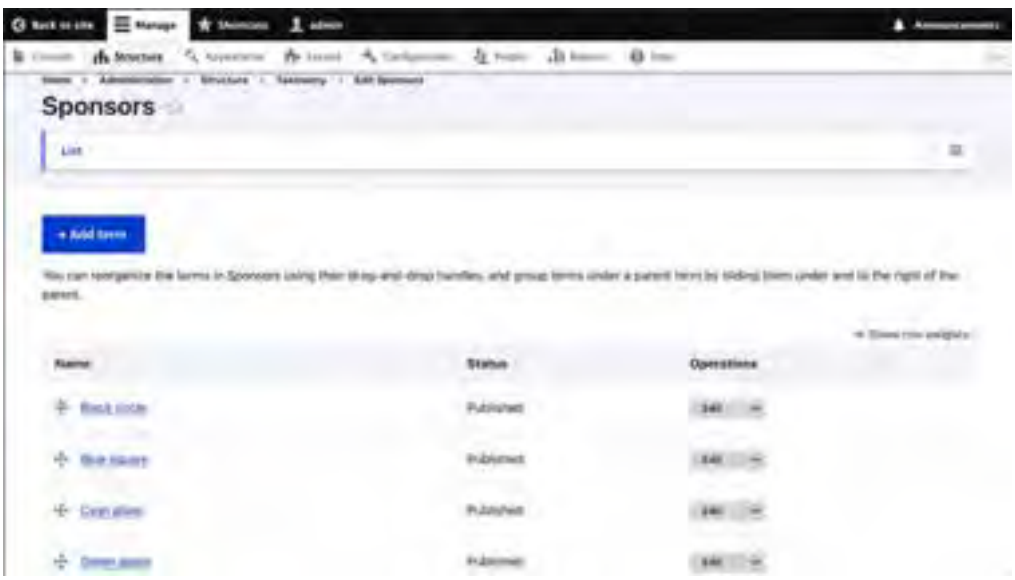


Figure 27. Migrated Taxonomy Terms

Migrating nodes as users

We could use the `migrate_plus.migration.upgrade_d7_node_speaker.yml` and `migrate_plus.migration.upgrade_d7_user.yml` files in the `ref_migrations` as a reference to migrate Drupal 7 nodes into Drupal 10 taxonomy terms. We could also review the `upgrade_d7_user` migration we created in the previous article. However, for this example, we will create the migration file from scratch again. It is good practice and will help us get a better understanding of the system.

As a reminder, we want to migrate nodes of type `speaker` as user entities. In Drupal 7, this content type has many fields attached to it. The corresponding Drupal 10 fields were added to the user entity in article 22 by applying recipes and manually creating some. Below is a summary of how Drupal 7 data will be migrated into Drupal 10:

- The node title will be migrated as the username (`name` property).
- The `field_email` field will be migrated as the user email (`mail` property).
- The `field_profile_picture` field will be migrated into the `user_picture` image field.
- The `field_website` and `field_biography` fields will be migrated into corresponding Drupal 10 fields with the same name.
- The `field_drupal_org_profile`, `field_linkedin_profile`, and `field_x_twitter_profile` will be combined into a single `field_social_media_links` field of type *social links*.
- The `field_favorite_quote` field collection will be migrated into the `field_favorite_quote` *paragraph*.

The last two elements of the list above will be covered in the next article. The rest will be addressed below. Additionally, all users migrated from speaker nodes will get the `Speaker` user role we created in article 22.

Unpublished nodes of type `speaker` will not be migrated. Also note that even though we are creating user entities, we do not have suitable password information in Drupal 7. After the migration, individual users can trigger a password reset operation or administrators can force this operation in bulk.

Create an `upgrade_d7_node_speaker_to_user` file in the `web/modules/custom/tag1_migration/migrations` folder of our Drupal 10 project. The same file name we will use as the migration ID. Below is the content of the file:

```yaml
id: upgrade_d7_node_speaker_to_user
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - user
 - tag1_content
label: 'Nodes (Speaker) to user accounts'
source:
 key: migrate
 plugin: d7_node
 node_type: speaker
 high_water_property:
   name: vid
   alias: nr
process:
 name:
   -
     plugin: get
     source: title
 mail:
   -
     plugin: sub_process
     source: field_email
     process:
       value: email
 created:
   -
     plugin: get
     source: created
 changed:
   -
     plugin: get
     source: changed
 status:
   -
     plugin: skip_on_empty
     source: status
     method: row
     message: 'Node was not migrated because it is unpublished.'
 init:
   -
     plugin: get
     source: '@mail'
 roles:
   -
     plugin: default_value
     default_value: [speaker]
 user_picture:
   -
```

```
      plugin: sub_process
      source: field_profile_picture
      process:
        target_id: fid
        alt: alt
        title: title
        width: width
        height: height
  field_biography:
    -
      plugin: get
      source: field_biography
  field_website:
    -
      plugin: sub_process
      source: field_website
      process:
        uri: value
        title: title
        options: attributes
destination:
 plugin: 'entity:user'
migration_dependencies:
 required:
   - upgrade_d7_file
 optional: {  }
```

Note that the source plugin fetches nodes and the destination plugin is configured to create users. Then, in the process section, we map available Drupal 7 node data to Drupal 10 user data.

This example uses different concepts and techniques we have covered throughout the series. We do not want to repeat ourselves too much, but here is a list of highlights from our example:

- **Some Drupal 7's fields use different property names to hold data compared to their Drupal 10's counterparts.** In such cases, the sub_process process plugin can be used to account for such changes in property names. This was needed to migrate Drupal 7's `Email` and `URL` fields into Drupal 10's `mail` property and `Link` field respectively.
- **The source and destination entity might share a property with the same name, but different meanings.** In this example, the `status` property exists both in Drupal 7 nodes and in Drupal 10 users. For nodes, `status` indicates the publication status: published or unpublished. For users, the `status` indicates whether the user account is active or blocked. In

our case, we use the `status` as retrieved from the node to determine if the record should be migrated or not using the skip_on_empty process plugin. When the node is published, it stores a value of `1`, which is sent to the destination `status` property meaning the account is active. *Reusing a property with the same will not always be possible.* It will depend on the meaning between the two entities and the process plugin chain used to assign the destination property.

- **The `init` user property stores the email address used for initial account creation.** We are reusing the value already assigned in the `mail` destination property to set the `init` property.
- **The `roles` user property expects a flat array array structure with the machine names of the roles the user will be assigned.** We use the default_value process plugin to assign the `Speaker` roles.

Before executing the `upgrade_d7_node_speaker_to_user` migration, make sure to account for potential entity ID conflicts as mentioned above. Now, rebuild caches for our new migration to be detected and execute it. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_speaker_to_user
ddev drush migrate:import upgrade_d7_node_speaker_to_user
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/people?role=speaker` and look at the list of migrated users.

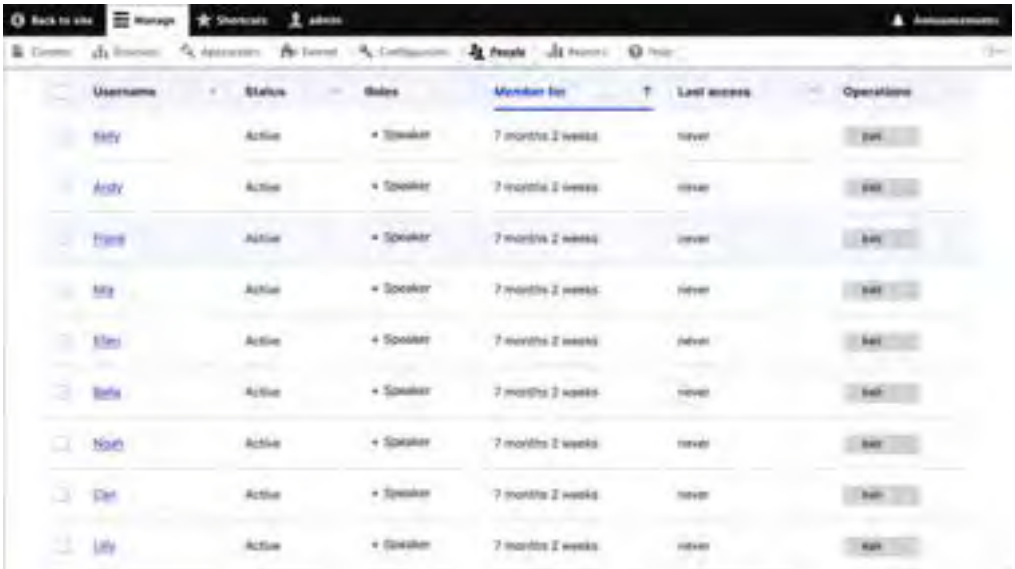


Figure 28. Migrated Users

Entity properties and fields for user and taxonomy terms entities

For reference, below is a list of the entity properties and fields attached to user and taxonomy terms based on the modules enabled in our example project.

The following are properties and fields in the `user` entity:

1. `uid` (type: `integer`): User ID. The user ID.
2. `uuid` (type: `uuid`): UUID. The user UUID.
3. `langcode` (type: `language`): Language code. The user language code.
4. `preferred_langcode` (type: `language`): Preferred language code. The user's preferred language code for receiving emails and viewing the site.
5. `preferred_admin_langcode` (type: `language`): Preferred admin language code. The user's preferred language code for viewing administration pages.
6. `name` (type: `string`): Name. The name of this user.
7. `pass` (type: `password`): Password. The password of this user (hashed).
8. `mail` (type: `email`): Email. The email of this user.
9. `timezone` (type: `string`): Timezone. The timezone of this user.

10. `status` (type: `boolean`): User status. Whether the user is active or blocked.
11. `created` (type: `created`): Created. The time that the user was created.
12. `changed` (type: `changed`): Changed. The time that the user was last edited.
13. `access` (type: `timestamp`): Last access. The time that the user last accessed the site.
14. `login` (type: `timestamp`): Last login. The time that the user last logged in.
15. `init` (type: `email`): Initial email. The email address used for initial account creation.
16. `roles` (type: `entity_reference`): Roles. The roles the user has.
17. `default_langcode` (type: `boolean`): Default translation. A flag indicating whether this is the default translation.
18. `field_biography` (type: `string_long`): Field field_biography. Attached to bundle(s): user.
19. `field_favorite_quote` (type: `entity_reference_revisions`): Field field_favorite_quote. Attached to bundle(s): user.
20. `field_social_media_links` (type: `social_links`): Field field_social_-media_links. Attached to bundle(s): user.
21. `field_website` (type: `link`): Field field_website. Attached to bundle(s): user.
22. `user_picture` (type: `image`): Field user_picture. Attached to bundle(s): user.

The following are properties and fields in the `taxonomy_term` entity:

1. `tid` (type: `integer`): Term ID. The term ID.
2. `uuid` (type: `uuid`): UUID. The term UUID.
3. `revision_id` (type: `integer`): Revision ID.
4. `langcode` (type: `language`): Language. The term language code.
5. `vid` (type: `entity_reference`): Vocabulary. The vocabulary to which the term is assigned.
6. `revision_created` (type: `created`): Revision create time. The time that the current revision was created.
7. `revision_user` (type: `entity_reference`): Revision user. The user ID of the author of the current revision.

8. `revision_log_message` (type: `string_long`): Revision log message. Briefly describe the changes you have made.
9. `status` (type: `boolean`): Published.
10. `name` (type: `string`): Name.
11. `description` (type: `text_long`): Description.
12. `weight` (type: `integer`): Weight. The weight of this term in relation to other terms.
13. `parent` (type: `entity_reference`): Term Parents. The parents of this term.
14. `changed` (type: `changed`): Changed. The time that the term was last edited.
15. `default_langcode` (type: `boolean`): Default translation. A flag indicating whether this is the default translation.
16. `revision_default` (type: `boolean`): Default revision. A flag indicating whether this was a default revision when it was saved.
17. `revision_translation_affected` (type: `boolean`): Revision translation affected. Indicates if the last edit of a translation belongs to current revision.
18. `field_logo` (type: `image`): Field field_logo. Attached to bundle(s): sponsors.

Next time, we'll update this node to user migration to populate the `field_favorite_quote` and `field_social_media_links` fields. This will require migrating paragraphs and creating a custom process plugin. Stay tuned.

27 - Paragraph migration. Creating custom process plugins

In the previous article, we started working on a node to user migration. Today, we expand on that example to accommodate more content model changes. First, we will learn how to migrate Drupal 7 field collections into Drupal 10 paragraphs. Then, we'll populate an entity reference revision field attached to the user entity to add relationships to the newly migrated paragraph entities. Finally, we will learn how to create a custom process plugin to combine three separate Drupal 7 fields into a single multi-value Drupal 10 field.

Entity ID and high water mark considerations

Even though today's migration will create paragraph entities in Drupal 10, the source data is Drupal 7 field collections. Note that while the paragraph module existed in Drupal 7, it was not common for a site to use field collections and paragraphs in the same installation. If your project makes use of both and you need to migrate the data of the two modules, you will have to review more Drupal 7 tables to determine a suitable value for the AUTO_INCREMENT value of Drupal 11's paragraph entity.

Our example only makes use of the field collections. From the `drupal7` folder, execute `ddev mysql` to open an interactive SQL shell. Then execute the following queries to help you determine which values to use for the AUTO_INCREMENT value:

```
-- Get the highest field collection id value.
SELECT item_id FROM field_collection_item ORDER BY item_id DESC LIMIT 1;

-- Get the highest field collection revision value.
SELECT revision_id FROM field_collection_item_revision ORDER BY revision_id D\
ESC LIMIT 1;
```

For brevity, we are going to show how to configure the `auto_increment_alter_content_entities` setting of the AUTO_INCREMENT Alter module

to apply the new AUTO_INCREMENT value. Refer to article 23 for more information on how this module works or how to perform the operation in custom code.

```
$settings['auto_increment_alter_content_entities'] = [
  'paragraph' => [500, 1500], // Alter the tables for the paragraph content e\
ntity.
];
```

Now, execute the command provided by the AUTO_INCREMENT Alter module to trigger the alter operation in the Drupal 10 project.

```
ddev drush auto-increment-alter:content-entities
```

As for the high water property, we need to identify one field returned by the source plugin whose value always increases. Paragraphs are revisionable entities. The automated upgrade path for paragraphs generates two content migrations for each field collection in the source site. One migrates the current revision and another migrates all past revisions. They use the d7_field_collection_item and d7_field_collection_item_revision source plugins respectively. Both source plugins retrieve data from the `field_collection_item` and `field_collection_item_revision` tables in Drupal 7.

Both migrations can use the `revision_id` column in the revision table as the high water mark as follows:

```
source:
 plugin: d7_field_collection_item
 high_water_property:
   name: revision_id
   alias: fr
```

Migrating field collections as paragraphs

The Paragraphs module offers an upgrade path for Drupal 7's Field collection module. Back in article 15, we explained how to migrate **paragraph types** out of field collections. As the name suggests, Drupal 7 field collections have fields attached to them. Adding those fields to the paragraphs entities was accounted

for in the multiple field-related migrations: storage, instance, widget, and formatters. With the migration of configuration taken care of, we can now focus on migrating content.

Paragraphs are attached to entity bundles (like content types) via *entity reference revision fields*. Because of this, **migrating paragraph content is a two step process**. First, you need to **migrate the paragraph entity data**. Second, you need to **migrate the entity reference revision data** that connects the paragraph with its host entity. Before we start writing migrations, let's review our Drupal 7 configuration and devise an approach to follow based on our migration plan.

In Drupal 7, we have the `field_favorite_quote` field collection which has two fields attached to it: `field_quote_name` and `field_quote_message`. The `field_favorite_quote` field collection is used in the `speaker` content type. In Drupal 10, the `field_favorite_quote` *field collection* was migrated as the `favorite_quote` *paragraph type* with the `field_quote_name` and `field_quote_message` still attached to it. Our migration plan says that `speaker` nodes should be migrated as Drupal 10 `user` entities. Back in article 22, we attached a `field_favorite_quote` *entity reference revisions field* to users, which allows referencing `favorite_quote` paragraphs entities.

There are two tasks we still need to complete: migrate the paragraphs entities and update the user migration to populate the `field_favorite_quote` reference field.

Paragraphs are revisionable entities. As part of the upgrade path from field collections, the paragraph module provides two content migrations:

- d7_field_collection.yml migrates the **current paragraph revision**.
- d7_field_collection_revisions.yml migrates **all past paragraph revisions**.

Each of these migrations use a deriver that generates a migration file for each Drupal 7 field collection. This is similar to how node migrations work – a separate migration is generated for each content type. In our example, after running the automatted migration, we ended up with the `upgrade_d7_field_collection_favorite_quote.yml` and `upgrade_d7_field_collection_revisions_favorite_quote.yml` migrations in the `ref_migrations` folder. Our migration plan says we do not need to migrate revisions for any content entity. Therefore, we will only migrate the current paragraphs revisions.

We use `upgrade_d7_field_collection_favorite_quote` to migrate the current revision for the one field collection that existed in Drupal 7. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_field_collection_favorite\
_quote.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_field_collection_fa\
vorite_quote.yml
ddev drush cache:rebuild
```

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `paragraph` and `tag1_content`.
- Add `key: migrate` under the source section.
- Add the `high_water_property` property as demonstrated above.
- Remove the migration dependencies. They currently list configuration migration. Early on we decided that our content migration will not depend on configuration migrations.

After the modifications, the `upgrade_d7_field_collection_favorite_-quote.yml` file should look like this:

```yaml
id: upgrade_d7_field_collection_favorite_quote
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - 'Field Collection Content'
 - paragraph
 - tag1_content
label: 'Field Collections (Favorite quote)'
source:
 key: migrate
 plugin: d7_field_collection_item
 field_name: field_favorite_quote
 high_water_property:
```

```
    name: revision_id
    alias: fr
process:
 type:
   -
     plugin: get
     source: bundle
 parent_id:
   -
     plugin: get
     source: parent_id
 parent_type:
   -
     plugin: get
     source: parent_type
 parent_field_name:
   -
     plugin: get
     source: field_name
 field_quote_name:
   -
     plugin: get
     source: field_quote_name
 field_quote_message:
   -
     plugin: get
     source: field_quote_message
destination:
 plugin: 'entity_reference_revisions:paragraph'
 default_bundle: favorite_quote
migration_dependencies:
 required: {  }
 optional: {  }
```

The generated migration has the `Field Collection Content` tag. This is important and will be explained in the next section. For now, make sure to preserve it.

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_field_collection_favorite_quote
ddev drush migrate:import upgrade_d7_field_collection_favorite_quote
```

If things are properly configured, you should not get any errors. But where

can you see the migrated paragraph entities? Out of the box, the paragraphs module does not provide a way to review their content entities from the user interface. Paragraphs are meant to be attached to a host entity via a reference field. It is by viewing the host entity that you can see the content of the paragraphs. The `upgrade_d7_field_collection_favorite_quote` migration generated content entities of the `favorite_quote` paragraph type. In our example, we will update the user migration to be able to see the migrated `favorite_quote` paragraphs referenced by the `field_favorite_quote` field.

In the meantime, there are other ways to make sure our paragraph migration created Drupal 10 content entities. One is to create a view that lists paragraph entities. This will be left as an exercise for the curious reader. Another is to run SQL queries against the Drupal 10 tables that store paragraph data. This is what we will do using a MySQL client.

From the `drupal10` folder, execute `ddev mysql` to open an interactive SQL shell. Then execute the following queries:

```sql
-- Query paragraph entity data.
SELECT * FROM paragraphs_item;
SELECT * FROM paragraphs_item_field_data;

-- Query field data attached to paragraph entities.
SELECT * FROM paragraph__field_quote_name;
SELECT * FROM paragraph__field_quote_message;
```

Technical note: Even though we are not migrating past revisions, the entity API still generates data in the revisions table for the paragraph entity and the fields attached to it. This is how Drupal works out of the box. That is why it is important to provide AUTO_INCREMENT values for revisions when the content entity supports it, even if you have no plans to migrate revisions.

Connecting paragraphs to their host entities

After confirming the field collection to paragraph migration worked, we need to update the migration of the host entity. In particular, we need to populate the references in the user entity to the newly migrated paragraph entities. To accomplish this, we need to update the `upgrade_d7_node_speaker_to_user` migration created in the previous article.

Before showing how to migrate the relationship to the paragraph entities, I would like to acknowledge that our example is a rather simple one. Paragraphs migrations can get quite complex when you need to migrate revision, translations, and nested paragraph relationships. When coming from field collections in Drupal 7, the automated upgrade path provided by the paragraph module is quite flexible and offers tools to account for many different scenarios.

That said, it is valid to extend migration plugins provided by the paragraphs module or write custom ones altogether. It is our hope that throughout the series you have gained a deeper understanding of the Migrate API to plan and execute custom migrations.

The one strategy I would advise against is to create paragraph entities on the fly when migrating their host entities. That violates the principles of ETL and the created entities might end up in the site even after clean up operations, like migration rollbacks. Instead, create separate migrations for each entity/bundle combination and add the relationships among them following an approach similar to what we are about to describe.

Back to our example, the generated migration already contains a process pipeline we can use to establish the relationship between paragraphs and their host entities. But where? In Drupal 7, the `speaker` content type uses the `field_favorite_quote` field collection. So, take a look at the `upgrade_d7_node_speaker` migration in the `ref_migrations` folder. The relevant part is the mapping of the `field_favorite_quote` field in the `process` section. Copy the snippet below from the generated `upgrade_d7_node_speaker` migration into the `upgrade_d7_node_speaker_to_user` migration we created in the previous article:

```
process:
 field_favorite_quote:
   -
     plugin: sub_process
     source: field_favorite_quote
     process:
       target_id:
         -
           plugin: paragraphs_lookup
           tags: 'Field Collection Content'
           source: value
         -
           plugin: extract
           index:
```

```
            - id
      target_revision_id:
        -
          plugin: paragraphs_lookup
          tags:
            - 'Field Collection Revisions Content'
            - 'Field Collection Content'
          tag_ids:
            'Field Collection Revisions Content':
              - revision_id
            'Field Collection Content':
              - value
        -
          plugin: extract
          index:
            - revision_id
```

Now, rebuild caches for our changes to be detected. Then rollback the `upgrade_d7_node_speaker_to_user` migration and import it again.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_speaker_to_user
ddev drush migrate:rollback upgrade_d7_node_speaker_to_user
ddev drush migrate:import upgrade_d7_node_speaker_to_user
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/people?role=speaker` and view or edit any of the migrated users with the `Speaker` role assigned to them. Clicking on the node for `Frank` will reveal a quote by Albert Einstein.

Wait, what just happened? Paragraph relationships use **entity reference revision fields**. To properly connect a paragraph field from the host entity to the paragraph entity, you need to *make sure to set values for the `target_id` and `target_revision_id` sub-fields.

In the snippet above, we are letting the automated upgrade path provided by the paragraph module do its job. It offers the paragraphs_lookup process plugin, which extends Drupal core's migration_lookup with extra functionality tailored to migrating data for entity reference revision fields. When paragraph entities are found, the process pipeline extracts their `id` and `revision_id` to populate the `target_id` and `target_revision_id` sub-fields of the `field_favorite_quote` field in the `user` entity. That is how the relationship between users and paragraphs is established.

Note the use of extra migration tags in the derived `d7_field_collection` and `d7_field_collection_revisions` migrations: `Field Collection Content` and `Field Collection Revisions Content` respectively. This limits the migrations that will be used in the lookup operation to those containing the listed tags. Feel free to review the ParagraphsLookup process plugin's code to better understand how the process pipeline is set up.

Migrate process plugin

Migrate process plugins are responsible for transforming source data into the format expected by the destination system. In our case, we are converting Drupal 7 data into a Drupal 10 suitable format. From a technical point of view, they leverage Drupal's Plugins API.

Below are some highlights regarding their implementation:

- Migrate process plugins are classes in the `Drupal\[module]\Plugin\migrate\proces` namespace and implement the MigrateProcessInterface interface.
- The ProcessPluginBase base class is available for convenience. It implements common methods invoked for process plugins.
- Most process plugins implement a transform method that takes care of the data manipulation operation. If such a method does not exist in the class, the process pipeline invoking the process plugin should specify a `method` key indicating which method in the process plugin's class should be executed. An example of this is the skip_on_empty which can be called using the string `row` or `process` as the value of `method` configuration option.
- If you need to inject a service, the plugin needs to implement the ContainerFactoryPluginInterface interface and its create method. Examples of this are the migration_lookup and machine_name process plugins.
- For discovery, they use PHP attributes. Before version 10.2 Drupal core used annotations for discovery. Conversions to PHP attributes have started and the plan is to eventually deprecate annotations altogether.
- A process plugin can signal that it supports handling multiple values by setting `handle_multiples` in its annotation. When set to `TRUE`, the plugin will expect an array as input and iterate over it, potentially changing the whole array. Examples of this are the sub_process and flatten process plugins. Conversely, a process plugin can signal that

its return value requires multiple handling by returning `TRUE` from the multiple method of the plugin class. Examples of this are the sub_process and explode process plugins.

- The plugin.manager.migrate.process service is the plugin manager responsible for process plugins. You can use it to get a list of available plugins based on enabled modules and obtain more details about their definition.

```
# List of migrate process plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.process')->getDefinitions()));"

# Details on a specific migrate process plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.process\
')->getDefinitions()['PROCESS_PLUGIN_ID']);"
```

Creating a custom process plugin

We are almost done with the speaker to user migration. The missing piece is migrating the `field_drupal_org_profile`, `field_linkedin_profile`, and `field_x_twitter_profile` URL fields in Drupal 7's `speaker` content type into a single Drupal 10 `field_social_media_links` social link field that accepts multiple values.

Back in article 16, we had a deep dive into how Drupal fields work – both from the perspective of PHP code and database tables. With regards to our task today, I explain how to combine multiple Drupal 7 URL fields into a single Drupal 10 social links field in this presentation. It covers a lot of ground from the technical side of things. I highly recommend revisiting article 16 and watching the video recording to have a better understanding of what we will do today.

In Drupal 7, the URL field has 3 sub-fields: `value`, `title`, `attributes`. In our example, the URL fields were configured to only store the URL value. As such, only the `value` sub-field is populated. An example value is: https://www.drupal.org/u/baltowen In Drupal 10, the social links field has 2 sub-fields: `social` and `link`. A single entry pointing to the same Drupal.org profile would store the values as `drupal` and `baltowen`, respectively.

We need to come up with a process pipeline that takes multiple URLs as stored in Drupal 7, breaks them into the platform/handle format used in Drupal 10, and returns the data to populate a multi-value social link field. *A custom process plugin is ideal for this scenario.*

Create a PHP file named `Tag1SocialLinks.php` inside our `tag1_migration` custom module's `/src/Plugin/migrate/process` folder. The path relative to the Drupal 10's project docroot is `web/modules/custom/tag1_migration/src/Plugin/migrate/process/Tag1SocialLinks.php`. The content of the file should be:

```php
<?php

namespace Drupal\tag1_migration\Plugin\migrate\process;

use Drupal\migrate\Attribute\MigrateProcess;
use Drupal\migrate\MigrateExecutableInterface;
use Drupal\migrate\ProcessPluginBase;
use Drupal\migrate\Row;

#[MigrateProcess(
  id: "tag1_social_links",
  handle_multiples: TRUE,
)]
class Tag1SocialLinks extends ProcessPluginBase {

  /**
   * {@inheritdoc}
   */
  public function transform($value, MigrateExecutableInterface $migrate_execu\
table, Row $row, $destination_property) {
    if (!is_array($value)) {
      $value = [$value];
    }

    $url_values = array_filter($value);

    $social_link_patterns = [
      'drupal' => '/^(http(s)?\:\/\/)?(www\.)?drupal\.org\/u\//',
      'twitter' => '/^(http(s)?\:\/\/)?(www\.)?(twitter|x)\.com\//',
      'linkedin' => '/^(http(s)?\:\/\/)?(www\.)?linkedin\.com\//',
    ];

    $result = [];
    foreach ($url_values as $url_value) {
      foreach ($social_link_patterns as $social_link_platform => $social_link\
_regex) {
        if (preg_match($social_link_regex, $url_value)) {
```

```
38            $result[] = [
39              'social' => $social_link_platform,
40              'link' => preg_replace($social_link_regex, '', $url_value),
41            ];
42          }
43        }
44      }
45
46      return $result;
47    }
48
49  }
```

Then, update the `upgrade_d7_node_speaker_to_user` migration with the following snippet in the process section:

```
1  process:
2   field_social_media_links:
3     -
4       plugin: tag1_social_links
5       source:
6         - field_drupal_org_profile/0/value
7         - field_linkedin_profile/0/value
8         - field_x_twitter_profile/0/value
```

Note that in our `upgrade_d7_node_speaker_to_user` migration, we are calling the `tag1_social_links`. This should match the `id` of the `MigrateProcess` PHP attribute used in our custom process plugin.

Now, rebuild caches for our new process plugin and the changes to the migration to take effect. Then rollback the `upgrade_d7_node_speaker_to_user` migration and import it again.

```
1  ddev drush cache:rebuild
2  ddev drush migrate:status upgrade_d7_node_speaker_to_user
3  ddev drush migrate:rollback upgrade_d7_node_speaker_to_user
4  ddev drush migrate:import upgrade_d7_node_speaker_to_user
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/people?role=speaker` and view or edit any of the migrated users with the `Speaker` role assigned to them. Clicking on the node for `Mia` will reveal links to her Drupal.org profile, LinkedIn, and X (Twitter).

A detailed review of the PHP code for the custom process plugin will be left as an exercise to the curious reader. What we want to highlight is the relationship between the data transformation logic and the data that is passed to the plugin itself. From our migration, we pass an array of 3 values: `field_drupal_org_profile/0/value`, `field_linkedin_profile/0/value`, and `field_x_twitter_profile/0/value`. Each of them attempts to retrieve the URL we are interested in processing by retrieving the `value` sub-field from the first delta. Do we need to extract the URL before sending data to the process plugin? Not necessarily. We could have extracted the URLs within the process plugin itself. This serves as a good reminder that *when implementing custom migration logic, we are the ones who decide what each component is responsible for doing*.

The above code will send an array with the following structure to our `tag1_social_links` custom process plugin:

```
[
 'https://drupal.org/u/baltowen',
 'https://www.linkedin.com/in/wendybaltodano',
 'https://x.com/baltowen',
]
```

After calling the `transform` method, our `tag1_social_links` custom process plugin will return an array with the following structure:

```
[
 0 => [
   'social' => 'drupal',
   'link' => 'baltowen',
 ],
 1 => [
   'social' => 'linkedin',
   'link' => 'in/wendybaltodano',
 ],
 2 => [
   'social' => 'twitter',
   'link' => 'baltowen',
 ],
]
```

This array will be stored in the `field_social_media_links` destination property. When the `entity:node` destination plugin calls the entity save

operation, the `field_social_media_links` field on the `user` entity will be populated with profile information for the three social media platforms.

Writing custom process plugins can make migrations easier to read and maintain over time. That said, make sure you are familiar with plugins that are available to avoid reinventing the wheel. This documentation page includes a list of process plugins in Drupal core and some contributed modules that are commonly used in custom migrations. The list is not exhaustive as it would be impractical to list all plugins across all contributed modules. When in doubt about what is available ask the Drupal API with the snippet we shared in the `Migrate process plugin` section above.

Next time, we'll learn to write custom source plugins. We will also walk you through populating regular entity revision fields; that is, fields that unlike paragraphs do not need to point to a specific revision of the referenced entity. All of this will be explained in the context of migrating a new entity type: media. Let's go!

© 2024 - 2025 Tag1 Consulting, Inc. | tag1consulting.com | Drupal Migration Experts

28 - Migrating media. Creating custom source plugins

In the previous article, we learned how to migrate paragraphs and create custom process plugins. Good exercise for the brain. Today, we will do some exercises for the body. Get ready for a strength training session – Drupal style – where we will learn about creating custom source plugins, extending existing ones, and writing media migrations from scratch.

Grab a bottle of water and a towel. Let's go!

Entity ID and high water mark considerations

To warm up, we will do some shoulder rolls. While doing so let's revisit entity IDs and high water mark considerations. We covered file migrations back in article 24. Much of what was presented there applies today. The Tag1 Team Talks podcast on file and media migrations also contains lots of useful information for today's topic. To avoid repeating ourselves, we'll summarize what you need to take into account to prevent entity ID conflicts and choose a high water mark.

You need to know which modules were used in the source site to provide media related functionality. Our example project uses Core's `file` and `image` modules plus the YouTube Field module to store references to external videos. Other Drupal 7 projects might use the File Entity module, which makes the file entity fieldable and allows it to have bundles similar to Drupal 10 media entities. Still other projects might use the D7 Media module and its vast ecosystem of related modules to provide a rich media management experience.

The reason we bring this up is because depending on how media was implemented in Drupal 7 and the content model you want to pursue in Drupal 10, you will have to look at different tables to determine the new AUTO_INCREMENT value for the media entity. Our Drupal 7 example project is relatively simple so checking the `file_managed` table will suffice. We showed how to do this in article 24.

For brevity, we are going to show *how* to configure the `auto_increment_alter_content_entities` setting of the AUTO_INCREMENT Alter module to apply the new AUTO_INCREMENT value. Refer to article 23 for more information of how this module works or how to perform the operation in custom code.

```
$settings['auto_increment_alter_content_entities'] = [
  'media' => [350], // Alter the tables for the media content entity.
];
```

Now execute the command provided by the AUTO_INCREMENT Alter module to trigger the alter operation in the Drupal 10 project.

```
ddev drush auto-increment-alter:content-entities
```

As for the high water property, it will depend on *where* your data comes from. Today's example includes two different sources:

- **Field API tables**. As we are going to discuss later, Drupal 7 fields use two tables: one for current revision data and another for past revision data. In theory, the `revision_id` column could be used as the high water mark. That said, the table for the current revision allows `NULL` values for the `revision_id` column, making it less than ideal for our purposes. Many times there will be a value, but its presence is not enforced at the database level. For simplicity, we will not define a high water mark in the migrations that read data directly from field API tables.
- **File entity tables**. For this, we can use the same high water mark configuration used in the `upgrade_d7_file` and `upgrade_d7_file_private` migrations:

```
source:
 key: migrate
 plugin: tag1_media_image
 high_water_property:
   name: fid
   alias: f
```

Migrate source plugin

Take some lightweight dumbbells. I'm using 10 pounds, approximately 4.5 kilos. To start, we will do three rounds of bicep curls. In between each round, we will casually talk about migrate source plugins.

Migrate source plugins are responsible for fetching data from one of many supported data repositories. In our example project, all migrations retrieve data from a Drupal 7 database. Other supported sources are: JSON and XML files, CSV files, Excel and LibreOffice files, etc. From a technical point of view, they leverage Drupal's Plugins API.

Below are some highlights regarding their implementation:

- Migrate source plugins are classes in the `Drupal\[module]\Plugin\migrate\source` namespace that implement the MigrateSourceInterface interface.
- The SourcePluginBase base class is available for convenience. It implements common methods invoked for source plugins.
- Many source plugins implement a prepareRow method that can add, edit, or delete data retrieved from the source. It is possible to use the retrieved data to further query the source and fetch extra information. For example, fieldable entities use this method to attach Drupal 7 Field API data to the entity being fetched. The `prepareRow` method can also be used to instruct that a record should not be processed by returning `FALSE`.
- If you need to inject a service, the plugin needs to implement the ContainerFactoryPluginInterface interface and its create method. Examples of this are the SqlBase, d7_node and spreadsheet. Notice that while `d7_node` does not explicitly implement the interface in its class declaration, it still can implement/overwrite the methods of the interface defined in one of its parent classes.
- For discovery, they still use Doctrine annotations at the time of publishing this article. When this Drupal core issue is committed, they will use PHP attributes for discovery of migrate source plugins.

- Many source plugins related to Drupal 7 to 10 migrations extend the SqlBase class and implement the `query` method. This lets you alter the SQL query used to retrieve data from the source database. You can add conditions, join tables, expand the list of fields to fetch, and much more. A common use case is limiting the number of records to retrieve. For example, you can decide only published nodes will be migrated to Drupal 10, leaving any unpublished content behind. This will be presented in the next article.
- The Migrate Drupal module offers a lot of functionality related to migrating data from Drupal 6 and 7. It even provides a way to introspect the current Drupal 10 installation to fetch content and configuration data via the content_entity and config source plugins respectively.
- The plugin.manager.migrate.source service is the plugin manager for migrate source plugins. You can use it to get a list of available source plugins and obtain more details about their definition.

```
# List of migrate source plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ate.source')->getDefinitions()));"

# Details on a specific migrate source plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migrate.source'\
)->getDefinitions()['SOURCE_PLUGIN_ID']);"
```

Creating custom source plugins from scratch

Put those dumbbells aside for a moment, because we are going to do body-weight exercise. Let's do three rounds of push-ups, 45 seconds each. Go at your own pace and feel free to kneel if necessary. When done, come back to learn how to create custom source plugins from scratch.

Per our migration plan, we want to migrate data stored in YouTube fields in Drupal 7 as remote video media entities in Drupal 10. In practice, there is only one field that would undergo this transformation: `field_video_recording`. The YouTube field module is available for Drupal 10 and provides an automated upgrade path via a field migration plugin. That said, reading field data to create media entities is a great example to learn how to create a custom source plugin and incorporate content model changes as part of the upgrade process.

Back in article 16, we had a deep dive into understanding how Drupal fields work, both from the perspective of PHP code and database tables. I also recommend watching this presentation that covers multiple examples of how to perform content model changes when migrating from Drupal 7.

In Drupal 7, YouTube fields have 2 sub-fields: `input` and `video_id`. Input stores the URL as submitted by the user. Example values are:

- `https://www.youtube.com/watch?v=HMYpxm-2o4c`
- `https://youtu.be/HMYpxm-2o4c`

Notice that both URLs link to the same video, but follow a different URL pattern. The `video_id` sub-field calls the youtube_get_video_id function to extract the video ID out of multiple accepted URL formats as defined in the module's README.txt file. For these two video URLs, the `video_id` value would be the same: `HMYpxm-2o4c`. When migrating data into Drupal 10 we will deduplicate records like these. Namely, we will only migrate unique video ID values.

In Drupal 10, a remote video media entity uses a plain text field to store the URL to the video: `field_media_oembed_video`. Plain text fields have a single `value` sub-field. Our custom source plugin will fetch the video ID and create a valid YouTube URL that can be assigned to the text field in the remote video media entity.

Create a PHP file named `Tag1YouTubeField.php` inside our `tag1_migration` custom module's `/src/Plugin/migrate/source` folder. The path relative to the Drupal 10's project docroot is `web/modules/custom/tag1_migration/src/Plugin/migrate/source/Tag1YouTubeField.php`. The content of the file should be:

```php
<?php

namespace Drupal\tag1_migration\Plugin\migrate\source;

use Drupal\Component\Plugin\Exception\InvalidPluginDefinitionException;
use Drupal\Core\State\StateInterface;
use Drupal\migrate\Plugin\migrate\source\SqlBase;
use Drupal\migrate\Plugin\MigrationInterface;
use Drupal\migrate\Row;

/**
 * Drupal 7 YouTube Field data.
 *
 * Available configuration keys:
 * - name: The name of the YouTube field.
 * - revisions: (optional) If TRUE, retrieve field revisions. Defaults to FAL\
SE.
 *
 * For additional configuration keys, refer to the parent classes.
 *
 * @see https://udrupal.com/migrate-source-plugins
 *
 * Example:
 *
 * @code
 *   source:
 *     plugin: tag1_youtube_field
 *     name: field_video
 *     revisions: TRUE
 * @endcode
 *
 * @see \Drupal\migrate\Plugin\migrate\source\SqlBase
 * @see \Drupal\migrate_drupal\Plugin\migrate\source\DrupalSqlBase
 * @see \Drupal\migrate_plus\Plugin\migrate\source\Table
 * @see \Drupal\migrate_source_csv\Plugin\migrate\source\CSV
 * @see \Drupal\media_migration\Plugin\migrate\source\VideoEmbedField
 *
 * @MigrateSource(
 *   id = "tag1_youtube_field",
 *   source_module = "youtube"
 * )
 */
class Tag1YouTubeField extends SqlBase {

  /**
   * YouTube field table.
   *
   * @var string
   */
  protected string $tableName;
```

```
51
52    /**
53     * Column name storing the YouTube video ID.
54     *
55     * @var string
56     */
57    protected string $videoIdColumnName;
58
59    /**
60     * YouTube URL prefix.
61     *
62     * @var string
63     */
64    protected string $urlPrefix = 'https://www.youtube.com/watch?v=';
65
66    /**
67     * {@inheritdoc}
68     */
69    public function __construct(array $configuration, $plugin_id, $plugin_defin\
70  ition, MigrationInterface $migration, StateInterface $state) {
71      if (empty($configuration['name'])) {
72        throw new \InvalidArgumentException("Table tag1_youtube is missing 'nam\
73  e' property configuration.");
74      }
75
76      parent::__construct($configuration, $plugin_id, $plugin_definition, $migr\
77  ation, $state);
78
79      // @see \Drupal\migrate_drupal\Plugin\migrate\source\d7\FieldableEntity::\
80  getFieldValues()
81      $this->tableName = ((bool) $configuration['revisions'] === TRUE ? 'field_\
82  revision_' : 'field_data_') . $configuration['name'];
83
84      // Retrieve the 'video_id' sub-field.
85      // @see youtube_field_schema() in Drupal 7's youtube.install file.
86      // @see youtube_get_video_id() in Drupal 7's youtube.inc file.
87      $this->videoIdColumnName = $configuration['name'] .  '_video_id';
88
89      if (!$this->getDatabase()->schema()->tableExists($this->tableName)) {
90        throw new InvalidPluginDefinitionException($plugin_id, "Source database\
91   table '{$this->tableName}' does not exist.");
92      }
93    }
94
95    /**
96     * {@inheritdoc}
97     */
98    public function query() {
99      $query = $this->select($this->tableName, 'yt')
100       ->distinct();
```

```
    $query->addField('yt', $this->videoIdColumnName, 'video_id');

    return $query;
  }

  /**
   * {@inheritdoc}
   */
  public function prepareRow(Row $row) {
    $video_id = $row->getSourceProperty('video_id');
    $row->setSourceProperty('video_url', $this->urlPrefix . $video_id);
  }

  /**
   * {@inheritdoc}
   */
  public function fields() {
    return [
      'video_id' => $this->t('YouTube video id.'),
      'video_url' => $this->t('YouTube video URL.'),
    ];
  }

  /**
   * {@inheritdoc}
   */
  public function getIds() {
    $ids['video_id']['type'] = 'string';
    $ids['video_id']['alias'] = 'yt';
    return $ids;
  }

}
```

Technical note: When the Core's `Migrate Drupal` module is enabled, it reads the value of `source_module` in the annotation and checks if the listed module is enabled in the source site. Because our custom source plugin reads data from the YouTube field module, we use its machine name `youtube`. If the module specified in `source_module` is not enabled, any migration using that source plugin will be filtered out and will not appear in the list of available migrations that can be executed. This is done with a combination of the `enforce_source_module_tags` configuration in `migrate_drupal.settings.yml` and tags being added to the `migration_tags` section of a migration definition file. By default, `Drupal 7` is one of the migration tags to enforce and it is included in the generated migrations.

A detailed review of the PHP code for the custom source plugin will be left

as an exercise to the curious reader. What we want to highlight is that **knowing the table structure of field API tables is key to accomplish our goal**. First, we need to determine if we want to retrieve data from the current revision only or include all past revisions data. In Drupal 7, every field created two tables based on its machine name: `field_data_FIELD_NAME` and `field_revision_FIELD_NAME`.

Our process plugin exposes two settings:

- `name`: Required. Stores a string indicating the name of the YouTube field in Drupal 7 to fetch data from. In our example that would be: `field_video_recording`.
- `revisions`: Optional. A boolean indicating whether revision data should be retrieved or not. Defaults to `FALSE`. In our example, we will not migrate revisions data.

With this information, the source plugin can figure out which Drupal 7 table to query: `field_data_field_video_recording`. But what about its structure? We need to find out the column name that stores the video ID data. From the root of the Drupal 7 project execute `ddev mysql` to access a command-line interface client MySQL. Then execute the following statement at the SQL prompt `DESCRIBE field_data_field_video_recording;`.

You will get an output similar to the following:

```
+-------------------------------+------------------+------+-----+---------+-\
------+
| Field                         | Type             | Null | Key | Default | \
Extra |
+-------------------------------+------------------+------+-----+---------+-\
------+
| entity_type                   | varchar(128)     | NO   | PRI |         | \
      |
| bundle                        | varchar(128)     | NO   | MUL |         | \
      |
| deleted                       | tinyint(4)       | NO   | PRI | 0       | \
      |
| entity_id                     | int(10) unsigned | NO   | PRI | NULL    | \
      |
| revision_id                   | int(10) unsigned | YES  | MUL | NULL    | \
      |
| language                      | varchar(32)      | NO   | PRI |         | \
      |
| delta                         | int(10) unsigned | NO   | PRI | NULL    | \
```

```
      |
| field_video_recording_input    | varchar(1024)    | YES  |     | NULL    | \
      |
| field_video_recording_video_id | varchar(15)      | YES  | MUL | NULL    | \
      |
+--------------------------------+------------------+------+-----+---------+-\
------+
```

Drupal 7 field API creates tables with a common structure. The following columns are present for all fields no matter their type: `entity_type`, `bundle`, `deleted`, `entity_id`, `revision_id`, `language`, and `delta`. Then, there will be one column for each sub-field as defined by the field type. In this case, we have `field_video_recording_input` and `field_video_recording_video_id`. Notice that the machine name of the field is prepended to the sub-field name with an underscore in between. The columns are the same for the `field_data_FIELD_NAME` and `field_revision_FIELD_NAME` tables. They differ in which columns are nullable and which act as primary keys.

In its `query` method, our custom process plugin builds a query to retrieve unique video ID values from current revision data of the `field_video_recording` field:

```
SELECT DISTINCT field_video_recording_video_id FROM field_data_field_video_re\
cording;
```

Then, in its `prepareRow` method, the custom source plugin reads the retrieved `video_id` value and creates a new **source property** named `video_url` with a valid YouTube URL. This can later be used in our migration to populate the `field_media_oembed_video` field for the remote video media entity.

Speaking of which, let's write that migration.

From Drupal 7 YouTube fields to Drupal 10 remote video media entities

Remember to stay hydrated during today's routine. Next, let's pick up those dumbbells for three rounds of triceps kickbacks. When completed, we will learn how to write a migration to create remote media entities from scratch.

Before doing so, it is important to understand what entities will be created, their base field definitions, and the fields attached to each bundle. We covered

this in great detail in article 26. Additionally, this article has a reference of base field definitions for Drupal 10 media entities. In short, we will create entities of type `media` of the `remote_video` bundle and populate its `field_media_oembed_video` plain text field with the `video_url` column retrieved by our custom source plugin.

Create an `upgrade_d7_media_remote_video.yml` file in the `web/modules/custom/tag1_migration/migrations` folder of our Drupal 10 project. We will use that same file name, without the file extension, as the migration ID. You can come up with any name as long as it is unique. Below is the content of the file:

```yaml
id: upgrade_d7_media_remote_video
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - media
 - tag1_content
label: 'Media (Remote video)'
source:
 key: migrate
 plugin: tag1_youtube_field
 name: field_video_recording
 revisions: FALSE
process:
 field_media_oembed_video: video_url
 status:
   plugin: default_value
   default_value: 1
 uid:
   plugin: default_value
   default_value: 1
 langcode:
   plugin: default_value
   default_value: en
destination:
 plugin: 'entity:media'
 default_bundle: remote_video
migration_dependencies:
 required: {  }
 optional: {  }
```

Now, rebuild caches for our new migration to be detected and trigger an import operation.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_media_remote_video
ddev drush migrate:import upgrade_d7_media_remote_video
```

Note: The first time you execute the `upgrade_d7_media_remote_video` migration, you might perceive that it runs slower than other migrations we have executed recently. Out of the box, when Drupal creates media entities, it tries to create thumbnails that serve as a preview when displaying media entities in listing pages. The thumbnail generation could be a slow process. Fortunately, you can configure media types to queue the thumbnail generation at a later time, speeding up the execution of the migration. This is configured on a per bundle basis. To enable this feature for remote videos go to `https://migration-drupal10.ddev.site/admin/structure/media/manage/remote_video` and enable the `Queue thumbnail downloads` option inside the `Publishing options` section. Thumbnail generation will now happen via a queue worker on cron execution.

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content/media?type=remote_video` and you shall be presented with multiple videos from our Tag1 Team Talks podcast series.

Note that in our `upgrade_d7_media_remote_video` migration, we are calling the `tag1_youtube_field`. This should match the `id` of the `MigrateSource` PHP annotation used in our custom source plugin. Then, we set the `name` configuration to the Drupal 7 field name to fetch data from `field_video_recording`. Finally, we set the `revisions` configuration to `FALSE` – meaning we only want to migrate data from current revisions. We could have opted for not including the `revisions` configuration (because the plugin assumes a `FALSE` default), but we decided to include it for completeness.

As it stands, our custom source plugin can read from one Drupal 7 field at a time. It is possible to write more advanced queries to be able to fetch data from multiple fields in one go. The Media Migration module provides many source plugins which advanced queries and logic in their `prepareRow` implementations. For reference, take a look at the YoutubeFieldSource and VideoEmbedField source plugins which retrieve data from the YouTube field and Video Embed Field Drupal 7 modules respectively. These source plugins build dynamic queries that leverage UNION clauses to fetch data from the multiple tables at the time.

The rest of our migration is relatively straightforward. We use the `entity:media` destination plugin to create media entities of type `remote_video`. In the `process` section, we assign the `video_url` fetched by our source plugin to the `field_media_oembed_video` plain text field. The remaining assignments in the process section are providing default values for the `status`, `uid`, and `langcode` base field definitions.

Extending existing source plugins

Catch your breath, and let's go down to the mat again. Use your dumbbells for three rounds of chest presses. When finished, I will teach how to extend existing source plugins.

Remember that per our migration plan, we also want to replace some of the image fields attached to content types for media reference fields. To accomplish this, we need to migrate file entities from Drupal 7 into media entities in Drupal 10. As covered in article 24, image fields in Drupal 7 are an extension of file fields. The data for both is stored in the `file_managed` table. In a stock Drupal 7 installation, an example record for this table would be:

```
*************************** 1. row ***************************
    fid: 1
    uid: 1
filename: druplicon.jpg
    uri: public://article/druplicon.jpg
filemime: image/jpeg
filesize: 2890
 status: 1
timestamp: 280299600
```

Technical note: If you are using the `7.x-2.x` branch of the File Entity module in Drupal 7, it alters the `file_managed` table to add an extra column: `type`. For the record above, the value for the `type` column would be `image`, which is one of the default file types provided by the module. The module also uses a queue triggered on cron to update the value of the new column based on the MIME type of the file.

We already used the d7_file plugin to migrate public and private files in article 24. Today, we are going to extend this core plugin and alter its query to

only retrieve images. Namely, we will use the `filemime` column in the `file_managed` table to limit the records to retrieve. If our project used the File Entity module, we could use the extra `type` column for filtering purposes.

Create a PHP file named `Tag1MediaImage.php` inside our `tag1_migration` custom module's `/src/Plugin/migrate/source` folder. The path relative to the Drupal 10's project docroot is `web/modules/custom/tag1_migration/src/Plugin/migrate/source/Tag1MediaImage.php`. The content of the file should be:

```php
<?php

namespace Drupal\tag1_migration\Plugin\migrate\source;

use Drupal\file\Plugin\migrate\source\d7\File;

/**
 * Retrieve permanent images
 *
 * @see \Drupal\file\Plugin\migrate\source\d7\File
 *
 * @MigrateSource(
 *   id = "tag1_media_image",
 *   source_module = "file"
 * )
 */
class Tag1MediaImage extends File {

  /**
   * {@inheritdoc}
   */
  public function query() {
    $query = parent::query();
    $query->condition('f.filemime', 'image/%', 'LIKE');
    $query->condition('f.status', 1);
    return $query;
  }

}
```

Note: Source plugins that only alter the parent's query might no longer be necessary once this core issue lands.

A detailed review of the PHP code for the custom source plugin will be left as an exercise to the curious reader. You do not need to do much to create a custom process plugin that extends an existing one. In this case, we are only overwriting the `query` method to add two conditions:

- That the `filemime` matches that pattern `image/%`. In SQL, the `%` character serves as a wildcard for pattern matching when paired with the LIKE operator.
- That the `status` is `1` which means the file is permanent.

Please note that we are adding conditions to the parent query provided by `d7_file`. So, you need to review that source plugin's `query` implementation to see how the initial query is built. At the risk of stating the obvious, by extending the `Drupal\file\Plugin\migrate\source\d7\File` class, we also inherit all the other methods defined in that class and up in its parent class hierarchy. The importance of this will be evident when writing the migration for creating image media entities.

Before switching focus to writing a media migration, consider how a custom source plugin that extends `d7_file` would look file if the Drupal 7 site used the File Entity module:

```php
<?php

namespace Drupal\tag1_migration\Plugin\migrate\source;

use Drupal\file\Plugin\migrate\source\d7\File;

/**
 * Retrieve permanent files optionally filtered by file type bundles.
 *
 * Available configuration keys:
 * - type: Only retrieve files matching the specified file type bundles. Can \
be
 *   set to a string or an array. If not declared then files of all file types
 *   bundles will be retrieved.
 *
 * @see \Drupal\file\Plugin\migrate\source\d7\File
 * @see file_entity_file_default_types() in Drupal 7's file_entity.module fil\
e.
 *
 * @MigrateSource(
 *   id = "tag1_d7_file",
 *   source_module = "file_entity"
 * )
 */
class Tag1File extends File {

  /**
   * {@inheritdoc}
```

```
29     */
30   public function query() {
31     $query = parent::query();
32
33     // Only migrate permanent files.
34     $query->condition('f.status', 1);
35
36     // Filter by file type bundle, if configured.
37     if (isset($this->configuration['type'])) {
38       $query->condition('f.type', (array) $this->configuration['type'], 'IN');
39     }
40
41     return $query;
42   }
43
44 }
```

The `tag1_d7_file` above can accept an optional `type` configuration to only retrieve files matching the specified file types. Then, you can use the plugin as follow:

```
1 source:
2  key: migrate
3  plugin: tag1_d7_file
4  type: image
```

Note to my future self: Thanks for providing the code for a source plugin I can copy/paste in the next migration project!

From Drupal 7 image fields to Drupal 10 image media entities

Don't give up just yet. We have one more exercise to go. Take your dumbbells for three rounds of overhead presses. Afterward we'll write another migration by hand.

Time to import Drupal 7 images as media entities in Drupal 10. Before doing so, you need to understand what entities will be created, their base field definitions, and the fields attached to each bundle. We covered this in great detail in article 26. Additionally, this article has a reference of base field definitions for Drupal 10 media entities. In short, we will create entities of type

`media` of the `image` bundle and populate its `field_media_image` image field with a reference from previously migrated files.

Create an `upgrade_d7_media_image.yml` file in the `web/modules/custom/tag1_migration/migrations` folder of our Drupal 10 project. We will use that same file name, without the file extension, as the migration ID. You can come up with any name as long as it is unique. Below is the content of the file:

```yaml
id: upgrade_d7_media_image
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - media
 - tag1_content
label: 'Media (Image)'
source:
 key: migrate
 plugin: tag1_media_image
 constants:
   source_base_path: NULL
 high_water_property:
   name: fid
   alias: f
process:
 name: filename
 status: status
 created: timestamp
 changed: timestamp
 field_media_image/alt: filename
 field_media_image/target_id:
   - plugin: migration_lookup
     source: fid
     migration: upgrade_d7_file
     no_stub: true
   - plugin: skip_on_empty
     method: row
     message: 'The file was not found.'
 uid:
   - plugin: migration_lookup
     source: uid
     migration: upgrade_d7_user
     no_stub: true
   - plugin: default_value
     default_value: 1
 langcode:
   plugin: default_value
```

```
    default_value: en
destination:
 plugin: 'entity:media'
 default_bundle: image
migration_dependencies:
 required:
   - upgrade_d7_file
   - upgrade_d7_user
 optional: {  }
```

Now, rebuild caches for our new migration to be detected and trigger an import operation.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_media_image
ddev drush migrate:import upgrade_d7_media_image
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content/media?type=image` and you will be presented with beautiful images created by Drupal 7's Devel Generate module.

Note that in our `upgrade_d7_media_image` migration, we are calling the `tag1_media_image`. This should match the `id` of the `MigrateSource` PHP annotation used in our custom source plugin. We recycled the configuration for `high_water_property` from the `upgrade_d7_file`.

What is that strange `source_base_path` constant with a value of `NULL`? If you do not include it, you will get the following warning for each record retrieved by our custom source plugin: `Undefined array key "constants" File.php:105`. Remember what we said earlier? By extending a source plugin, you inherit all its methods. Well, that warning comes from implementation of the `prepareRow` method in the `d7_file` source plugin. That plugin assumes a source constant named source_base_path exists for setting the `filepath` of files to import. Our media migration makes no use of such source property, but the expectation that the constant exists is very much present. So, to suppress the warning, we can provide the source constant and set it to a `NULL` value.

We use the `entity:media` destination plugin to create media entities of type `image`. In the `migration_dependencies` section, we add `upgrade_d7_file` and `upgrade_d7_user` as required dependencies, because we will perform migration lookups against them. The process section contains mapping

for multiple base field definitions, which we hope will be straightforward to understand now. The key part of this migration is populating the `field_media_image` field.

In Drupal 10, image fields are entity reference fields. As noted in this article, this field type has five sub-fields:

1. `target_id`: An integer representing the ID of the target entity.
2. `alt`: Alternative image text, for the image's `alt` attribute.
3. `title`: Image title text, for the image's `title` attribute.
4. `width`: The width of the image in pixels.
5. `height`: The height of the image in pixels.

Our source data does not contain information for `width` and `height`. When not specified, the ImageItem class responsible for providing image fields loads the file to determine its dimensions. If available, setting the value for these sub-fields will yield a performance boost, because there will be no need to calculate the image's dimensions.

While not ideal, we are using the filename as the `alt` attribute. For `title`, we are not even going to pretend to have a sensible value to use. Maybe a good compromise would be generating image descriptions and alt-text with AI like Drupal's project founder Dries Buytaert considered doing for his website.

Finally, the most important sub-field to set for any entity reference field: `target_id`. Let's revisit the process pipeline to set it:

```
process:
 field_media_image/target_id:
   - plugin: migration_lookup
     source: fid
     migration: upgrade_d7_file
     no_stub: true
   - plugin: skip_on_empty
     method: row
     message: 'The file was not found.'
```

For image fields, `target_id` should be set to the referenced file ID. Our source plugin provides the `fid` source property that can be used for this purpose. It might be tempting to assign a copy `fid` directly into `field_media_image/target_id`. But that neglects the fact that it is possible for records to exist in Drupal 7's `file_managed` table, but the corresponding files are not

present on disk. We will not speculate about what would cause such a situation to happen, but I have seen this many times in real life projects. So, to safeguard us from creating invalid media entities that point to non-existent files, we perform a `migration_lookup` against the `upgrade_d7_file` migration. If that migration was not able to retrieve the file from Drupal 7 for any reason, we leverage the `skip_on_empty` process plugin to bail out from creating the media entity.

If the above sounds familiar, it is because a similar technique is used to connect paragraphs to their host entities as explained in the previous article. The difference is that paragraph fields are of type **entity reference revisions** and require setting the `target_revision_id` sub-field in addition to `target_id`. Fields of type **entity reference** – like files, images, taxonomy term references, user references, etc. – only require setting the `target_id` sub-field.

Time to cool down after an intense upper body exercise routine. Those migration muscles might be aching, but hey… no pain no gain!

Now you know how to create custom source plugins, extend existing ones, and use them to create media entities. In the next article, we will work our lower body… I mean, we will work on migrating yet another entity: nodes. We might or might not create one more source plugin. Only time will tell.

29 - Migrating nodes - Part 1

In the previous article, we learned to create custom source plugins and leveraged them to migrate media entities. Today, we'll migrate nodes and create another custom source plugin to prevent unpublished nodes from being imported. After applying the content model changes stipulated in our migration plan, our example Drupal 10 project ended up having five content types. In this article, we'll cover two: basic pages and articles. We'll also revisit a key topic for Drupal migration projects: relationships among entities.

Entity ID and high water mark considerations

Nodes are revisionable entities. When the migration project involves content model changes, you need to consider IDs from all entities in the source site that will be used for creating nodes in the destination site. Our example project includes conversions from nodes to other entities like users and taxonomy terms. But we're not creating Drupal 10 nodes from multiple Drupal 7 entity types. Therefore, reviewing the Drupal 7 node tables is enough to determine a suitable AUTO_INCREMENT value for Drupal 10's node entity. From the `drupal7` folder, execute `ddev mysql` to open an interactive SQL shell. Then execute the following queries:

```
-- Get the highest node ID value.
SELECT nid FROM node ORDER BY nid DESC LIMIT 1;

-- Get the highest node revision ID value.
SELECT vid FROM node_revision ORDER BY vid DESC LIMIT 1;
```

For brevity, we're showing you how to configure the `auto_increment_alter_content_entities` setting of the AUTO_INCREMENT Alter module to apply the new AUTO_INCREMENT value. Refer to article 23 for more information of how this module works or how to perform the operation in custom code.

```
$settings['auto_increment_alter_content_entities'] = [
  'node' => [450, 1000], // Alter the tables for the node content entity.
];
```

Now execute the command provided by the AUTO_INCREMENT Alter module to trigger the alter operation in the Drupal 10 project.

```
ddev drush auto-increment-alter:content-entities
```

As for the high water property, we need to identify one field returned by the source plugin whose value always increases. Drupal includes three source plugins, explained later, to fetch Drupal 7 node data. All of them retrieve the node revision ID (`vid`) column from Drupal 7's `node_revision` table. Today, we'll create a custom `tag1_d7_node` source plugin that extends one of the core ones. Unless we manually remove elements from the query, anything from the parent class will be available in our custom source plugin.

Therefore, we can configure the `high_water_property` as follows:

```
source:
 plugin: tag1_d7_node
 high_water_property:
   name: vid
   alias: nr
```

Filtering out records from being imported

Our migration plan says there is no need to migrate unpublished nodes – no matter their content type. In article 25, we discussed how to filter out records using process plugins and we explained how to do it using source plugins in article 28. Now that we have seen two options, we would like to revisit the topic again to provide a bit more context.

From the perspective of the migrate API, there are four primary ways to filter out records to import.

1. **The `query` method in the source plugin**. This is the most performant option since the records are filtered out *before* they reach Drupal for processing.

2. **The `prepareRow` method in the source plugin**. At this point, you have the record as retrieved by the source query. You can use data from retrieved fields to fetch more data from the **source site** to determine if a particular record should be migrated or not. For example, when using the organic groups module in Drupal 7, you can check which group a node belongs to determine if it should be migrated or not. Returning `FALSE` from this method tells the Migrate API that the records should *not* be migrated.
3. **As part of the process pipeline**. At this point, you have the record as returned by the `prepareRow` method, which can add/change/remove data retrieved by the source query. We saw an example of this in the previous article when creating media entities from files. The media entity was created *only if* the corresponding file was imported in the file migration. Some process plugins that provide logic to skip records from being imported are skip_on_empty, skip_on_value, static_map, and gate.
4. **As part of the destination plugin**. At this point, you have the transformed data as returned by the process pipeline. While it's not common to filter out records this late in the import process, you can use it as a *last line of defense* to prevent migrating invalid entries. For example, the EntityContentBase destination plugin, base for multiple destination plugins that create content entities, exposes an `validate` configuration option. When set to `TRUE`, the Migrate API will call the validate method of the entity API before importing a record. In practice, this means all validation constraints are enforced at the entity and field level.

Technical note: With the exception of the first option, the identifiers of the records that were filtered out are recorded in the migration map tables. It's also possible to leave a message indicating why a record was skipped. Looking at the *map* and *message* tables for a migration can be useful to understand why some content was not migrated when the expectation was for it to be imported.

When deciding on which option to implement, remember to abide by ETL principles. **Only use data pertinent to each phase when building the logic to filter out data.** For example, a source plugin can inspect Drupal 7 data, but it should not check Drupal 10 data to determine if a record should be imported or not. This is similar to the advice of not creating entities on the fly as part of the process pipeline.

Extending the node source plugin to filter out unpublished nodes

Create a PHP file named `Tag1Node.php` inside our `tag1_migration` custom module's `/src/Plugin/migrate/source` folder. The path relative to the Drupal 10's project docroot is `web/modules/custom/tag1_migration/src/-Plugin/migrate/source/Tag1Node.php`. The content of the file should be:

```php
<?php

namespace Drupal\tag1_migration\Plugin\migrate\source;

use Drupal\node\Plugin\migrate\source\d7\Node;

/**
 * Customizations for node migrations.
 *
 * @MigrateSource(
 *   id = "tag1_d7_node",
 *   source_module = "node",
 * )
 */
class Tag1Node extends Node {

  /**
   * {@inheritdoc}
   */
  public function query() {
    $query = parent::query();
    // Exclude unpublished content.
    $query->condition('n.status', 1);
    return $query;
  }

}
```

As a reminder, the query method allows you to alter the SQL query used to retrieve data from the source database. You can add conditions, join tables, expand the list of fields to fetch, and much more. Note that custom source plugins that only alter the parent's query might no longer be necessary once this core issue is fixed.

Technical note: The `drush migrate:status` command shows a report for each migration indicating the *total* number of records, how many have been

imported, and many are still *unprocessed*. These numbers are accurate when the migrations have not been executed and you import all records at once. When running incremental migrations, it's possible for things to change in the source site in between runs, like nodes being deleted or added. That can lead to some strange calculations – like a negative number for unprocessed records. In the report:

- **Total** is the count of records as returned by the source query at that point in time.
- **Imported** is the number of records already processed as recorded in the migration map tables.
- **Unprocessed** is the difference between total and unprocessed at that point in time.

Before writing a migration that makes use of our custom source plugin, I would like to point out that Drupal core offers three source plugins to retrieve node data:

1. d7_node: Retrieves only the current revision.
2. d7_node_revision: Retrieves only past revisions.
3. d7_node_complete: Retrieves all current and past revisions.

For more information on using these plugins, read the change record on classic vs complete node migrations. In all cases, it's possible to fetch node translations, if available. The first two options require setting the `translations` configuration option to `TRUE`. The last option fetches translations without having to specify extra configuration options. To account for translations when creating Drupal 10 content entities, set the `translations` configuration option in the **destination plugin**, assuming it extends EntityContentBase. Below is how a node migration would be configured to import translations:

```yaml
source:
 key: migrate
 plugin: d7_node
 node_type: page
 translations: TRUE
process: ...
destination:
 plugin: 'entity:node'
 default_bundle: page
 translations: TRUE
```

Migrating basic pages

We use `upgrade_d7_node_page` to migrate basic pages. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```bash
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_page.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_node_page.yml
ddev drush cache:rebuild
```

If you do not have a `migrate_plus.migration.upgrade_d7_node_page.yml` file in the `ref_migrations` migrations folder, it's likely that you have a file named `migrate_plus.migration.upgrade_d7_node_complete_page.yml` instead. That would mean that you used the node complete approach, instead of the classic one, when performing the automated migration. Feel free to proceed with our instructions using that file. Ultimately, we are going to update the source plugin to use our custom one. The file name or plugin ID has no effect on what data is retrieved. That will depend on which source plugin is used and how it is configured.

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node` and `tag1_content`.
- Add `key: migrate` under the source section.

- Change the source plugin to configuration to use our custom `tag1_d7_node` plugin.
- Add the `high_water_property` property as demonstrated above.
- Update the migration dependencies so that only a *required dependency* on `upgrade_d7_user` remains.

We also need to account for changes in *text formats*. In article 22 we decided not to migrated Drupal 7 text formats, but leverage those that Drupal 10 provides out of the box. In practice this means that the `filtered_html` format used in Drupal 7 no longer exists. In Drupal 10, the migration of any rich text field will have to be updated to map `filtered_html` to a valid text format in the new site. Using the snippet below we replace it with the `restricted_html` text format in the `body` field:

```
process:
  body:
    -
      plugin: sub_process
      source: body
      process:
        value: value
        summary: summary
        format:
          -
            plugin: static_map
            source: format
            map:
              filtered_html: restricted_html
            bypass: TRUE
```

Note: You can use the snippet process proposed for the migrate plus module to create reusable process pipelines for use in your migrations.

After the modifications, the `upgrade_d7_node_page.yml` file should look like this:

```yaml
id: upgrade_d7_node_page
class: Drupal\migrate\Plugin\Migration
migration_tags:
  - 'Drupal 7'
  - Content
  - node
  - tag1_content
label: 'Nodes (Basic page)'
source:
  key: migrate
  plugin: tag1_d7_node
  node_type: page
  high_water_property:
    name: vid
    alias: nr
process:
  nid:
    -
      plugin: get
      source: tnid
  vid:
    -
      plugin: get
      source: vid
  langcode:
    -
      plugin: default_value
      source: language
      default_value: und
  title:
    -
      plugin: get
      source: title
  uid:
    -
      plugin: get
      source: node_uid
  status:
    -
      plugin: get
      source: status
  created:
    -
      plugin: get
      source: created
  changed:
    -
      plugin: get
      source: changed
  promote:
```

```
    -
      plugin: get
      source: promote
  sticky:
    -
      plugin: get
      source: sticky
  revision_uid:
    -
      plugin: get
      source: revision_uid
  revision_log:
    -
      plugin: get
      source: log
  revision_timestamp:
    -
      plugin: get
      source: timestamp
  comment_node_page/0/status:
    -
      plugin: get
      source: comment
  body:
    -
      plugin: sub_process
      source: body
      process:
        value: value
        summary: summary
        format:
          -
            plugin: static_map
            source: format
            map:
              filtered_html: restricted_html
            bypass: TRUE
destination:
  plugin: 'entity:node'
  default_bundle: page
migration_dependencies:
  required:
    - upgrade_d7_user
  optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_page
ddev drush migrate:import upgrade_d7_node_page
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content?type=page` and look at the list of migrated basic page nodes. More important though is what you do not see. Drupal 7's "Test page" node with `nid` 16 should not exist in the migrated site. If it appears, make sure you update the source plugin to use `tag1_d7_node`, rebuild caches, rollback, and import again.

After running a migration, I would normally explain interesting or complex elements. There is not much to highlight for this one other than it's the 21st migration we write so far in the series. Can you believe that? **Congratulations!** You now have a lot more experience than me when I worked on my first migration project.

Migrating articles

We use `upgrade_d7_node_article` to migrate articles. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_article.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_node_article.yml
ddev drush cache:rebuild
```

If you do not have a `migrate_plus.migration.upgrade_d7_node_article.yml` file in the `ref_migrations` migrations folder, it's likely that you have a file named `migrate_plus.migration.upgrade_d7_node_complete_page.yml` instead. That would mean that you used the node complete approach, instead of the classic one, when performing the automated migration. Feel free to proceed with our instructions using that file. Ultimately, we are going to update the source plugin to use our custom one. The file name or plugin ID has no effect on what data is retrieved. That will depend on which source plugin is used and how it is configured.

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node` and `tag1_content`.
- Add `key: migrate` under the source section.
- Change the source plugin to configuration to use our custom `tag1_d7_node` plugin.
- Add the `high_water_property` property as demonstrated above.
- Update the migration dependencies so that `upgrade_d7_taxonomy_term` and `upgrade_d7_user` are listed as *required dependencies*.
- Apply the same treatment to *account for text format changes* in the `body` as we did in the page migration.

After the modifications, the `upgrade_d7_node_article.yml` file should look like this:

```yaml
id: upgrade_d7_node_article
class: Drupal\migrate\Plugin\Migration
migration_tags:
  - 'Drupal 7'
  - Content
  - node
  - tag1_content
label: 'Nodes (Article)'
source:
  key: migrate
  plugin: tag1_d7_node
  node_type: article
  high_water_property:
    name: vid
    alias: nr
process:
  nid:
    -
      plugin: get
      source: tnid
  vid:
    -
      plugin: get
      source: vid
  langcode:
    -
      plugin: default_value
      source: language
      default_value: und
  title:
```

```
31      -
32        plugin: get
33        source: title
34    uid:
35      -
36        plugin: get
37        source: node_uid
38    status:
39      -
40        plugin: get
41        source: status
42    created:
43      -
44        plugin: get
45        source: created
46    changed:
47      -
48        plugin: get
49        source: changed
50    promote:
51      -
52        plugin: get
53        source: promote
54    sticky:
55      -
56        plugin: get
57        source: sticky
58    revision_uid:
59      -
60        plugin: get
61        source: revision_uid
62    revision_log:
63      -
64        plugin: get
65        source: log
66    revision_timestamp:
67      -
68        plugin: get
69        source: timestamp
70    comment_node_article/0/status:
71      -
72        plugin: get
73        source: comment
74    body:
75      -
76        plugin: sub_process
77        source: body
78        process:
79          value: value
80          summary: summary
```

```
        format:
          -
            plugin: static_map
            source: format
            map:
              filtered_html: restricted_html
            bypass: TRUE
  field_tags:
    -
      plugin: sub_process
      source: field_tags
      process:
        target_id: tid
  field_image:
    -
      plugin: sub_process
      source: field_image
      process:
        target_id: fid
        alt: alt
        title: title
        width: width
        height: height
destination:
  plugin: 'entity:node'
  default_bundle: article
migration_dependencies:
  required:
    - upgrade_d7_taxonomy_term
    - upgrade_d7_user
  optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. We run `migrate:status` to make sure we can connect to Drupal 7. Then, we run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_article
ddev drush migrate:import upgrade_d7_node_article
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content?type=article` and look at the list of migrated article nodes. More important, though, is what you do not see. Drupal 7's "Test article" node with `nid` 71 should **not** exist in the migrated site. If it appears, make sure you update the source plugin to use `tag1_d7_node`, rebuild caches, rollback, and import again.

Understanding migration lookup operations

Review the migrated articles. Do you notice something missing? Correct! None of the nodes have images. The generated migration does not account for content model changes. In our Drupal 10 site, we decided to use media reference fields instead of regular image fields to store the articles' images. Those paying close attention will also notice that the machine name of the field attached to the article content type is different. In Drupal 7, it's named `field_image` while in Drupal 10 it's `field_media_image`. To account for the change in field name and type, replace the `field_image` assignment in the `upgrade_d7_node_article.yml` file with the snippet below and add the `upgrade_d7_media_image` to the list of required migration dependencies.

```
process:
 field_media_image:
   -
     plugin: sub_process
     source: field_image
     process:
       target_id:
         -
           plugin: migration_lookup
           source: fid
           migration: upgrade_d7_media_image
           no_stub: true
```

Let's explain how we came up with this process pipeline. The Drupal 7 source plugin will return a field named `field_image` with a structure similar to the following:

```
[
 0 => [
   'fid' => 1885,
   'alt' => 'The quick brown fox jumps over the lazy dog',
   'title' => NULL,
   'width' => 1280,
   'width' => 720,
 ],
 1 => [
   'fid' => 344,
   'alt' => 'Vegard and Bård in fox costumes',
   'title' => 'What does the fox say?',
   'width' => 2013,
```

```
14     'width' => 903,
15   ],
16 ]
```

In Drupal 10, the `field_media_image` is of type media reference which expects a single sub-field to be set: `target_id`. In the previous article, we created Drupal 10 media entities from Drupal 7 image files. We can perform a lookup operation against the `upgrade_d7_media_image` migration to obtain the media entity ID that corresponds to Drupal 7's file ID.

But wait? Is there anything special about `fid` that the `migration_lookup` plugin knows how to convert that to a media entity ID? What if sent a different value like the `alt` attribute? Would that work? No, there's nothing special about `fid`. Using `alt` or any other of the sub-fields of `field_image` would work.

The way the migration_lookup plugin works is by mapping the passed *source value(s)* to the *identifier(s) of the migration(s)* over which the lookup operation will be performed. In this case, we are checking against the `upgrade_d7_media_image` migration. Ok, but where do we specify that `fid` is its identifier? Migrations are like onions. They have layers. Follow along:

1. The `upgrade_d7_media_image` migration uses the `tag1_media_image` source plugin.
2. The `tag1_media_image` source plugin extends Drupal core's `d7_file` source plugin.
3. The `d7_file` source plugin implements the getIds method of the MigrateSourceInterface to define the list of source fields that uniquely identify a source row.
4. In said `getIds` method, the file `fid` is designated as the only source field.

When configuring the lookup operation against `upgrade_d7_media_image`, the name `fid` is not special in itself. What is important is that **the value that we are using for the source configuration of the lookup operation represents a file ID**.

Also note that we are looking against `upgrade_d7_media_image`, not `upgrade_d7_file`. The former creates media entities. The latter creates file entities. In `upgrade_d7_node_article` we are populating the `field_media_image` field, which is a media reference field. So, we need the *media entity IDs*,

not *file entity IDs*, to properly assign the relationship to the media entity. If this sounds confusing, welcome to the club. Take time to understand it and make a mental model of the relationship among entities.

For reference, below is part of the code of the class that defines the `d7_file` source plugin:

```
/**
 * Drupal 7 file source from database.
 *
 * @see \Drupal\file\Plugin\migrate\source\d7\File
 *
 * @MigrateSource(
 *   id = "d7_file",
 *   source_module = "file"
 * )
 */
class File extends DrupalSqlBase {

  /**
   * {@inheritdoc}
   */
  public function getIds() {
    $ids['fid']['type'] = 'integer';
    $ids['fid']['alias'] = 'f';
    return $ids;
  }

}
```

Technical note: It's possible that multiple fields might be necessary to uniquely identify a source record. For example, the d7_node_complete source plugin uses the combination of three fields as a unique identifier: `nid`, `vid`, `language`. In such cases, you can pass an array of values to the `source` configuration of the `migration_lookup` process plugin.

After modifying the `upgrade_d7_node_article.yml` file, rollback the migration, rebuild caches, and execute the import operation again. Then, visit any of the articles to view a beautiful image created by Drupal 7's Devel Generate module.

```
ddev drush migrate:rollback upgrade_d7_node_article
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_article
ddev drush migrate:import upgrade_d7_node_article
```

Understanding the relationships among Drupal entities

Being able to establish relationships among entities is the foundation of the flexible and powerful content modeling capabilities of Drupal. As noted in article 5, there are two types of relationships among entities: *explicit* and *implicit*.

Explicit relationships are entity reference fields to nodes, users, taxonomy terms, files, media entities, paragraphs, groups, commerce products, etc. **Implicit relationships** are those established by base field definitions, including the content type for a node and the user who created it.

In article 7, we talked about two types of Drupal entities: *content* and *configuration*. Both explicit and implicit relationships can point to either content or configuration entities. As an example, consider one of the migrated articles nodes:

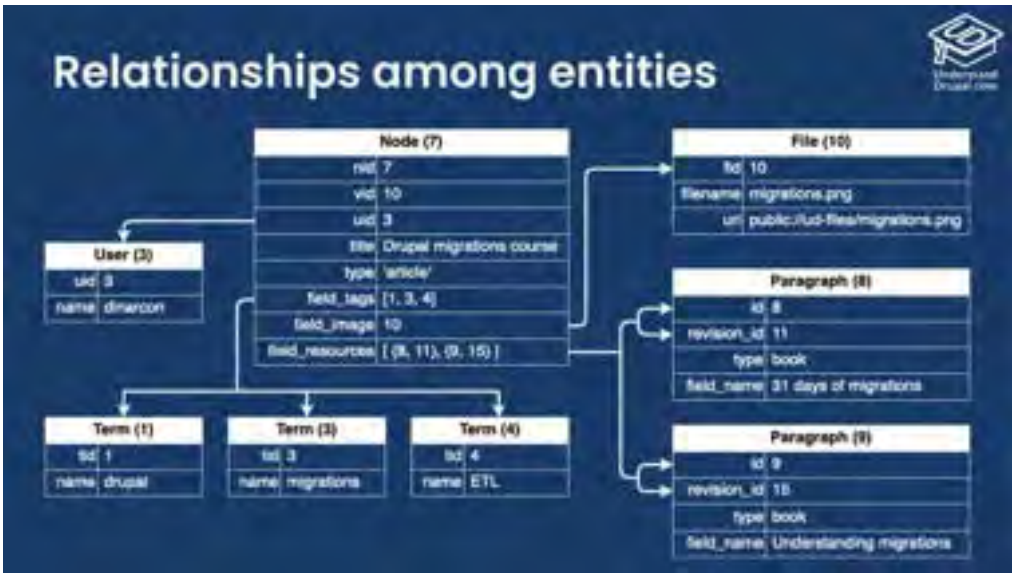
1. The `uid` *base field definition* is an *implicit relationship* to the `user` *content entity*.
2. The `type` *base field definition* is an *implicit relationship* to the `node_type` *configuration entity*.
3. The `field_tags` *field* is an *explicit relationship* to the `tags` bundle of the `taxonomy_term` *content entity*
4. The `field_media_image` *field* is an *explicit relationship* to the `image` bundle of the `media` *content entity*.

Having multiple layers of relationships among entities is common. Expanding the example above, the `image` bundle of the `media` *content entity* has an entity reference field named `field_media_image`. It establishes an *explicit relationship* to the `file` *content entity*.

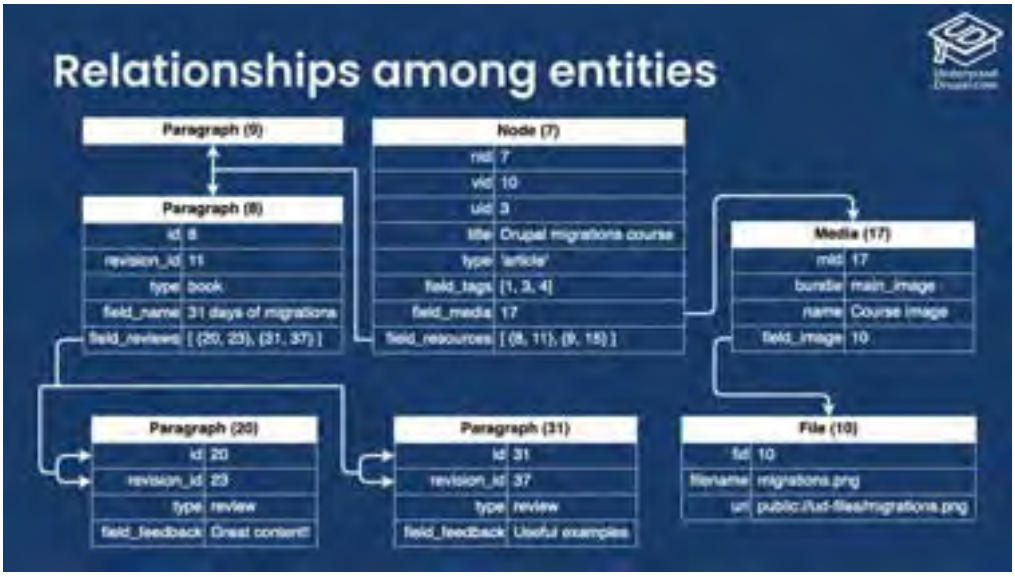
I did not mean to write a tongue-twister. Calling things by their names is helpful in keeping track of the multiple levels of relationships among Drupal entities. Also helpful are visual queues. The Entity Relationship Diagrams

module lets you visualize the entity structure of your Drupal site. Database tools like MySQL Workbench can reverse engineer your database to generate entity-relationship diagrams. If nothing else, use a pen and paper to sketch the relationships between entities. If your drawing skills are like mine, a digital sketchbook might be a better option.

Consider the diagram below from my Upgrading to Drupal 10 using the Migrate API presentation:



Among other relationships, it shows a direct connection between the node and file entities via an image field. If we wanted to change the content model to use media entities, there would be a connection from node to media entities via a *media reference field* and then a connection from media to file entities via an *image field*. The diagram below demonstrates the new connections among node, media, and file entities.



Before wrapping up this section, I would like to highlight something that we have been doing all along throughout the series. Each of the 20+ migrations we created is responsible for importing only one type of entity each. Focusing on the ones responsible for creating content, we started with files because it has no dependencies on other migrations. We later continued with users, taxonomy terms, paragraphs, media, and finally nodes. This order is very much intentional.

Migrations executed later perform lookup operations against migrations that came before to establish relationships among entities.

Technical note: The file entity has a `uid` *base field definition* that establishes a relationship to the user content entity. It's used to keep track of who uploaded the file. The `upgrade_d7_file` migration is the first one we executed. When the file entities are created, the value for `uid` is written to the database even though the referenced users might not exist in the system yet. Later on when we execute the `upgrade_d7_user` migration, those previously established relationships are still there and now point to valid user entities.

Today's custom source plugin and migrations were less complex than those covered in previous articles. That gave us a chance to elaborate on two very important topics: how to filter out records from being imported and to better understand the relationships among Drupal entities. In the next article, we'll write the migrations for the three content types that are pending to import.

30 - Node migrations - Part 2

In the last article, we started migrating nodes. Today, we'll continue with the three content types that are pending to import. This is the 30th article in the series so the migrations should feel familiar. To keep things interesting, in addition to explaining how to perform those node migrations, we'll also discuss other topics, like how the high water property is set up, strategies to perform quality assurance, and the benefits of enabling entity validation in content migrations. Ready?

Alias configuration in high water property

We normally start each article with a note on how to avoid entity ID conflicts and setting up a high water mark for the migrations that will be covered. Today, we continue with node migrations so what we explained last time applies here, too. Instead, let's talk about something that we have seen many times but never stopped to explain. **The `alias` configuration when setting up the high water property.**

Consider the following snippet copied from our node migrations:

```
source:
 key: migrate
 plugin: tag1_d7_node
 node_type: page
 high_water_property:
   name: vid
   alias: nr
```

The `high_water_property` consists of an array of two values. As explained in other articles, `name` is set to a timestamp or serial field returned by the source plugin. In this case, `vid` is a reference to a base field definition of the node entity that represents the node's revision ID. But, what is the purpose of the `alias` configuration and where does the `nr` value come from?

It's possible that the source plugin retrieves data from multiple tables. An alias is assigned to each table when assembling the query in the source plugin.

Among other things, this allows for the disambiguation of fields in case the same field name exists in more than one table.

Consider the following snippet containing part of the `query` implementation of the `d7_node` source plugin:

```
/**
 * Drupal 7 node source from database.
 *
 * @see \Drupal\node\Plugin\migrate\source\d7\Node
 *
 * @MigrateSource(
 *   id = "d7_node",
 *   source_module = "node"
 * )
 */
class Node extends FieldableEntity {

  /**
   * The join options between the node and the node_revision table.
   */
  const JOIN = '[n].[vid] = [nr].[vid]';

  /**
   * {@inheritdoc}
   */
  public function query() {
    // Select node in its last revision.
    $query = $this->select('node_revision', 'nr')
      ->fields('n', [
        'nid',
        'type',
        'language',
        'status',
        'created',
        'changed',
        'comment',
        'promote',
        'sticky',
        'tnid',
        'translate',
      ])
      ->fields('nr', [
        'vid',
        'title',
        'log',
        'timestamp',
      ]);
    $query->addField('n', 'uid', 'node_uid');
```

```
    $query->addField('nr', 'uid', 'revision_uid');
    $query->innerJoin('node', 'n', static::JOIN);

    // More code.

    return $query;
  }

}
```

Refer to this documentation page to learn more about *dynamic queries* in Drupal. What is important to note is that the source plugin queries the `node` and `node_revision` tables from Drupal 7. Both tables have a `vid` field. In fact, they have many fields in common: `nid`, `uid`, `title`, `status`, `comment`, `promote`, `sticky`.

When the query is built, we need a way to indicate which table to fetch a field from and that is what the table alias is used for. In the code above, the alias is the second argument to the `select` and `innerJoin` methods. **Those table aliases are the ones we can use when setting up the `high_water_property`**. You can see from the snippet above that `vid` is being retrieved from the `node_revision` table whose alias is `nr`.

Technical note: It's possible to retrieve a field common to multiple tables as long as you define *field aliases* to be able to tell them apart. In the snippet above, `uid` from the `node` table is retrieved as `node_uid` while `uid` from the `node_revision` is retrieved as `revision_uid`.

Migrating venue nodes

Now that we've clarified the alias for the high water property, let's continue by migrating venues. We're using `upgrade_d7_node_venue` so copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_venue.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_node_venue.yml
ddev drush cache:rebuild
```

If you do not have a `migrate_plus.migration.upgrade_d7_node_venue.yml` file in the `ref_migrations` migrations folder, it's likely that you have a file named `migrate_plus.migration.upgrade_d7_node_complete_page.yml` instead. That would mean that you used the node complete approach, instead of the classic one, when performing the automated migration. Feel free to proceed with our instructions using that file. Ultimately, we are going to update the source plugin to use our custom one. The file name or plugin ID has no effect on what data is retrieved. That will depend on which source plugin is used and how it is configured.

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node` and `tag1_content`.
- Add `key: migrate` under the source section.
- Change the source plugin to configuration to use our custom `tag1_d7_node` plugin.
- Add the `high_water_property` property as demonstrated above.
- **Update the mapping** of the `field_media_image` media field in the `process` section as we did in the previous article.
- Update the migration dependencies so that `upgrade_d7_media_image` and `upgrade_d7_user` are listed as *required dependencies*.

We also need to account for changes in *text formats*. In article 22, we decided not to migrate Drupal 7 text formats and instead leverage those that Drupal 10 provides out of the box. In practice, this means that the `filtered_html` format used in Drupal 7 no longer exists. In Drupal 10, the migration of any rich text field will have to be updated to map `filtered_html` to a valid text format in the new site. Using the snippet below, we replace it with the `restricted_html` text format in the `field_additional_information` field:

```
process:
 field_additional_information:
   -
     plugin: sub_process
     source: field_additional_information
     process:
       value: value
       format:
         -
           plugin: static_map
           source: format
           map:
             filtered_html: restricted_html
           bypass: TRUE
```

Note: You can use the snippet process proposed for the migrate plus module to create reusable process pipelines for use in your migrations.

After the modifications, the `upgrade_d7_node_venue.yml` file should look like this:

```
id: upgrade_d7_node_venue
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - node
 - tag1_content
label: 'Nodes (Venue)'
source:
 key: migrate
 plugin: tag1_d7_node
 node_type: venue
 high_water_property:
   name: vid
   alias: nr
process:
 nid:
   -
     plugin: get
     source: tnid
 vid:
   -
     plugin: get
     source: vid
 langcode:
   -
     plugin: default_value
```

```
28        source: language
29        default_value: und
30    title:
31      -
32        plugin: get
33        source: title
34    uid:
35      -
36        plugin: get
37        source: node_uid
38    status:
39      -
40        plugin: get
41        source: status
42    created:
43      -
44        plugin: get
45        source: created
46    changed:
47      -
48        plugin: get
49        source: changed
50    promote:
51      -
52        plugin: get
53        source: promote
54    sticky:
55      -
56        plugin: get
57        source: sticky
58    revision_uid:
59      -
60        plugin: get
61        source: revision_uid
62    revision_log:
63      -
64        plugin: get
65        source: log
66    revision_timestamp:
67      -
68        plugin: get
69        source: timestamp
70    comment_node_venue/0/status:
71      -
72        plugin: get
73        source: comment
74    field_address:
75      -
76        plugin: addressfield
77        source: field_address
```

```
 field_media_image:
   -
     plugin: sub_process
     source: field_image
     process:
       target_id:
         -
           plugin: migration_lookup
           source: fid
           migration: upgrade_d7_media_image
           no_stub: true
 field_phone:
   -
     plugin: get
     source: field_phone
 field_additional_information:
   -
     plugin: sub_process
     source: field_additional_information
     process:
       value: value
       format:
         -
           plugin: static_map
           source: format
           map:
             filtered_html: restricted_html
           bypass: TRUE
destination:
 plugin: 'entity:node'
 default_bundle: venue
migration_dependencies:
 required:
   - upgrade_d7_media_image
   - upgrade_d7_user
 optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_venue
ddev drush migrate:import upgrade_d7_node_venue
```

If things are properly configured, you should not get any errors. Go to `https://migration-drupal10.ddev.site/admin/content?type=venue`

and look at the list of migrated venue nodes. More important, though, is what you do *not* see. Drupal 7's "Test venue" node with `nid` 21 should not exist in the migrated site. If it appears, make sure you update the source plugin to use `tag1_d7_node`, rebuild caches, rollback, and import again.

You should see the media reference field populated. If that is not the case, verify that you replaced mapping of `field_image` as generated by the migration with the `process` pipeline for the `field_media_image` as described in the previous article. Remember that our migration plan asks for media reference fields in place of image fields. The generated migration would not account for that automatically so we need to make this change ourselves.

Another thing to check is that address information was migrated properly. In Drupal 7, there were multiple options to store address-related information. Two examples are the address field and location modules. Our example Drupal 7 project uses the former. In Drupal 10, we use the address module, which provides an automated upgrade path for the `address field` module. If your Drupal 7 project uses the `location` module, you can look at the Location Migration module for reference. This article includes more information on migrating address fields in Drupal 10.

Migrating session nodes

We use `upgrade_d7_node_session` to migrate sessions. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_session.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_node_session.yml
ddev drush cache:rebuild
```

If you do not have a `migrate_plus.migration.upgrade_d7_node_session.yml` file in the `ref_migrations` migrations folder, it's likely that you have a file named `migrate_plus.migration.upgrade_d7_node_session.yml` instead. That would mean that you used the node complete approach, instead of the classic one, when performing the automated migration. Feel free to proceed with our instructions using that file. Ultimately, we are going to update the source plugin to use our custom one. The file name or plugin ID has

no effect on what data is retrieved. That will depend on which source plugin is used and how it is configured.

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node` and `tag1_content`.
- Add `key: migrate` under the source section.
- Change the source plugin to configuration to use our custom `tag1_d7_node` plugin.
- Add the `high_water_property` property as demonstrated above.
- Apply the same treatment to **account for text format changes** in the `field_description` field as we did with the `field_additional_information` field in the venue migration.
- Update the migration dependencies so that `upgrade_d7_file`, `upgrade_d7_media_remote_video`, `upgrade_d7_node_speaker_to_user`, `upgrade_d7_taxonomy_term`, and `upgrade_d7_user` are listed as *required dependencies*.

Before running the migration, there are two changes we need to make to accommodate content model changes. The first is for the *speakers* field. In Drupal 7, speakers were *nodes*. Back in article 26, speakers were migrated as *users*. The name of the field in the `session` content type is the same between both versions, but they point to different types of entities. Update the process pipeline for `field_speakers` with the following code snippet:

```
process:
 field_speakers:
   -
     plugin: sub_process
     source: field_speakers
     process:
       target_id:
         -
           plugin: migration_lookup
           source: target_id
           migration: upgrade_d7_node_speaker_to_user
           no_stub: true
```

In the previous article, we explained how migration lookup operations work. You can also refer to article 16 and this presentation to better understand the database table structure for Drupal fields and how that is relevant to field mappings in migrations. In the snippet above, we are using the `target_id` sub-field of Drupal 7's `field_speakers` to perform a lookup operation against the `upgrade_d7_node_speaker_to_user` migration we created in article 26. This will return the `uid` of the migrated user and assign it to the `target_id` sub-field of Drupal 10's `field_speakers`, establishing the relationship to the user entity as expected in the new site.

The second change we need to do is related to the field that stores the video recording for the session. In Drupal 7, we used the YouTube field module. In Drupal 10, we decided to leverage media entities, which are supported out of the box. In article 28, we learned how to create remote video media entities in Drupal 10 out of Drupal 7 field data. The name of the field also changed from `field_video_recording` to `field_media_remote_video`. Replace the mapping of `field_video_recording` from the generated migration with the snippet below:

```
process:
 field_media_remote_video:
   -
     plugin: sub_process
     source: field_video_recording
     process:
       target_id:
         -
           plugin: migration_lookup
           source: video_id
           migration: upgrade_d7_media_remote_video
           no_stub: true
```

The `upgrade_d7_media_remote_video` migration uses the `video_id` sub-field of Drupal 7's YouTube field module as its identifier. So, we use that as the source for the lookup operation to retrieve corresponding *media ID* and assign it to the `target_id` sub-field of the `field_media_remote_video` field in the new site.

After the modifications, the `upgrade_d7_node_session.yml` file should look like this:

```yaml
id: upgrade_d7_node_session
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - node
 - tag1_content
label: 'Nodes (Session)'
source:
 key: migrate
 plugin: tag1_d7_node
 node_type: session
 high_water_property:
   name: vid
   alias: nr
process:
 nid:
   -
     plugin: get
     source: tnid
 vid:
   -
     plugin: get
     source: vid
 langcode:
   -
     plugin: default_value
     source: language
     default_value: und
 title:
   -
     plugin: get
     source: title
 uid:
   -
     plugin: get
     source: node_uid
 status:
   -
     plugin: get
     source: status
 created:
   -
     plugin: get
     source: created
 changed:
   -
     plugin: get
     source: changed
 promote:
```

```
51      -
52        plugin: get
53        source: promote
54    sticky:
55      -
56        plugin: get
57        source: sticky
58    revision_uid:
59      -
60        plugin: get
61        source: revision_uid
62    revision_log:
63      -
64        plugin: get
65        source: log
66    revision_timestamp:
67      -
68        plugin: get
69        source: timestamp
70    comment_node_session/0/status:
71      -
72        plugin: get
73        source: comment
74    field_speakers:
75      -
76        plugin: sub_process
77        source: field_speakers
78        process:
79          target_id:
80            -
81              plugin: migration_lookup
82              source: target_id
83              migration: upgrade_d7_node_speaker_to_user
84              no_stub: true
85    field_description:
86      -
87        plugin: sub_process
88        source: field_description
89        process:
90          value: value
91          format:
92            -
93              plugin: static_map
94              source: format
95              map:
96                filtered_html: restricted_html
97              bypass: TRUE
98    field_media_remote_video:
99      -
100       plugin: sub_process
```

```
      source: field_video_recording
      process:
        target_id:
          -
            plugin: migration_lookup
            source: video_id
            migration: upgrade_d7_media_remote_video
            no_stub: true
  field_slides:
    -
      plugin: sub_process
      source: field_slides
      process:
        target_id: fid
        display: display
        description: description
  field_topics:
    -
      plugin: sub_process
      source: field_topics
      process:
        target_id: tid
destination:
  plugin: 'entity:node'
  default_bundle: session
migration_dependencies:
  required:
    - upgrade_d7_file
    - upgrade_d7_media_remote_video
    - upgrade_d7_node_speaker_to_user
    - upgrade_d7_taxonomy_term
    - upgrade_d7_user
  optional: {  }
```

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
ddev drush cache:rebuild
ddev drush migrate:status upgrade_d7_node_session
ddev drush migrate:import upgrade_d7_node_session
```

If things are properly configured, you should not get any errors and *all fields should be populated.* Go to `https://migration-drupal10.ddev.site/admin/content?type=session` and look at the list of migrated session nodes. More important though is what you do not see.

Drupal 7's "Test session" node with `nid` 46 should *not* exist in the migrated site. If it appears, make sure you update the source plugin to use `tag1_d7_node`, rebuild caches, rollback, and import again.

Migrating event nodes

Next, we use `upgrade_d7_node_event` to migrate events. Copy it from the reference folder into our `tag1_migration` custom module and rebuild caches for the migration to be detected.

```
cd drupal10
cp ref_migrations/migrate_plus.migration.upgrade_d7_node_event.yml \
  web/modules/custom/tag1_migration/migrations/upgrade_d7_node_event.yml
ddev drush cache:rebuild
```

If you do not have a `migrate_plus.migration.upgrade_d7_node_event.yml` file in the `ref_migrations` migrations folder, it's likely that you have a file named `migrate_plus.migration.upgrade_d7_node_event.yml` instead. That would mean that you used the node complete approach, instead of the classic one, when performing the automated migration. Feel free to proceed with our instructions using that file. Ultimately, we are going to update the source plugin to use our custom one. The file name or plugin ID has no effect on what data is retrieved. That will depend on which source plugin is used and how it is configured.

Note that while copying the file, we also changed its name and placed it in a `migrations` folder inside our `tag1_migration` custom module. After copying the file, make the following changes:

- Remove the following keys: `uuid`, `langcode`, `status`, `dependencies`, `field_plugin_method`, `cck_plugin_method`, and `migration_group`.
- Add two migration tags: `node` and `tag1_content`.
- Add `key: migrate` under the source section.
- Change the source plugin to configuration to use our custom `tag1_d7_node` plugin.
- Add the `high_water_property` property as demonstrated above.
- Apply the same treatment to **account for text format changes** in the `field_description` field as we did with the `field_additional_information` field in the venue migration.

- Update the migration dependencies so that `upgrade_d7_node_session`, `upgrade_d7_node_sponsor_to_taxonomy_term`, `upgrade_d7_node_venue`, and `upgrade_d7_user` are listed as *required dependencies*.

Before running the migration, we need to accommodate another content model change. In Drupal 7, sponsors were *nodes* but, remember, speakers were migrated as taxonomy terms in article 26. The name of the field in the `event` content type is the same between both versions, but they point to different types of entities. Update the process pipeline for `field_sponsors` with the following code snippet:

```
process:
 field_sponsors:
   -
     plugin: sub_process
     source: field_sponsors
     process:
       target_id:
         -
           plugin: migration_lookup
           source: target_id
           migration: upgrade_d7_node_sponsor_to_taxonomy_term
           no_stub: true
```

After the modifications, the `upgrade_d7_node_event.yml` file should look like this:

```
id: upgrade_d7_node_event
class: Drupal\migrate\Plugin\Migration
migration_tags:
 - 'Drupal 7'
 - Content
 - node
 - tag1_content
label: 'Nodes (Event)'
source:
 key: migrate
 plugin: tag1_d7_node
 node_type: event
 high_water_property:
   name: vid
   alias: nr
process:
 nid:
```

```
18      -
19        plugin: get
20        source: tnid
21    vid:
22      -
23        plugin: get
24        source: vid
25    langcode:
26      -
27        plugin: default_value
28        source: language
29        default_value: und
30    title:
31      -
32        plugin: get
33        source: title
34    uid:
35      -
36        plugin: get
37        source: node_uid
38    status:
39      -
40        plugin: get
41        source: status
42    created:
43      -
44        plugin: get
45        source: created
46    changed:
47      -
48        plugin: get
49        source: changed
50    promote:
51      -
52        plugin: get
53        source: promote
54    sticky:
55      -
56        plugin: get
57        source: sticky
58    revision_uid:
59      -
60        plugin: get
61        source: revision_uid
62    revision_log:
63      -
64        plugin: get
65        source: log
66    revision_timestamp:
67      -
```

```
68        plugin: get
69        source: timestamp
70    comment_node_event/0/status:
71      -
72        plugin: get
73        source: comment
74    field_description:
75      -
76        plugin: sub_process
77        source: field_description
78        process:
79          value: value
80          format:
81            -
82              plugin: static_map
83              source: format
84              map:
85                filtered_html: restricted_html
86              bypass: TRUE
87    field_sessions:
88      -
89        plugin: get
90        source: field_sessions
91    field_venue:
92      -
93        plugin: get
94        source: field_venue
95    field_type:
96      -
97        plugin: get
98        source: field_type
99    field_sponsors:
100     -
101       plugin: sub_process
102       source: field_sponsors
103       process:
104         target_id:
105           -
106             plugin: migration_lookup
107             source: target_id
108             migration: upgrade_d7_node_sponsor_to_taxonomy_term
109             no_stub: true
110   field_date:
111     -
112       plugin: sub_process
113       source: field_date
114       process:
115         value:
116           plugin: format_date
117           from_format: 'Y-m-d H:i:s'
```

```
118          to_format: Y-m-d
119          source: value
120  destination:
121   plugin: 'entity:node'
122   default_bundle: event
123  migration_dependencies:
124   required:
125     - upgrade_d7_node_session
126     - upgrade_d7_node_sponsor_to_taxonomy_term
127     - upgrade_d7_node_venue
128     - upgrade_d7_user
129   optional: {  }
```

Technical note: You can improve the process pipeline for the `field_sessions` and `field_venue` fields by performing lookup operations against the `upgrade_d7_node_session` and `upgrade_d7_node_venue` migrations, respectively. This will make sure that only nodes that were successfully imported into Drupal 10 will be referenced. Revisit the previous article for a detailed explanation of how migration lookup operations work.

Now, rebuild caches for our changes to be detected and execute the migration. Run `migrate:status` to make sure we can connect to Drupal 7. Then, run `migrate:import` to perform the import operations.

```
1  ddev drush cache:rebuild
2  ddev drush migrate:status upgrade_d7_node_event
3  ddev drush migrate:import upgrade_d7_node_event
```

If things are properly configured, you should not get any errors and *all fields should be populated.* Go to `https://migration-drupal10.ddev.site/admin/content?type=event` and look at the list of migrated event nodes. More important though is what you do not see. Drupal 7's "Test event" node with `nid` 73 should *not* exist in the migrated site. If it appears, make sure you update the source plugin to use `tag1_d7_node`, rebuild caches, rollback, and import again.

Before wrapping up this section, I would like to point out that the event content type includes a date field. The automated migration created a process pipeline that imports its data. Refer to this article to learn more about migrating dates in Drupal 10.

Speaking of dates, take a look at nodes with `nid` 62 and 63 in the new site. They have no date set even though it is a required field. Is it an error in the

migration? Let's use this opportunity to discuss quality assurance and entity validation in Drupal migrations.

Quality assurance in Drupal migrations

Like other aspects of a Drupal project, the result of the migration needs to be thoroughly tested for quality assurance. Back in article 18, we saw an example where working on new migrations uncovered the need to make manual adjustments to previously migrated configuration. While writing today's migrations, I noticed that I had not accounted for text format changes in the migration of basic pages and articles nodes. Upon realizing the omission, I went back and updated the previous entry in the series.

Going back to the missing dates in the migrated `event` nodes, all nodes except those with `nid` 62 and 63 have a value for the date field. Any guesses about what the issue with those two nodes could be? Where would you start debugging the issue? Maybe the generated process pipeline is wrong after all?

What if I told you there is nothing wrong with our migration? To prove this, review the same nodes in Drupal 7 and you see that our source site also has no date set for those nodes even though the field is required. How that happened is not as relevant as the fact that it's impossible to migrate data that never existed. This may seem like a farfetched example, but I have seen this happen in real life projects. That is, people asking why a field for a specific node or user is not set. It's because the field for that particular node or user has no value.

There are multiple ways to assess the correctness of your migrations. Below is a non-exhaustive lists of tools and techniques I have used in the past:

- **Revisit the Drupal 7 site audit template**. Introduced in article 3 and further explored in article 8, populating the template is one of the very first things I do in every migration project. It can be used to keep track of decisions about what elements to migrate, what to skip, and what will undergo content model changes. One time when the template was extremely useful was when upgrading an application that used over 100 fields in a single content type. I was able to use the field instance report in the template to filter the list of fields per type. That helped me keep track of the fields that needed to be migrated and make sure I have accounted for them all.

- **Use the Migrate QA module**. It keeps track of every single migrated content entity, allows them to be reviewed by multiple stakeholders, and flags them depending on their approval status. Refer to the module's documentation for more information on how to implement it.
- **Identify content with data in the fields to validate**. For this, you can create a view and include a filter criteria that ensures the fields under testing have a value. If done in Drupal 10, remember that views are configuration entities. If used exclusively for testing the migration, it's very likely that you do not want to preserve the view. Make sure to delete it when you are done testing so it does not get exported with the rest of your site's configuration. Alternatively, you can query data from Drupal 7 and 10 databases directly. This requires some understanding of Drupal's database structure, but it's a very efficient way to find entities that match exactly your test criteria.
- **Create test content in the source site**. Usually done in a local copy of the site, you can create test content where all the fields that you are interested in testing are populated. If you happen to have conditional fields, or business logic that affect how multiple fields behave, creating sample content will save you time. This approach is useful to narrow down the number of imported content that needs to be reviewed after executing the migrations.

No matter which approach you take, make sure to involve the client in the quality assurance process. They will have a deeper understanding of the content model and any business logic that might be in place. Among other things, you can request the client create a list of the most important pages or sections of the website or application. The development team can pay extra attention to the migrations involved in producing content for the list provided by the client.

Throughout all your testing efforts, make sure to strike a balance. At some point, there will be diminishing returns. Consider time and budget when deciding how much and for how long you should test the migrations.

Content entity validation

There is one thing you can do to dramatically improve the quality of the migrated content: **enable entity validation**. All the destination plugins we have

used so far in the series are subclasses of the EntityContentBase destination plugin, which exposes the `validate` configuration option. When set to `TRUE`, the Migrate API will call the validate method of the entity API before importing a record. In practice, this means all validation constraints are enforced at the entity and field level.

The advantage of this is that you are guaranteed to have *content that respects the validation put in place in the new site*. The disadvantage is that the *checks are very strict and content that does not comply is not migrated at all*. Consider the following examples of migrations with entity validation on:

- **Required fields**: Drupal will refuse to import content if a single required field is missing (or it has an invalid value).
- **Text formats**: Drupal will check if the author of the node has *permission to use* the text format specified in rich text fields.
- **Relationship among migrations**: Drupal will fail to import a media entity that points to a file whose *status* is not permanent. This example is interesting because migrating non-permanent files (those with a `status` value of `0`) does not produce a validation error in the *file migration*. But attempting to *migrate media entities* that point to non-permanent files will fail validation.

In all these cases, the whole content entity (node, user, taxonomy term, media, etc.) will be skipped. Having entity validation enabled is useful to detect errors, especially early on in the project. But it often becomes impractical because of issues in the source data that cannot be fixed. In our example, setting a date for the two nodes would be quick and easy. In real life projects, it might be impractical to retroactively populate fields for content that had been created over the years. In some cases, it would be straight out impossible. Let's say a new required field was introduced to a content type years after it was initially created. Old content might not have a suitable value to use for the new field.

On this topic, I fondly remember migrating a Drupal 6 project with very little field validation in place. This was a humbling learning experience because things I took for granted from Drupal 7 did not exist in Drupal 6. The muscle memory I had developed around the structure of the database no longer applied. Content types and CCK fields had a different table structure back then. Moreover, a lot of content was entered in free-form text fields. We had to parse

unstructured data and perform complex data transformations to make it fit the rigid content structure defined in the new site.

Before closing, I want to clarify that by no means I pretend to paint a bad picture about entity validation. I strive to use it in all my migration projects. I turn it off only when it's impossible or impractical to keep it enabled. Even in those cases, it already helped identify errors in my migration or inconsistencies in the source data. When a content entity fails to import due to validation errors, it logs a detailed report to the migration *message table*, which can be retrieved using the `drush migration:messages` command.

Are you up for a challenge? Enable entity validation for all the content migrations in the series and fix any issues that might arise. For the node migrations, you would configure the destination plugin like this:

```
destination:
 plugin: 'entity:node'
 default_bundle: DRUPAL_10_CONTENT_TYPE_MACHINE_NAME
 validate: true
```

Today, we wrote our 25th migration. Congrats on making it this far! Your perseverance is worthy of admiration. There is only one more article to go. Ready for the season finale?

31 - Debugging tips, performance considerations, Drupal CMS, AI-assisted migrations and more!

Welcome to the last article in the series. Today, we'll wrap up by covering how to debug migrations, performance considerations, and the importance of checking your site for broken links before launch. We'll also briefly discuss migrating into Drupal CMS and using AI to assist with the migration process. Before doing so, let's talk about other migrations that you might need to implement in your project.

More migrations

We wrote 25 migrations throughout the series, but no need to stop here. Below is a list of common – and not so common – migrations that might need to be implemented:

- **URL aliases:** By default, content entity migrations do not include importing their URL aliases. A dedicated d7_url_alias migration exists for that purpose. It queries Drupal 7's `url_alias` table and creates Drupal 10's URL alias (`path_alias`) entities.
- **Pathauto:** The pathauto_settings and pathauto_patterns migrations are responsible for importing pathauto's module configuration and URL patterns, respectively.
- **Redirects:** The d7_path_redirect migration will query Drupal 7's `redirect` table and create Drupal 10's `redirect` entities.
- **Menus:** The menu and menu_links migrations are responsible for importing menu (configuration) and menu items (content) entities, respectively.
- **Metatags:** The metatag module provides some configuration-related migrations. They include adding a `metatags` field to entities which gets populated when importing the content. For example, if the metatag module is enabled in Drupal 10 before starting the automated upgrade

process, the generated `upgrade_d7_node_page` migration will account for importing metatags data for nodes of type basic page.

- **Blocks:** The d7_custom_block migration takes care of importing block content entities into the `basic` block type. That block type comes out of the box with the `standard` installation profile or can be added by applying basic_block_type recipe. Note that blocks have two related configuration entities: `block_content_type` and `block`. The first is the block type. The second stores block placement data within a theme. The block_content_type migrations and d7_block included in the block_content and block modules account for importing those configuration entities.
- **Comments:** The comment module provides multiple migrations to import comment-related data. Most of them handle importing configuration. The d7_comment migration is the one responsible for importing content entities. Note that the `comment` entity in Drupal 10 is fieldable and can have multiple bundles. That is, different *comment types* might exist similar to having multiple *content types*, each with a different set of attached fields. Unless you are customizing your migration, you might end up with multiple comment migrations after an automated migration. Refer to this documentation page to learn more about some caveats associated with migrating comments.

In all cases above, we link to the migration plugins included with each module. When running the automated migration process with the Migrate Upgrade module you will get derived migrations prefixed, by default, by `upgrade_`. For example, the `d7_url_alias` migration will be generated as `upgrade_d7_url_alias`. In some cases, migration derivers are implemented, like in the case of nodes where a separate migration is created for each content type. For instance, `upgrade_d7_node_article` and `upgrade_d7_node_page`.

Some upgrade paths are not fully automated, but there are modules that can help get you started. Below are some examples:

- For `webforms` use the webform migrate module.
- For `views` use the views migration module.
- For `media` use the media migration or migrate media handler module. The Tag1 Team Talks podcast on file and media migrations contains lots of useful information on migrating media.

Debugging migrations

Migrations have a lot of moving pieces and, as noted in previous articles, they interact with other APIs in Drupal. For example, while running the migration for a content type, you will interact with the Entity and Field APIs. If content validation is enabled, the entity validation API will also come into play. Another important factor to consider is which modules you have enabled in Drupal 10 at the time the migrations are executed. Hook implementations and event subscribers will be called either by the Migrate API itself or other subsystems.

Debugging is an essential tool for development in general. To learn more about this topic, check out this presentation by Randy Ray. For migration specific tips, review this documentation page.

When it comes to debugging Drupal migrations, I often perform the following four actions.

1. Check if the migration's definition is what you expect

What do I mean by this? That the migration YAML file you see in your editor accurately represents what the Migrate API is executing during import operations. Migrations are plugins and as such they can be modified dynamically. Altering a migration plugin allows you to replace part of the migration definition with values read from settings, configuration, or environment variables. An example would be injecting API keys into migrations that interact with third-party services. This prevents exposing the API keys in plain text, as part of the migration file, and committing them to the code repository.

Modifying migrations dynamically is usually done by implementing hook_migration_plugins_alter, a technique used by modules like migrate plus, migrate_tools, paragraphs, media migration, commerce migrate, among others. There is nothing wrong in altering migrations dynamically. In fact, it's to be expected given there is an API for it. Problems could arise when you are altering migrations in your custom code without considering that other enabled modules might also be doing the same.

You can use the migration plugin manager to check the final state of a migration after all dynamic modifications. That is what ultimately gets called when the import operation is triggered. Back in article 16, we explained how to

obtain the definition of a migration plugin, along with other plugin types used by the Migrate API.

As a reminder, refer to the following snippet:

```
# List of migration plugin ids.
ddev drush php:eval "print_r(array_keys(\Drupal::service('plugin.manager.migr\
ation')->getDefinitions()));"

# Details on a specific migration plugin id.
ddev drush php:eval "print_r(\Drupal::service('plugin.manager.migration')->ge\
tDefinitions()['MIGRATION_ID']);"
```

2. Check the query executed against Drupal 7

All the migrations presented in this series use source plugins that fetch data from a Drupal 7 database. If you suspect that something is not right with the data being retrieved, it's a good idea to check the query the migration executes against the Drupal 7 site. You can do this by implementing hook_query_TAG_-alter.

In our custom `tag1_migration` module, the following code will log the query executed by our migration:

```
/**
 * Implements hook_query_TAG_alter().
 *
 * // @see \Drupal\migrate\Plugin\migrate\source\SqlBase::prepareQuery
 */
function tag1_migration_query_migrate_alter(AlterableInterface $query) {
  /** @var \Drupal\migrate\Plugin\MigrationInterface $migration */
  $migration = $query->getMetaData('migration');
  $migration_tags = $migration->getMigrationTags();

  // Only act when a migration uses a tag related to our project.
  if (count(array_intersect(['tag1_content', 'tag1_configuration'], $migratio\
n_tags)) === 0) {
    return FALSE;
  }

  $placeholder_ids = array_keys($query->getArguments());
  $placeholder_values = [];

  foreach ($query->getArguments() as $arg) {
```

```
    $placeholder_values[] = "'$arg'";
  }

  $query_string = str_replace($placeholder_ids, $placeholder_values, (string)\
 $query);
  $query_string = preg_replace('/["\[\]{}]/', '', $query_string);

  \Drupal::logger('my_custom_module')->debug('The @migration_id migration exe\
cuted the following query: @query', [
    '@migration_id' => $migration->getPluginId(),
    '@query' => $query_string,
  ]);
}
```

A detailed review of the PHP code for hook implementation will be left as an exercise to the curious reader. The one thing I want to highlight is that the snippet above would **only log the query assembled by the `query` method of the source plugin**. Most notably, it would not log any queries that could be executed in the `prepareRow` of the source plugin. In fact, executing a single migration can trigger multiple queries against the source site. The snippet above focuses on the one that is arguably most important.

If you need to inspect extra data fetched by the `prepareRow` method or implementations of hook_migrate_prepare_row, it's best to resort to a debugger.

3. Use a debugger like XDebug

Using a proper debugger like XDebug is imperative to debug Drupal migrations effectively. In addition to identifying and fixing errors, debuggers are a fantastic tool to study and understand how a system works. Over the years, I have dedicated a good amount of time step-debugging migration-related code to understand how the Migrate API works and how it interacts with other Drupal sub-systems.

When debugging migrations, the most common locations where I add breakpoints are:

- **The import method of the \Drupal\migrate\MigrateExecutable class.** In this method, the source plugin retrieves data, the process pipeline is executed, and the processed data is sent to the destination plugin, which in our examples creates Drupal entities. My preferred location to add

a breakpoint is in the line where the import method is invoked on the destination plugin. At this point, you can inspect the `$row` variable to review all data retrieved by the source plugin, including alterations while preparing the row, and the result of applying the process pipeline.

- **The transform method in process plugins.** Step-debugging this method will let you see which data is sent to it, how it is manipulated, and what is the returned value. This is particularly useful when a process plugin exposes configuration options that affect how they behave. This technique also lets you investigate why you do not get the expected results after chaining multiple plugins. A common cause of confusion in process plugin chains is knowing whether a process plugin can process multiple values or not (as explained in article 27).

Technical note: It's possible for a process plugin to not include a `transform` method. When this is the case, you need to specify a value for the `method` configuration option. The base class for process plugins then checks if the indicated method exists in the class that defined the process plugin and calls it. An example of this is skip_on_empty, which needs to be configured with either method: row or method: process.

When using a debugger to troubleshoot code, make sure to turn it off when you finish reviewing what needs to be debugged. XDebug and similar tools add overhead to execution of the application. In the case of migrations, having a debugger on while running a full migration can slow down the process significantly. That is why Drush disables Xdebug by default. If you are running your migration from the command line using Drush, you will need to use the `--xdebug` flag like this:

```
drush migrate:import upgrade_d7_user --xdebug
```

I also want to highlight two modules that can assist with debugging migrations:

1. **Migrate Devel** helps with inspecting retrieved source data, the result of the process pipeline, and the destination ID values.
2. **Migrate Sandbox** helps assemble and check complex process pipelines with either test data or real records retrieved by a migration source plugin.

4. Inspecting migration ID map tables

Sometimes you do not notice issues in your migrations until later on. In these cases, migration ID map tables are available to help troubleshoot. These tables can be used to understand why an entity was not migrated or why a field was not populated as expected.

When a migration is executed, the Migrate API creates two tables automatically: an ID map table and a messages table. We discussed migration messages in article 14. As for ID map tables, they are used to keep track of which entities from the source have already been processed. For each source record, the system tracks whether the import process was successful or not. In cases where the import succeeds, destination ID identifiers are also stored for each source record in the map table.

To better explain the above, consider the `upgrade_d7_user` migration from article 25. Assume we modify it slightly so that entity IDs are not preserved. In its process pipeline, the migration skips Drupal 7 users that were blocked. Below is a sample of how the ID map table for this migration would look like:

```
-- SELECT sourceid1, destid1, source_row_status, rollback_action, last_import\
ed
-- FROM migrate_map_upgrade_d7_user
-- WHERE sourceid1;

+-----------+---------+-------------------+-----------------+---------------+
| sourceid1 | destid1 | source_row_status | rollback_action | last_imported |
+-----------+---------+-------------------+-----------------+---------------+
|        13 |      17 |                 0 |               0 |    1736882479 |
|        19 |    NULL |                 2 |               0 |    1736882479 |
+-----------+---------+-------------------+-----------------+---------------+
```

To get started, the default name for the ID map table is the migration ID prefixed by `migrate_map_`. For `upgrade_d7_user` the corresponding ID table is `migrate_map_upgrade_d7_user`. If a migration ID is too long, the ID map and messages table names will be truncated. Check this core issue if you stumble into issues with mapping lookup operations.

The source plugin for the user migration defines a single unique identifier: Drupal 7's user ID. The destination plugin that creates Drupal 10 users also leverages the user ID as its unique identifier. The above table indicates that Drupal 7's user with `uid` 13 was imported into Drupal 10 as user with `uid` 17.

And Drupal 7's user with `uid` 19 was not migrated. While not enforced by the API, it's possible to have a note in the messages table indicating why the record was skipped (as discussed in article 14).

The values of the `source_row_status` columns correspond to constants defined in the MigrateIdMapInterface:

- The value 0 indicates that the import of the row was successful.
- The value 1 indicates that the row needs to be updated.
- The value 2 indicates that the import of the row was ignored.
- The value 3 indicates that the import of the row failed.

The values of the `rollback_action` columns correspond to constants defined in the MigrateIdMapInterface:

- The value 0 indicates that the data for the row is to be deleted upon rollback of its migration.
- The value 1 indicates that the data for the row is to be preserved upon rollback of its migration.

The `last_imported` column contains a UNIX timestamp of the last time the row was imported. The ID map table also contains a `source_ids_hash`, which stores a hash of the source IDs. Another column named `hash` might contain a hash of all source row data for detecting changes. This column is populated when the track_changes configuration option is set to `TRUE` in your source plugin.

Stepping back to the topic of troubleshooting migrations, the migrate ID map and messages table are very useful. They store key information about all the records processed by the migrations and let you know if they were successfully imported or not. Many times I have been asked why an entity was not migrated or why a field was not populated. This does not necessarily mean a problem in the migration. Some elements might be excluded on purpose and the client just needs to be reminded of the conditions that led to such an outcome.

Once the migration is finished, you could delete the migration tables. However, I highly recommend keeping them for a while after launch. They are the paper trail of your migration efforts. If you decide to delete these tables, take a look at this documentation page for information on how to do it.

Migration performance

Over the years, I have had the opportunity to work on very large migration projects. In some cases, the sites have been running for more than a decade. The volume of content that has accumulated is so large that running a full migration can take multiple days to complete. The tips below will yield greater performance gains for large migrations, but they can also be applied in most projects.

Let's start our discussion on performance by considering how we execute the migrations. The migration runners present in Drush core and modules like Migrate Tools allow you to execute multiple migrations in a single command. Some ways to do this is using the `--tag`, `--group`, or `--execute-dependencies` flags, depending on the runner of choice. Instead of doing this, it's better to execute each migration individually. I usually create a bash script where I list the migrations that need to be executed like in the following snippet:

```bash
#!/usr/bin/env bash

echo "Migration started at $(date)"

drush migrate:import upgrade_d7_file

drush migrate:import upgrade_d7_user

drush migrate:import upgrade_d7_taxonomy_term_tags

drush migrate:import upgrade_d7_node_article
drush migrate:import upgrade_d7_node_page

drush migrate:import upgrade_d7_url_alias

echo "Migration completed at $(date)"
```

Make sure to list the migrations in the proper order they need to be executed. If you want to take it a step further, you can adjust the script to run some migrations in parallel. Running migrations for the same destination entity type in at the same time is generally fine. If you go this route, you need to be mindful about dependencies among migrations, especially when there are lookup operations in the process pipeline.

Before going further, let's pause for a bit to discuss a topic that we have not given much attention to: **migration runners**. By now, executing `drush migrate:import` might be second nature to you. But where does this command come from? And, are there alternatives? The import command, and other migration related ones, have been part of Drush core since version 10.4.

Before that, a common way to run migrations was using the Migrate Tools module. In addition to providing Drush commands, this module also offers a user interface to running migrations. Other runners include the modules Migrate Upgrade, Migrate Scheduler, Migrate Manifest, Migrate Cron, and Migrate source UI. Reading about the features of each of these modules will be left as an exercise to the curious reader. The reason I bring this topic now is to share an anecdote related to migration runners and performance.

Some time ago I was working on a very large project. A few content types had millions of nodes associated with them. Executing those migrations would take hours – and that was expected. What was strange is that after the last node was processed, the migration seemed to halt for a few extra hours.

With the help of a colleague, the XDebug profiler, and the profile data visualization tool KCachegrind, we were able to identify the bottleneck. In short, the migration runner we were using had a post-import operation that looped over all the records written to the migration's ID map table. In practice, this meant performing extra operations on millions of records. After reviewing the code, it was determined that the extra operation was not needed in our project. By changing from one migration runner to another, we were able to save a significant amount of time when running a full migration.

The moral of this story is **take time to understand the tools you use, both for executing the migration and for debugging issues when they inevitably arise**. Nowadays, if you use DDEV (which we highly recommend) you can visually profile your application using XHGui – a graphical interface for XHProf.

In the above example, we see how skipping a post-import operation saved us a lot of time. Are there other things we can bypass to speed up the migration executing time? Yes. In most cases, it will depend on your specific setup, what modules are enabled at the time the migration is executed, and how they are configured.

Remember that the migrate API will interact with other Drupal APIs. This means enabled modules can react to entity CRUD operations. As an example, consider a site using a Search API. **You should delay the indexing of content**

after the migration is fully completed. Some of the ways to accomplish this are:

1. When adding an index, configure it so that elements are not indexed immediately.
2. Disable the index via a configuration override.
3. Disable the index using Drush commands. This can be incorporated in the same script that is used to execute the migrations as follows:

```
#!/usr/bin/env bash

echo "Migration started at $(date)"

drush search-api:disable-all

# Execute migrations with drush migrate:import ...

drush search-api:enable-all

echo "Migration completed at $(date)"
```

Delaying indexing of migrating content will have a greater performance boost when you're using external search backends like Apache Solr. You can do something similar with any operation that will talk to an external API when entities are being created in Drupal. Some internal Drupal operations can be quite expensive too. For example, modules that define access controls might require node access permissions need to be rebuilt. Other operations might not be as expensive but might improve performance, like deferring the generation of thumbnails for media entities (as described in article 28).

But do not take any of these examples as universal truths. Assess the requirements of *your* projects and act accordingly. **While creating the migration scripts, you might want to focus on the integrity of the data**. Once the data is migrated, you then proceed to implement and test other aspects of the project like search.

In the snippet above, we took advantage of Drush commands provided by the Search API module to disable and enable indexing at the appropriate time during the migration process. Not every module provides commands that can be used to toggle certain features on demand. An option is to disable and re-enable modules that perform expensive operations. However, this

is far from ideal, because we are changing the site's configuration and its overall behaviour. A better alternative would be to disable some hooks by leveraging hook_module_implements_alter. If you follow this path, consider using Migrate Boost, which temporarily disables hooks only while running migrations.

Is there anything else we can disable? Yes. Before covering that, let me reiterate. **Do not disable or remove things blindly**. Carefully consider the implications of the actions you take. We are providing general advice here. At the very minimum, you should **measure the impact of these changes, one at a time, to determine if it's making a real impact on improving the performance in your migration project**.

That said, you can disable entity caching and migration discovery. The first can lead to performance improvements, because Drupal will skip caching the recently migrated entities. Caches can be warmed in bulk after the migration is complete. Disabling the migration discovery caches is mostly a convenience while developing migration. By doing so, you do not need to constantly rebuild caches or sync configuration when using Migrate Plus's migration configuration entities. To disable these cache bins, copy `web/sites/example.settings.local.php` as `web/sites/default/settings.local.php`. Make sure this file is included by `web/sites/default/settings.php`.

Back in the newly created `settings.local.php` file have this code:

```
$settings['container_yamls'][] = DRUPAL_ROOT . '/sites/development.services.y\
ml';
$settings['cache']['bins']['entity'] = 'cache.backend.null';
$settings['cache']['bins']['discovery_migration'] = 'cache.backend.memory';
```

At the risk of stating the obvious, the above should only be done during local development. *Never disable entity cache in a production environment* unless you are completely certain of the implications of doing so.

The last thing I want to discuss on the topic of performance is where to execute the migrations. Ideally, the Drupal 7 database is located in the same server where the Drupal 10 site is hosted. If possible, the Drupal 7 uploaded files should also be located in the same server (as discussed in article 24). Depending on the size of the project, and data privacy requirements, it's possible to run the migrations in a local environment in one of the developer's machines. Alternatively, you can provision a temporary server where the migration can be executed.

If possible, avoid running the migration directly in your hosting provider. This isn't about not running migrations in a production environment. Instead, do not have two projects hosted on different servers and run a migration directly between them. Only twice I have seen migrations being executed directly in staging servers within the client's infrastructure. This was a hard requirement derived from very strict data protection policies.

Before launch, check your site for broken links

Launching a site requires thoughtful considerations on multiple fronts: accessibility, infrastructure, security, performance, search engine optimization, marketing automation, up-time monitoring, third-party services integration, and more. Covering all of them is outside the scope of the series. We recommend taking a look at the checklist for launching a site included in the Drupal.org documentation guide on administering sites to get you started.

That said, there is one thing you *must* do before going live with the new site: **account for URL changes**.

Once I read the case of an ecommerce platform losing over 50% of its revenue after updating their site. They attributed the loss to a change in the URL structure for products. After years of being highly ranked in search engines, the URLs that were indexed had changed with no redirects in place. This might seem as an extreme, unlikely example until things go wrong. In the context of a custom Drupal 7 to 10 migration, changes in URL structure are common. This can be caused by:

- **Changes in entity types**. For example, creating users from nodes. A redirect would need to be added from the old node to the new user. Consider both the node's canonical URL (e.g. `/node/7`) as well as the node's path alias (e.g. `/hello-world`).
- **Changes in pathauto patterns**. For example, a content type whose pathauto pattern changed. When the node migration happens, if the pathauto module is enabled in the new site, the node will receive a path alias based on the new patterns.

To mitigate the impact of broken links, consider the following:

- **Pay close attention to the URL alias, pathauto, and redirect migrations.** It's very likely that you will have to customize them. Also likely that you will have to create new, custom migrations altogether. In a recent project I used a custom migration to create Drupal 11 redirects out of Drupal 7 path aliases for content types whose URL pattern had changed. This involved a custom source plugin that joined data from Drupal 7's `url_alias` and `node` tables. The data retrieved was used by a custom migration to create `redirect` entities in the new site.
- **Involve members from multiple teams.** While this series has been focused on the technical aspects of migrations, upgrading a site to a new platform does not happen in a vacuum. Large projects tend to have dedicated teams focused on analytics and marketing efforts. Whether that is the case for your project or not, try to get a report of the most visited pages and sections in the site so that the technical team can pay extra attention to them. Involve the quality assurance team to assist with manual and automated reviews. Speaking of which...
- **Leverage tools for automated checks.** Within Drupal, the Link checker can find broken links in content entities. There is also a report available at `/admin/reports/page-not-found` that lists the top requested URLs that do not exist on the site. Outside of Drupal, you can use reports from your analytics software to identify broken links. Also, website crawling software can flag other types of errors. For instance, they can identify duplicate content being served from multiple URLs. Some tools will assist you in taking preventive action while others will help in correcting oversights in your migration. As a general recommendation, have tools at your disposal to periodically check for broken links before and after the migration.

Drupal CMS and AI-assisted migrations

The series has focused on creating custom migrations for projects built on top of Drupal core. By now, it's likely that you have heard about Drupal CMS, a ready-to-use platform created to empower marketing teams, content creators and site builders. Dries Buytaert, Drupal founder and project lead, says the goal of the new platform is to be the gold standard for no-code website building.

When you install Drupal CMS, you get features such as advanced media management, SEO tools, AI-driven website building, consent management,

analytics, search, automatic updates, and more. To learn more about Drupal CMS as a product, read its official documentation and join the conversation in the multiple Drupal slack channels: `drupal-cms-development`, `drupal-cms-support`, `drupal-cms-marketplace`, and `drupal-cms-templates`.

One of the landmark features of Drupal CMS is the use of recipes to provide an out-of-the-box experience following community best practices. Among other things, **a newly installed Drupal CMS site will have a known and consistent content model**. While there are no promises that the content model will remain unchanged, by the time features are included in Drupal CMS they are very polished. The benefit of having a stable content model is that it's easier to build tools to migrate content into it.

At DrupalCon Atlanta 2025, a new initiative was launched to provide automated migrations into Drupal CMS and Experience Builder – from paragraphs, layout builder, and more. Some of these automated migrations will not even use the Migrate API! Instead, they will leverage artificial intelligence (AI) to help with the task.

Going back in time to DrupalCon Barcelona 2024, the DriesNote included a demo on migrating content into Drupal using AI. By DrupalCon Atlanta 2025, AI integration into Drupal CMS has evolved significantly beyond content migration. Remember that Drupal CMS is built on top of Drupal core. As such, these AI features can be leveraged on any Drupal project. AI-assisted migrations, as well as AI-assisted Drupal development, are a reality today.

Anything that I write about artificial intelligence in Drupal is likely to be outdated in the near future. If leveraging AI for Drupal development is something that interests you, I recommend this presentation by Fabian Franz and Marco Molinari. Also, follow the development of the AI module, check its official documentation, and join the conversation in the `#ai` channel in Drupal slack.

To assess how AI can help today with migrations that leverage the Migrate API, I asked one of the most popular AI-assistants some of the questions we answered during this migration series. Things like how to migrate Drupal 7 nodes as Drupal 10 users, how to create a custom source plugin that excludes unpublished nodes, how to create a custom process plugin that combines data from multiple Drupal 7 fields into a single Drupal 10 field, and more. The results were mixed. Some could achieve the desired results with a little tweaking while others were objectively wrong.

AI tools can provide a lot of help, but do not trust them blindly. While they can guide you in the right direction, at times their output is not the most efficient solution. AI is a promising technology, and it will continue improving. Check it out for yourself. As of today, though, these tools are not a substitute for taking the time to learn the Drupal's Migrate API with a guide like ours.

Thank you!

And that concludes the Drupal 7 to Drupal 10 migration guide. Thank you very much for joining us in this learning experience! We hope you learned a lot and gained the knowledge you need to migrate your next project with confidence.

If you need personalized help with your project, the Tag1 Team is ready to assist. Contact us to learn how we can help you with your migration.